



BITS Pilani
Pilani Campus

Computer Organization & Architecture

R

Computer Architecture and Organization-1



- **Architecture** is those attributes visible to the programmer and having direct impact on the logical execution of the program
 - Instruction set, Number of bits used for data types, I/O mechanisms, Memory addressing techniques
 - e.g. x86 architecture, IBM/360 architecture
- **Organization** is implementation of computer system in terms of its interconnection of functional units
 - Control signals
 - Interfaces between computer and peripherals
 - Memory technology

Computer Architecture and Organization-2



- **Architecture Question?**
 - Is there a multiply/division instruction available?
- **Organization Question?**
 - Is multiplication implemented by separate hardware or is it done by repeated addition?



Computer Organization vs. Computer Architecture

- For example, it is an architectural design issue whether a computer will have a multiply instruction.
- It is an organizational issue whether that instruction will be implemented by a special multiply unit or by a mechanism that makes repeated use of the add unit of the system
- Many computer manufacturers offer a family of computer models, all with the same architecture but with differences in organization.
- Different models in the family have different price and performance characteristics.
- A prominent example of both these phenomena is the IBM system /370 architecture. It is first introduced in 1970 and included a number of models.
- This gives code compatibility
 - At least backwards
- Organization differs between different versions



Structure and Functions of various components of a computer system.

- Structure is the way in which components relate to each other.
- ✓ Structure = static relations among components
- Function is the operation of individual components as part of the structure.
- ✓ Function = dynamic behaviour of each component
- In terms of description, we have two choices, starting at the bottom and building up to a complete description or beginning with a top view and decomposing the system into the subparts.
- The computer system will be described from top down.

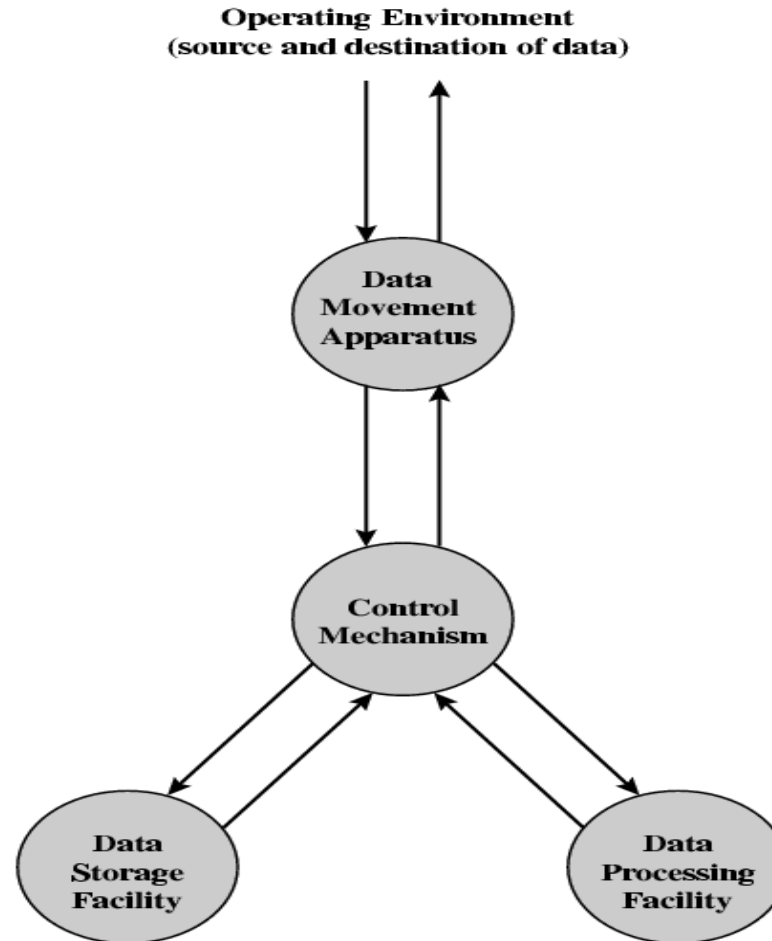
How to Describe Computer System?



- Computer is a complex system!!!
 - Contains millions of electronics components
- Function is the operation of individual components as part of the structure
- Structure is the way in which components are related to each other

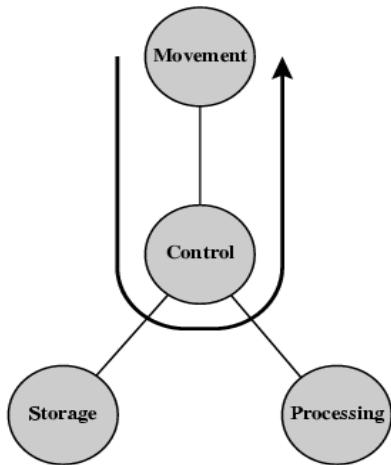
Functional View of Computer

- All computer functions are:
 - Data processing
 - Data storage
 - Data movement
 - Control

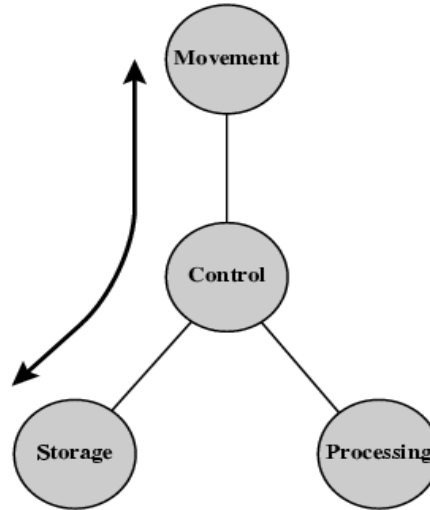


Computer Operations

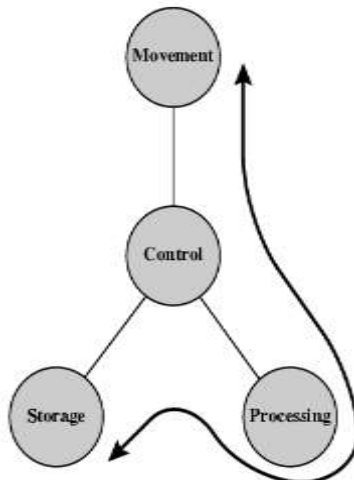
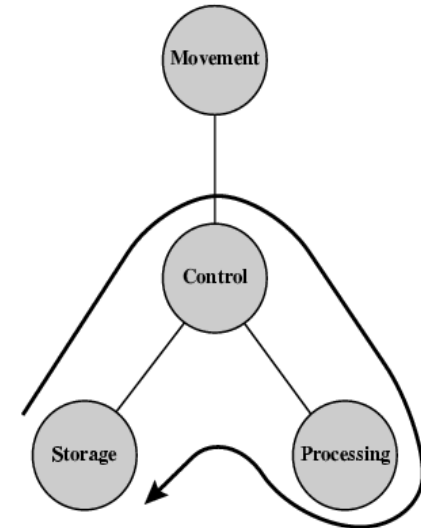
Operations (a) Data movement



Operations (b) Storage



Operation (c) Processing from/to storage

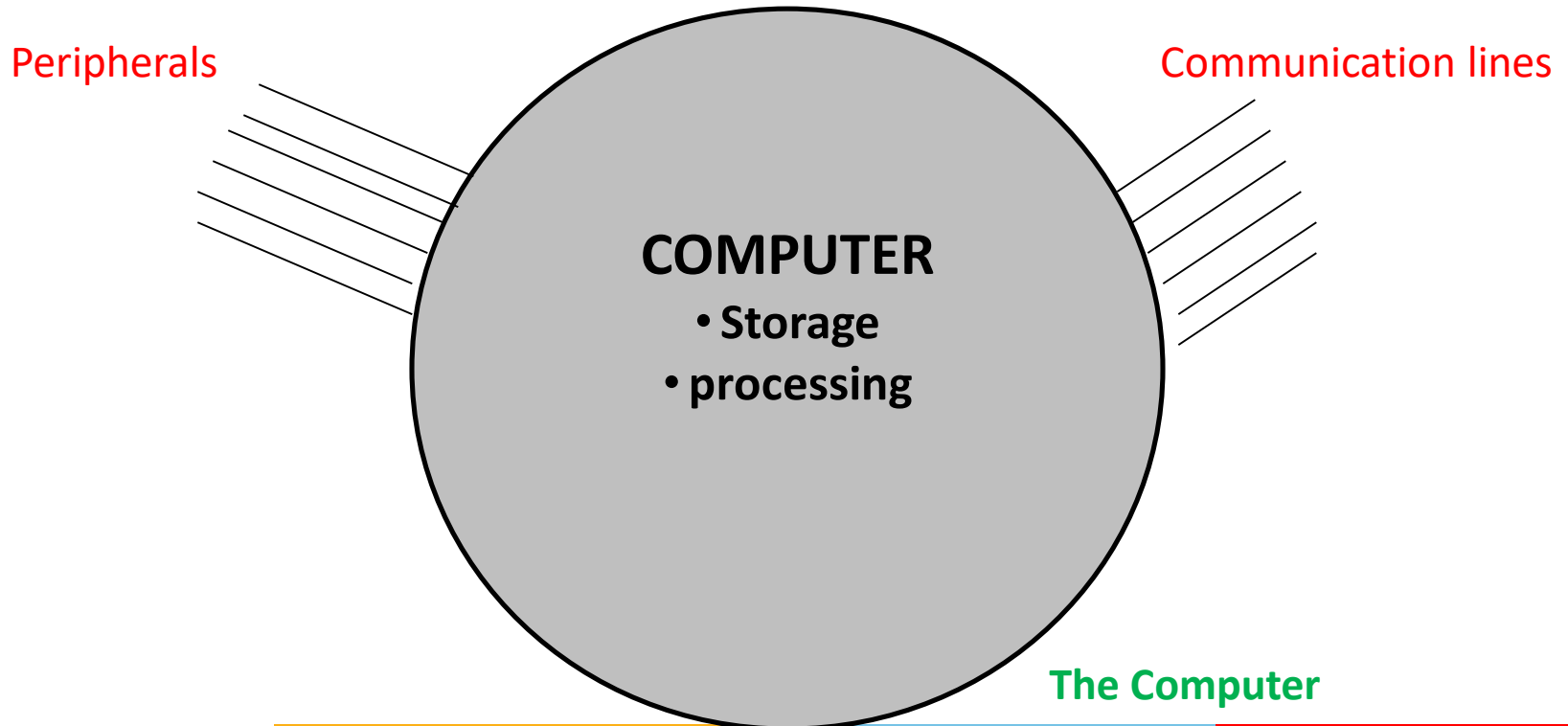


Operation (d)
Processing from storage to I/O

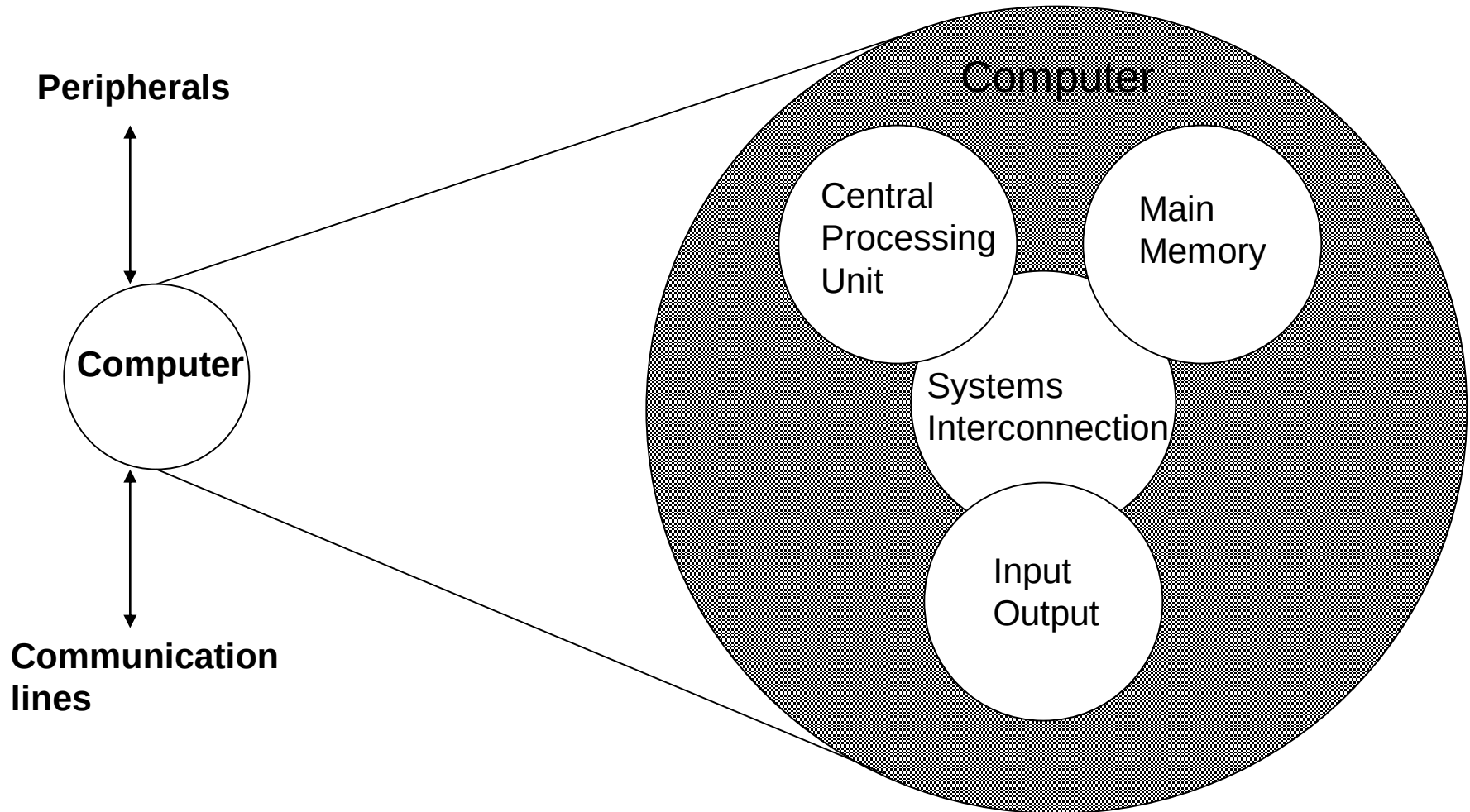
Structure



- The figure shows the simplest depiction of a computer.
- The computer interacts in some fashion with its external environment.
- In general, all of its linkages to the external environment can be classified as peripheral devices or communication lines.

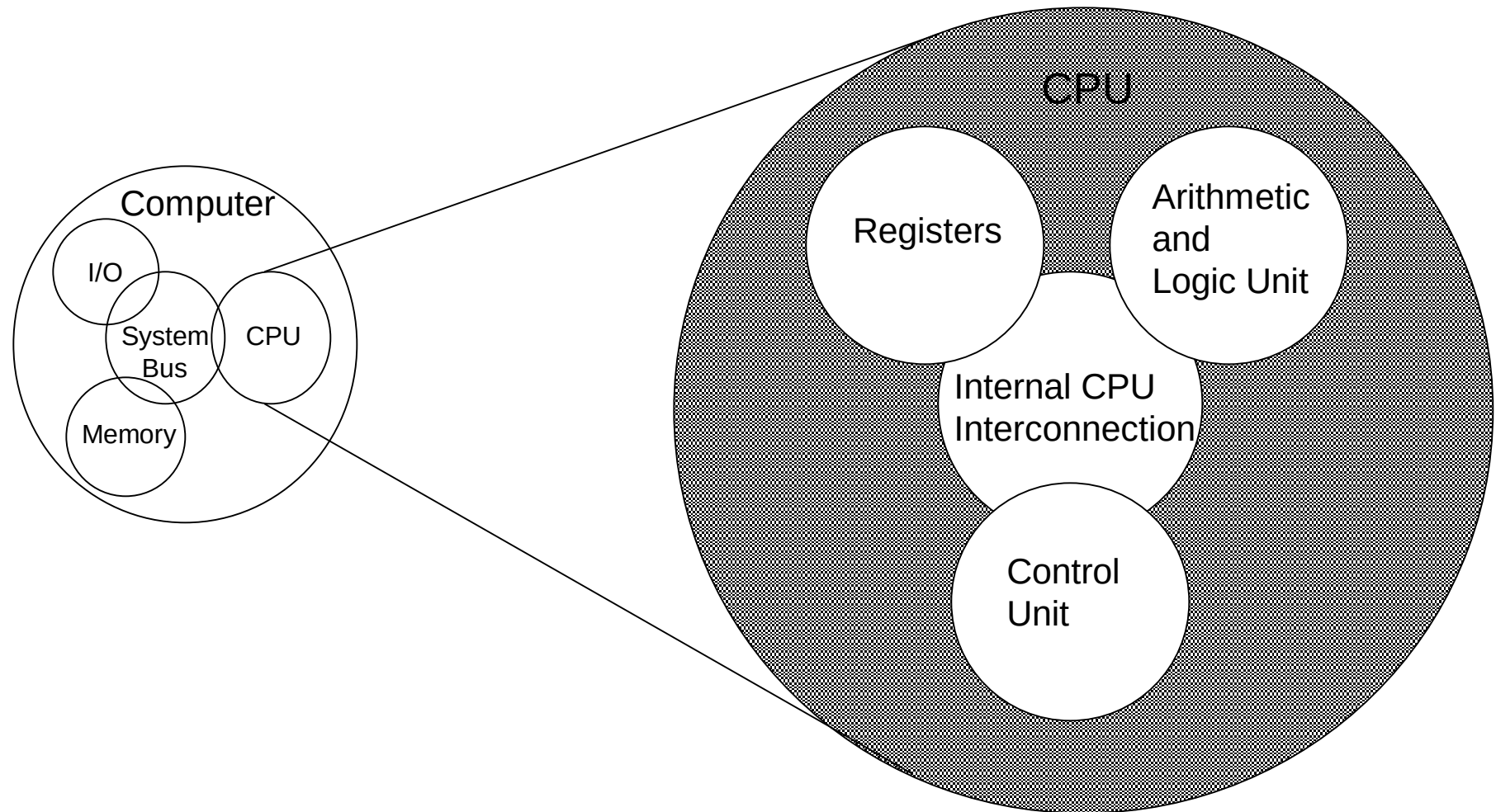


Structural View of a Computer



- There are four main structural components:
- **Central processing unit (CPU):** controls the operation of the computer and performs its data processing functions; often simply referred to as **processor**.
- **Main memory:** Stores data.
- **I/O:** Moves data between the computer and its external environment.
- **System Interconnection:** Some mechanism that provides for communication among CPU, main memory and I/O.

Structural View of the CPU





Structural View of the CPU

- A common example of system interconnection is by means of a **system bus**, consisting of number of conducting wires to which all the other components attach.
- The most complex component is the CPU and its major structural components are as follows:
- **Control unit:** Controls the operation of the CPU and hence the computer.
- **Arithmetic and logic unit (ALU):** Performs the computers data processing functions.
- **Registers:** Provides storage internal to the CPU.
- **CPU Interconnection:** Some mechanism that provides for communication among the control unit, ALU and registers.



Brief History of Computers

- ENIAC was the first general purpose electronic digital computer
 - By John Mauchly and John Eckert in 1946
- Von Neumann Machine (1946) called as IAS(Institute of advanced study)
 - Based on stored program concept
- PDP-1 Computer (1957)
 - Developed by Digital Equipment Corporation (DEC)
 - First step towards mini computers
- IBM SYSTEM/360 (1964)
 - First planned family of computers

ENIAC - background

- Electronic Numerical Integrator And Computer
- Eckert and Mauchly
- University of Pennsylvania
- Trajectory tables for weapons
- Started 1943
- Finished 1946
 - Too late for war effort
- Used until 1955

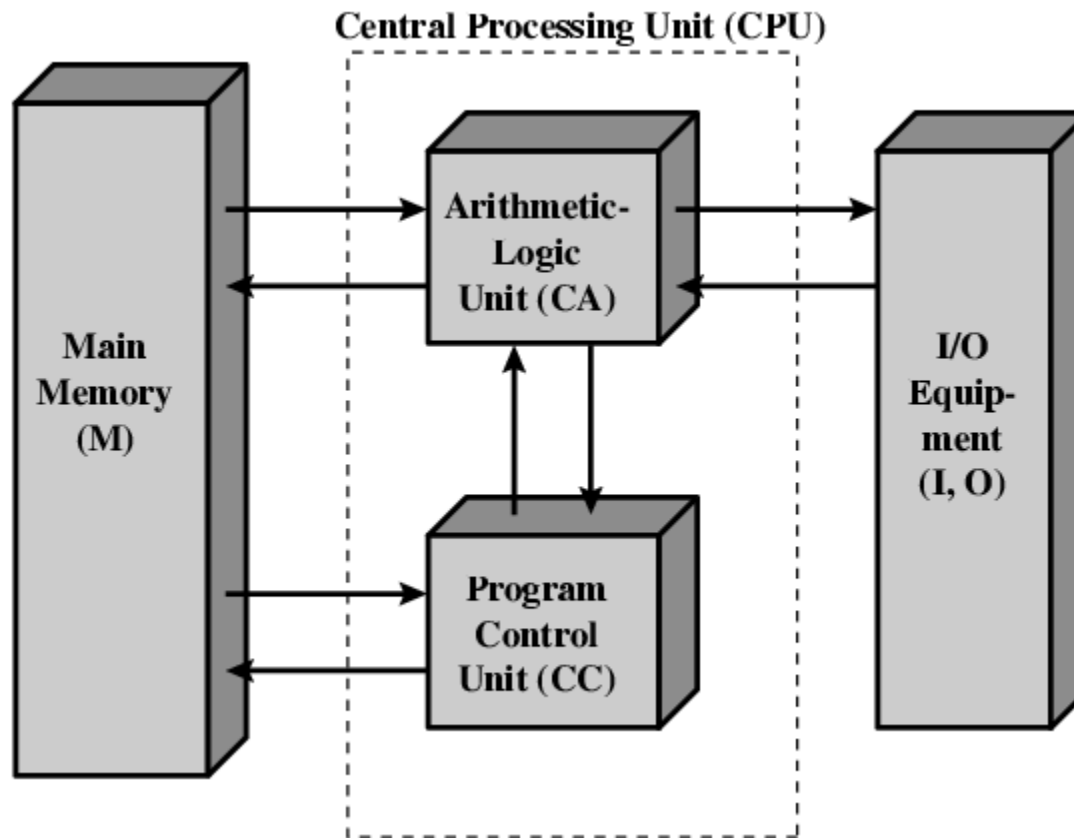
ENIAC - details

- Decimal (not binary)
- 20 accumulators of 10 digits
- Programmed manually by switches
- 18,000 vacuum tubes
- 30 tons
- 15,000 square feet
- 140 kW power consumption
- 5,000 additions per second

von Neumann/Turing

- Stored Program concept
- Main memory storing programs and data
- ALU operating on binary data
- Control unit interpreting instructions from memory and executing
- Input and output equipment operated by control unit
- Princeton Institute for Advanced Studies
 - IAS (Institute for Advanced Study)
- Completed 1952

Structure of von Neumann machine

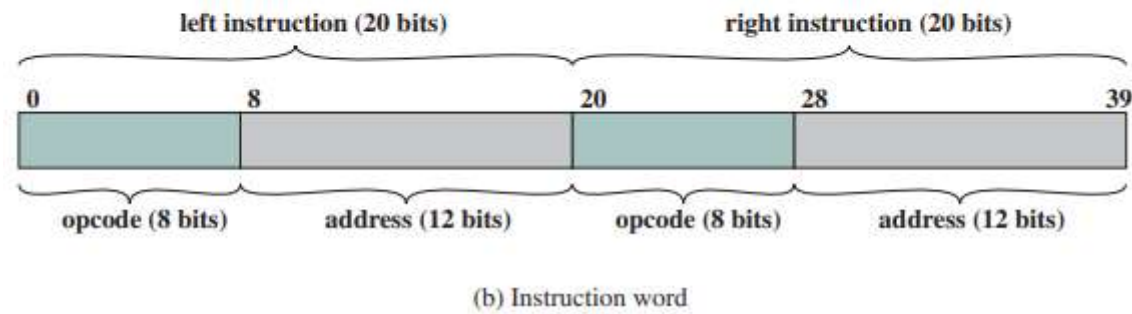
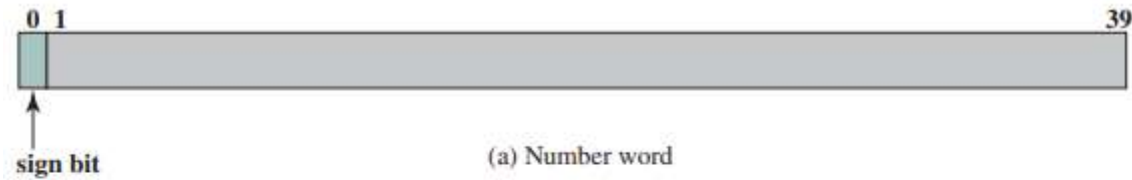




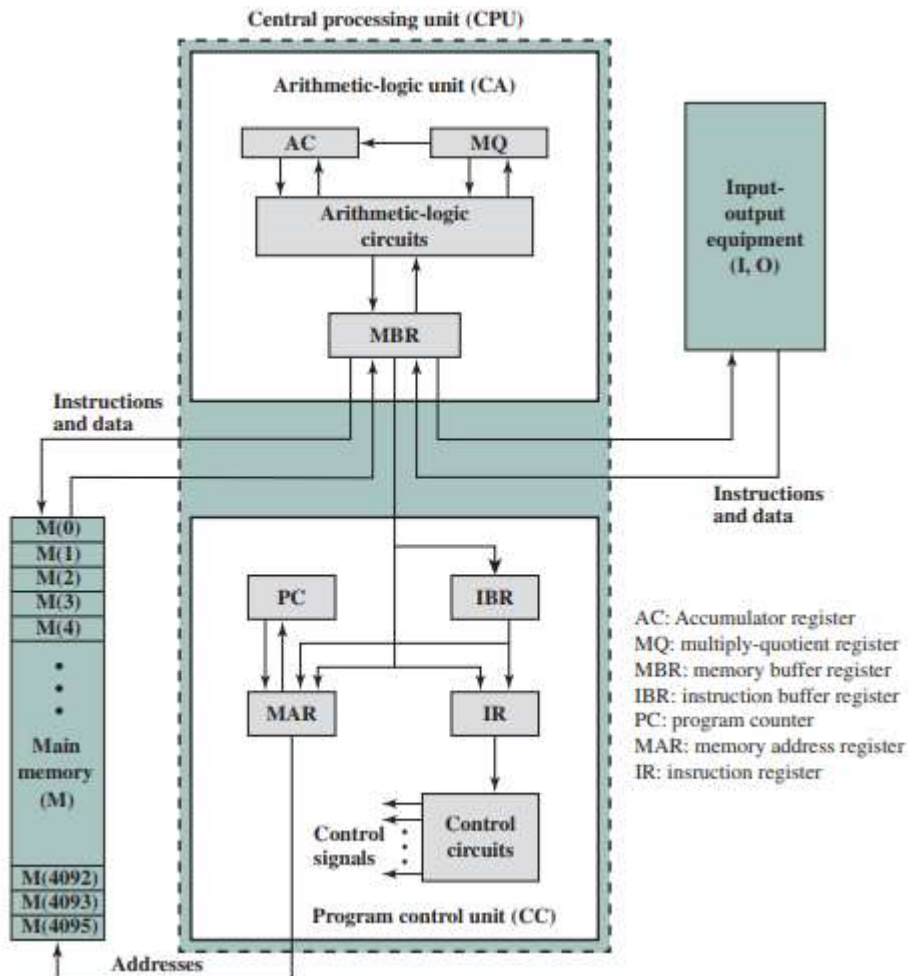
IAS - details

- 1000 x 40 bit words
 - Binary number
 - memory of the IAS consists of 4,096 storage locations, called words, of 40 binary digits (bits) each
 - 2 x 20 bit instructions
- Set of registers (storage in CPU)
 - Memory Buffer Register
 - Memory Address Register
 - Instruction Register
 - Instruction Buffer Register
 - Program Counter
 - Accumulator
 - Multiplier Quotient

IAS - details



Structure of IAS – detail





- 1947 - Eckert-Mauchly Computer Corporation
- UNIVAC I (Universal Automatic Computer)
- US Bureau of Census 1950 calculations
- Became part of Sperry-Rand Corporation
- Late 1950s - UNIVAC II
 - Faster
 - More memory
- IBM
- Punched-card processing equipment
- 1953 - the 701
 - IBM's first stored program computer
 - Scientific calculations
- 1955 - the 702
 - Business applications
- Lead to 700/7000 series

Transistors

- Replaced vacuum tubes
- Smaller
- Cheaper
- Less heat dissipation
- Solid State device
- Made from Silicon (Sand)
- Invented 1947 at Bell Labs
- William Shockley et al

Transistor Based Computers

- Second generation machines
- NCR & RCA produced small transistor machines
- IBM 7000
- DEC - 1957
 - Produced PDP-1(**Programmed Data Processor**)



Microelectronics

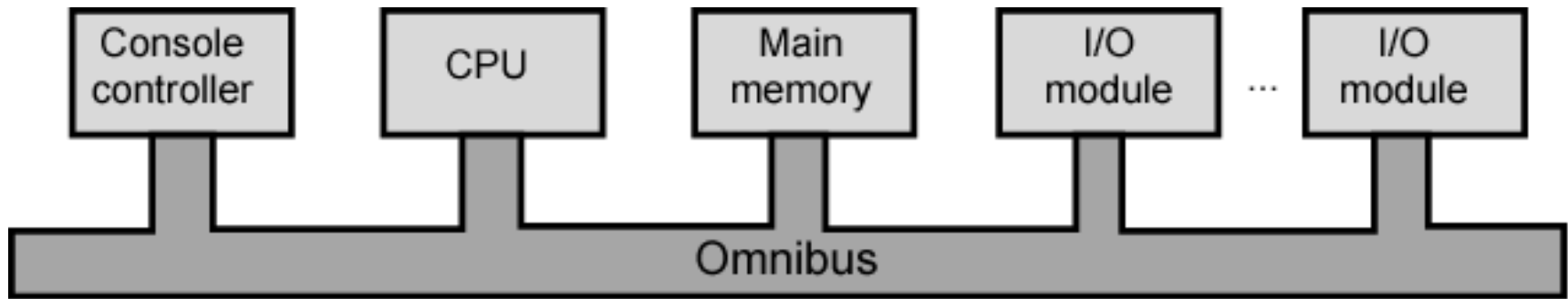
- Literally - “small electronics”
- A computer is made up of gates, memory cells and interconnections
- These can be manufactured on a semiconductor
- e.g. silicon wafer

DEC PDP-8



- 1964
- First minicomputer (after miniskirt!)
- Did not need air conditioned room
- Small enough to sit on a lab bench
- \$16,000
 - \$100k+ for IBM 360
- Embedded applications & OEM(original equipment manufacturers)
- BUS STRUCTURE

DEC - PDP-8 Bus Structure



models of the PDP-8 used a structure that became virtually universal for microcomputers: the bus structure

IBM 360 series

- 1964
- Replaced (& not compatible with) 7000 series
- First planned “family” of computers
 - Similar or identical instruction sets
 - Similar or identical O/S
 - Increasing speed
 - Increasing number of I/O ports (i.e. more terminals)
 - Increased memory size
 - Increased cost
- Multiplexed switch structure

Semiconductor Memory

- 1970
- Fairchild
- Size of a single core
 - i.e. 1 bit of magnetic core storage
- Holds 256 bits
- Non-destructive read
- Much faster than core
- Capacity approximately doubles each year

- 1971 - 4004
 - First microprocessor
 - All CPU components on a single chip
 - 4 bit
- Followed in 1972 by 8008
 - 8 bit
 - Both designed for specific applications
- 1974 - 8080
 - Intel's first general purpose microprocessor



Computer Generations

- Vacuum tube (1946-1957) & Transistor (1958-1964)
- Integrated Circuits
 - Small scale integration - 1965 on
 - Up to 100 devices on a chip
 - Medium scale integration - to 1971
 - 100 - 3,000 devices on a chip
 - Large scale integration - 1971-1977
 - 3,000 - 100,000 devices on a chip
 - Very large scale integration - 1978 -1991
 - 100,000 - 100,000,000 devices on a chip
 - Ultra large scale integration – 1991 onwards
 - Over 100,000,000 devices on a chip



x86 Evolution-1

- 1971 - 4004
 - First microprocessor of 4 bit
 - All CPU components on a single chip
 - Followed in 1972 by 8008 (8 bit processor)
 - Both designed for specific applications
- 8080
 - First general purpose microprocessor
 - Process/move 8 bit data at a time
 - Used in first personal computer – Altair

x86 Evolution-2

- 8086
 - Much more powerful (16 bit data)
 - Instruction cache, pre-fetch few instructions
 - 8088 (8 bit external bus) used in first IBM PC
- 80286
 - 16 MByte memory addressable
 - Up from 1MB (in 8086)
- 80386
 - 32 bit processor with multitasking support



x86 Evolution-3

- 80486
 - Sophisticated powerful cache and instruction pipelining
 - Built in maths co-processor
- Pentium
 - Superscalar
 - Multiple instructions executed in parallel
- Pentium Pro
 - Increased superscalar organization
 - Aggressive register renaming
 - Branch prediction and Data flow analysis



x86 Evolution-4

- Pentium II
 - MMX technology, graphics, video & audio processing
- Pentium III
 - Additional floating point instructions for 3D graphics
- Pentium 4
 - Further floating point and multimedia enhancements
- Itanium Series
 - 64 bit with Hardware enhancements to increase speed
- What's next???
 - Multi core architectures



Fetch and Execute cycles



What is a program?

- A logical group of instructions to achieve a specific task.
- An instruction is a sequence of steps
- For each step an arithmetic or logical operation or data movement is done.

Addition of Two Numbers

Start

Load R1, A

Load R2, B

$R3 = R1 + R2$

Store C, R3

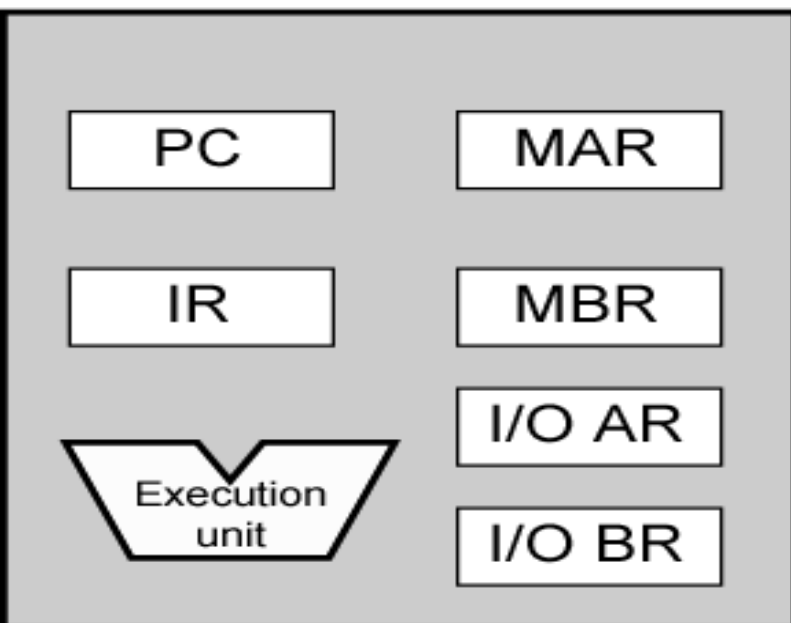
End of program



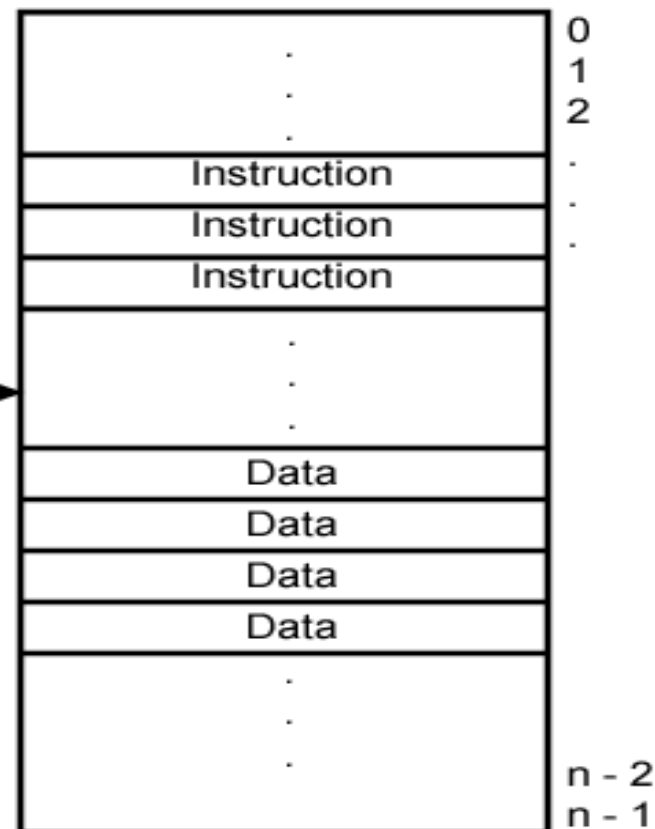
Components (start here)

- The Control Unit and the Arithmetic and Logic Unit constitute the Central Processing Unit
- Data and instructions need to get into the system and results out
 - Input/output
- Temporary storage of code and results is needed
 - Main memory

CPU

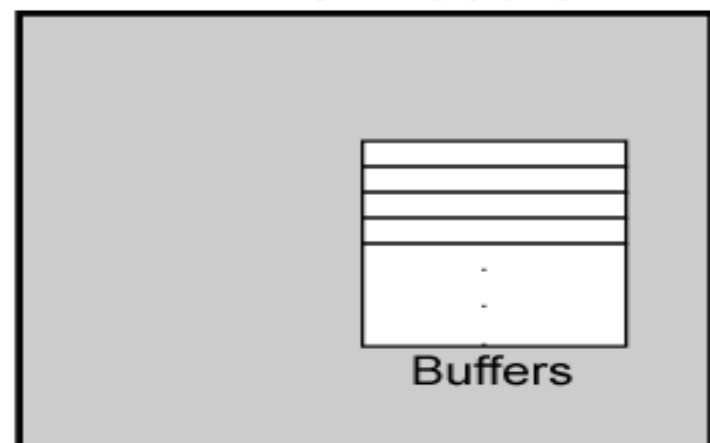


Main Memory



System Bus

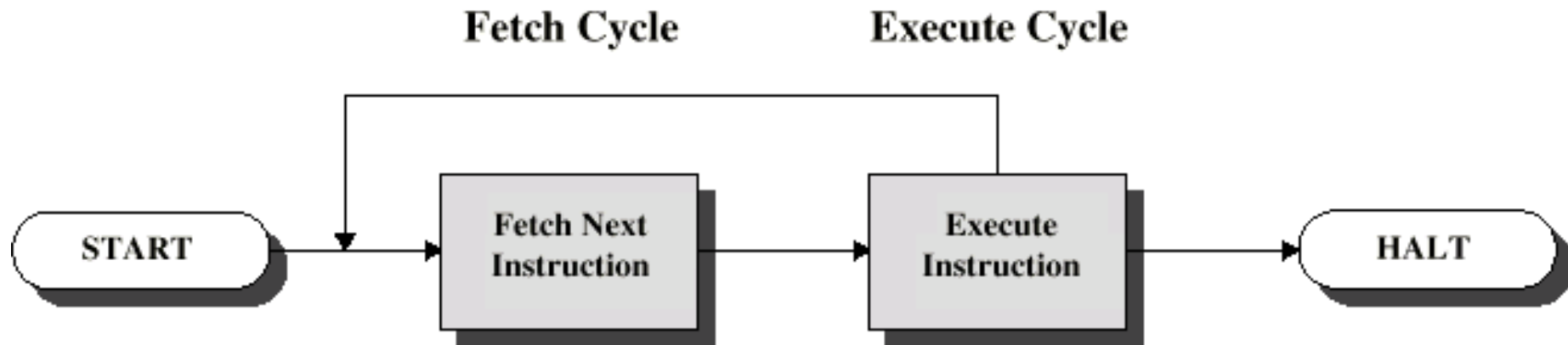
I/O Module



- PC = Program counter
- IR = Instruction register
- MAR = Memory address register
- MBR = Memory buffer register
- I/O AR = Input/output address register
- I/O BR = Input/output buffer register

Instruction Cycle

- Two steps:
 - Fetch
 - Execute





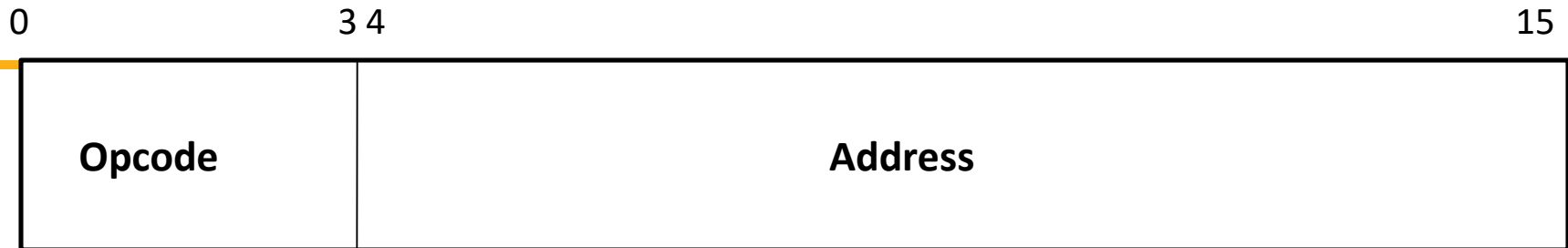
Fetch Cycle

- Program Counter (PC) holds address of next instruction to be fetched
- Processor fetches instruction from memory location pointed to by PC
- Increment PC
 - Unless told otherwise
- Instruction loaded into Instruction Register (IR)
- Processor interprets instruction and performs required actions



Execute Cycle

- Processor-memory
 - data transfer between CPU and main memory
- Processor I/O
 - Data transfer between CPU and I/O module
- Data processing
 - Some arithmetic or logical operation on data
- Control
 - Alteration of sequence of operations
 - e.g. jump
- Combination of above

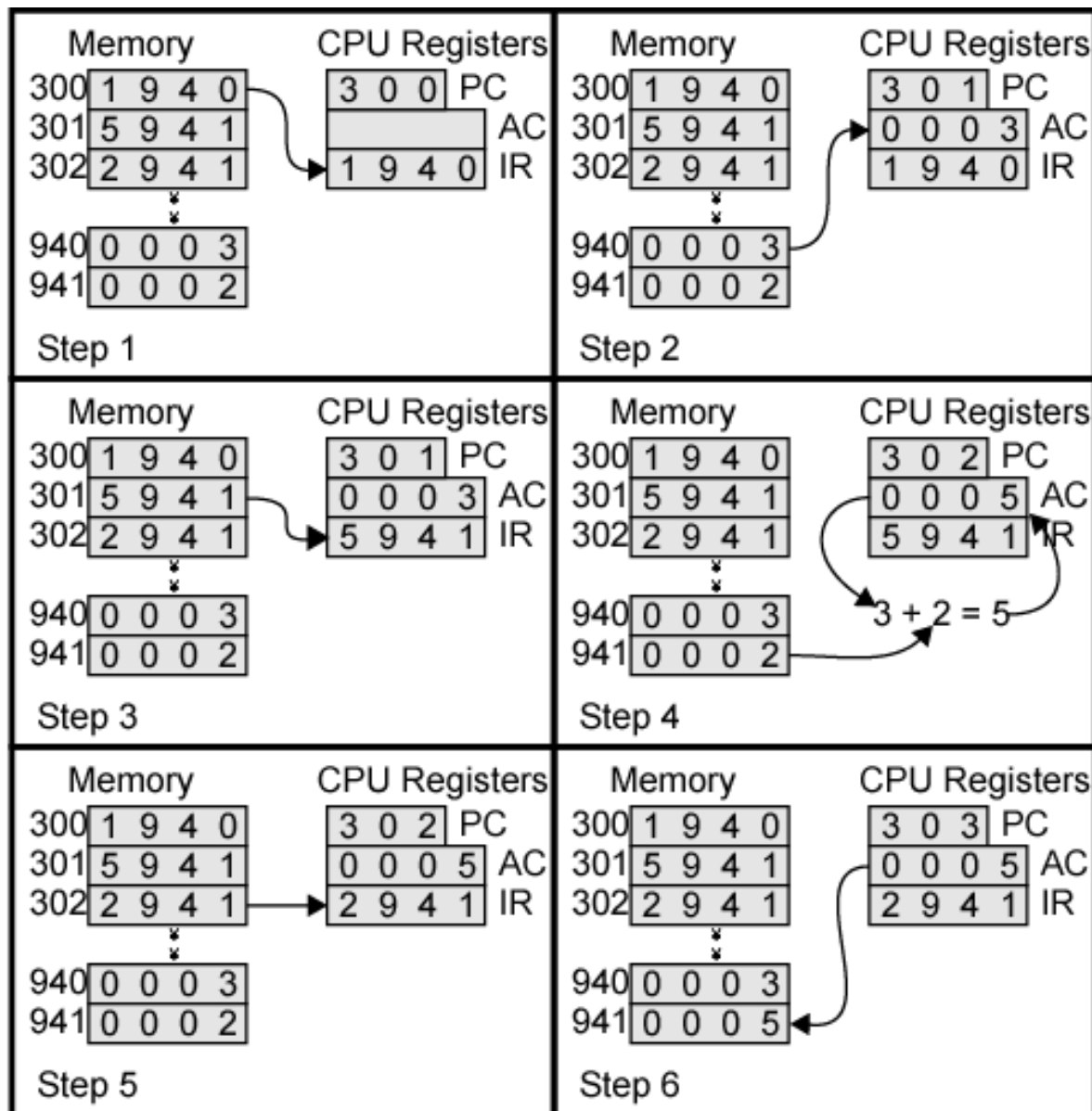


Instruction format

- The instruction format provides 4 bits for the opcode, so that there can be as many as $2^4 = 16$ different opcodes.
- ,and up to $2^{12} = 4096$ (4K) words of memory can be directly addressed.

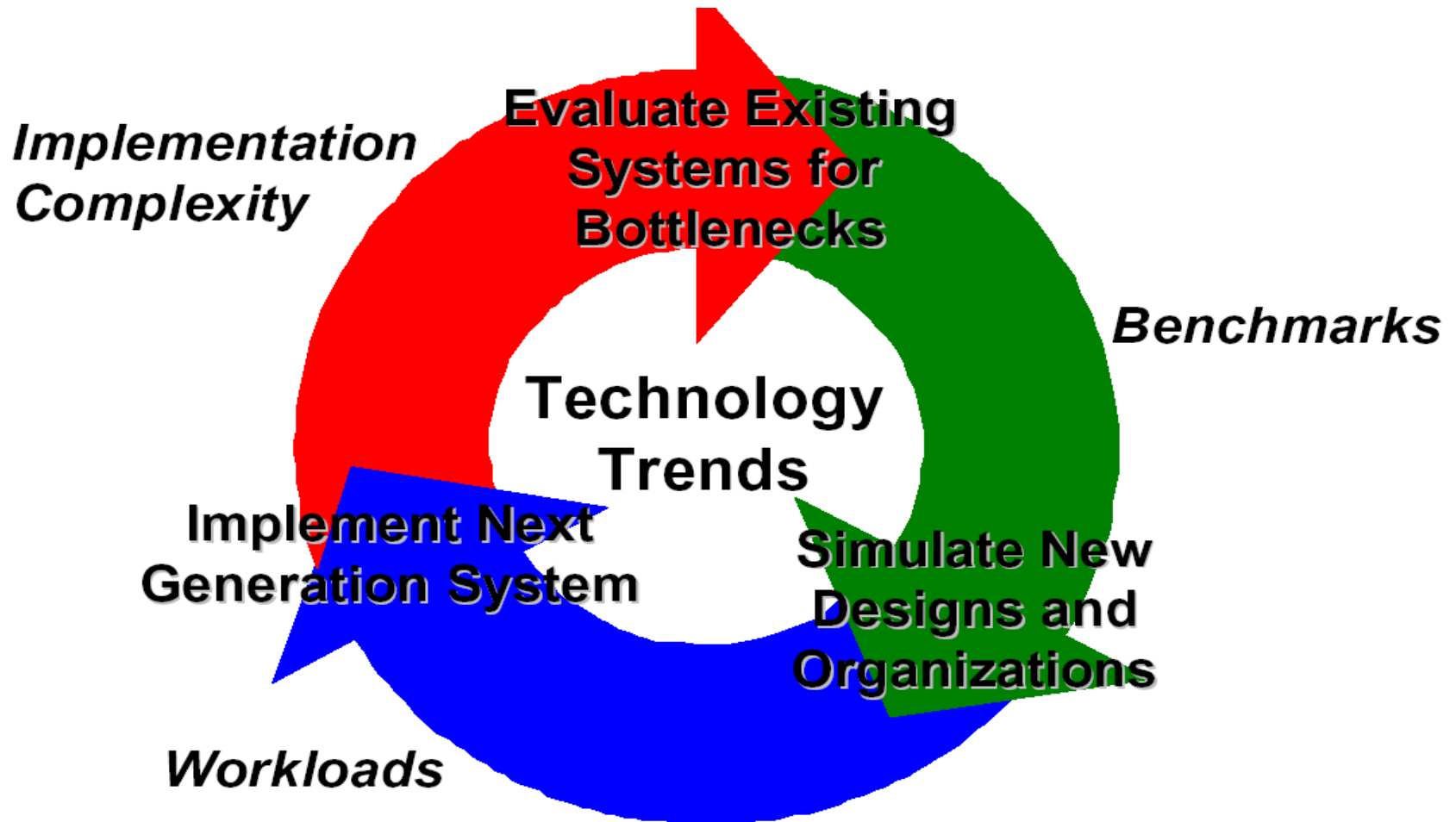
opcode	function
0001	Load AC from memory
0010	Store AC to Memory
0101	Add to AC from Memory

Example of Program Execution



The program fragment shown adds the contents of the memory word at address 940 to the contents of the memory word at address 941 and stores the result in the latter location

The Role of Performance



$$CPI = \frac{CPU \text{ clock cycles for the program}}{Instruction \text{ count}}$$

$$CPU \text{ time} = Instruction \text{ count} \times CPI \times Clock \text{ cycle time}$$

$$= \frac{Instruction \text{ count} \times CPI}{Clock \text{ rate}}$$

CPU Performance Equation



- Multiple aspects to performance: helps to isolate them
- Latency = seconds / program =
 - $(\text{insns} / \text{program}) * (\text{cycles} / \text{insn}) * (\text{seconds} / \text{cycle})$
 - **Insns / program**: dynamic insn count
 - Impacted by program, compiler, ISA
 - **Cycles / insn**: CPI
 - Impacted by program, compiler, ISA, micro-arch
 - **Seconds / cycle**: clock period (Hz)
 - Impacted by micro-arch, technology
- For low latency (better performance) minimize all three
 - Difficult: often pull against one another
 - Example we have seen: RISC vs. CISC ISAs



PERFORMANCE

Performance is one of the key parameter to evaluate a system.

If we say one system is better than another we consider

- Response Time
- Throughput

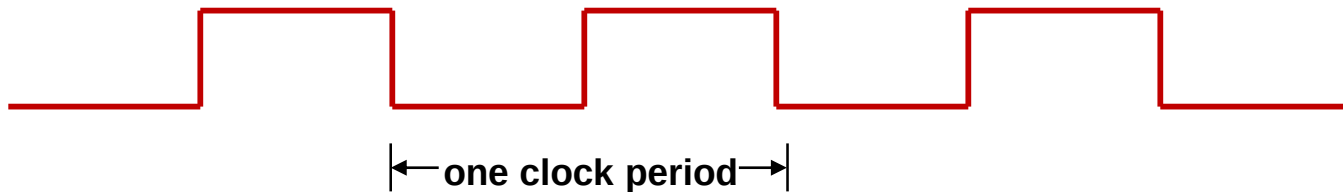


Computer Performance

- Performance is one of the key parameter to-
 - Evaluate processor hardware
 - Measure requirement for new systems
- When we say one computer has better performance than another, what does it mean?
 - Criterion for the performance?
 - **Response Time** (Single computer), **Throughput** (Data center)

Clock Rate

- Operation performed by processor are governed by system clock (**fundamental level of processor speed measurement**)
 - Generated by quartz crystal



- A clock cycle is the basic unit of time to execute one operation.
- Clock Rate (**clock cycles per second** in MHz or GHz) is inverse of clock cycle time (**clock period**)



Is Clock Rate Enough?

- As we know, instruction execution takes several discrete steps
 - Fetching, decoding, ALU operation, fetching data etc.
 - It takes multiple clock cycles for its execution
- Different instructions takes different number of cycles for their execution
 - LOAD, ADD, SUB, JUMP etc.
- Thus clock speed doesn't tell the whole story!!!



Performance: Application Specific

- Performance
 - How a processor performs when executing a given application
 - Application performance depends upon
 - Speed of the processor
 - Instruction set
 - Choice of language
 - Efficiency of the compiler
 - Programming skill of the user

CPU Performance



- To maximize performance, need to minimize execution time

$$\text{performance} = 1 / \text{execution_time}$$

If X is n times faster than Y, then

$$\frac{\text{performance}_x}{\text{performance}_y} = \frac{\text{execution_time}_y}{\text{execution_time}_x} = n$$

Cycles Per Instruction (CPI)

- For any given processor, number of cycles required varies for different types of instructions
 - e.g. load, store, branch, add, mul etc.
- Hence CPI is not a constant value for a processor
- Needs to calculate average CPI for processor

CPU Performance and it's Factors



	I_c	p	m	k	τ
Instruction set architecture	X	X			
Compiler technology	X	X	X		
Processor implementation		X			X
Cache and memory hierarchy				X	X

- Program execution time (T) = $I_c \times \text{CPI} \times \tau$
 - τ = cycle time, I_c = no. of instructions in the program
- Instruction execution also requires memory access
 - $T = I_c \times [p + (m \times k)] \times \tau$
 - p = processor cycles, m = memory references
 - k = ratio of memory cycles to processor cycles
- These performance factors are influenced by
 - ISA,
 - compiler, (how effective the compiler is in producing an efficient machine language program from a high-level language program)
 - processor implementation
 - and memory hierarchy

MIPS Rate

- Common measure of performance
 - Millions Instructions Per Second (MIPS) rate
 - $I_c / (\text{Execution time} \times 10^6)$
 - This can be written as:
 - $= I_c / \text{cycle time} \times (I_c \times \text{CPI}) \times 10^6$
 - $= f / \text{CPI} \times 10^6$

In class Example1

- Consider the execution of a program that results in the execution of 2 million instructions on a 400-MHz processor. The program consists of four major types of instructions. The instruction mix and the CPI for each instruction type are given below, based on the result of a program trace experiment

Instruction Type	CPI	Instruction Mix (%)
Arithmetic and logic	1	60
Load/store with cache hit	2	18
Branch	4	12
Memory reference with cache miss	8	10

The average CPI when the program is executed on a uniprocessor with the above trace results is

$$CPI = 0.6 + (2 * 0.18) + (4 * 0.12) + (8 * 0.1) = 2.24.$$

$$MIPS \text{ rate} = I_c / T * 10^6 = f / CPI * 10^6$$

The corresponding MIPS rate is $(400 * 10^6) / (2.24 * 10^6) \approx 178$.

Example of Performance Measure



Airplane	Passenger capacity	Cruising range (miles)	Cruising speed (m.p.h)	Passenger throughput (Passenger $\times$ m.p.h)
Boeing 777	375	4630	610	228,750
Boeing 747	470	4150	610	286,700
BAC/Sud Concorde	132	4000	1350	178,200
Douglas DC-8-50	146	8720	544	79,424



Limitation of MIPS Rate

- MIPS rate or instruction execution rate is also inadequate to measure CPU performance. Why?
 - Because of differences in ISA
 - Ex. To execute a high level language statement $A=B+C$ (A,B and C are in memory) may need different number of low level instructions for different ISA

Amdahl's Law Formula



- For program running on single processor
 - Fraction f of code infinitely parallelizable with no scheduling overhead
 - Fraction $(1-f)$ of code inherently serial
 - T is total execution time for program on single processor
 - N is number of processors that fully exploit parallel portions of code

$$\text{Speedup} = \frac{\text{time to execute program on a single processor}}{\text{time to execute program on } N \text{ parallel processors}} = \frac{T(1-f) + Tf}{T(1-f) + \frac{Tf}{N}} = \frac{1}{(1-f) + \frac{f}{N}}$$

- Conclusions
 - f small, parallel processors has little effect
 - $N \rightarrow \infty$, speedup bound by $1/(1-f)$
 - Diminishing returns for using more processors



$$\frac{1}{(1 - P) + \frac{P}{S}}$$

How much will an optimization improve performance?

P = proportion of running time affected by optimization

S = speedup

What if I speedup 25% of a program's execution by 2x?

1.14x speedup

What if I speedup 25% of a program's execution by ∞ ?

1.33x speedup

In class Example2



Suppose we have made the following measurements:

Frequency of FP operations = 50%

Average CPI of FP operations = 4.0

Average CPI of other instructions = 1.33

Frequency of FPSQR = 20%

CPI of FPSQR = 20

Assume that the two design alternatives are to decrease the CPI of FPSQR to 2 or to decrease the average CPI of all FP operations to 2.5. Compare these two design alternatives using the processor performance equation.



Apply Amdahl's Law to the above question and observe the result

$$\text{Speedup}_{\text{enhanced only new FPSQR}} = \frac{CPI_{\text{old FPSQR}}}{CPI_{\text{new FPSQR}}} = \frac{20}{2} = 10$$

$$\text{Speedup}_{\text{enhanced only new FP}} = \frac{CPI_{\text{old FP}}}{CPI_{\text{new FPS}}} = \frac{4}{2.5} = 1.6$$

Amdahl's Law eqn:

$$\text{speedup}_{\text{overall}} = \frac{\text{exexution time}_{\text{old}}}{\text{execution time}_{\text{new}}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{speedup}_{\text{enhanced}}}}$$

$$\text{With FPSQR Speedup}_{\text{overall}} = \frac{1}{(1 - 0.2) + \frac{0.2}{10}} = \frac{1}{0.82} = 1.219 \cong 1.22$$

$$\text{With FP Speedup}_{\text{overall}} = \frac{1}{(1 - 0.5) + \frac{0.5}{1.6}} = \frac{1}{0.8125} = 1.23$$

We get the same result using Amdahl's Law

In class Example3

A benchmark program is run on a 40 MHz processor. The executed program consists of 100,000 instruction executions, with the following instruction mix and clock cycle count:

Instruction Type	Instruction Count	Cycles per Instruction
arithmetic	45000	1
Data transfer	32000	2
Floating point	15000	2
Control transfer	8000	2

Determine the effective CPI, MIPS rate, and execution time for this program.

In class Example3 contnd



SOL: $CPI = 1.55$, MIPS RATE = 25.8,
execution time = 38.75ms

In class Example4



Q2. Early examples of CISC and RISC are the VAX 11/780 and the IBM RS/6000, respectively.

Using a typical benchmark program, the following machine characteristics results:

The final column shows that the VAX required 12 times longer than the IBM measured in CPU time.

- What is the relative size of the instruction count of the machine code for this benchmark program running on the two machines?
- What are the CPI (Cycles Per Instruction) values for the two machines?

Processor	Clock Frequency	Performance	CPU time
VAX 11/780	5 MHz	1MIPS	12 x seconds
IBM RS/6000	25 MHz	18 MIPS	x seconds



Sol: We can express the MIPS rate as: $[(\text{MIPS rate})/10^6] = I_c/T$.

So that:

$$I_c = T \times [(\text{MIPS rate})/10^6].$$

a) The ratio of the instruction count of the RS/6000 to the VAX is

$$[x \times 18]/[12x \times 1] = 1.5$$

b) For the Vax, $CPI = (5 \text{ MHz})/(1 \text{ MIPS}) = 5$

For the RS/6000, $CPI = 25/18 = 1.39$

Benchmark Programs



- It is a collection of a programs that provides representative test of a computer in a particular application area
 - e.g. SPEC (System Performance Evaluation Corporation) benchmark suites
 - SPEC CPU 2006 is used for measuring performance for the computational based applications



Review Questions

- Differentiate between computer organization and architecture?
- What are the four main functions of a computer?
- What are the basic structural components of a computer?
- Describe the computer generations in brief.
- What was the first general purpose microprocessor?
- Consider two implementation of the same ISA. Computer A clock cycle time of 250 ns and a CPI of 2 for a program. Computer B has a clock cycle time of 500 ns and a CPI of 1.2 for the same program. Which computer is faster for this program and by how much?
- A program runs on computer A with a 2 GHz clock in 10 seconds. Another computer B with 4 GHz run this program in 6 seconds. To accomplish this, computer B will require P times as many clock cycles as computer A to run the program. Find the value of P.

Practice questions



- Practice question



Performance revision

Consider two different machines, with two different instruction sets, both of which have a clock rate of 200 MHz. The following measurements are recorded on the two machines running a given set of benchmark programs:

Instruction Type	Instruction Count (millions)	Cycles per Instruction
Machine A		
Arithmetic and logic	8	1
Load and store	4	3
Branch	2	4
Others	4	3
Machine B		
Arithmetic and logic	10	1
Load and store	8	2
Branch	2	4
Others	4	3

- Determine the effective CPI, MIPS rate, and execution time for each machine.
- Comment on the results.

Performance revision

clock rate of 200 MHz. :

Instruction Type	Instruction Count (millions)	Cycles per Instruction
Machine A		
Arithmetic and logic	8	1
Load and store	4	3
Branch	2	4
Others	4	3

- Determine the effective CPI, MIPS rate, and execution time for each machine.
- Comment on the results.

$$CPI_A = \frac{\sum CPI_i \times I_i}{I_c}$$

$$= \frac{(8 \times 1 + 4 \times 3 + 2 \times 4 + 4 \times 3) \times 10^6}{(8 + 4 + 2 + 4) \times 10^6} = 2.12$$



Performance revision

clock rate of 200 MHz. :

Instruction Type	Instruction Count (millions)	Cycles per Instruction
Machine A		
Arithmetic and logic	8	1
Load and store	4	3
Branch	2	4
Others	4	3

- Determine the effective CPI, MIPS rate, and execution time for each machine.
- Comment on the results.

Time for 1 cycle = $1/f$ Hz

Time for 1 instruction = $\text{CPI} \times 1/f$

$\text{CPI} \times 1/f$ secs for 1 instruction

There fore in 1 sec no of instructions = f / CPI

And this MIPS = $f / (\text{CPI} \times 10^6)$



Performance revision

clock rate of 200 MHz. :

Instruction Type	Instruction Count (millions)	Cycles per Instruction
Machine A		
Arithmetic and logic	8	1
Load and store	4	3
Branch	2	4
Others	4	3

- a. Determine the effective CPI, MIPS rate, and execution time for each machine.
- b. Comment on the results.

And this MIPS= $f / (CPI \times 10^6)$

$$MIPS_A = \frac{f}{CPI_A \times 10^6} = \frac{200 \times 10^6}{2.22 \times 10^6} = 90$$

$$CPU_A = \frac{I_c \times CPI_A}{f} = \frac{18 \times 10^6 \times 2.2}{200 \times 10^6} = 0.2 \text{ s}$$



Performance revision

clock rate of 200 MHz. :

Instruction Type	Instruction Count (millions)	Cycles per Instruction
Machine B		
Arithmetic and logic	10	1
Load and store	8	2
Branch	2	4
Others	4	3

- Determine the effective CPI, MIPS rate, and execution time for each machine.
- Comment on the results.

$$CPI_B = \frac{\sum CPI_i \times I_i}{I_c} = \frac{(10 \times 1 + 8 \times 2 + 2 \times 4 + 4 \times 3) \times 10^6}{(10 + 8 + 2 + 4) \times 10^6} \approx 1.92$$

$$MIPS_B = \frac{f}{CPI_B \times 10^6} = \frac{200 \times 10^6}{1.92 \times 10^6} = 104$$

$$CPU_B = \frac{I_c \times CPI_B}{f} = \frac{24 \times 10^6 \times 1.92}{200 \times 10^6} = 0.23 \text{ s}$$

Thank You!



COMPUTER ORGANISATION AND ARCHITECTURE



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

R.K.Tiwary



Contact Session 3

Computer System Components and Interconnections

IMP Note to Self





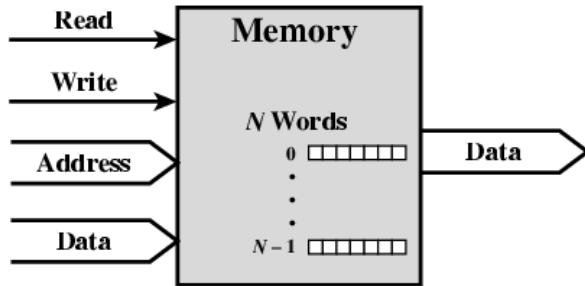
IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**

Inter Connections

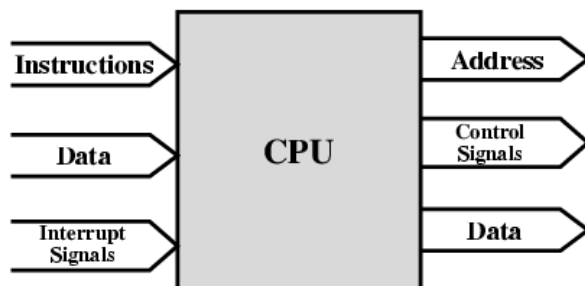
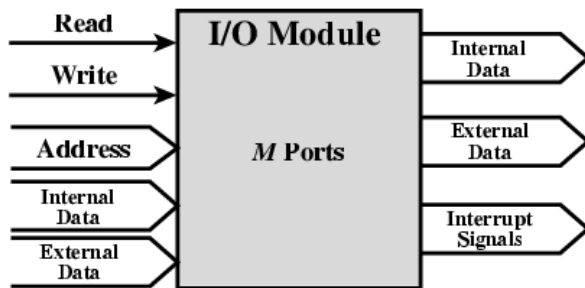
- All the units must be connected
- Different type of connection for different type of unit
 - Memory
 - Input/Output
 - CPU

Computer Modules



■ I/O module: I/O is functionally similar to memory. There are two operations; read and write.

- I/O module may control more than one external device.
- Refer to each of the interfaces to an external device as a port and give each a unique address (e.g., 0, 1, ..., M-1).
- In addition, there are external data paths for the input and output of data with an external device.
- Finally, an I/O module may be able to send interrupt signals to the processor.





CPU & MEMORY Connection

- Reads instruction and data
- Writes out data (after processing)
- Sends control signals to other units
- Receives (& acts on) interrupts

Memory Connection

- Receives and sends data
- Receives addresses (of locations)
- Receives control signals
 - Read
 - Write
 - Timing

Input/Output Connection



- Similar to memory from computer's viewpoint
- Output
 - Receive data from computer
 - Send data to peripheral
- Input
 - Receive data from peripheral
 - Send data to computer
- Receive control signals from computer
- Send control signals to peripherals
 - e.g. spin disk
- Receive addresses from computer
 - e.g. port number to identify peripheral
- Send interrupt signals (control)

Input/Output Connection



- Similar to memory from computer's viewpoint
- Output
 - Receive data from computer
 - Send data to peripheral
- Input
 - Receive data from peripheral
 - Send data to computer
- Receive control signals from computer
- Send control signals to peripherals
 - e.g. spin disk
- Receive addresses from computer
 - e.g. port number to identify peripheral
- Send interrupt signals (control)



Concept of BUS



Buses

- There are a number of possible interconnection systems
- it has gradually given way to various point-to-point interconnection structures, which now dominate computer system design
- Single and multiple BUS structures are most common
- e.g. Control/Address/Data bus (PC)
- e.g. Unibus (DEC-PDP)

A communication pathway connecting two or more devices is a bus.

- Parallel lines on circuit boards
- Ribbon cables
- Strip connectors on motherboards
- Sets of wires



Different Bus

Data Bus

- Carries data
 - Remember that there is no difference between “data” and “instruction” at this level
- Width is a key determinant of performance
 - 8, 16, 32, 64 bit

Address bus

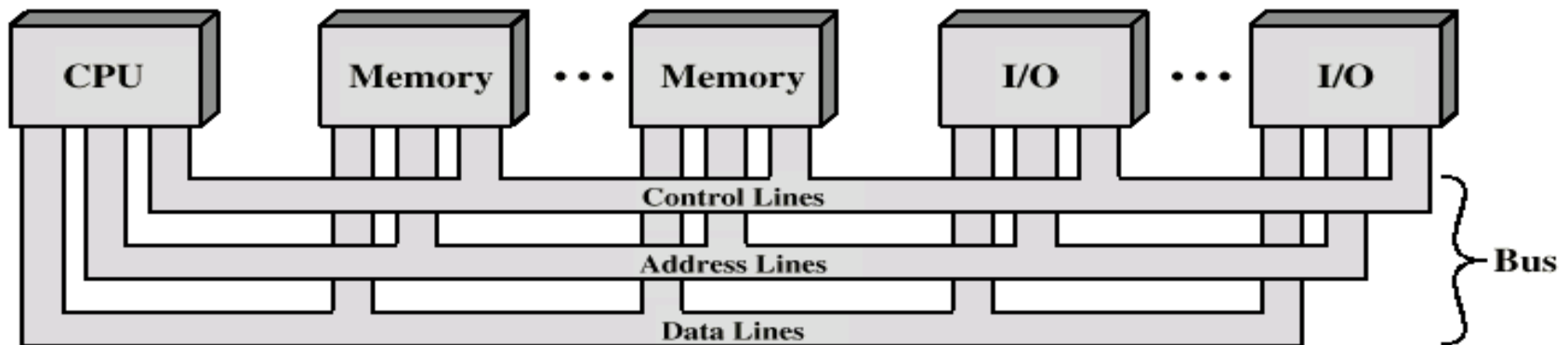
- Identify the source or destination of data

e.g. CPU needs to read an instruction (data) from a given location in memory
- Bus width determines maximum memory capacity of system

e.g. 8080 has 16 bit address bus giving 64k address space

Control Bus

- Control and timing information
 - Memory read/write signal
 - Interrupt request
 - Clock signals





Single Bus Problems

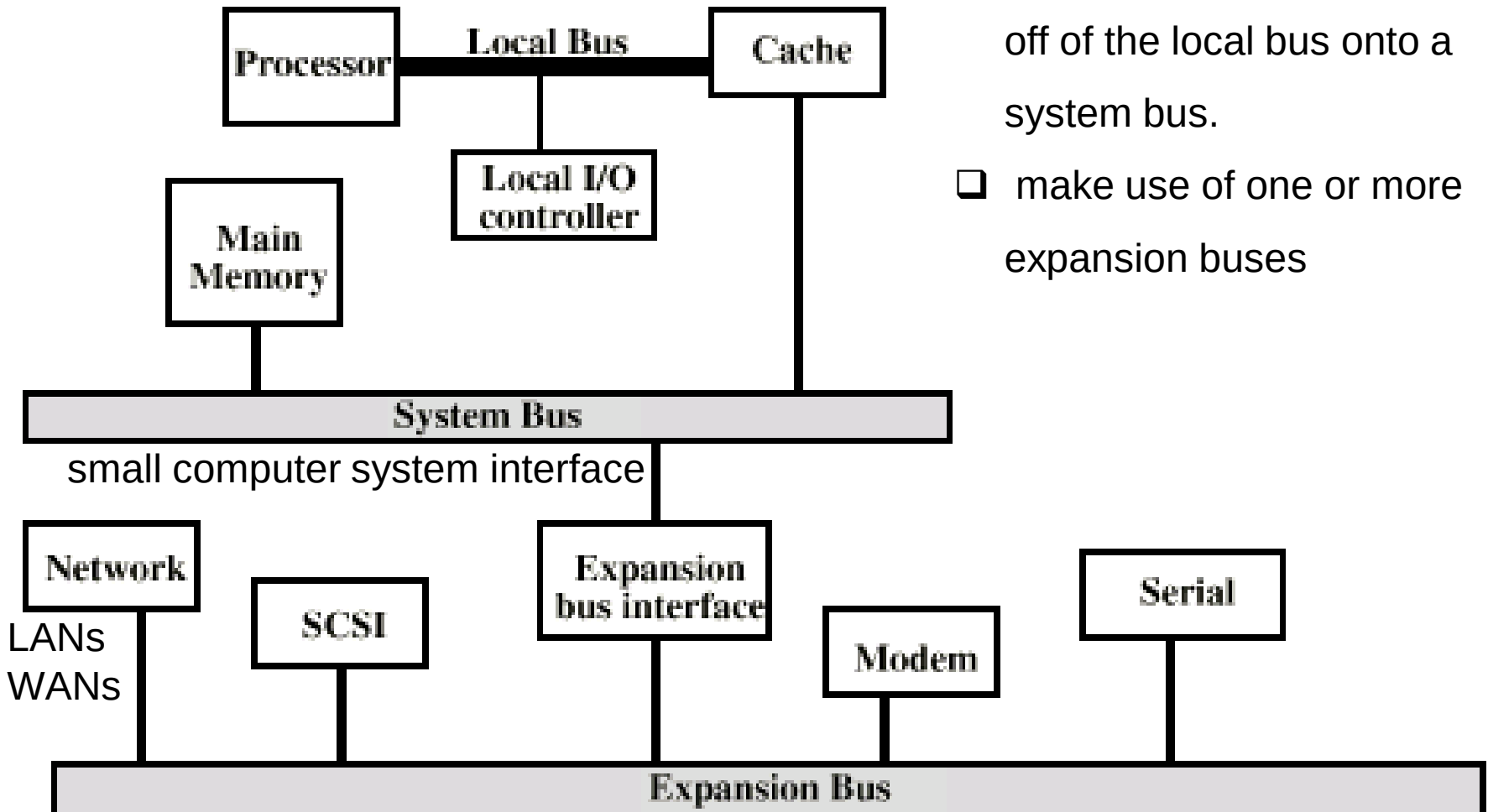
- Lots of devices on one bus leads to:
 - Propagation delays
 - Long data paths mean that co-ordination of bus use can adversely affect performance
 - If aggregate data transfer approaches bus capacity
- Most systems use multiple buses to overcome these problems



Multiple Bus Hierarchies

- A great number of devices on a bus will cause performance to suffer
- ✓ Propagation delay - the time it takes for devices to coordinate the use of the bus
- ✓ The bus may become a bottleneck as the aggregate data transfer demand approaches the capacity of the bus (in available transfer cycles/second)
- Traditional Hierarchical Bus Architecture
 - ✓ Use of a cache structure insulates CPU from frequent accesses to main memory
 - ✓ Main memory can be moved off local bus to a system bus
 - ✓ Expansion bus interface
 - buffers data transfers between system bus and I/O controllers on expansion bus
 - insulates memory-to-processor traffic from I/O traffic

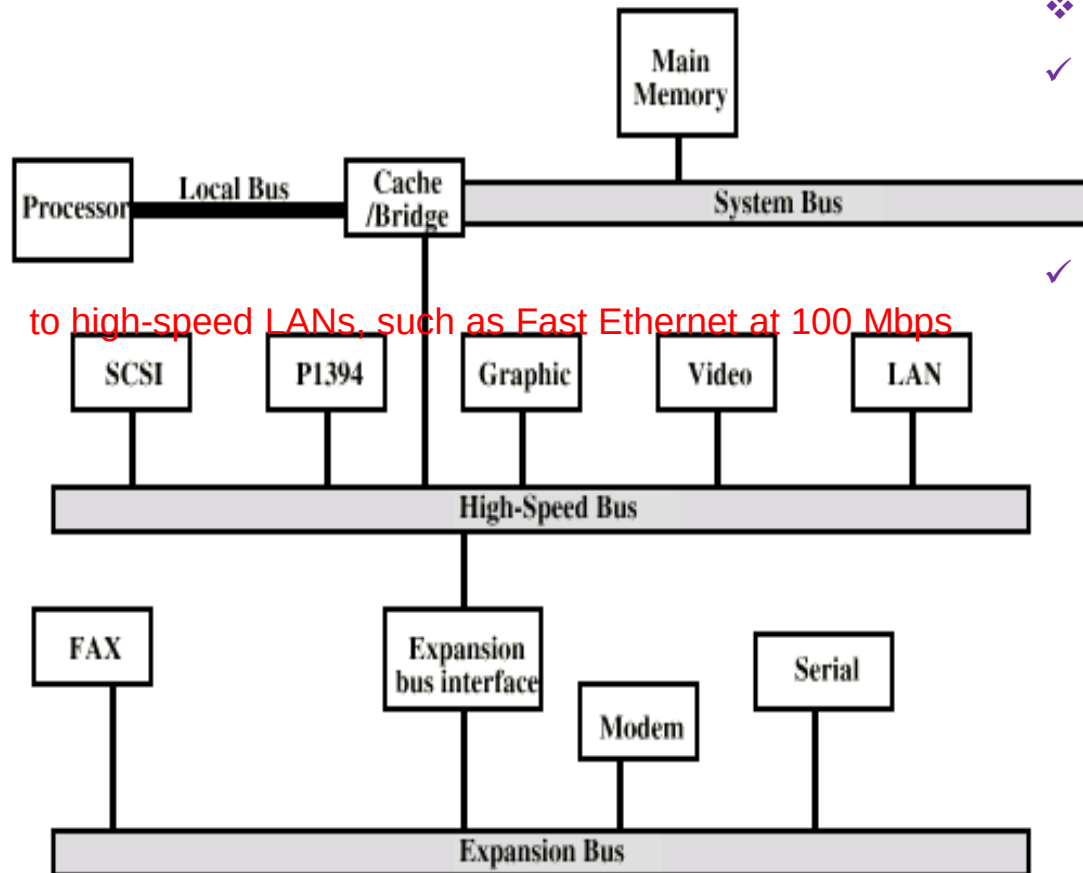
Traditional (ISA Industry Standard Architecture) (with cache)



- ❑ Main memory can be moved off of the local bus onto a system bus.
- ❑ make use of one or more expansion buses



High Performance Bus



- ❖ High-performance Hierarchical Bus Architecture
- ✓ Traditional hierarchical bus breaks down as higher and higher performance is seen in the I/O devices
- ✓ Incorporates a high-speed bus
 - specifically designed to support high-capacity I/O devices
 - brings high-demand devices into closer integration with the processor and at the same time is independent of the processor
 - Changes in processor architecture do not affect the high-speed bus, and vice versa
 - Sometimes known as a mezzanine architecture



Bus Types

- Dedicated
 - Separate data & address lines
- Multiplexed
 - Shared lines
 - Address valid or data valid control line
 - Advantage - **fewer lines**
 - Disadvantages
 - More complex control
 - Reduction in performance (cannot take place in parallel)

Type	Bus Width
Dedicated	Address
Multiplexed	Data
Method of Arbitration	Data Transfer Type
Centralized	Read
Distributed	Write
Timing	Read-modify-write
Synchronous	Read-after-write
Asynchronous	Block



Bus Arbitration

- More than one module controlling the bus
- e.g. CPU and DMA controller
- Only one module may control bus at one time
- Arbitration may be centralised or distributed
- Centralised
 - Single hardware device controlling bus access
 - Bus Controller
 - May be part of CPU or separate
- Distributed
 - Each module may claim the bus
 - Control logic on all modules

What is Bus Arbitration, Bus Mastering and DMA?



- Bus Arbitration – an elaborate system for resolving bus control conflicts and assigning priorities to the requests for control of the bus.
- Bus Mastering – a method of enabling different device controllers on the bus to ‘talk’ to one another, without having to go through the CPU.
- DMA(Direct Memory Access) – a method of transferring data from a hard disk to main memory without having to go through the CPU.



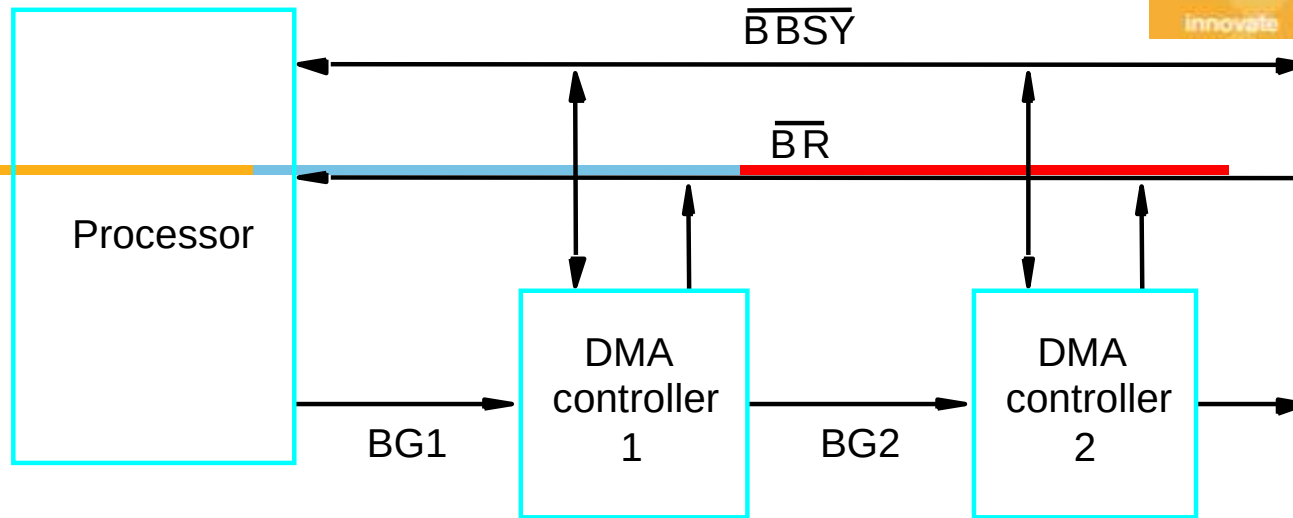
- **Centralized**

Centralized bus arbitration requires hardware (arbiter) that will grant the bus to one of the requesting devices. This hardware can be part of the CPU or it can be a separate device on the motherboard.

- **Decentralized**

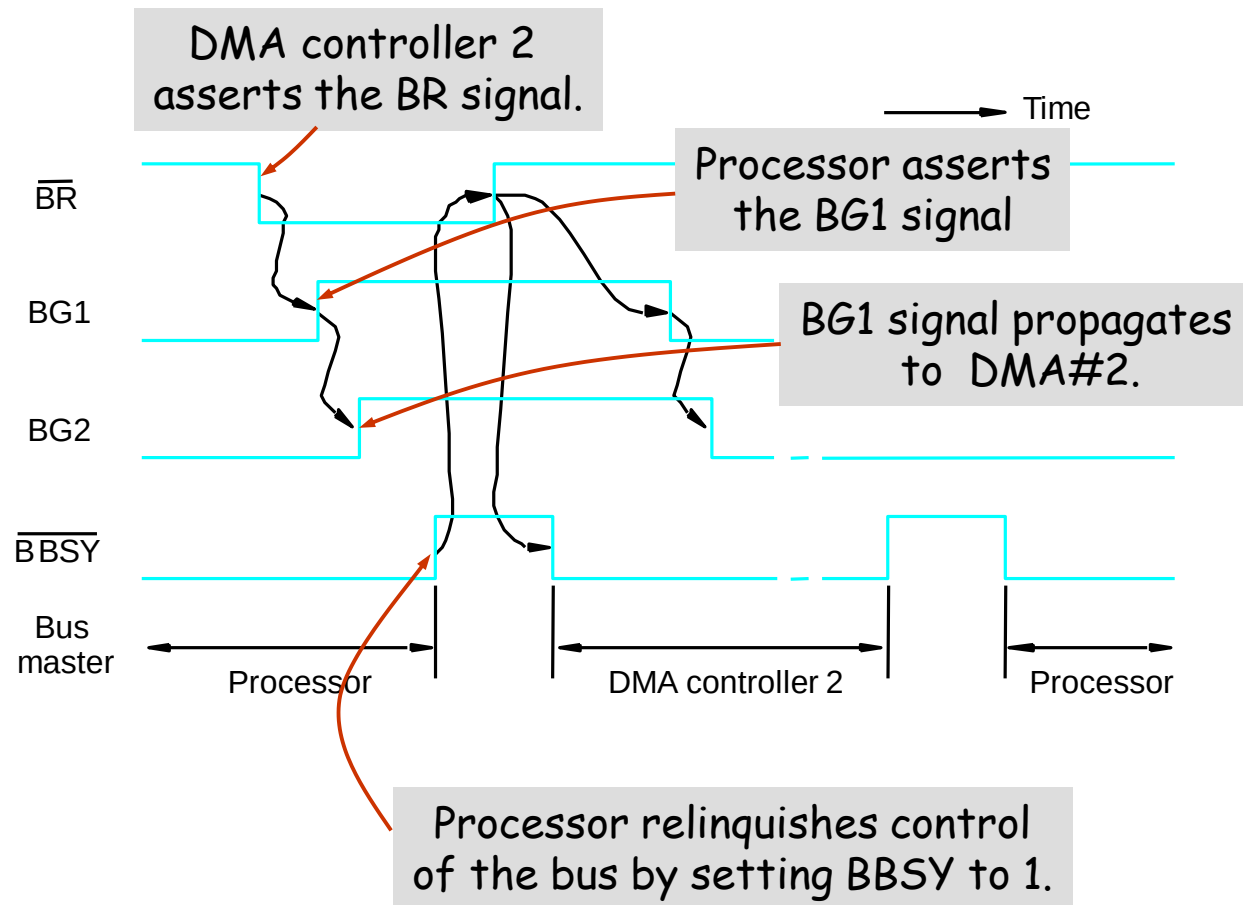
Decentralized arbitration there isn't an arbiter, so the devices have to decide who goes next. This makes the devices more complicated, but saves the expense of having an arbiter.

Centralized arbitration



- Bus arbiter may be the processor or a separate unit connected to the bus.
- Normally, the processor is the bus master, unless it grants bus membership to one of the DMA controllers.
- DMA controller requests the control of the bus by asserting the Bus Request (BR) line.
- In response, the processor activates the Bus-Grant1 (BG1) line, indicating that the controller may use the bus when it is free.
- BG1 signal is connected to all DMA controllers in a daisy chain fashion.
- BBSY signal is 0, it indicates that the bus is busy. When BBSY becomes 1, the DMA controller which asserted BR can acquire control of the bus.

Centralized arbitration (contd..)





- Centralized arbitration scheme with one Bus-Request (BR) line and one Bus-Grant (BG) line forming a daisy chain.
- Several pairs of BR and BG lines are possible, perhaps one per device as in the case of interrupts.
- Bus arbiter has to ensure that only one request is granted at any given time.
- It may do so according to a fixed priority scheme, or a rotating priority scheme.
- Rotating priority scheme:
 - There are four devices, and initial priority is 1,2,3,4.
 - After the request from device 1 is granted, the priority changes to 2,3,4,1.



- All devices waiting to use the bus share the responsibility of carrying out the arbitration process.
 - Arbitration process does not depend on a central arbiter and hence distributed arbitration has higher reliability.
- Each device is assigned a 4-bit ID number.
- All the devices are connected using 5 lines, 4 arbitration lines to transmit the ID, and one line for the Start-Arbitration signal.
- To request the bus a device:
 - Asserts the Start-Arbitration signal.
 - Places its 4-bit ID number on the arbitration lines.
- The pattern that appears on the arbitration lines is the logical-OR of all the 4-bit device IDs placed on the arbitration lines.

Distributed arbitration



- Device A has the ID 5 and wants to request the bus:
 - Transmits the pattern 0101 on the arbitration lines.
- Device B has the ID 6 and wants to request the bus:
 - Transmits the pattern 0110 on the arbitration lines.
- Pattern that appears on the arbitration lines is the logical OR of the patterns:
 - Pattern 0111 appears on the arbitration lines.

Arbitration process:

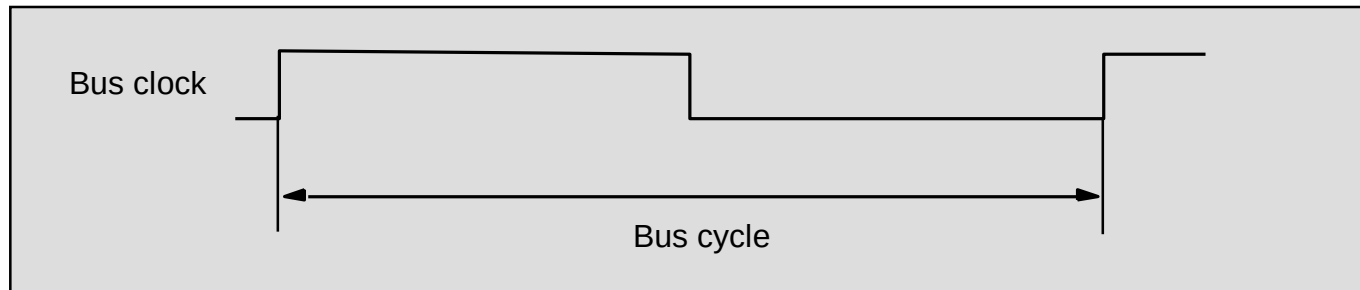
- Each device compares the pattern that appears on the arbitration lines to its own ID, starting with MSB.
- If it detects a difference, it transmits 0s on the arbitration lines for that and all lower bit positions.
- Device A compares its ID 5 with a pattern 0101 to pattern 0111.
- It detects a difference at bit position 2, as a result, it transmits a pattern 0100 on the arbitration lines.
- The pattern that appears on the arbitration lines is the logical-OR of 0100 and 0110, which is 0110.
- This pattern is the same as the device ID of B, and hence B has won the arbitration.



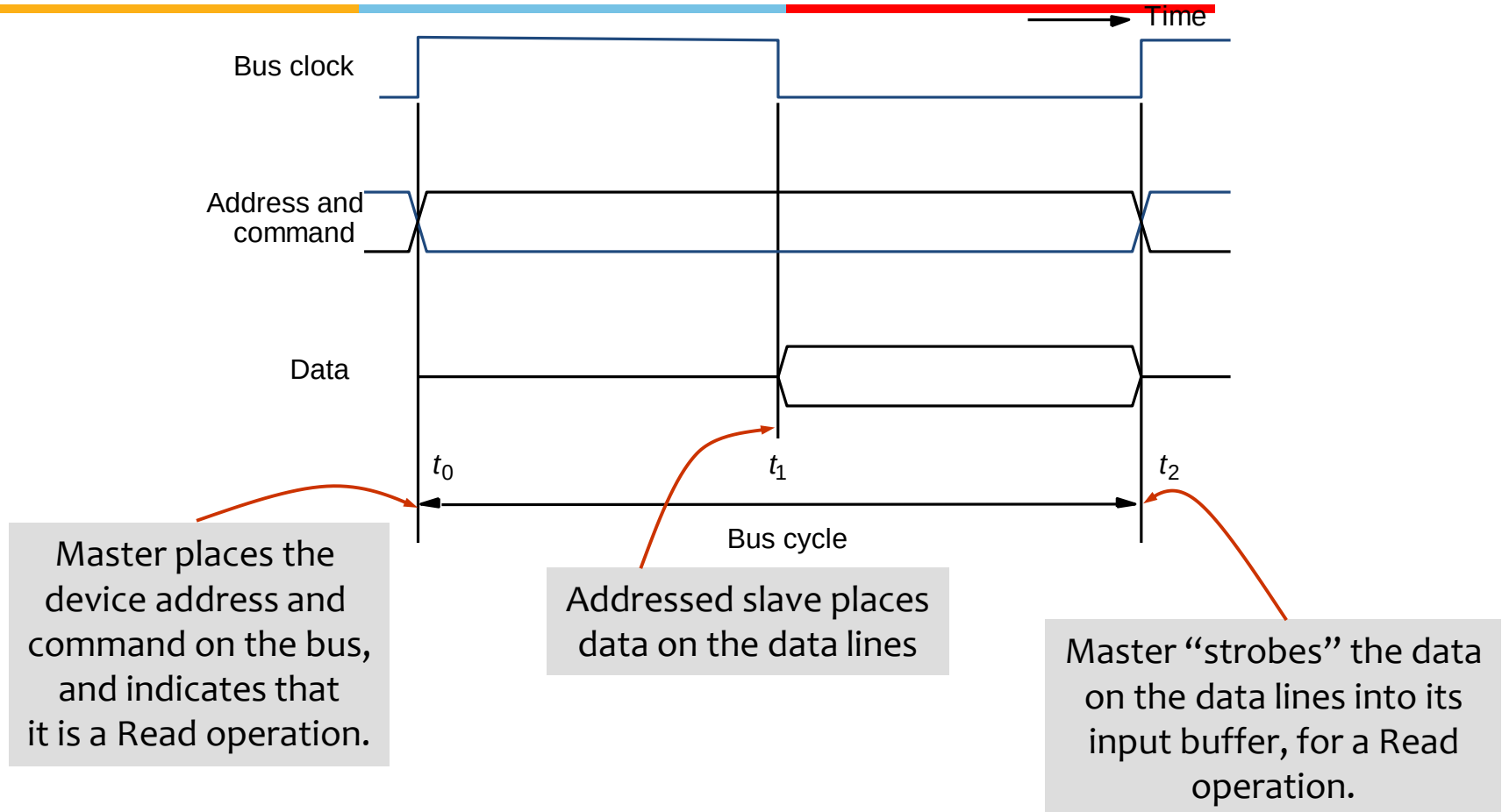
Timing

- Co-ordination of events on bus
- Synchronous
 - Events determined by clock signals and synchronized on leading edge of clock
 - All devices can read clock line
 - Usually a single cycle for an event
- Asynchronous
 - The occurrence of one event on a bus depends on the occurrence of a previous event
 - Events on the bus are not synchronized with the clock

Synchronous bus



Synchronous bus(contnd.)



- In case of a Write operation, the master places the data on the bus along with the address and commands at time t_0 .
- The slave strobes the data into its input buffer at time t_2 .



Synchronous bus(contnd.)

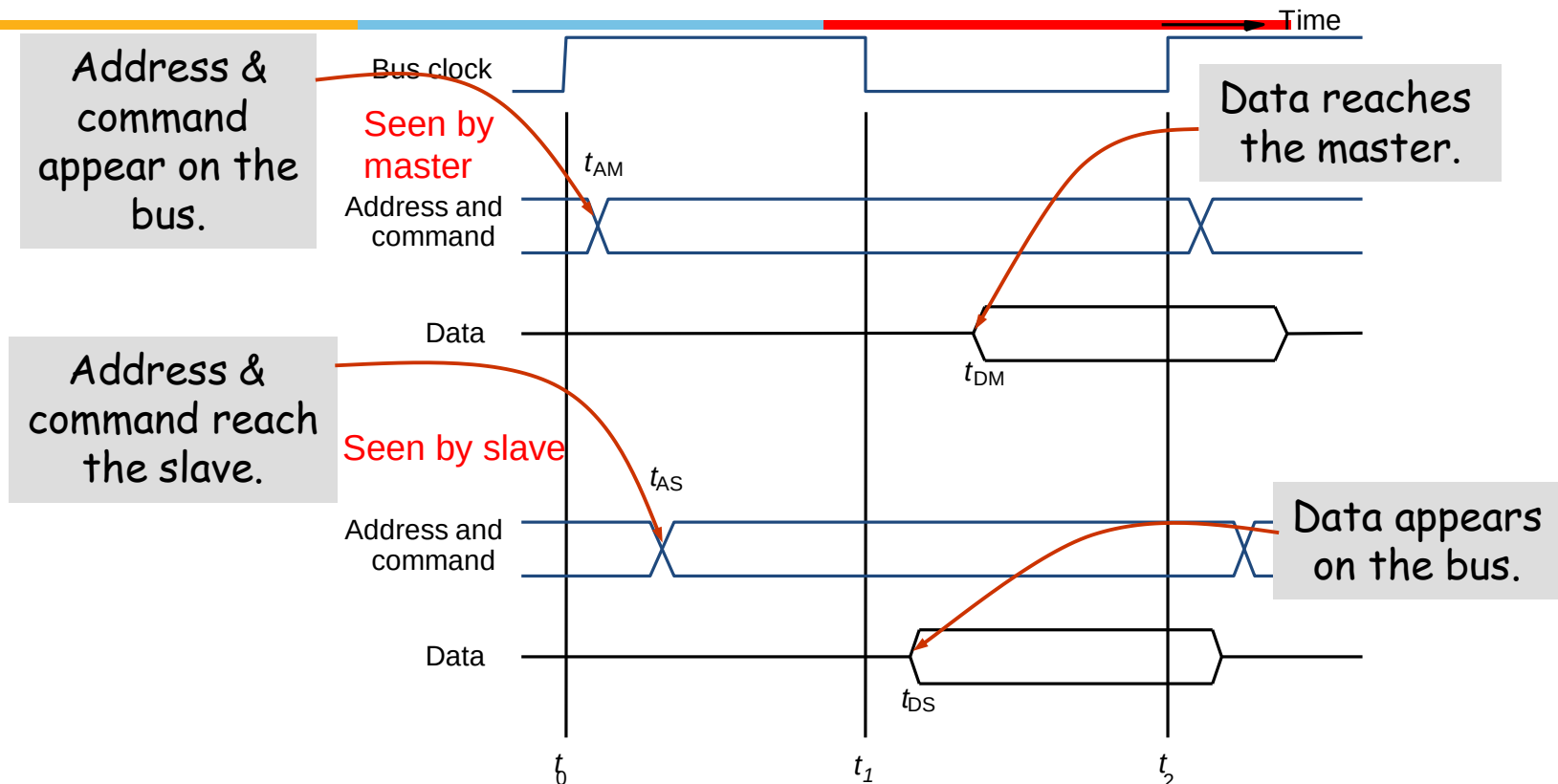
- Once the master places the device address and command on the bus, it takes time for this information to propagate to the devices:
 - This time depends on the physical and electrical characteristics of the bus.
- Also, all the devices have to be given enough time to decode the address and control signals, so that the addressed slave can place data on the bus.
- Width of the pulse $t_1 - t_0$ depends on:
 - Maximum propagation delay between two devices connected to the bus.
 - Time taken by all the devices to decode the address and control signals, so that the addressed slave can respond at time t_1 .



Synchronous bus(contnd.)

- At the end of the clock cycle, at time t_2 , the master strobes the data on the data lines into its input buffer if it's a Read operation.
 - “Strobe” means to capture the values of the data and store them into a buffer.
- When data are to be loaded into a storage buffer register, the data should be available for a period longer than the setup time of the device.
- Width of the pulse $t_2 - t_1$ should be longer than:
 - Maximum propagation time of the bus plus
 - Set up time of the input buffer register of the master.

Synchronous bus(contnd.)



- Signals do not appear on the bus as soon as they are placed on the bus, due to the propagation delay in the interface circuits.
- Signals reach the devices after a propagation delay which depends on the characteristics of the bus.
- Data must remain on the bus for some time after t_2 equal to the hold time of the buffer.



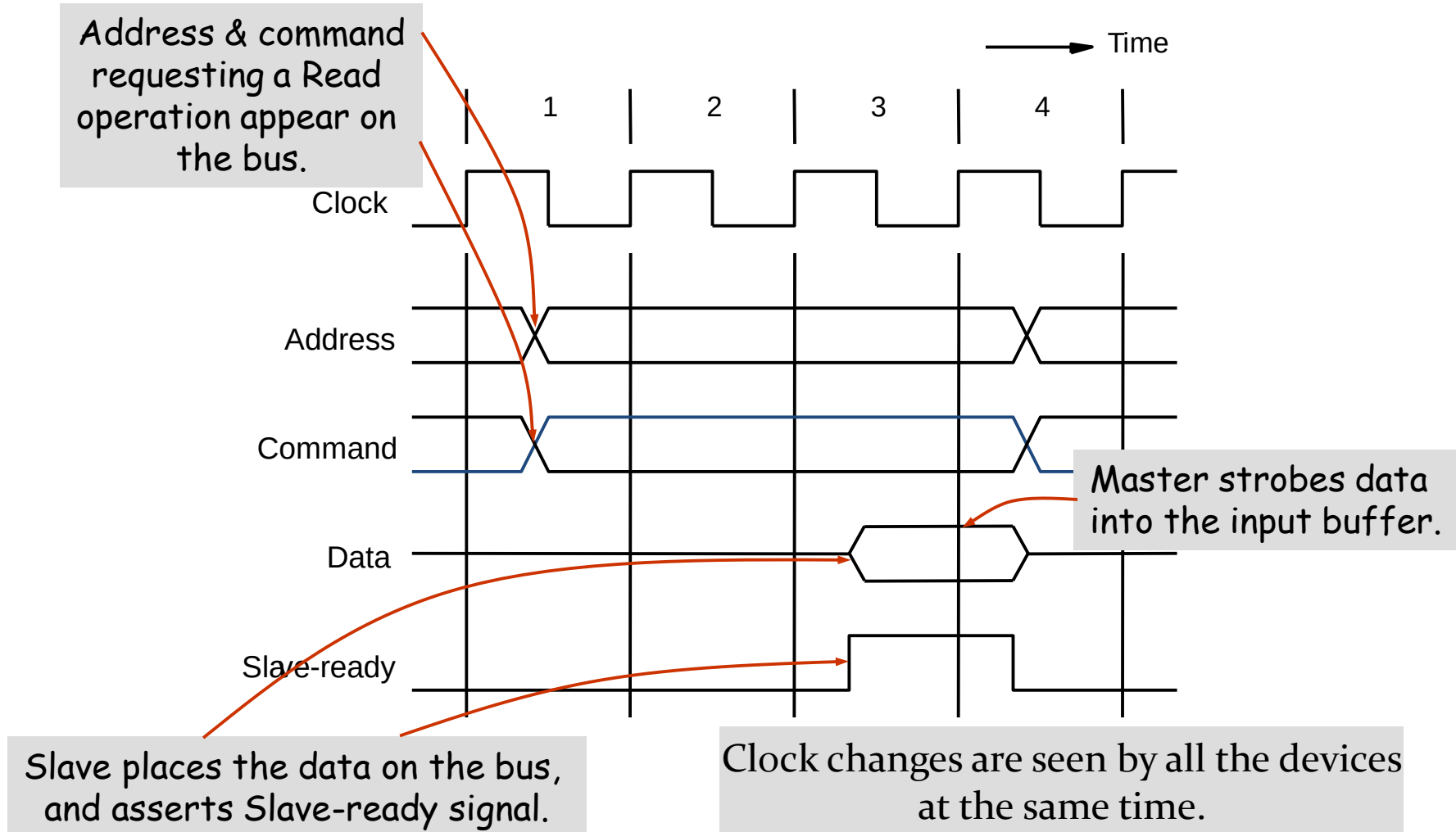
Synchronous bus(contnd.)

- Most buses have control signals to represent a response from the slave.
- Control signals serve two purposes:
 - Inform the master that the slave has recognized the address, and is ready to participate in a data transfer operation.
 - Enable to adjust the duration of the data transfer operation based on the speed of the participating slaves.
- High-frequency bus clock is used:
 - Data transfer spans several clock cycles instead of just one clock cycle as in the earlier case.

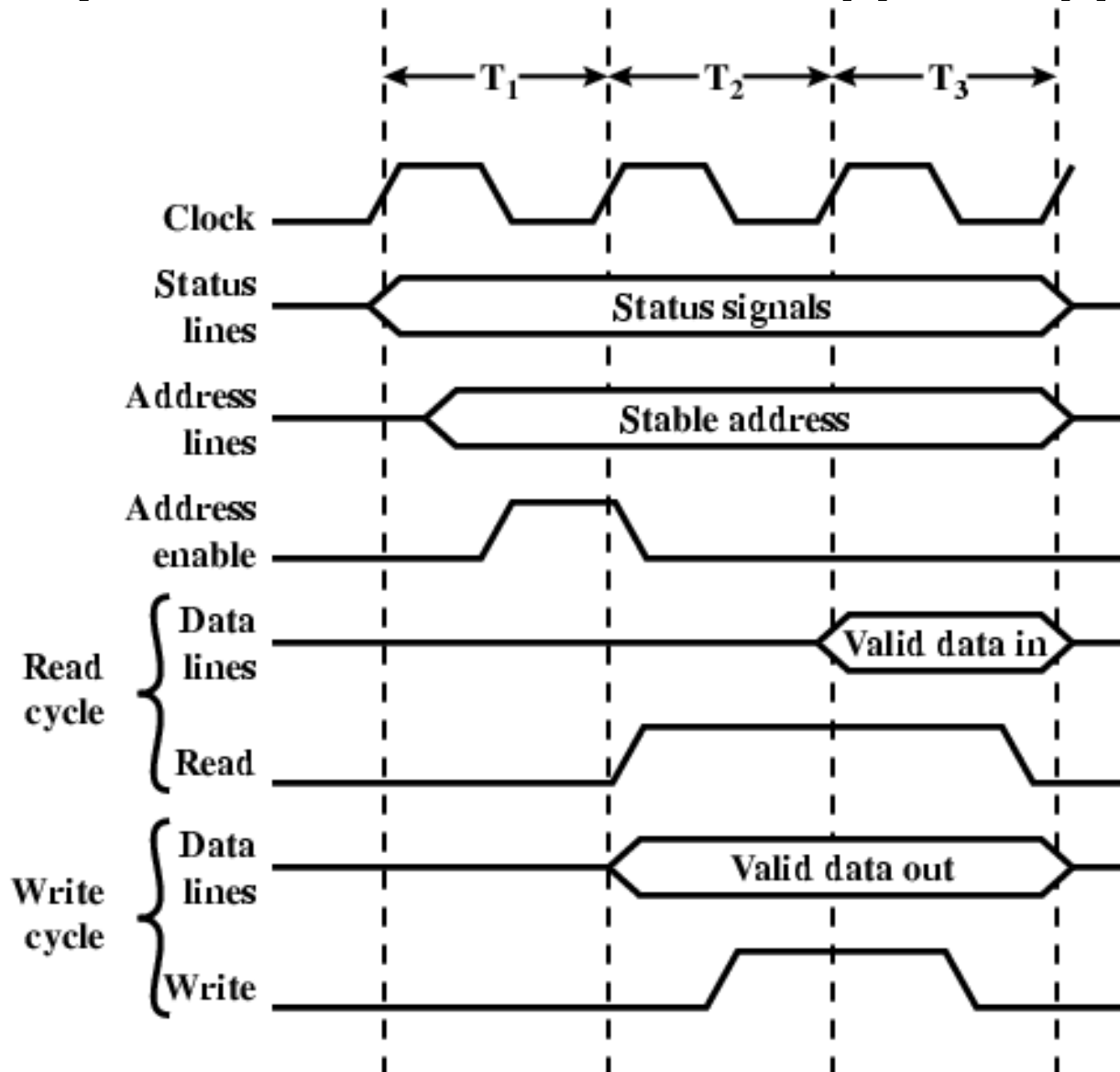
Synchronous bus(contnd.)



Slave-ready signal is an acknowledgement from the slave to the master to confirm that the valid data has been sent. Depending on when the slave-ready signal is asserted, the duration of the data transfer can change.



Synchronous Timing Diagram

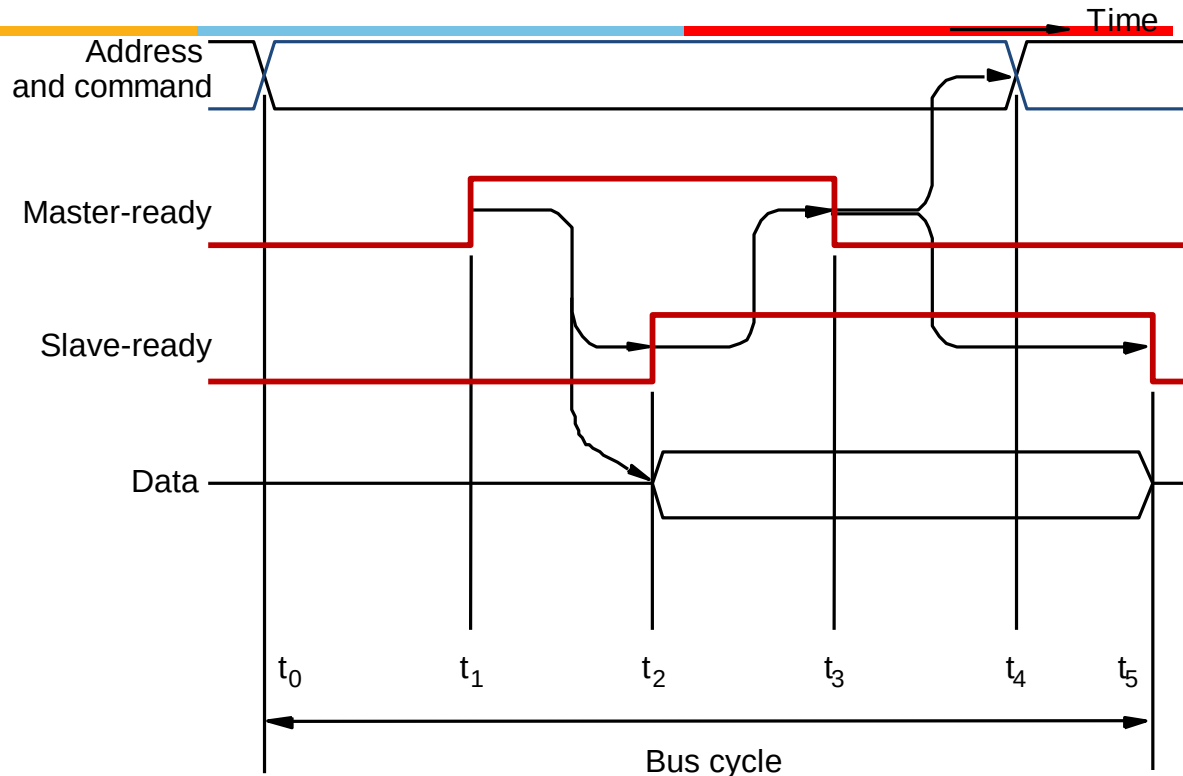




Asynchronous bus

- Data transfers on the bus is controlled by a handshake between the master and the slave.
- Common clock in the synchronous bus case is replaced by two timing control lines:
 - Master-ready,
 - Slave-ready.
- Master-ready signal is asserted by the master to indicate to the slave that it is ready to participate in a data transfer.
- Slave-ready signal is asserted by the slave in response to the master-ready from the master, and it indicates to the master that the slave is ready to participate in a data transfer.

Asynchronous bus(contnd.)



t_0 - Master places the address and command information on the bus.

t_1 - Master asserts the Master-ready signal. Master-ready signal is asserted at t_1 instead of t_0 .

t_2 - Addressed slave places the data on the bus and asserts the Slave-ready signal.

t_3 - Slave-ready signal arrives at the master.

t_4 - Master removes the address and command information.

t_5 - Slave receives the transition of the Master-ready signal from 1 to 0. It removes the data and the Slave-ready signal from the bus.

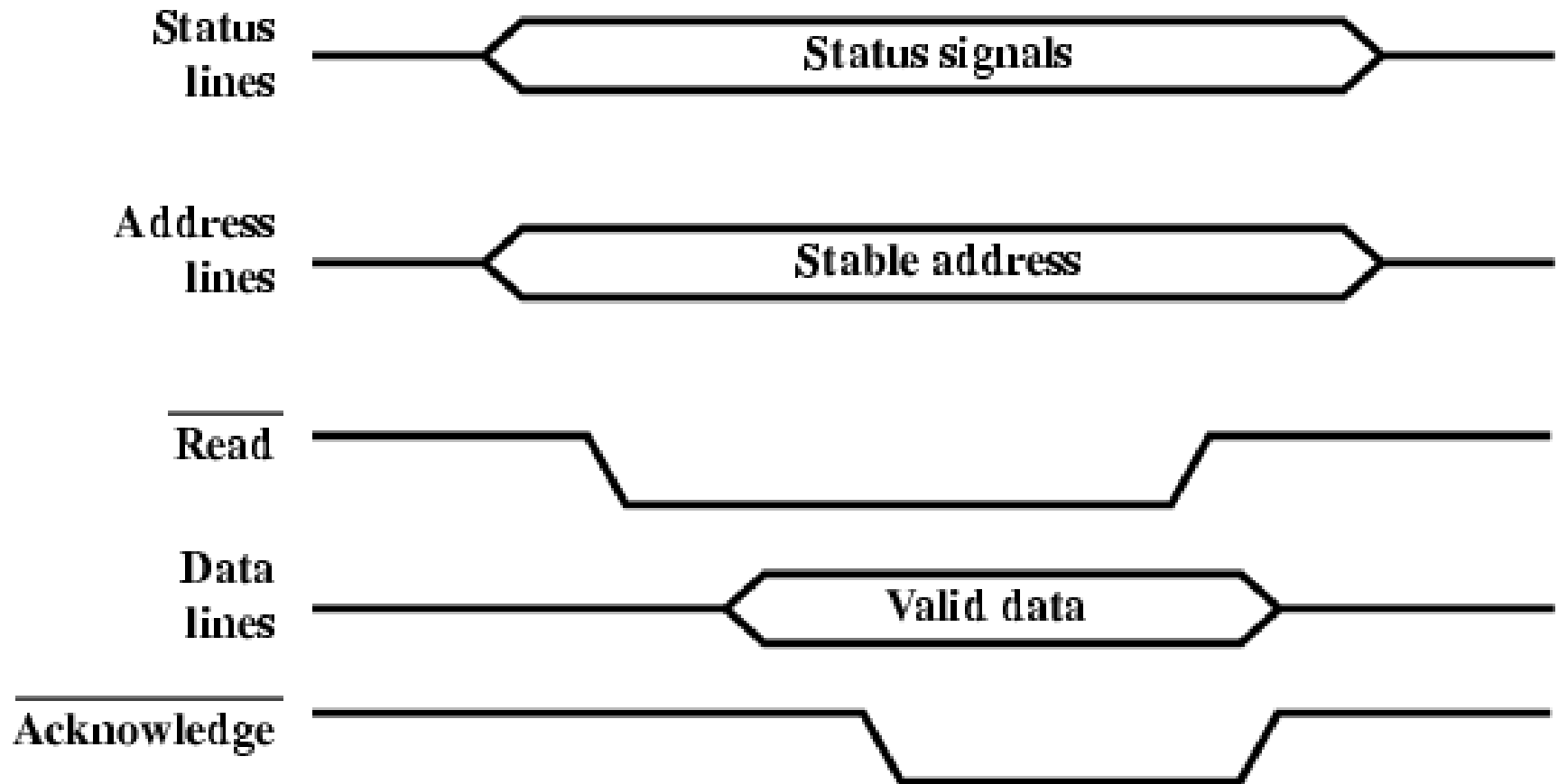


Asynchronous vs. Synchronous bus

- **Advantages of asynchronous bus:**
 - Eliminates the need for synchronization between the sender and the receiver.
 - Can accommodate varying delays automatically, using the Slave-ready signal.
- **Disadvantages of asynchronous bus:**
 - Data transfer rate with full handshake is limited by two-round trip delays.
 - Data transfers using a synchronous bus involves only one round trip delay, and hence a synchronous bus can achieve faster rates.

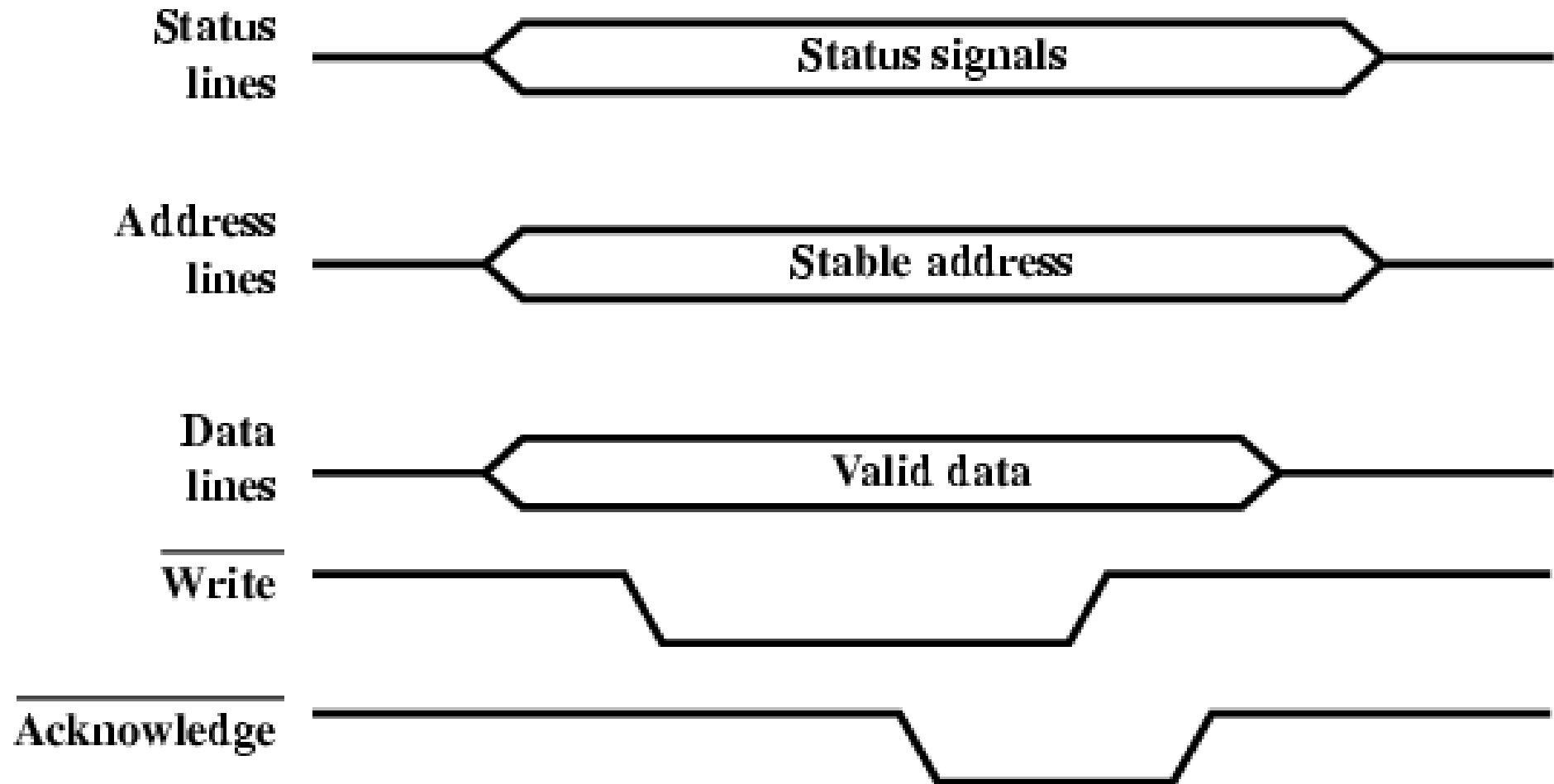


Asynchronous Timing – Read Diagram





Asynchronous Timing – Write Diagram

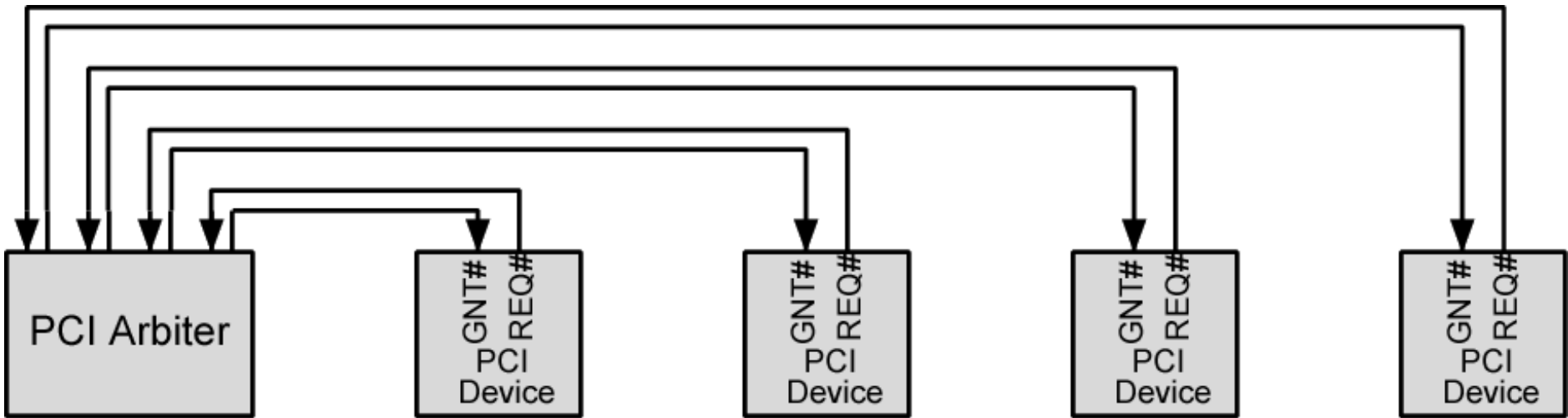




PCI Bus

- *Peripheral Component Interconnect*
- Introduced in 1992
- Low-cost bus
- Processor independent
- Plug-and-play capability
- In today's computers, most memory transfers involve a burst of data rather than just one word. The PCI is designed primarily to support this mode of operation.
- The bus supports three independent address spaces: memory, I/O, and configuration.
- we assumed that the master maintains the address information on the bus until data transfer is completed. But, the address is needed only long enough for the slave to be selected. Thus, the address is needed on the bus for one clock cycle only, freeing the address lines to be used for sending data in subsequent clock cycles. The result is a significant cost reduction.
- A master is called an initiator in PCI terminology. The addressed device that responds to read and write commands is called a target.

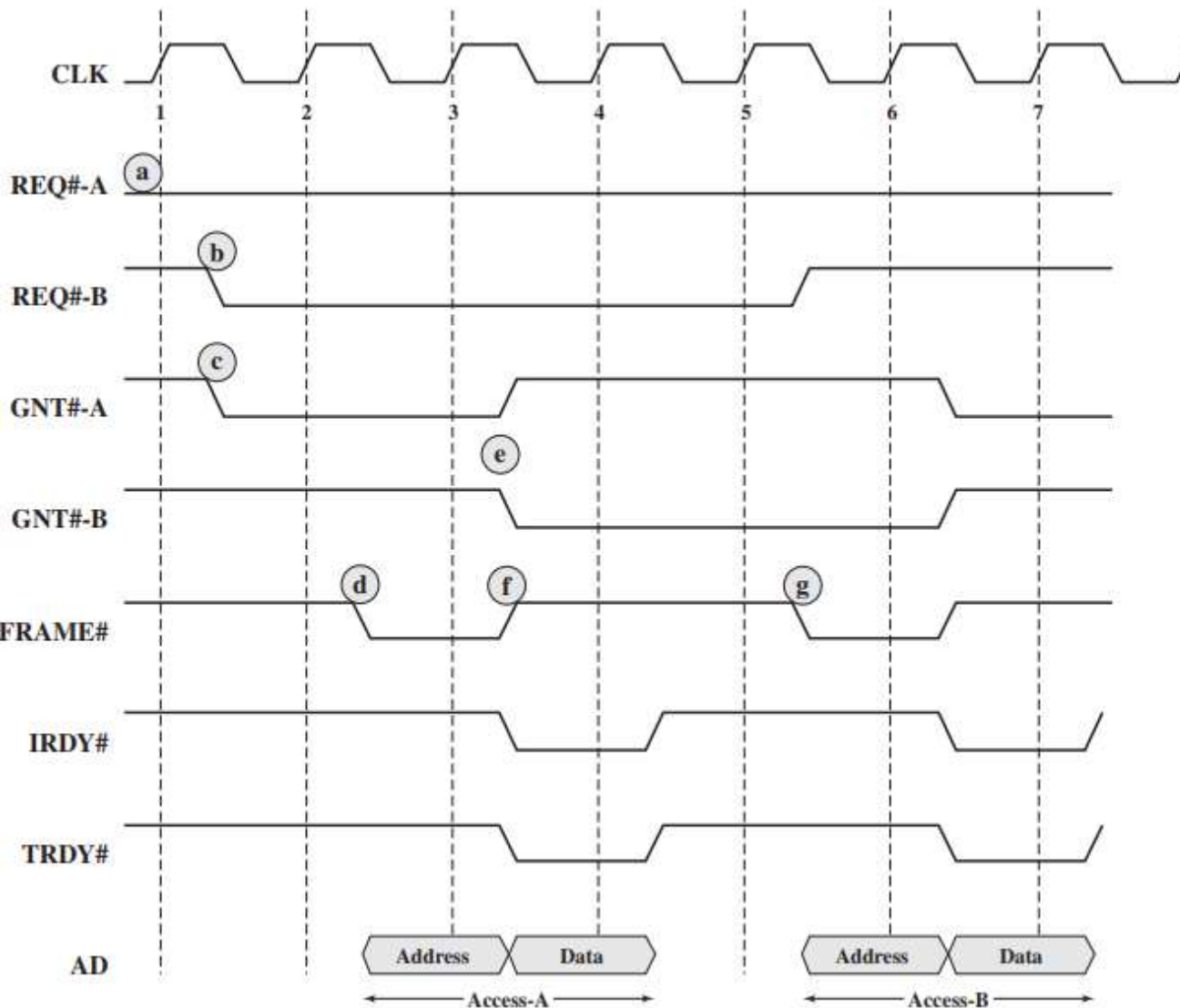
PCI Bus Arbiter



- ❑ PCI makes use of a centralized, synchronous arbitration scheme
- ❑ each master has a unique request (REQ) and grant (GNT) signal
- ❑ does not dictate a particular arbitration algorithm
- ✓ a first-come-first-served approach
- ✓ a round-robin approach
- ✓ or some sort of priority scheme.

PCI Bus Arbitration

devices A and B are arbitrating



IRDY# s/t/s Initiator Ready.

Driven by current bus master (initiator of transaction). During a read, indicates that the master is prepared to accept data; during a write, indicates that valid data are present on AD.

TRDY# s/t/s Target Ready.

Driven by the target (selected device). During a read, indicates that valid data are present on AD; during a write, indicates that target is ready to accept data



- a. At some point prior to the start of clock 1, A has asserted its REQ signal. The arbiter samples this signal at the beginning of clock cycle 1.
- b. During clock cycle 1, B requests use of the bus by asserting its REQ signal.
- c. At the same time, the arbiter asserts GNT-A to grant bus access to A.
- d. Bus master A samples GNT-A at the beginning of clock 2 and learns that it has been granted bus access. It also finds IRDY and TRDY de-asserted, indicating that the bus is idle. Accordingly, it asserts FRAME and places the address information on the address bus and the command on the C/BE bus (not shown). It also continues to assert REQ-A, because it has a second transaction to perform after this one



e. The bus arbiter samples all REQ lines at the beginning of clock 3 and makes an arbitration decision to grant the bus to B for the next transaction. It then asserts GNT-B and deasserts GNT-A. B will not be able to use the bus until it returns to an idle state.

f. A deasserts FRAME to indicate that the last (and only) data transfer is in progress. It puts the data on the data bus and signals the target with IRDY. The target reads the data at the beginning of the next clock cycle.

g. At the beginning of clock 5, B finds IRDY and FRAME deasserted and so is able to take control of the bus by asserting FRAME. It also deasserts its REQ line, because it only wants to perform one transaction.

Subsequently, master A is granted access to the bus for its next transaction.

Program

PAdd	LAdd	Instruction	Bas
☐ 0000	0000	MOV #40, R02	000
☐ 0006	0006	TRANS::	000
☐ 0006	0006	STB R02, @R02	000
☒ → 0011	0011	CMP #103, R02	000
☐ 0017	0017	JEQ \$STOP	000
☐ 0021	0021	INC R02	000
☐ 0024	0024	JMP \$TRANS:	000
☐ 0028	0028	STOP:	000
☐ 0028	0028	HLT	000

<https://yasmin-cpu-os-simulator.software.informer.com/6.1/>

IMP Note to Self





BITS Pilani
Pilani Campus



RKTiwary
COA -IS ZC353



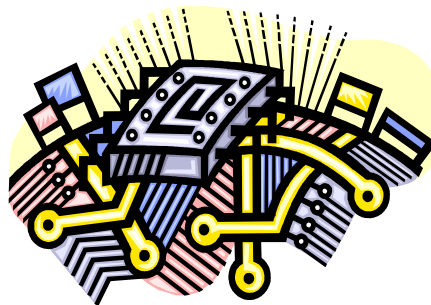
Chapter 9

- Computer Arithmetic

Arithmetic & Logic Unit (ALU)



- ❑ Part of the computer that actually performs arithmetic and logical operations on data
- ❑ All of the other elements of the computer system are there mainly to bring data into the ALU for it to process and then to take the results back out
- ❑ Based on the use of simple digital logic devices that can store binary digits and perform simple Boolean logic operations



ALU Inputs and outputs

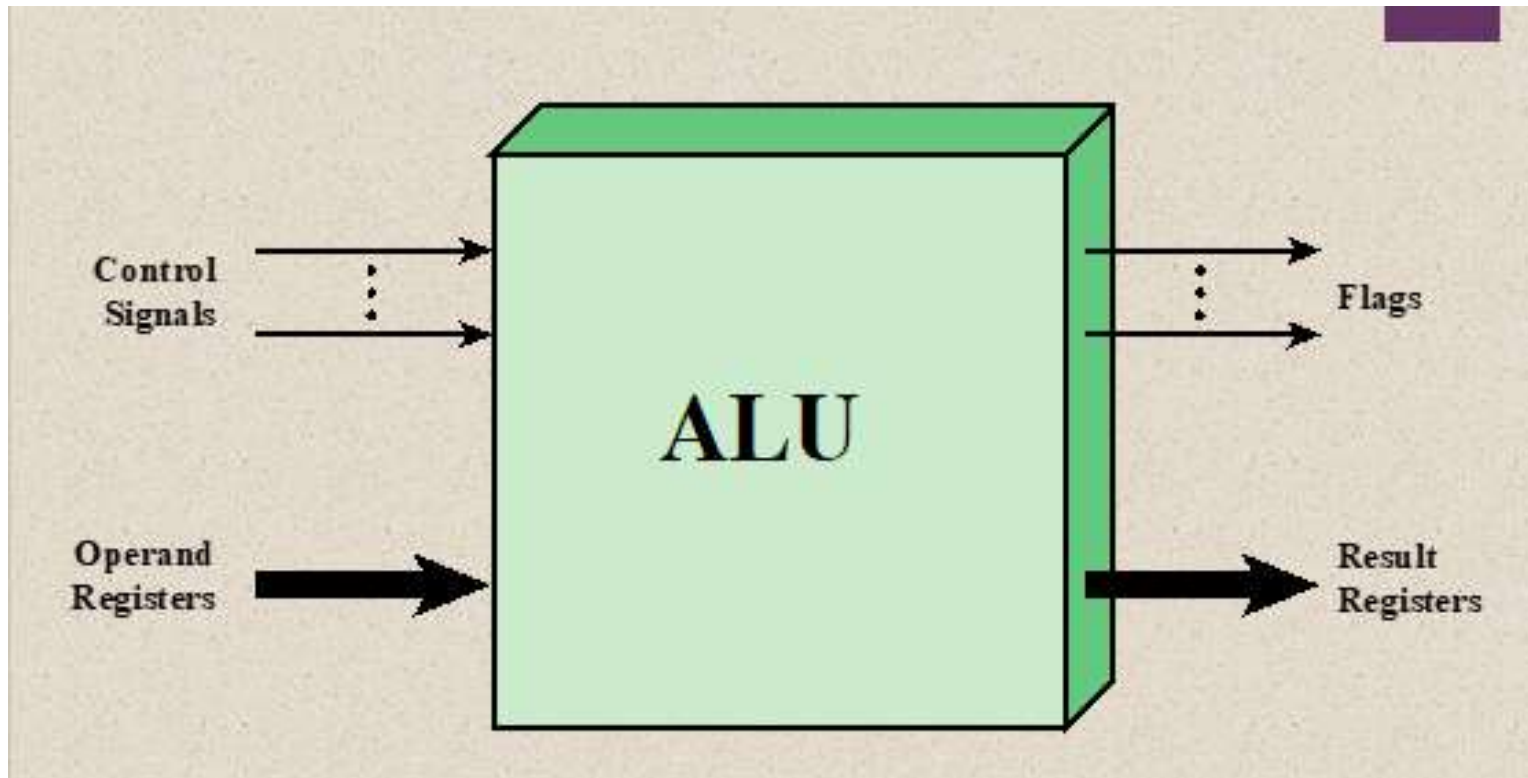


Fig 9.1

Integer Representation



In the binary number system arbitrary numbers can be represented with:

- The digits zero and one
- The minus sign (for negative numbers)
- The period, or **radix point** (for numbers with a fractional component)
- For purposes of computer storage and processing we do not have the benefit of special symbols for the minus sign and radix point
- Only binary digits (0,1) may be used to represent numbers





Signed Nos 2's complement representation

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

Eg +6= 0110
-6= 1010

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$



Consider any binary number in 2's complement

101101

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$



Consider any binary number in 2's complement

101101

$$= -2^{6-1}.1 + 2^4.0 + 2^3.1 + 2^2.2 + 2^1.0 + 2^0.1$$

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$



Consider any binary number in 2's complement

101101

$$\begin{aligned} &= -2^{6-1}.1 + 2^4.0 + 2^3.1 + 2^2.1 + 2^1.0 + 2^0.1 \\ &= -32 + 0 + 8 + 4 + 0 + 1 = -19 \end{aligned}$$

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$



Consider any binary number in 2's complement
101101

$$= -2^{6-1} \cdot 1 + 2^4 \cdot 0 + 2^3 \cdot 1 + 2^2 \cdot 1 + 2^1 \cdot 0 + 2^0 \cdot 1$$

$$= -32 + 0 + 8 + 4 + 0 + 1 = -19$$

The 2's complement of 101101 is 010011 = 19

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$



Consider any binary number in 2's complement
101101

$$= -2^{6-1} \cdot 1 + 2^4 \cdot 0 + 2^3 \cdot 1 + 2^2 \cdot 1 + 2^1 \cdot 0 + 2^0 \cdot 1$$

$$= -32 + 0 + 8 + 4 + 0 + 1 = -19$$

The 2's complement of 101101 is 010011 = 19

Now if a 1 is appended then 1101101 ?

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$



Consider any binary number in 2's complement
101101

$$= -2^{6-1}.1 + 2^4.0 + 2^3.1 + 2^2.1 + 2^1.0 + 2^0.1$$

$$= -32 + 0 + 8 + 4 + 0 + 1 = -19$$

The 2's complement of 101101 is 010011 = 19

Now if a 1 is appended then 1101101 ?

$$1101101 = -2^{7-1}.1 + 2^5.1 + 2^4.0 + 2^3.1 + 2^2.1 + 2^1.0 + 2^0.1$$

$$= -64 + 32 + 0 + 8 + 4 + 0 + 1$$

$$= -19$$

Addition and Subtraction

OVERFLOW RULE: If two numbers are added, and they are both positive or both negative, then overflow occurs if and only if the result has the opposite sign.

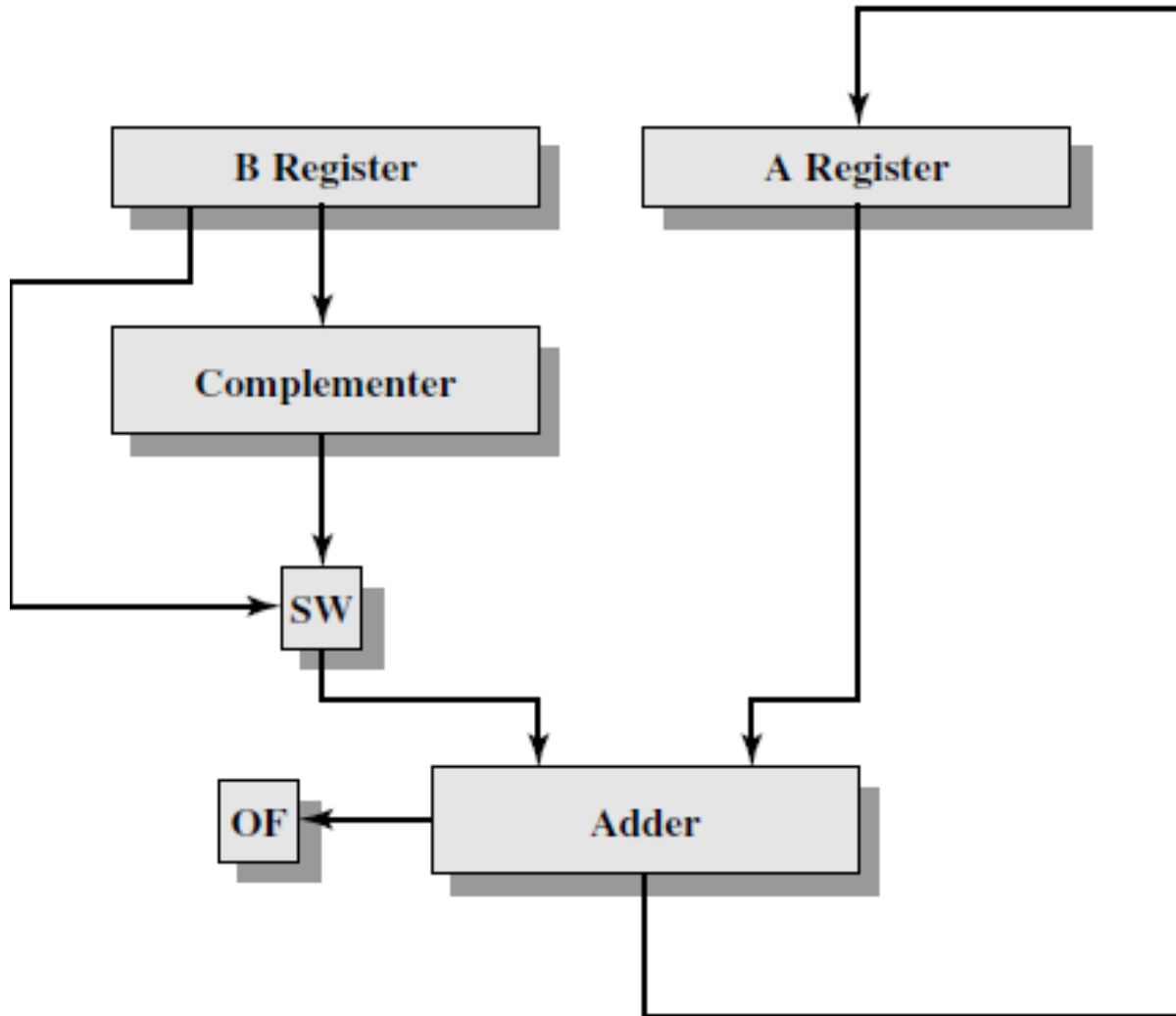
$\begin{array}{r} 1001 = -7 \\ + \underline{0101} = 5 \\ 1110 = -2 \\ (a) (-7) + (+5) \end{array}$	$\begin{array}{r} 1100 = -4 \\ + \underline{0100} = 4 \\ 10000 = 0 \\ (b) (-4) + (+4) \end{array}$
$\begin{array}{r} 0011 = 3 \\ + \underline{0100} = 4 \\ 0111 = 7 \\ (c) (+3) + (+4) \end{array}$	$\begin{array}{r} 1100 = -4 \\ + \underline{1111} = -1 \\ 11011 = -5 \\ (d) (-4) + (-1) \end{array}$
$\begin{array}{r} 0101 = 5 \\ + \underline{0100} = 4 \\ 1001 = \text{Overflow} \\ (e) (+5) + (+4) \end{array}$	$\begin{array}{r} 1001 = -7 \\ + \underline{1010} = -6 \\ 10011 = \text{Overflow} \\ (f) (-7) + (-6) \end{array}$

Addition and Subtraction

SUBTRACTION RULE: To subtract one number (subtrahend) from another (minuend), take the twos complement (negation) of the subtrahend and add it to the minuend.

$\begin{array}{r} 0010 = 2 \\ + 1001 = -7 \\ \hline 1011 = -5 \end{array}$ (a) M = 2 = 0010 S = 7 = 0111 -S = 1001	$\begin{array}{r} 0101 = 5 \\ + 1110 = -2 \\ \hline 10011 = 3 \end{array}$ (b) M = 5 = 0101 S = 2 = 0010 -S = 1110
$\begin{array}{r} 1011 = -5 \\ + 1110 = -2 \\ \hline 11001 = -7 \end{array}$ (c) M = -5 = 1011 S = 2 = 0010 -S = 1110	$\begin{array}{r} 0101 = 5 \\ + 0010 = 2 \\ \hline 0111 = 7 \end{array}$ (d) M = 5 = 0101 S = -2 = 1110 -S = 0010
$\begin{array}{r} 0111 = 7 \\ + 0111 = 7 \\ \hline 1110 = \text{Overflow} \end{array}$ (e) M = 7 = 0111 S = -7 = 1001 -S = 0111	$\begin{array}{r} 1010 = -6 \\ + 1100 = -4 \\ \hline 10110 = \text{Overflow} \end{array}$ (f) M = -6 = 1010 S = 4 = 0100 -S = 1100

Block Diagram of Hardware for Addition and Subtraction



OF= Overflow bit
 SW =Switch (select
 addition or
 subtraction)

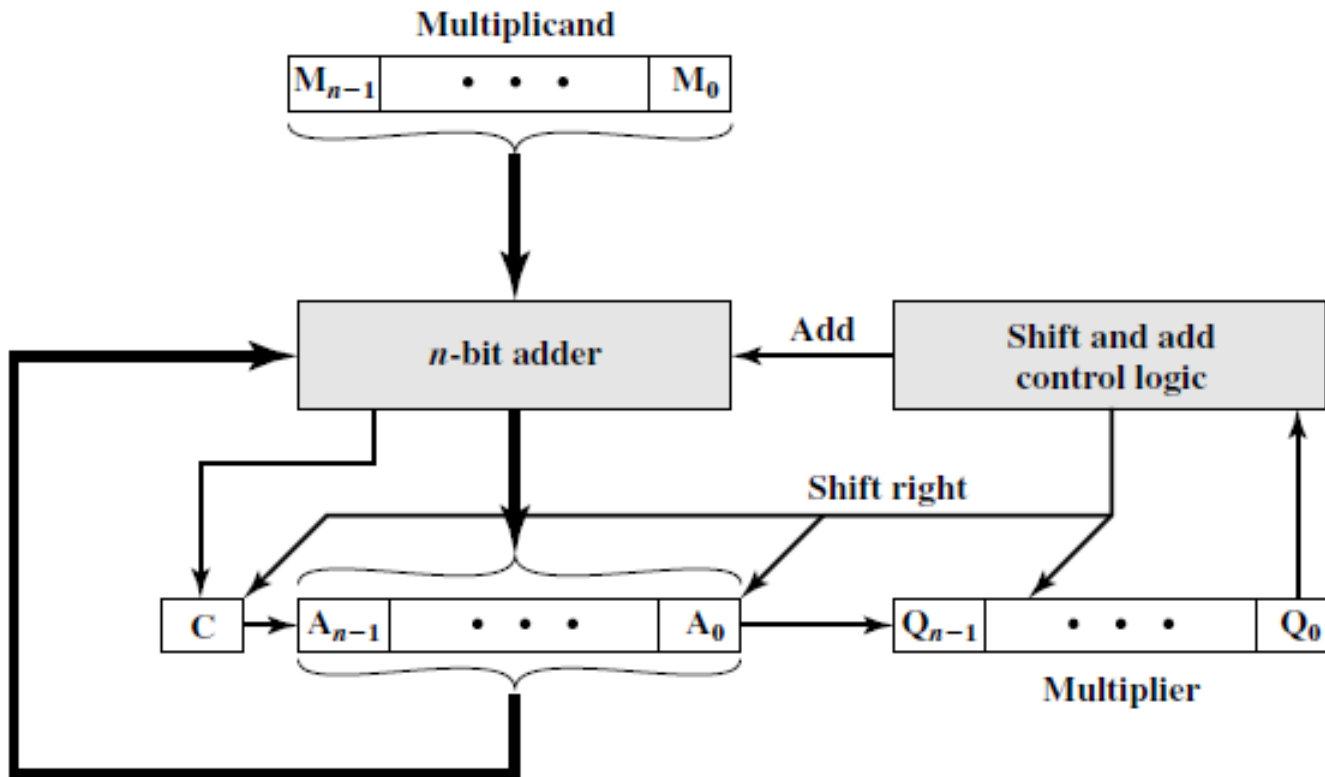


Multiplication of UNSIGNED INTEGERS

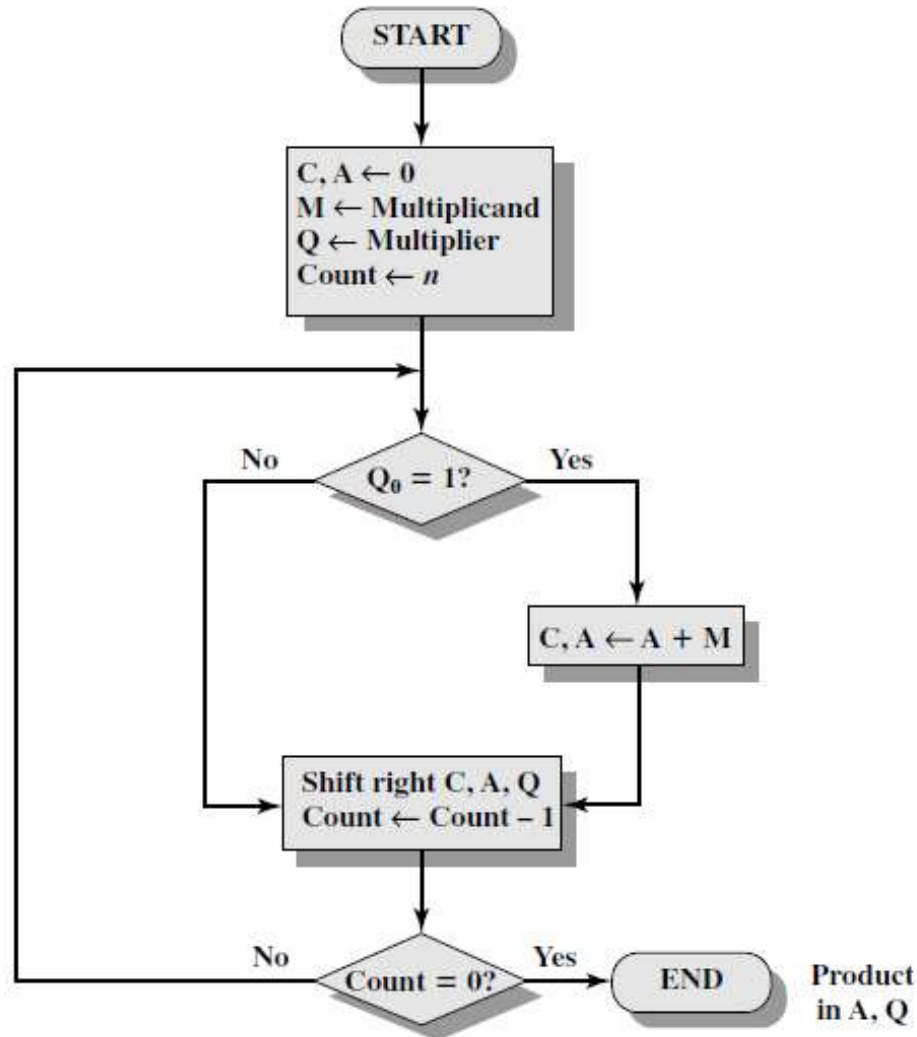
1. Multiplication involves the generation of partial products, one for each digit in the multiplier. These partial products are then summed to produce the final product.
2. The partial products are easily defined. When the multiplier bit is 0, the partial product is 0. When the multiplier is 1, the partial product is the multiplicand.
3. The total product is produced by summing the partial products. For this operation, each successive partial product is shifted one position to the left relative to the preceding partial product.
4. The multiplication of two n -bit binary integers results in a product of up to $2n$ bits in length (e.g., $11 \times 11 = 1001$).

1011	Multiplicand (11)
$\times 1101$	Multiplier (13)
1011	} Partial products
0000	
1011	
1011	
10001111	Product (143)

Multiplication of UNSIGNED INTEGERS



Multiplication of UNSIGNED INTEGERS



Multiplication of UNSIGNED INTEGERS



C	A	Q	M		
0	0000	1101	1011	Initial value	

Multiplication of UNSIGNED INTEGERS

C	A	Q	M		
0	0000	1101	1011	Initial value	
0	1011	1101	1011	Add	First cycle
0	0101	1110	1011	Shift	



Multiplication of UNSIGNED INTEGERS

C	A	Q	M		
0	0000	1101	1011	Initial value	
0	1011	1101	1011	Add	1 st cycle
0	0101	1110	1011	Shift	
0	0010	1111	1011	shift	2 nd cycle

Multiplication of UNSIGNED INTEGERS

C	A	Q	M		
0	0000	1101	1011	Initial value	
0	1011	1101	1011	Add	1 st cycle
0	0101	1110	1011	Shift	
0	0010	1111	1011	shift	2 nd cycle
0	1101	1111	1011	Add	

Multiplication of UNSIGNED INTEGERS

C	A	Q	M		
0	0000	1101	1011	Initial value	
0	1011	1101	1011	Add	1 st cycle
0	0101	1110	1011	Shift	
0	0010	1111	1011	shift	2 nd cycle
0	1101	1111	1011	Add	3 rd cycle
0	0110	1111	1011	shift	

Multiplication of UNSIGNED INTEGERS

C	A	Q	M		
0	0000	1101	1011	Initial value	
0	1011	1101	1011	Add	1 st cycle
0	0101	1110	1011	Shift	
0	0010	1111	1011	shift	2 nd cycle
0	1101	1111	1011	Add	3 rd cycle
0	0110	1111	1011	shift	
1	0001	1111	1011	Add	

Multiplication of UNSIGNED INTEGERS

C	A	Q	M		
0	0000	1101	1011	Initial value	
0	1011	1101	1011	Add	1 st cycle
0	0101	1110	1011	Shift	
0	0010	1111	1011	shift	2 nd cycle
0	1101	1111	1011	Add	3 rd cycle
0	0110	1111	1011	shift	
1	0001	1111	1011	Add	4 th cycle
0	1000	1111	1011	Shift	143

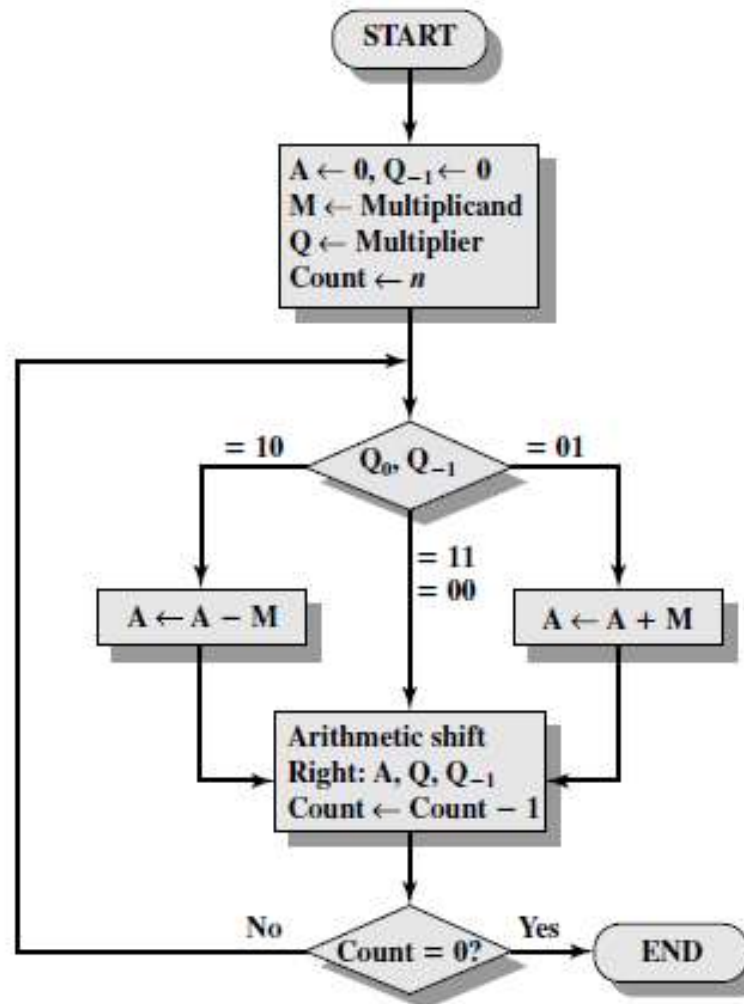
Multiplication of SIGNED INTEGERS

11 (1011) by 13 (1101) to get 143 (10001111)

-5(1011) times -3 (1101) equals -113 (10001111)

$ \begin{array}{r} 1001 \quad (9) \\ \times 0011 \quad (3) \\ \hline 00001001 \quad 1001 \times 2^0 \\ 00010010 \quad 1001 \times 2^1 \\ \hline 00011011 \quad (27) \end{array} $	$ \begin{array}{r} 1001 \quad (-7) \\ \times 0011 \quad (3) \\ \hline 11111001 \quad (-7) \times 2^0 = (-7) \\ 11110010 \quad (-7) \times 2^1 = (-14) \\ \hline 11101011 \quad (-21) \end{array} $
---	--

Booth Multiplier



Multiplication of -8 X 5



01 A+M
10 A-M

A	Q(multiplier)	Q-1	M(Multiplicand)	Comments
00000	00101	0	11000	Initial value

Multiplication of -8 X 5



01 A+M
10 A-M

A	Q(multiplier)	Q-1	M(Multiplicand)	Comments
00000	00101	0	11000	Initial value
01000	00101	0	11000	A-M

Multiplication of -8 X 5



01 A+M
10 A-M

A	Q(multiplier)	Q-1	M(Multiplicand)	Comments
00000	00101	0	11000	Initial value
01000	00101	0	11000	A-M
00100	00010	1	11000	

Multiplication of -8 X 5



01 A+M
10 A-M

A	Q(multiplier)	Q-1	M(Multiplicand)	Comments
00000	00101	0	11000	Initial value
01000	00101	0	11000	A-M
00100	00010	1	11000	
11100	00010	1	11000	A+M

Multiplication of -8 X 5



01 A+M
10 A-M

A	Q(multiplier)	Q-1	M(Multiplicand)	Comments
00000	00101	0	11000	Initial value
01000	00101	0	11000	A-M
00100	00010	1	11000	
11100	00010	1	11000	A+M
11110	00001	0	11000	

Multiplication of -8 X 5



01 A+M
10 A-M

A	Q(multiplier)	Q-1	M(Multiplicand)	Comments
00000	00101	0	11000	Initial value
01000	00101	0	11000	A-M
00100	00010	1	11000	
11100	00010	1	11000	A+M
11110	00001	0	11000	
00110	00001	0	11000	A-M

Multiplication of -8 X 5



01 A+M
10 A-M

A	Q(multiplier)	Q-1	M(Multiplicand)	Comments
00000	00101	0	11000	Initial value
01000	00101	0	11000	A-M
00100	00010	1	11000	
11100	00010	1	11000	A+M
11110	00001	0	11000	
00110	00001	0	11000	A-M
00011	00000	1	11000	

Multiplication of -8 X 5



01 A+M
10 A-M

A	Q(multiplier)	Q-1	M(Multiplicand	Comments
00000	00101	0	11000	Initial value
01000	00101	0	11000	A-M
00100	00010	1	11000	
11100	00010	1	11000	A+M
11110	00001	0	11000	
00110	00001	0	11000	A-M
00011	00000	1	11000	
11011	00000	1	11000	A-M

Multiplication of -8 X 5



01 A+M
10 A-M

A	Q(multiplier)	Q-1	M(Multiplicand	Comments
00000	00101	0	11000	Initial value
01000	00101	0	11000	A-M
00100	00010	1	11000	
11100	00010	1	11000	A+M
11110	00001	0	11000	
00110	00001	0	11000	A-M
00011	00000	1	11000	
11011	00000	1	11000	A-M
11101	10000	0	11000	

Multiplication of -8 X 5



01 A+M
10 A-M

A	Q(multiplier)	Q-1	M(Multiplicand	Comments
00000	00101	0	11000	Initial value
01000	00101	0	11000	A-M
00100	00010	1	11000	
11100	00010	1	11000	A+M
11110	00001	0	11000	
00110	00001	0	11000	A-M
00011	00000	1	11000	
11011	00000	1	11000	A-M
11101	10000	0	11000	
11110	11000	0	11000	shift

Multiplication of -8 X 5



01 A+M
10 A-M

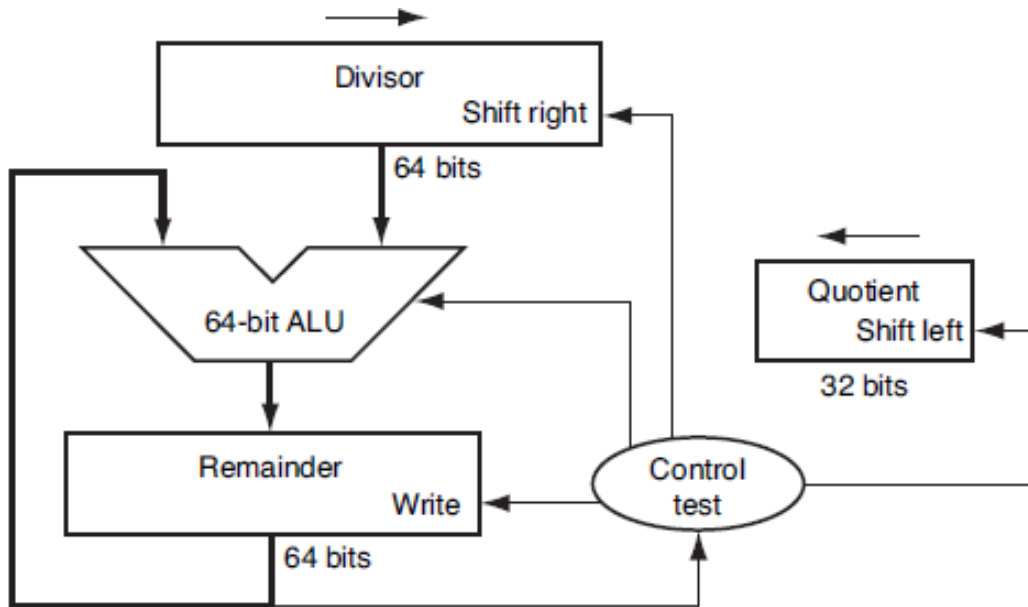
A	Q(multiplier)	Q-1	M(Multiplicand)	Comments
00000	00101	0	11000	Initial value
01000	00101	0	11000	A-M
00100	00010	1	11000	
11100	00010	1	11000	A+M
11110	00001	0	11000	
00110	00001	0	11000	A-M
00011	00000	1	11000	
11011	00000	1	11000	A-M
11101	10000	0	11000	
11110	11000	0	11000	shift
11110	11000	-40	-512+472	Final result

The basic grammar school division algorithm

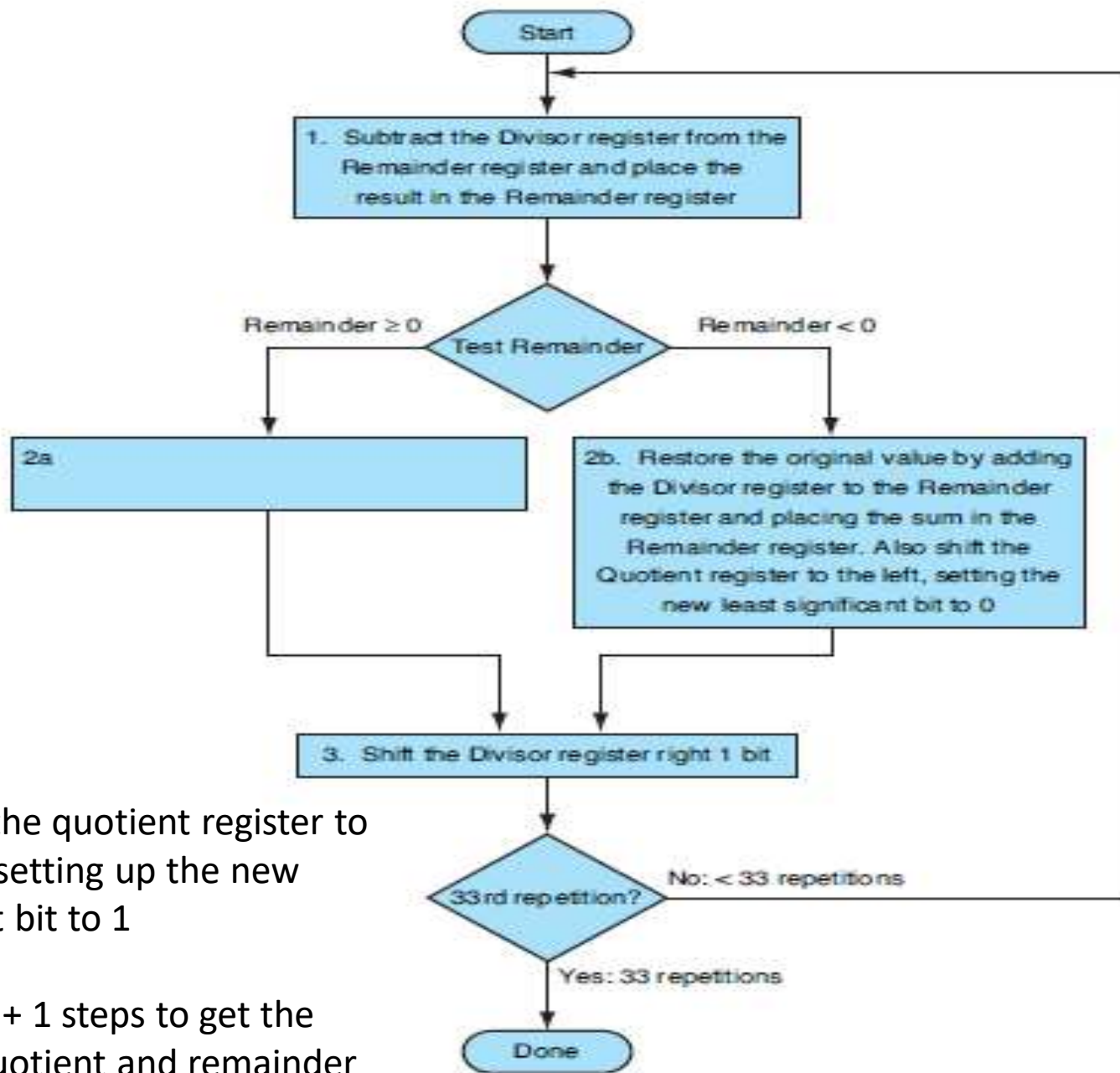
Division Algorithm and Hardware



$$\text{Dividend} = \text{Quotient} \times \text{Divisor} + \text{Remainder}$$



- ❖ The Divisor register, ALU, and Remainder register are all 64 bits wide, with only the Quotient register being 32 bits.
- ❖ The 32-bit divisor starts in the left half of the Divisor register and is shifted right 1 bit each iteration.
- ❖ The remainder is initialized with the dividend. Control decides when to shift the Divisor and Quotient registers and when to write the new value into the Remainder register.



2a: shift the quotient register to the left, setting up the new rightmost bit to 1

it takes $n + 1$ steps to get the proper quotient and remainder

Division 7/2

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1: Rem = Rem – Div	0000	0010 0000	①110 0111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0010 0000	0000 0111
	3: Shift Div right	0000	0001 0000	0000 0111
2	1: Rem = Rem – Div	0000	0001 0000	①111 0111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0001 0000	0000 0111
	3: Shift Div right	0000	0000 1000	0000 0111
3	1: Rem = Rem – Div	0000	0000 1000	①111 1111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0000 1000	0000 0111
	3: Shift Div right	0000	0000 0100	0000 0111
4	1: Rem = Rem – Div	0000	0000 0100	①000 0011
	2a: Rem $\geq$ 0 $\Rightarrow$ sll Q, Q0 = 1	0001	0000 0100	0000 0011
	3: Shift Div right	0001	0000 0010	0000 0011
5	1: Rem = Rem – Div	0001	0000 0010	①000 0001
	2a: Rem $\geq$ 0 $\Rightarrow$ sll Q, Q0 = 1	0011	0000 0010	0000 0001
	3: Shift Div right	0011	0000 0001	0000 0001

Shift left logical (sll)

9/4

n	step	Quotient	Divisor	remainder
0	Initial value	0000	0100 0000	0000 1001

9/4

[illegible]

[illegible]

n	step	Quotient	Divisor	remainder
0	Initial value	0000	0100 0000	0000 1001
1	1: Rem= Rem-Div	0000	0100 0000	1100 0000
	2b: Rem<0 => +Div, sll Q, Q0=0	0000	0100 0000	0000 1001
	3: Shift Div Right	0000	0010 0000	0000 1001
2	1: Rem= Rem-Div	0000	0010 0000	1110 1001
	2b: Rem<0 => +Div, sll Q, Q0=0	0000	0010 0000	0000 1001
	3: Shift Div Right	0000	0001 0000	0000 1001

n	step	Quotient	Divisor	remainder
0	Initial value	0000	0100 0000	0000 1001
1	1: Rem= Rem-Div	0000	0100 0000	1100 0000
	2b: Rem<0 => +Div, sll Q, Q0=0	0000	0100 0000	0000 1001
	3: Shift Div Right	0000	0010 0000	0000 1001
2	1: Rem= Rem-Div	0000	0010 0000	1110 1001
	2b: Rem<0 => +Div, sll Q, Q0=0	0000	0010 0000	0000 1001
	3: Shift Div Right	0000	0001 0000	0000 1001
3	1: Rem= Rem-Div	0000	0001 0000	1111 1001
	2b: Rem<0 => +Div, sll Q, Q0=0	0000	0001 0000	0000 1001
	3: Shift Div Right	0000	0000 1000	0000 1001

n	step	Quotient	Divisor	remainder
0	Initial value	0000	0100 0000	0000 1001
1	1: Rem= Rem-Div	0000	0100 0000	1 100 0000
	2b: Rem<0 => +Div, sll Q, Q0=0	0000	0100 0000	0000 1001
	3: Shift Div Right	0000	0010 0000	0000 1001
2	1: Rem= Rem-Div	0000	0010 0000	1 110 1001
	2b: Rem<0 => +Div, sll Q, Q0=0	0000	0010 0000	0000 1001
	3: Shift Div Right	0000	0001 0000	0000 1001
3	1: Rem= Rem-Div	0000	0001 0000	1 111 1001
	2b: Rem<0 => +Div, sll Q, Q0=0	0000	0001 0000	0000 1001
	3: Shift Div Right	0000	0000 1000	0000 1001
4	1: Rem= Rem-Div	0000	0000 1000	0 000 0001
	2a: Rem>0 => sll Q, Q0=1	0001	0000 1000	0000 0001
	3: Shift Div Right	0001	0000 0100	0000 0001
5	1: Rem= Rem-Div	0001	0000 0100	1 111 1101
	2b: Rem<0 => +Div, sll Q, Q0=0	0010	0000 0100	0000 0001
	3: Shift Div Right	0010	0000 0010	0000 0001



Remember the signs of the divisor and dividend and then negate the quotient if the signs disagree.

$$\text{Dividend} = \text{Quotient} \times \text{Divisor} + \text{Remainder}$$

Look at the example of

$$\pm 11 \div \pm 5$$

$$\frac{+11}{+5} = +2 \text{ and } +1;$$

$$\frac{-11}{+5} = -2 \text{ and } -1$$

The 1st case is easy

$+11 \div +5$: Quotient = +2, Remainder = +1

Checking the results:

$$11 = 2 \times 5 + (+1) = 10 + 1$$



Dividend = Quotient $\times$ Divisor + Remainder

$$\pm 11 \div \pm 5 \quad \frac{+11}{+5} = +2 \text{ and } +1; \quad \frac{-11}{+5} = -2 \text{ and } -1$$

2nd case If we change the sign of the dividend, the quotient must change as well:

$$-11 \div +5: \text{Quotient} = -2$$

Rewriting our basic formula to calculate the remainder:

$$\begin{aligned} \text{Remainder} &= (\text{Dividend} - \text{Quotient} \times \text{Divisor}) = -11 - (-2 \times +5) \\ &= -11 - (-10) = -1 \end{aligned}$$

So,

$$-11 \div +5: \text{Quotient} = -2, \text{Remainder} = -1$$



$$\text{Dividend} = \text{Quotient} \times \text{Divisor} + \text{Remainder}$$

$$\pm 11 \div \pm 5 \qquad \frac{+11}{+5} = +2 \text{ and } +1; \qquad \frac{-11}{+5} = -2 \text{ and } -1$$

Checking the results again:

$$-11 = -2 \cdot 5 + (-1) = -10 - 1$$

The same result is obtained if

$$-11 = -3 \cdot 5 + (4) = -15 + 4$$

absolute value of the quotient would then change

How to avoid this?



Signed Division

$$\text{Dividend} = \text{Quotient} \times \text{Divisor} + \text{Remainder}$$

$$\pm 11 \div \pm 5$$

$$\frac{+11}{+5} = +2 \text{ and } +1;$$

$$\frac{-11}{+5} = -2 \text{ and } -1$$

This anomalous behavior is avoided by following the rule that the dividend and remainder must have the same signs, no matter what the signs of the divisor and quotient.

$$\begin{aligned} &\frac{+11}{-5} = -2 \text{ and } +1 \\ ; &\frac{-11}{-5} = +2 \text{ and } -1 \end{aligned}$$

$$+11 \div -5: \text{Quotient} = -2, \text{Remainder} = +1$$

$$-11 \div -5: \text{Quotient} = +2, \text{Remainder} = -1$$

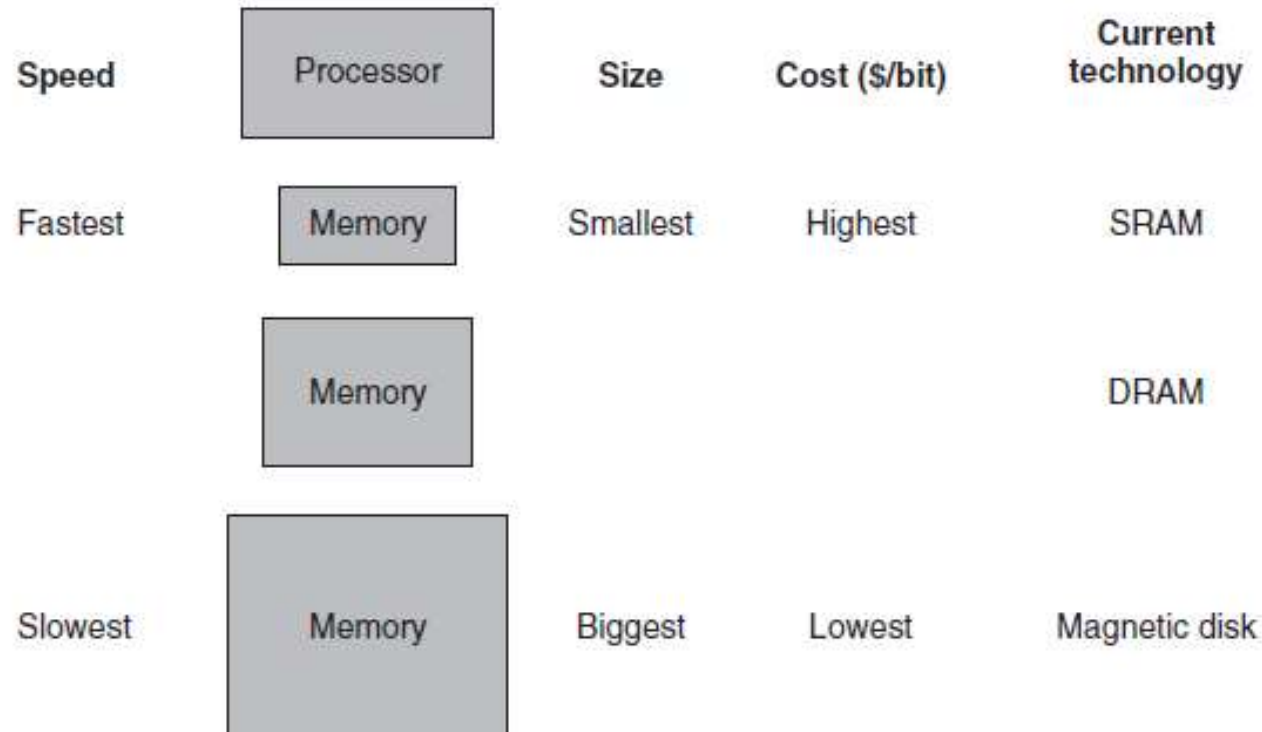
Thus the correctly signed division algorithm negates the quotient if the signs of the operands are opposite and makes the sign of the nonzero remainder match the dividend



BITS Pilani
Pilani Campus

RKTiwary
COA -IS ZC353

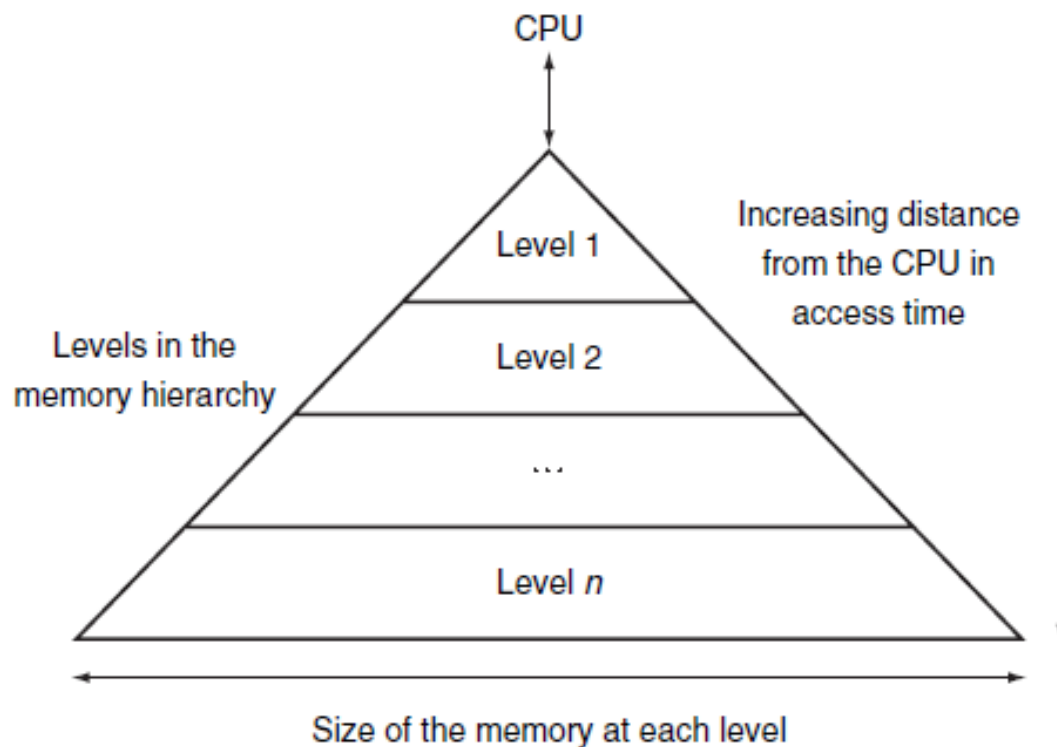
The basic structure of a memory hierarchy



The Basics of Caches



Example: Library



Important Terms



- ❖ **Hit time &** hit rate
- ❖ **Miss Penalty &** miss rate
- ❖ block (or line)
- ❖ temporal locality
- ❖ spatial locality



cache miss: A request for data from the cache that cannot be fulfilled because the data is not present in the cache

CPU time =

$(\text{CPU execution clock cycles} + \text{Memory-stall clock cycles}) \times \text{Clock cycle time}$

Assume the miss rate of an instruction cache is 3% and the miss rate of the data cache is 5%. If a processor has a CPI of 3 without any memory stalls and the miss penalty is 120 cycles for all misses,

Determine

how much faster a processor would run with a perfect cache that never missed. Assume the frequency of all loads and stores is 40%.



Assume the miss rate of an instruction cache is 3% and the miss rate of the data cache is 5%. If a processor has a CPI of 3 without any memory stalls and the miss penalty is 120 cycles for all misses,

Determine

how much faster a processor would run with a perfect cache that never missed. Assume the frequency of all loads and stores is 40%.

$$\text{Total Miss penalty} = I_c \times (0.03 \times 0.6 \times 120 + 0.05 \times 0.4 \times 120) \times 100 = (2.16 + 2.4)I_c = 4.56I_c$$

$$\text{For Perfect cache execution time} = 3 \times I_c = 3I_c$$

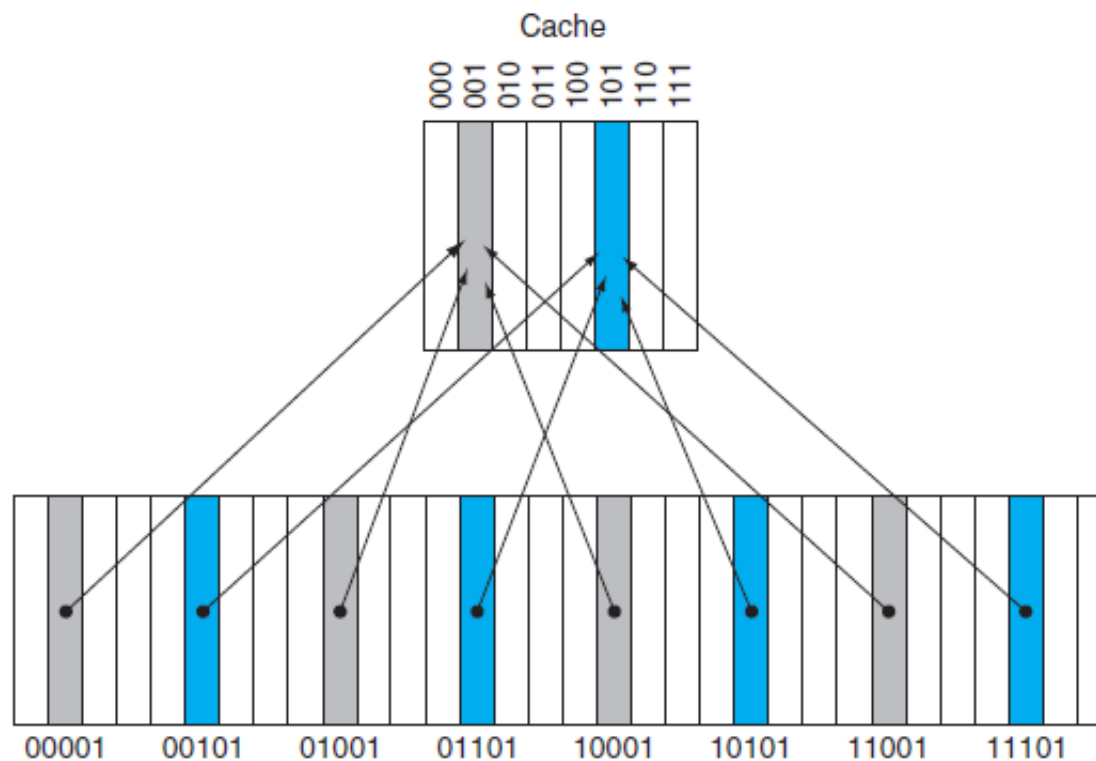
$$\text{Total with imperfect} = 3 \times 0.92I_c + 4.56I_c = 2.76I_c + 4.56I_c = 7.32I_c$$

$$\text{Speed up} = 7.32/3 = 2.44$$

Direct Mapped cache



Cache always smaller
(Block address) modulo (Number of blocks in the cache)



Example:



Decimal address of reference	Binary address of reference	Hit or miss in cache	Assigned cache block (where found or placed)
22	10110_{two}	miss (000)	$(10\mathbf{110}_{\text{two}} \bmod 8) = \mathbf{110}_{\text{two}}$
26	11010_{two}	miss (000)	$(11\mathbf{010}_{\text{two}} \bmod 8) = \mathbf{010}_{\text{two}}$
22	10110_{two}	hit	$(10\mathbf{110}_{\text{two}} \bmod 8) = \mathbf{110}_{\text{two}}$
26	11010_{two}	hit	$(11\mathbf{010}_{\text{two}} \bmod 8) = \mathbf{010}_{\text{two}}$
16	10000_{two}	miss (000)	$(10\mathbf{000}_{\text{two}} \bmod 8) = \mathbf{000}_{\text{two}}$
3	00011_{two}	miss (000)	$(00\mathbf{011}_{\text{two}} \bmod 8) = \mathbf{011}_{\text{two}}$
16	10000_{two}	hit	$(10\mathbf{000}_{\text{two}} \bmod 8) = \mathbf{000}_{\text{two}}$
18	10010_{two}	miss (000)	$(10\mathbf{010}_{\text{two}} \bmod 8) = \mathbf{010}_{\text{two}}$
16	10000_{two}	hit	$(10\mathbf{000}_{\text{two}} \bmod 8) = \mathbf{000}_{\text{two}}$

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Decimal address of reference	Binary address of reference	Hit or miss in cache	Assigned cache block (where found or placed)
22	10110_{two}	miss (22)	$(10110_{\text{two}} \bmod 8) = 110_{\text{two}}$
26	11010_{two}	miss (26)	$(11010_{\text{two}} \bmod 8) = 010_{\text{two}}$
22	10110_{two}	hit	$(10110_{\text{two}} \bmod 8) = 110_{\text{two}}$
26	11010_{two}	hit	$(11010_{\text{two}} \bmod 8) = 010_{\text{two}}$
16	10000_{two}	miss (16)	$(10000_{\text{two}} \bmod 8) = 000_{\text{two}}$
3	00011_{two}	miss (3)	$(00011_{\text{two}} \bmod 8) = 011_{\text{two}}$
16	10000_{two}	hit	$(10000_{\text{two}} \bmod 8) = 000_{\text{two}}$
18	10010_{two}	miss (18)	$(10010_{\text{two}} \bmod 8) = 010_{\text{two}}$
16	10000_{two}	hit	$(10000_{\text{two}} \bmod 8) = 000_{\text{two}}$

Index	V	Tag	Data
000	N		
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		



Decimal address of reference	Binary address of reference	Hit or miss in cache	Assigned cache block (where found or placed)
22	10110_{two}	miss (000)	$(10\textcolor{teal}{110}_{\text{two}} \bmod 8) = \textcolor{teal}{110}_{\text{two}}$
26	11010_{two}	miss (000)	$(11\textcolor{teal}{010}_{\text{two}} \bmod 8) = \textcolor{teal}{010}_{\text{two}}$
22	10110_{two}	hit	$(10\textcolor{teal}{110}_{\text{two}} \bmod 8) = \textcolor{teal}{110}_{\text{two}}$
26	11010_{two}	hit	$(11\textcolor{teal}{010}_{\text{two}} \bmod 8) = \textcolor{teal}{010}_{\text{two}}$
16	10000_{two}	miss (000)	$(10\textcolor{teal}{000}_{\text{two}} \bmod 8) = \textcolor{teal}{000}_{\text{two}}$
3	00011_{two}	miss (000)	$(00\textcolor{teal}{011}_{\text{two}} \bmod 8) = \textcolor{teal}{011}_{\text{two}}$
16	10000_{two}	hit	$(10\textcolor{teal}{000}_{\text{two}} \bmod 8) = \textcolor{teal}{000}_{\text{two}}$
18	10010_{two}	miss (000)	$(10\textcolor{teal}{010}_{\text{two}} \bmod 8) = \textcolor{teal}{010}_{\text{two}}$
16	10000_{two}	hit	$(10\textcolor{teal}{000}_{\text{two}} \bmod 8) = \textcolor{teal}{000}_{\text{two}}$

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
e 011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
f 010	Y	10_{two}	Memory (10010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

How many total bits are required for a directed mapped cache with four byte blocks, if the cache contains 64 Kbytes of data and the address length of the processor is 32 bits? Assume that byte addressing is used.



How many total bits are required for a directed mapped cache with four byte blocks, if the cache contains 64 Kbytes of data and the address length of the processor is 32 bits? Assume that byte addressing is used.

With four byte blocks, the 64 Kbyte cache contains a total of $64K/4 = 16K$ blocks. The cache has the following parameters

- (a) Byte select size: 2 bits (since $2^2 = 4$ bytes/block)
- (b) Cache index size: 14 bits (since $2^{14} = 16K$ blocks)
- (c) Cache tag size: 16 bits (remaining bits from 32)
- (d) Block size: 32 bits (4 bytes)
- (e) Number of blocks: 16K blocks (cache size/block size)

Assuming that the cache has a valid bit, a cache tag, and data for each block, the size of the cache is:

$$\begin{aligned}\text{cache bits} &= \text{number of blocks} \times (\text{block size} + \text{tag size} + 1) \\ &= 2^{14} \times (32 + 16 + 1) \\ &= 802,816 \text{ bits}\end{aligned}$$

How many bits are required if the block size is 32 bytes?

How many bits are required if the block size is 32 bytes?



With 32 byte blocks, the 64 Kbyte cache contains a total of $64K/32 = 2K$ blocks.

The cache has the following parameters

(a) Byte select size: 5 bits (since $2^5 = 32$ bytes/block)

(b) Cache index size: 11 bits (since $2^{11} = 2K$ blocks)

(c) Cache tag size: 16 bits (remaining bits from 32)

(d) Block size: 256 bits (32 bytes)

(e) Number of blocks: 2K blocks (cache size/block size)

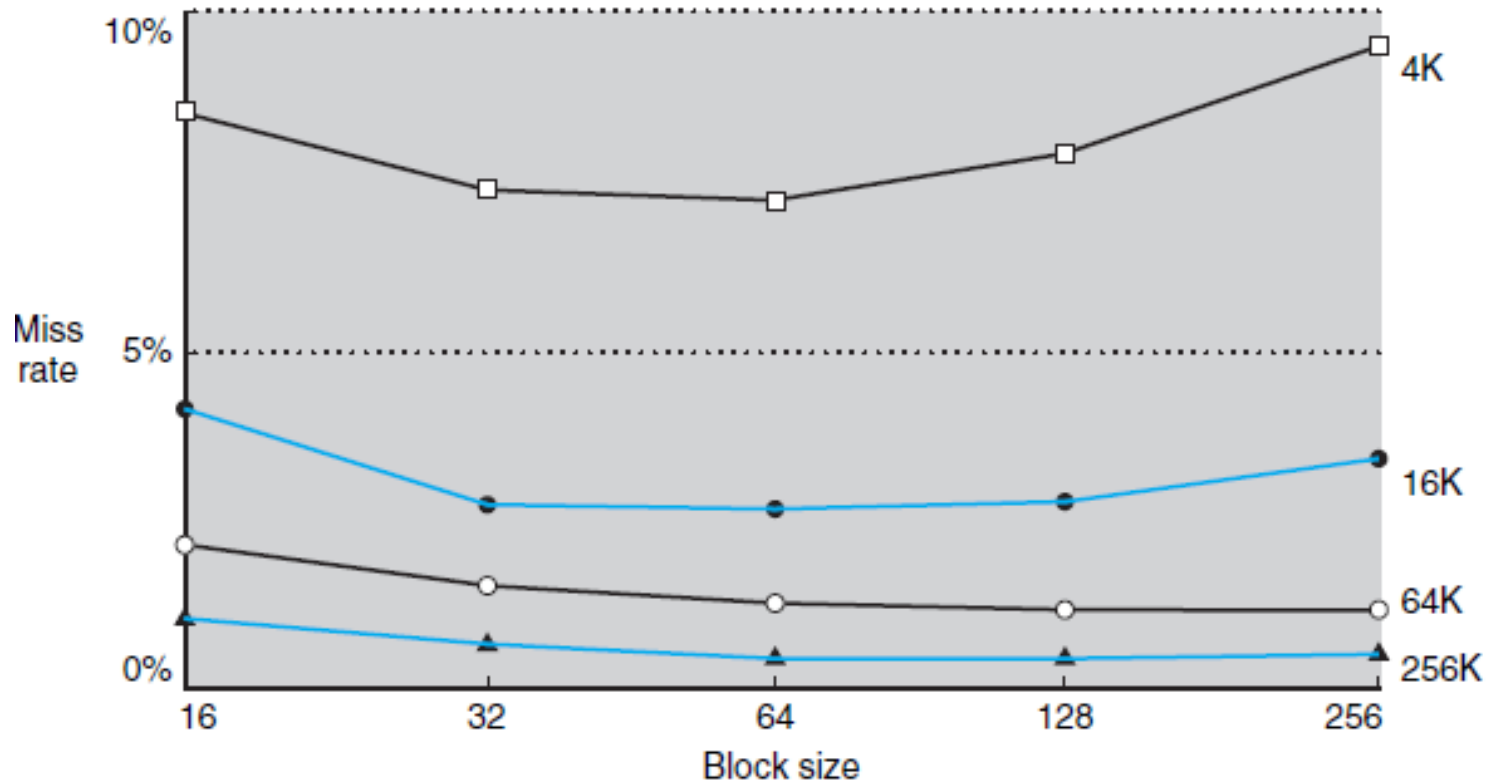
cache bits = number of blocks $\times$ (block size + tag size + 1)

$= 2^{11} \times (256 + 16 + 1)$

$= 559,104$ bits

Note: Increasing the block size tends to decrease the number of bits needed to implement the cache

Miss rate versus block size



Mapping an Address to a Multiword Cache Block



Consider a cache with 32 blocks and a block size of 8 bytes. To what block number does byte address 600 map?

Explore?

Address of the block is $\frac{\text{Byte Address}}{\text{Bytes per block}}$

This block contains all addresses of 8 bytes

$$\left\lfloor \frac{\text{Byte Address}}{\text{Byte per block}} \right\rfloor \times \text{Bytes per block}$$

$$\left\lfloor \frac{600}{8} \right\rfloor = 75 \text{ is the block address}$$

This maps to $(75 \text{ modulo } 32) = 11$

In fact this contains all address between 600 to 607

Analyse Directly Mapped cache organization using different cache size and block size : Explain what cache hit and miss rates are



Write a new program in the CPU simulator and name it as call it
Cache understanding1

```
MOV #0, R02  
STB R02, @R02  
CMP #63, R02  
JEQ 28  
INR R02  
JMP 6  
HLT
```

Question: Can you think of another instruction in place of INR R02?

It can be ADD #01, R02.

JEQ 28 instruction?

Configure the cache

Block Size = 4

Cache Type = Direct Mapped

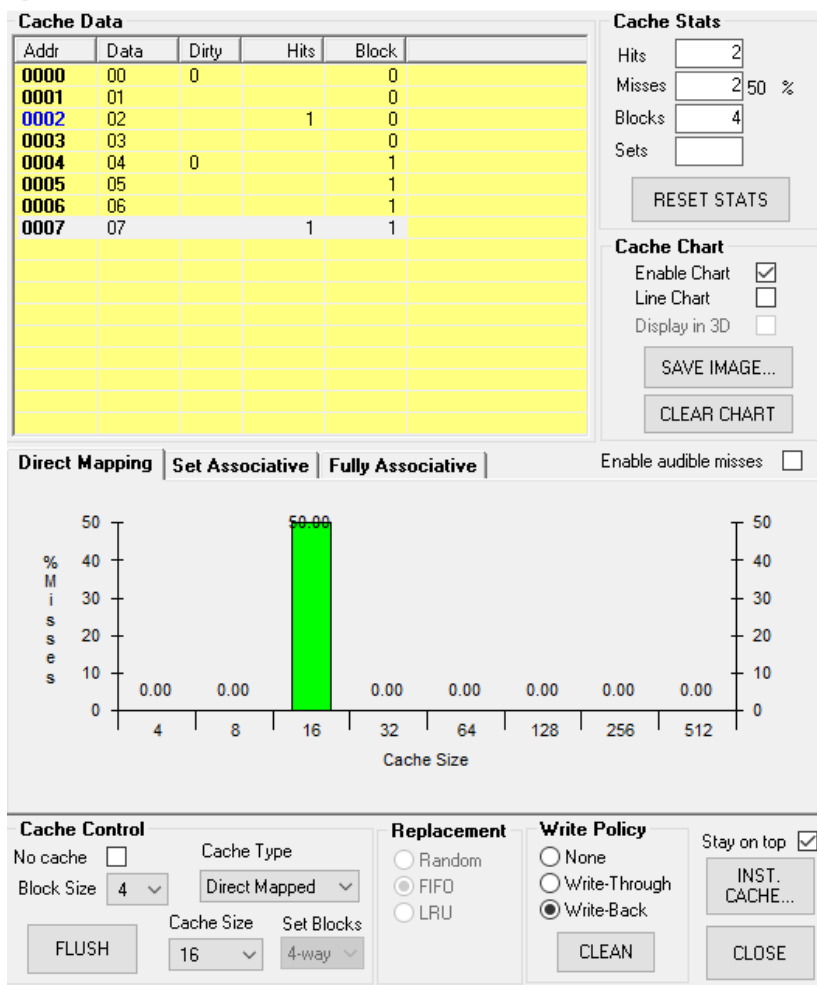
Cache Size = 16

Write Policy = Write-Back

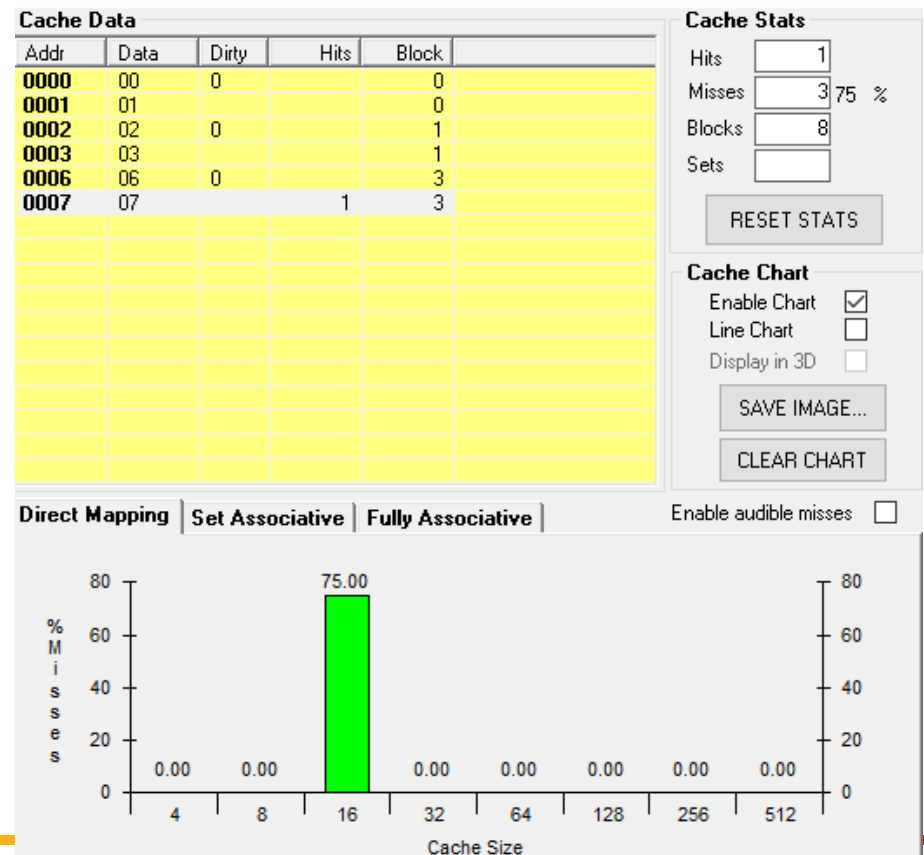
0024	0024	JMP 6	0000	2
0028	0028	LDB 1, R03	0000	0
0034	0034	LDB 2, R04	0000	0
0040	0040	LDB 6, R05	0000	0
0046	0046	LDB 7, R05	0000	0
0052	0052	HLT	0000	2

lead

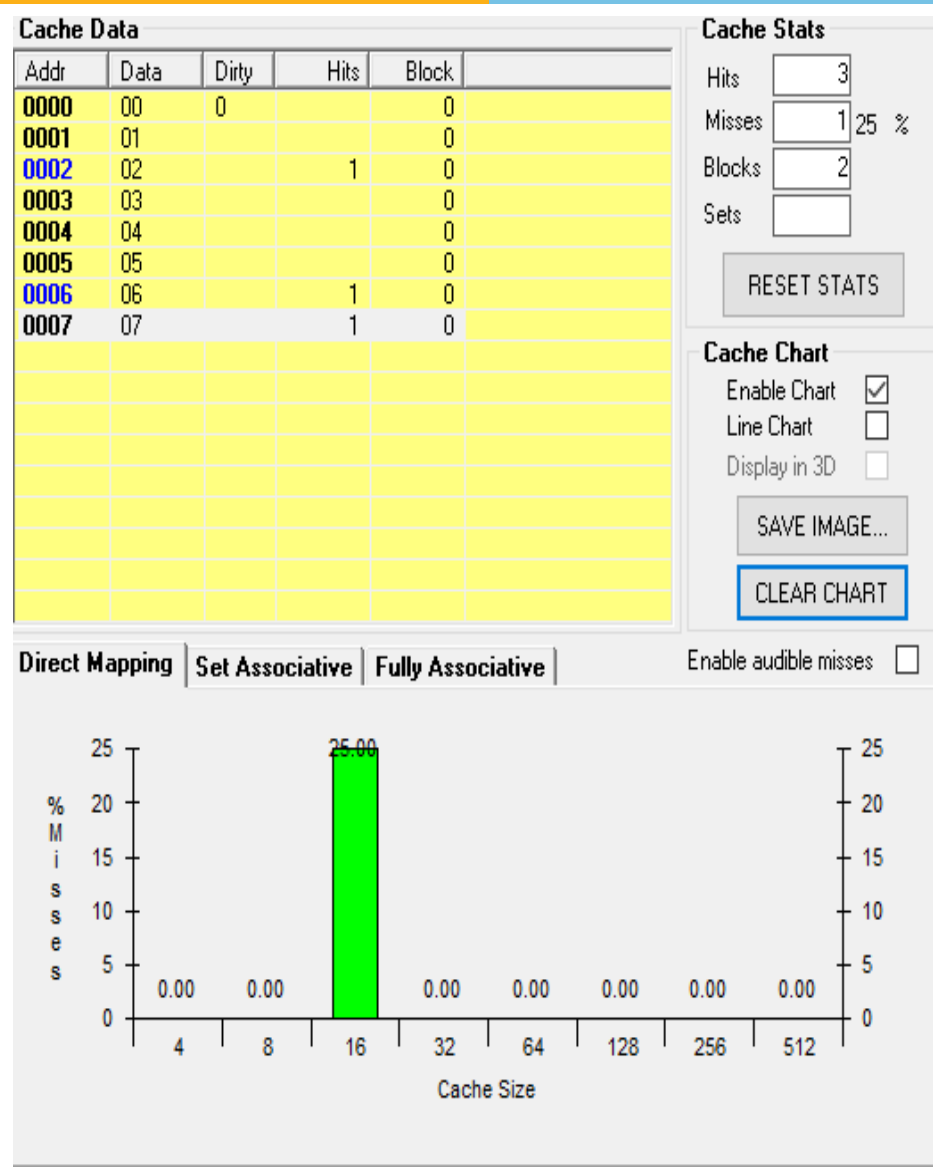
Data Cache: CPU 0



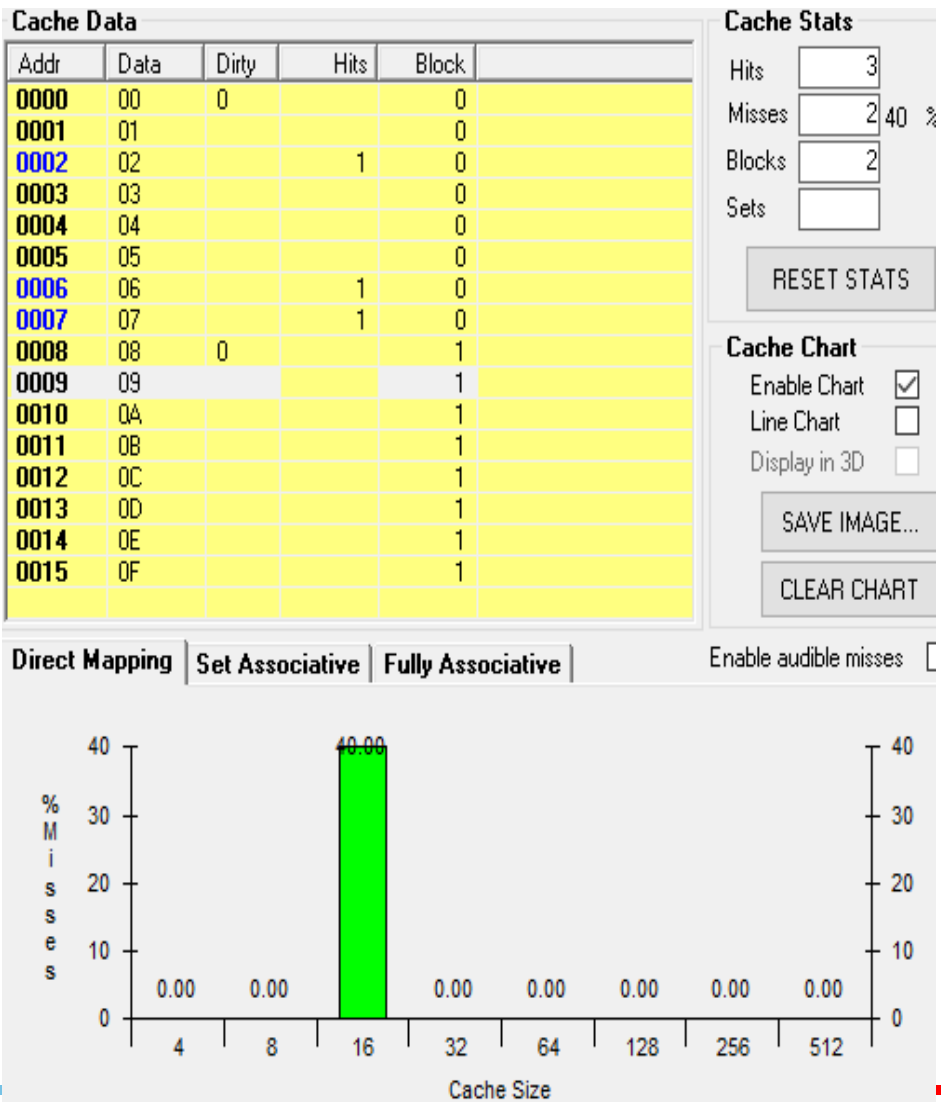
Change Block size =2



Change Block size =8



Insert Instruction LDB 9, R08





BITS Pilani
Pilani Campus



RKTiwary
COA -IS ZC353

Reducing Cache Misses by More Flexible Placement of Blocks



Fully associative cache:

A cache structure in which a block can be placed in any location in the cache.

Set-associative cache:

A cache that has a fixed number of locations (at least two) where each block can be placed.

A set-associative cache with n locations for a block is called an n -way set-associative cache

The location of a memory block whose address is 12



Direct mapped



Direct-mapped cache
(Block number) modulo (Number of *blocks* in the cache)

In direct-mapped placement, there is only one cache block where memory block 12 can be found, and that block is given by $(12 \text{ modulo } 8) = 4$.

The location of a memory block whose address is 12



Set associative



Set-associative cache, set number
(Block number) modulo (Number of sets in the cache)

In a two-way set-associative cache, there would be four sets, and memory block 12 must be in set $(12 \bmod 4) = 0$;

the memory block could be in either element of the set

The location of a memory block whose address is 12



Fully associative



Set-associative cache, set number
(Block number) modulo (Number of sets in the cache)

In a fully associative placement,
the memory block for block address 12 can
appear in any of the eight cache blocks.

One-way set associative (direct mapped)

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

A direct-mapped cache :a one-way set-associative cache:

A fully associative: cache with m entries is simply an m -way set-associative cache; it has one set with m blocks, and an entry can reside in any block within that set





Example

Assume there are three small caches, each consisting of four one-word blocks. One cache is fully associative, a second is two-way set-associative, and the third is direct-mapped. Find the number of misses for each cache organization given the following sequence of block addresses: 0, 8, 0, 6, and 8.

Block Adrs	Cache block	Memory Adr	Hit/Miss	Content of the cache			
				Block0	Block1	Block2	Block3
0	(0 modulo 4)=0	0	miss	0			
6	(6 modulo 4)=2	8	Miss	8			
8	(8 modulo 4)=0	0	Miss	0			
		6	Miss	0		6	
		8	miss	8			



Set Associative

Two way means there are two sets with indices 0 and 1

Block Adrs	Cache Set
0	$(0 \text{ modulo } 2)=0$
6	$(6 \text{ modulo } 2)=0$
8	$(8 \text{ modulo } 2)=0$

Set-associative caches usually replace the least recently used block within a set

Memory Adr	Hit/Miss	Content of the cache			
		set0	set0	Set1	Set1
0	miss	0			
8	Miss	0	8		
0	Hit	0	8		
6	Miss	0	6		
8	miss	8			

Fully Associative



Memory Adr	Hit/Miss	Content of the cache			
		Block0	Block1	Block2	Block3
0	miss	0			
8	Miss	0	8		
0	Hit	0	8		
6	Miss	0	8	6	
8	Hit	0	8	6	



What is when the block size is 8

For two way set Associative we have 2 Sets (0,1)

Memory Adr	Hit/ Miss	Contents of cache blocks after reference							
		Set0	Set0	Set0	Set0	Set1	Set1	Set1	Set1
0	M	0							
8	M	0	8						
0	H	0	8						
6	M	0	8	6					
8	H	0	8	6					

there would be no replacements in the two-way set-associative cache;

and it would have the same number of misses as the fully associative cache.
Check for yourself.: if we had 16 blocks, all 3 caches would have the same number of misses

How much of a reduction in the miss rate is achieved by associativity?

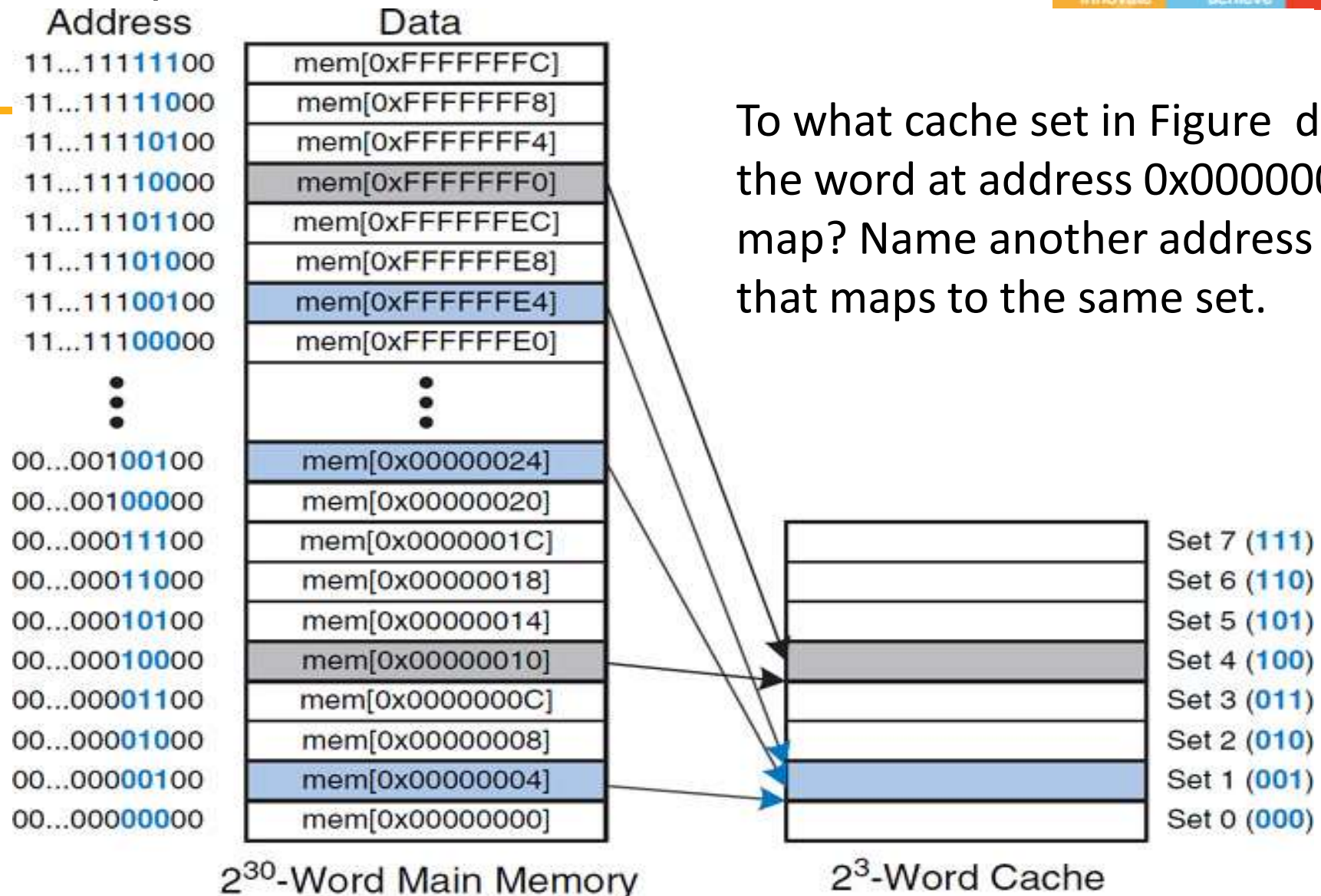


for a 64 KB data cache with a 16-word block

Associativity	Data miss rate
1	10.3%
2	8.6%
4	8.3%
8	8.1%

Going from one-way(direct) to two-way associativity decreases the miss rate by about 15%, but there is little further improvement in going to higher associativity.

To explore



Solution



0x00000014 H

Representing only lower byte : 0001 0100

The two least significant bits of the address are 00, because the address is word aligned.

So the alignment is 101 00

The next three bits are 101, so the word maps to set 5.

The other address are 0001 0100 to xxx1 0100

Or for that matter in Hex xxxxxx14 and in increment of 20H

Words at addresses 0x34, 0x54, 0x74, ..., 0xFFFFFFFF4 all map to this same set.



Locating a Block in the Cache



- ☐ Index value is used to select the set
- ☐ tags of all the blocks in the set must be searched
- ☐ associativity increases
 - ✓ Block per set increases
 - ✓ Associativity increases by 2
 - Number of blocks increase by 2
 - Index bit reduces by 1 bit
 - increases the size of the tag by 1 bit



EXAMPLE:

Suppose we have a processor with a base CPI of 1.0, assuming all references hit in the primary cache, and a clock rate of 4 GHz.

Assume a main memory access time of 100 ns, including all the miss handling.

Suppose the miss rate per instruction at the primary cache is 2%.

How much faster will the processor be if we add a secondary cache that has a 5 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 0.5%?



Solution

The miss penalty to main memory is $= \frac{100ns}{0.25ns} = 400 \text{ cycles}$

The effective CPI with one level of caching is given by

Total CPI = Base CPI + Memory-stall cycles per instruction

For the processor with one level of caching,

$$\begin{aligned} \text{Total CPI} &= 1.0 + \text{Memory-stall cycles per instruction} \\ &= 1.0 + 2\% \times 400 = 9 \end{aligned}$$

With two levels of caching, a miss in the primary (or first-level) cache can be satisfied either by the secondary cache or by main memory. The miss penalty for an access to the second-level cache is $= \frac{5ns}{0.25ns} = 20 \text{ cycles}$

Thus, for a two-level cache, total CPI is the sum of the stall cycles from both levels of cache and the base CPI:

Total CPI = 1 + Primary stalls per instruction + Secondary stalls per instruction

$$= 1 + 2\% \times 20 + 0.5\% \times 400$$

$$= 1 + 0.4 + 2.0 = 3.4$$

Thus, the processor with the secondary cache is faster by

$$9.0/3.4 = 2.6$$



Comparison of different cache mapping, Directly Mapped cache, set associative and fully associative

First flush the contents of the cache by clicking on the FLUSH button. Then enter the following instructions after the last LDB instruction in the above program:

LDB 0, R00

LDB 4, R00

LDB 17, R00

LDB 20, R00

Now select the option

Block Size = 4

Cache Type = Set Associative

Cache Size = 16

Set Blocks = 2-way

Write Policy = Write-Back



Example

Increasing associativity requires more comparators and more tag bits per cache block.

Assuming a cache of 4K blocks, a 16-byte block size, and a 32-bit address, find the total number of sets and the total number of tag bits for caches that are

- direct mapped,
- two-way and four-way set associative,
- and fully associative.

Assuming a cache of 4K blocks, a 16-Byte block size, and a 32-bit address



Sol:

For Direct

$(28 - 12) \times 4K = 16 \times 4K = 64 \text{ K tag bits}$

for a two-way set-associative cache

2K sets

total number of tag bits is $(28 - 11) \times 2 \times 2K = 34 \times 2K = 68 \text{ kBits}$

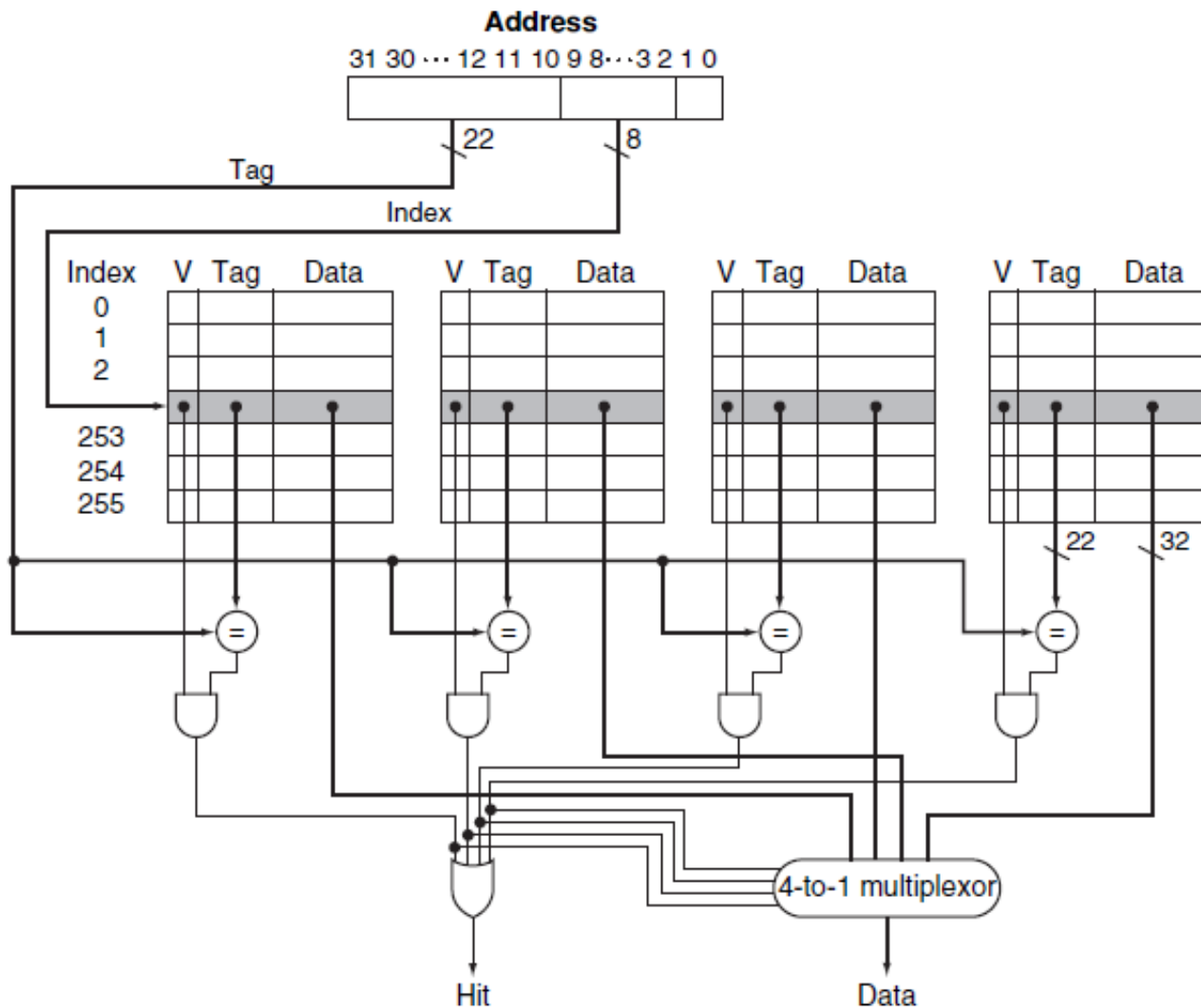
four-way set-associative cache

total number is $(28 - 10) \times 4 \times 1K = 72 \times 1K = 72 \text{ K tag bits}$.

fully associative cache,

there is only one set with 4K blocks, and the tag is 28 bits, leading to $28 \times 4K \times 1 = 112K \text{ tag bits}$

four-way set-associative cache





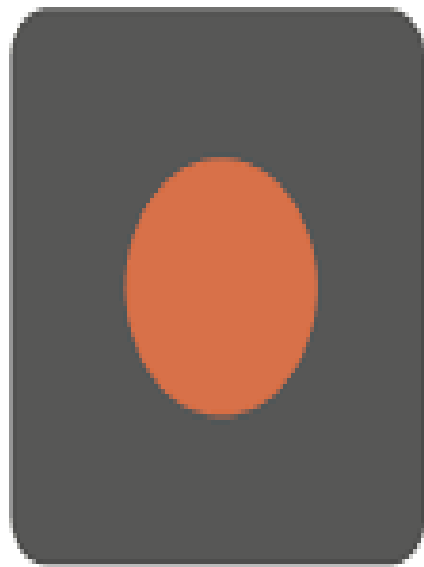
BITS Pilani
Pilani Campus



RKTiwary

COA - L5

IMP Note to Self



Start
Recording

IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**



Real Numbers

Numbers with fractions

Could be done in pure binary

- $1001.1010 = 2^4 + 2^0 + 2^{-1} + 2^{-3} = 9.625$

Where is the binary point?

Fixed?

- Very limited

Moving?

- How do you show where it is?

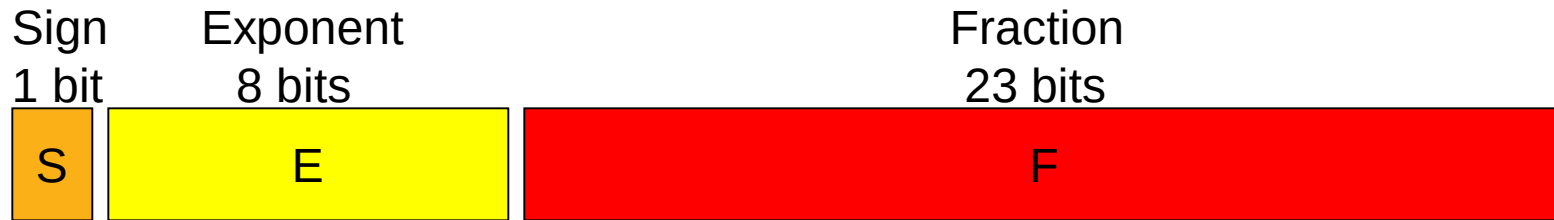


Floating Point

- Normalized scientific notation: single non-zero digit to the left of the decimal (binary) point – example: 3.5×10^9
- $1.010001 \times 2^{-5}_{\text{two}} = (1 + 0 \times 2^{-1} + 1 \times 2^{-2} + \dots + 1 \times 2^{-6}) \times 2^{-5}_{\text{ten}}$
- A standard notation enables easy exchange of data between machines and simplifies hardware algorithms – the IEEE 754 standard defines how floating point numbers are represented



Sign and Magnitude Representation



- More exponent bits → wider range of numbers (not necessarily more numbers – recall there are infinite real numbers)
- More fraction bits → higher precision
- Register value = $(-1)^S \times F \times 2^E$
- Since we are only representing normalized numbers, we are guaranteed that the number is of the form 1.xxxx..
Hence, in IEEE 754 standard, the 1 is implicit
Register value = $(-1)^S \times (1 + F) \times 2^E$



Sign and Magnitude Representation



- Largest number that can be represented:
- Smallest number that can be represented:

0.000000001_{ten} or $1.0_{\text{ten}} \times 10^{-9}$ (seconds in a nanosecond)

$3,155,760,000_{\text{ten}}$ or $3.15576_{\text{ten}} \times 10^9$ (seconds in a typical century)



Biased Exponent Representation

- ❖ How to represent a signed exponent? Choices are ...
 - ✧ Sign + magnitude representation for the exponent
 - ✧ Two's complement representation
 - ✧ Biased representation
- ❖ IEEE 754 uses **biased representation** for the **exponent**
 - ✧ Value of exponent
$$= \text{val}(E) = E - \text{Bias} \text{ (Bias is a constant)}$$
- ❖ Recall that exponent field is **8 bits** for **single precision**
 - ✧ E can be in the range **0 to 255**
 - ✧ $E = 0$ and $E = 255$ are **reserved for special use** (discussed later)
- ✧ $E = 1$ to **254** are used for **normalized** floating point numbers
- ✧ **Bias = 127** (half of **254**), $\text{val}(E) = E - 127$
- ✧ $\text{val}(E=1) = -126$, $\text{val}(E=127) = 0$, $\text{val}(E=254) = 127$

Biased Exponent – Cont'd

❖ Value of a Normalized Floating Point Number is

$$(-1)^S \times (1.F)_2 \times 2^{E - \text{Bias}}$$

$$(-1)^S \times (1.f_1f_2f_3f_4 \dots)_2 \times 2^{E - \text{Bias}}$$

$$(-1)^S \times (1 + f_1 \times 2^{-1} + f_2 \times 2^{-2} + f_3 \times 2^{-3} + f_4 \times 2^{-4} \dots)_2 \times 2^{E - \text{Bias}}$$

Examples of Single Precision Float

❖ What is the decimal value of this **Single Precision** float?

❖ 1 0 1 1 1 1 0 0 0 1 0

❖ **Solution:**

✧ Sign = 1 is negative

✧ Exponent = $(01111100)_2 = 124$, $E - \text{bias} = 124 - 127 = -3$

✧ Significand = $(1.0100 \dots 0)_2 = 1 + 2^{-2} = 1.25$ (**1. is implicit**)

✧ Value in decimal = $-1.25 \times 2^{-3} = -0.15625$

Or Significand = $(1.0100 \dots 0)_2 \times 2^{-3} = 0.0010100 = -0.15625$

❖ What is the decimal value of?

❖ 0 1 0 0 0 0 0 1 0 0 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

❖ **Solution:**

Implicit

✧ Value in decimal = $+(1.01001100 \dots 0)_2 \times 2^{130-127}$

$= (1.01001100 \dots 0)_2 \times 2^3 = (1010.01100 \dots 0)_2 = 10.375$

Converting FP Decimal to Binary

❖ Convert -0.8125 to binary in single precision

❖ **Solution:**

✧ Fraction bits can be obtained using multiplication by 2

▪ $0.8125 \times 2 = 1.625$

▪ $0.625 \times 2 = 1.25$

▪ $0.25 \times 2 = 0.5$

▪ $0.5 \times 2 = 1.0$

▪ Stop when fractional part is 0

$$0.8125 = (0.1101)_2 = \frac{1}{2} + \frac{1}{4} + \frac{1}{16} = \frac{13}{16}$$

✧ Fraction $= (0.1101)_2 = (1.101)_2 \times 2^{-1}$ (Normalized)

✧ Exponent $= -1 + \text{Bias} = 126$ (single precision) = 7E H

1 0 1 1 1 1 1 0 1 0 1 0

Largest Normalized Float

❖ What is the **Largest normalized** float?

❖ **Solution for Single Precision:**

❖ 0 1 1 1 1 1 1 0 1

✧ Exponent – bias = $254 - 127 = 127$ (**largest exponent for SP**)

✧ Significand = $(1.111 \dots 1)_2 = \text{almost } 2$

✧ Value in decimal $\approx 2 \times 2^{127} \approx 2^{128} \approx 3.4028 \dots \times 10^{38}$

❖ **Overflow:** exponent is **too large** to fit in the exponent field



Smallest Normalized Float

❖ What is the **smallest (in absolute value) normalized float**?

❖ **Solution for Single Precision:**

0 0 0 0 0 0 0 0 1 0

✧ Exponent – bias = $1 - 127 = -126$ (**smallest exponent for SP**)

✧ Significand = $(1.000 \dots 0)_2 = 1$

✧ Value in decimal = $1 \times 2^{-126} = 1.17549 \dots \times 10^{-38}$

❖ **Underflow:** exponent is **too small** to fit in exponent field



Zero, Infinity, and NaN

❖ Zero

- ✧ Exponent field $E = 0$ and fraction $F = 0$
- ✧ $+0$ and -0 are possible according to sign bit S

❖ Infinity

- ✧ Infinity is a special value represented with **maximum E** and **$F = 0$**
- For **single precision** with 8-bit exponent: **maximum $E = 255$**
- ✧ Infinity can result from overflow or division by zero
- ✧ $+\infty$ and $-\infty$ are possible according to sign bit S

❖ NaN (Not a Number)

- ✧ NaN is a special value represented with **maximum E** and **$F \neq 0$**
- ✧ Result of exceptional situations, such as $0/0$ or $\text{sqrt}(\text{negative})$
- ✧ Operation on a NaN results is NaN: $\text{Op}(X, \text{NaN}) = \text{NaN}$

Summary of IEEE 754 Encoding

Single-Precision	Exponent = 8	Fraction = 23	Value
Normalized Number	1 to 254	Anything	$\pm (1.F)_2 \times 2^{E-127}$
Denormalized Number	0	nonzero	$\pm (0.F)_2 \times 2^{-126}$
Zero	0	0	± 0
Infinity	255	0	$\pm \infty$
NaN	255	nonzero	NaN

Double-Precision	Exponent = 11	Fraction = 52	Value
Normalized Number	1 to 2046	Anything	$\pm (1.F)_2 \times 2^{E-1023}$
Denormalized Number	0	nonzero	$\pm (0.F)_2 \times 2^{-1022}$
Zero	0	0	± 0
Infinity	2047	0	$\pm \infty$
NaN	2047	nonzero	NaN

Floating Point Addition Example

Consider Adding (Single-Precision Floating-Point):

$$+ 1.111001000000000000000010_2 \times 2^4$$

$$+ 1.1000000000000000110000101_2 \times 2^2$$

❖ Cannot add significands ... Why?

✧ Because **exponents are not equal**

❖ How to make exponents equal?

✧ **Shift the significand of the lesser exponent right**

✧ Difference between the two exponents = $4 - 2 = 2$

✧ So, **shift right** second number by **2** bits and increment exponent

$$1.1000000000000000110000101_2 \times 2^2 =$$

$$0.0110000000000000001100001 \mathbf{01}_2 \times \mathbf{2^4}$$

Floating-Point Addition – cont'd

❖ Now, **ADD the Significands:**

$$+ 1.111001000000000000000010 \times 2^4$$

$$+ 1.1000000000000000110000101 \times 2^2$$

$$+ 1.111001000000000000000010 \times 2^4$$

$$+ 0.011000000000000000110000101 \times 2^4 \text{ (shift right)}$$

$$+ \mathbf{10.010001000000000000110001101} \times 2^4 \text{ (result)}$$

❖ Addition produces a **carry bit**, result is NOT normalized

❖ **Normalize Result (shift right and increment exponent):**

$$+ \mathbf{10.010001000000000000110001101} \times 2^4$$

$$= + \mathbf{1.001000100000000000110001101} \times 2^5$$

Rounding

- ❖ Single-precision requires only 23 fraction bits
- ❖ However, Normalized result can contain additional bits

1.001000100000000000110001 | **1 01** × **2⁵**

Round Bit: R = 1 *Sticky Bit: S = 1*

- ❖ Two extra bits are needed for rounding
- ✧ **Round bit:** appears just after the normalized result
- ✧ **Sticky bit:** appears after the round bit (**OR of all additional bits**)

- ❖ Since **RS = 11**, increment fraction to round to nearest

$$\mathbf{1.00100010000000000110001} \times 2^5$$

1.00100010000000000110010 $\times 2^5$ (Rounded)

✦ Subtraction is converted into addition

✧ If the sign bits of the operands are different

❖ Consider Adding:

$$+ 1.00000000101100010001101 \times 2^{-6}$$

$$- 1.000000000000000010011010 \times 2^{-1}$$

$$+ 0.00001000000001011000100 \ 01101 \times 2^{-1} \text{ (shift right 5 bits)}$$

$$- 1.000000000000000010011010 \times 2^{-1}$$

$$0 \ 0.00001000000001011000100 \ 01101 \times 2^{-1}$$

$$1 \ 0.111111111111111101100110 \times 2^{-1} \text{ (2's complement)}$$

$$1 \ 1.00001000000001000101010 \ 01101 \times 2^{-1} \text{ (ADD)}$$

$$- 0.1111011111111111011010101 \ 10011 \times 2^{-1} \text{ (2's complement)}$$

❖ 2's complement of result is required if result is negative



Subtraction FP contnd.

$$\begin{aligned} &+ 1.00000000101100010001101 \times 2^{-6} \\ &- 1.000000000000000010011010 \times 2^{-1} \\ &- 0.11110111111110111010101 \mathbf{10011} \times 2^{-1} \text{ (result is negative)} \end{aligned}$$

❖ Result should be normalized

✧ For subtraction, we can have **leading zeros**. To normalize, count the number of leading zeros, then shift result left and decrement the exponent accordingly.

Guard bit

$$\begin{aligned} &- \mathbf{0.11110111111110111010101} \mathbf{1} \mathbf{0011} \times 2^{-1} \\ &- \mathbf{1.11101111111110111010101} \mathbf{1} \mathbf{0011} \times 2^{-2} \text{ (Normalized)} \end{aligned}$$

Guard bit

❖ **Guard bit**: guards against loss of a fraction bit

✧ Needed for subtraction, when result has a leading zero and should be normalized.

Subtraction FP contnd.

- ❖ Next, normalized result should be rounded

Guard bit

$$- 0.111101111111110111010101 \mathbf{1} 0 011 \times 2^{-1}$$

$$- \mathbf{1.111011111111110111010101} \mathbf{1} \mathbf{0} \mathbf{011} \times 2^{-2} \text{ (Normalized)}$$

Round bit: R=0

Sticky bit: S = 1

- ❖ Since **R = 0**, it is more accurate to **truncate** the result even if **S = 1**. We simply discard the extra bits.

$$- 1.111011111111110111010101 \mathbf{0} \mathbf{011} \times 2^{-2} \text{ (Normalized)}$$

$$- 1.111011111111110111010101 \times 2^{-2} \text{ (Rounded to nearest)}$$

Exponent -2 is biased to $-2+127=125=7D$ H

- ❖ IEEE 754 Representation of Result

1 0 1 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 0 1 1



Rounding to Nearest Even

- ❖ Normalized result has the form: **1.** $f_1 f_2 \dots f_l$ **R S**
- ✧ The **round bit R** appears after the last fraction bit f_l
- ✧ The **sticky bit S** is the OR of all remaining additional bits
- ❖ **Round to Nearest Even**: default rounding mode
- ❖ Four cases for **RS**:
 - ✧ **RS = 00** → Result is Exact, no need for rounding
 - ✧ **RS = 01** → **Truncate** result by discarding **RS**
 - ✧ **RS = 11** → **Increment** result: ADD 1 to last fraction bit
 - ✧ **RS = 10** → Tie Case (either truncate or increment result)
- ✓ **Check Last fraction bit f_l** (**f_{23}** for single-precision or **f_{52}** for double)
- ✓ If f_l is **0** then **truncate** result to keep fraction even
- ✓ If f_l is **1** then **increment** result to make fraction even

Floating Point Multiplication Example



- ❖ Consider multiplying:
 $-1.110\ 1000\ 0100\ 0000\ 1010\ 0001_2 \times 2^{-4}$
 $\times 1.100\ 0000\ 0001\ 0000\ 0000\ 0000_2 \times 2^{-2}$
- ❖ Unlike addition, we **add the exponents** of the operands
- ✧ Result exponent value = $(-4) + (-2) = -6$
- ❖ Using the biased representation: $E_Z = E_X + E_Y - Bias$
- ✧ $E_X = (-4) + 127 = 123$ (**$Bias = 127$ for single precision**)
- ✧ $E_Y = (-2) + 127 = 125$
- ✧ $E_Z = 123 + 125 - 127 = 121$ (**value = -6**)
- ❖ Sign bit of product can be computed independently
- ❖ Sign bit of product = $Sign_X \mathbf{XOR} Sign_Y = \mathbf{1}$ (**negative**)

FP Multiplication contnd.

❖ Now multiply the significands:

(Multiplicand)

1.11010000100000010100001

(Multiplier)

× 1.10000000001000000000000

111010000100000010100001

111010000100000010100001

1.11010000100000010100001

10.1011100011111011111100110010100001000000000000

❖ 24 bits × 24 bits → 48 bits (double number of bits)

❖ Multiplicand × 0 = 0 Zero rows are eliminated

❖ Multiplicand × 1 = Multiplicand (shifted left)



FP Multiplication contnd.

❖ Normalize Product:

$$-10.10111000111110111111001100... \times 2^{-6}$$

Shift right and increment exponent because of **carry bit**

$$= -1.010111000111110111111001100... \times 2^{-5}$$

❖ Round to Nearest Even: (keep only 23 fraction bits)

$$1.010111000111110111111100 \mid 1\ 100... \times 2^{-5}$$

Round bit = **1**, Sticky bit = **1**, so increment fraction

$$\text{Final result} = -1.0101110001111101111110**1** \times 2^{-5}$$

❖ IEEE 754 Representation

1 0 1 1 1 1 0 1 0 0 1 0 1 1 1 0 0 0 1 1 1 1 1 0 1 1 1 1 1 1 0 1



Examples

Final representation: $(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$

- Represent -0.75_{ten} in single and double-precision formats

Single: $(1 + 8 + 23)$

Double: $(1 + 11 + 52)$

- What decimal number is represented by the following single-precision number?

1 1000 0001 01000...0000



By RKT
Instruction Architecture
Instruction sets & Addressing mode



Objectives

- ❑ Machine instructions and program execution, including branching and subroutine call and return operations.
- ❑ Addressing methods for accessing register and memory operands.
- ❑ Assembly language for representing machine instructions, data, and programs.
- ❑ Program-controlled Input/Output operations.

Memory Location, Addresses, and Operation

Memory consists of many millions of storage cells, each of which can store 1 bit.

Data is usually accessed in n -bit groups. n is called word length.

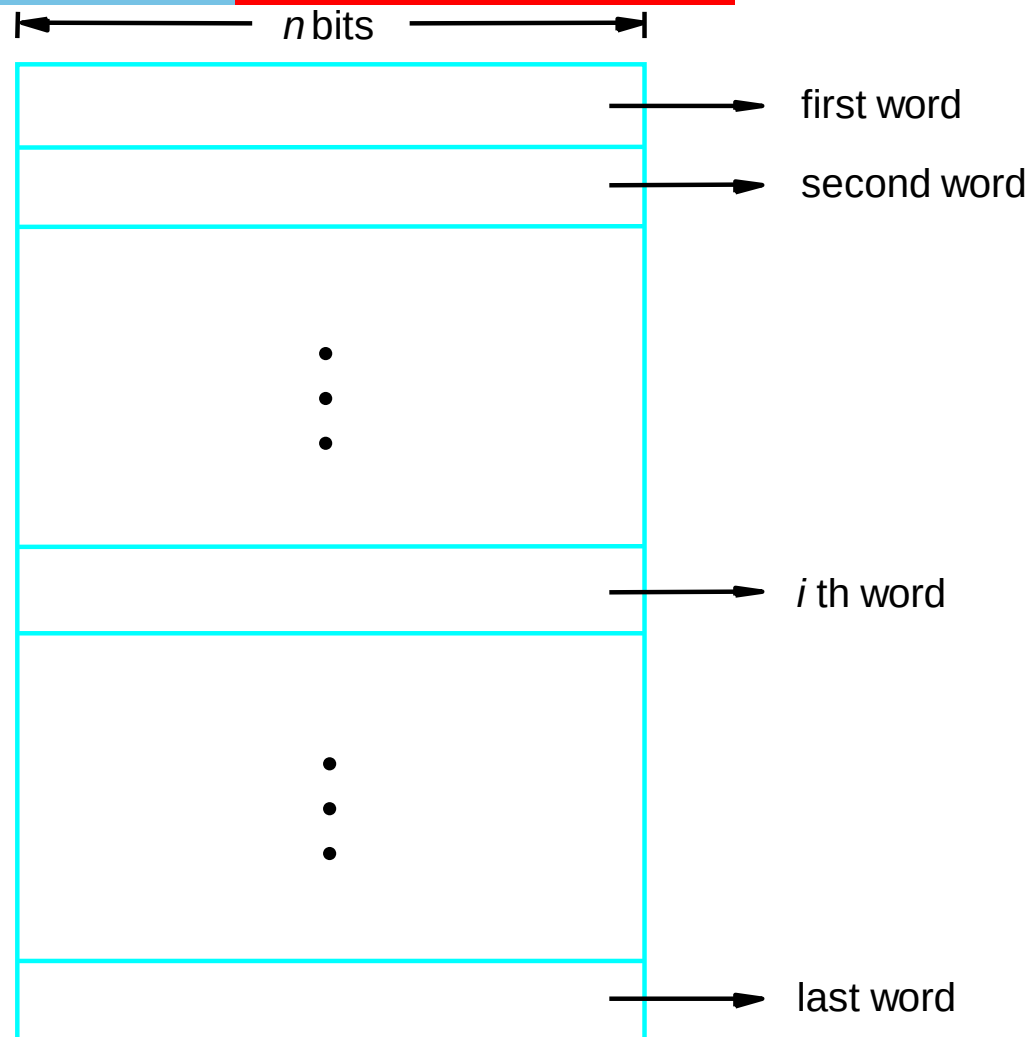
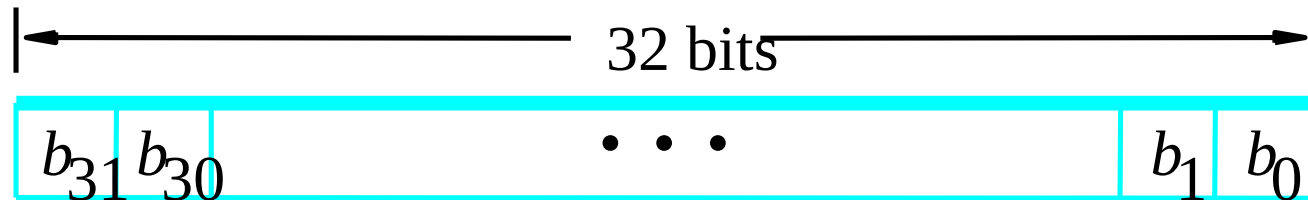


Figure 2.5. Memory words.

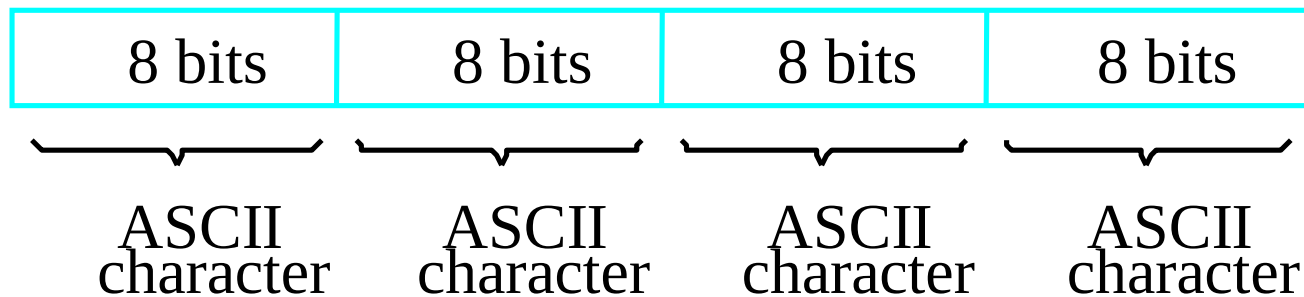
Memory Location, Addresses, and Operation

32-bit word length example



Sign bit: $b_{31} = 0$ for positive numbers
 $b_{31} = 1$ for negative numbers

(a) A signed integer



(b) Four characters



Memory Location, Addresses, and Operation

- ❑ To retrieve information from memory, either for one word or one byte (8-bit), addresses for each location are needed.
- ❑ A k -bit address memory has 2^k memory locations, namely 0 to 2^k-1 , called memory space.
- ❑ 24-bit memory: $2^{24} = 16,777,216 = 16\text{M}$ ($1\text{M}=2^{20}$)
- ❑ 32-bit memory: $2^{32} = 4\text{G}$ ($1\text{G}=2^{30}$)
- ❑ $1\text{K(kilo)}=2^{10}$
- ❑ $1\text{T(tera)}=2^{40}$



Memory Location, Addresses, and Operation

- ❑ It is impractical to assign distinct addresses to individual bit locations in the memory.
- ❑ The most practical assignment is to have successive addresses refer to successive byte locations in the memory – byte-addressable memory.
- ❑ Byte locations have addresses 0, 1, 2, ...
- ❑ If word length is 32 bits, they successive words are located at addresses 0, 4, 8,...



Big-Endian and Little-Endian Assignments

Big-Endian: lower byte addresses are used for the most significant bytes of the word

Little-Endian: opposite ordering. lower byte addresses are used for the less significant bytes of the word

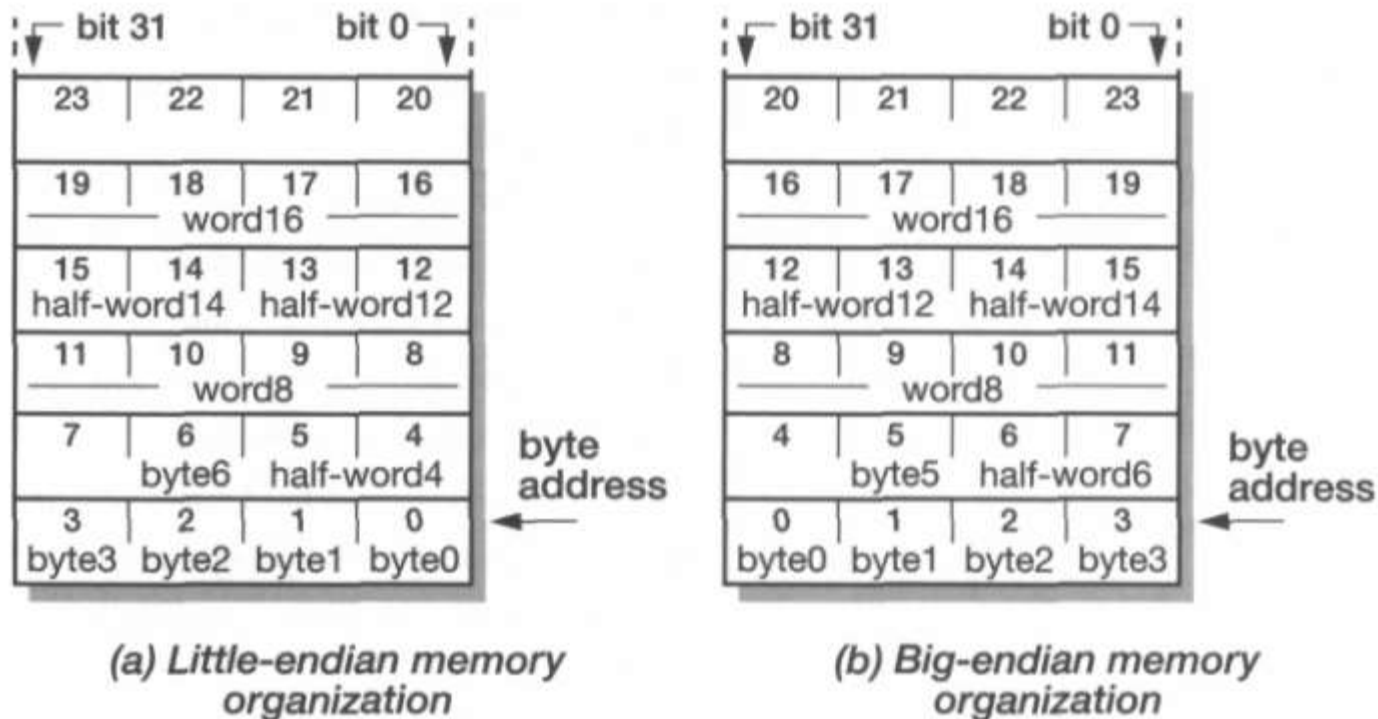
Word address	Byte address			
0	0	1	2	3
4	4	5	6	7
$2^k - 4$	$2^k - 4$	$2^k - 3$	$2^k - 2$	$2^k - 1$

(a) Big-endian assignment

	Byte address			
0	3	2	1	0
4	7	6	5	4
$2^k - 4$	$2^k - 1$	$2^k - 2$	$2^k - 3$	$2^k - 4$

(b) Little-endian assignment

Little- and big-endian memory organizations

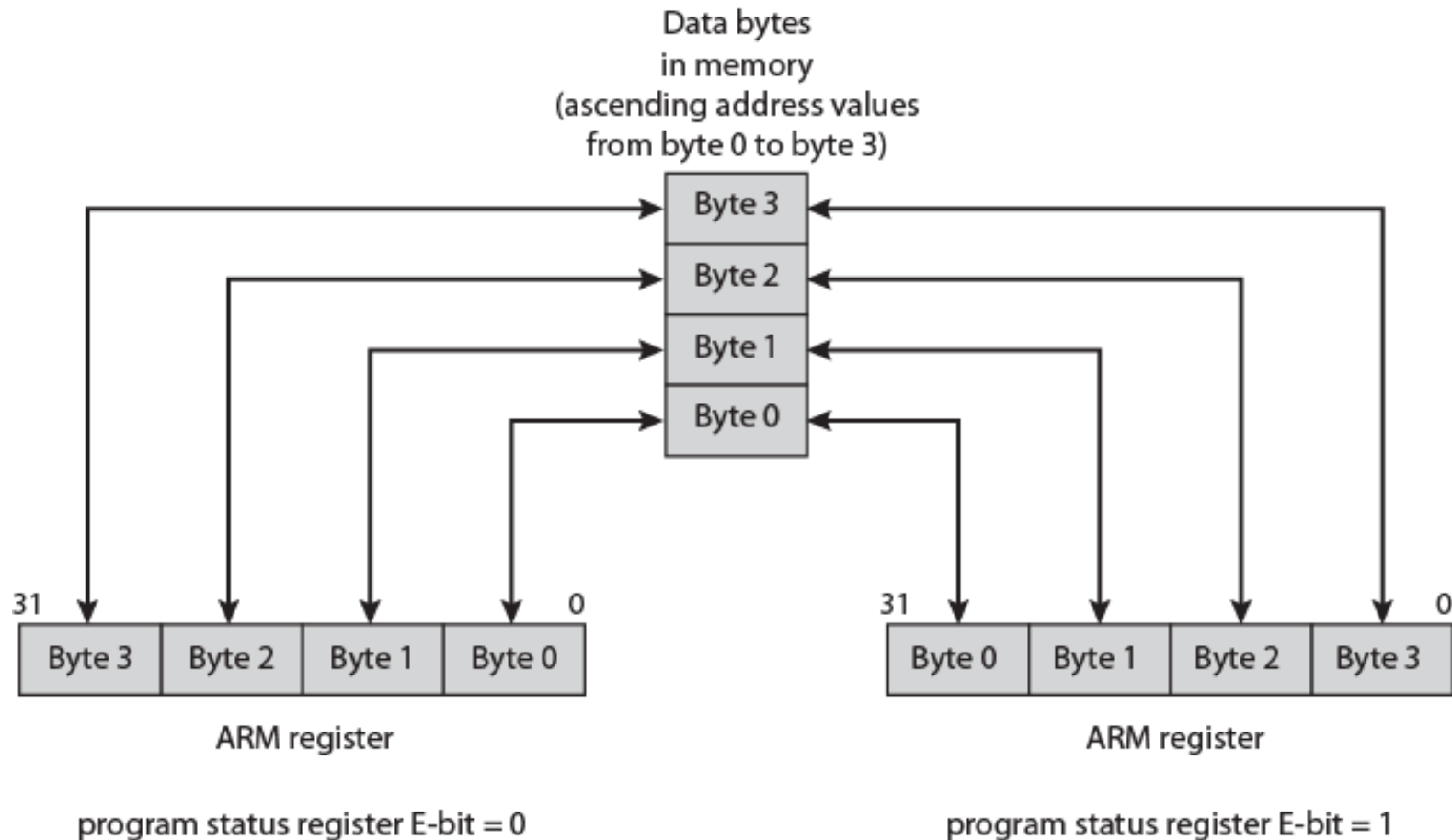


Most ARM chips can be configured to work with either memory arrangement, though they default to little-endian

ARM Endian Support

E-bit in system control register

Under program control



Memory Location, Addresses, and Operation



- ❑ Address ordering of bytes
- ❑ Word alignment
- ❑ Words are said to be aligned in memory if they begin at a byte address. that is a multiple of the number of bytes in a word.
 - 16-bit word: word addresses: 0, 2, 4,....
 - 32-bit word: word addresses: 0, 4, 8,....
 - 64-bit word: word addresses: 0, 8,16,....

Memory Operation

Load (or Read or Fetch)

- Copy the content. The memory content doesn't change.
- Address – Load
- Registers can be used

Store (or Write)

- Overwrite the content in memory
- Address and Data – Store
- Registers can be used

“Must-Perform” Operations



- ❑ Data transfers between the memory and the processor registers
- ❑ Arithmetic and logic operations on data
- ❑ Program sequencing and control
- ❑ I/O transfers

Assembly Language Notation

- ❑ Represent machine instructions and programs.
- ❑ Move LOC, $R1 = R1 \leftarrow [LOC]$
- ❑ Add R1, R2, $R3 = R3 \leftarrow [R1] + [R2]$
- ❑ or
- ❑ Add R1, R2, $R3 = R1 \leftarrow [R2] + [R3]$



CPU Organization

- ❑ Single Accumulator
 - ✓ Result usually goes to the Accumulator
 - ✓ Accumulator has to be saved to memory quite often
- ❑ General Register
 - ✓ Registers hold operands thus reduce memory traffic
 - ✓ Register book keeping
- ❑ Stack
 - ✓ Operands and result are always in the stack

Instruction Formats



Three-Address Instructions

- ADD R1, R2, R3 $R1 \leftarrow R2 + R3$ or $R3 \leftarrow R1 + R2$

Two-Address Instructions

- ADD R1, R2 $R1 \leftarrow R1 + R2$

One-Address Instructions

- ADD M $AC \leftarrow AC + M[AR]$

Zero-Address Instructions

- ADD $TOS \leftarrow TOS + (TOS - 1)$

RISC Instructions

- Lots of registers. Memory is restricted to Load & Store

Instruction Formats



Example: Evaluate $(A+B) * (C+D)$

Three-Address

- | | | | |
|----|-----|-----------|-------------------------------|
| 1. | ADD | R1, A, B | ; $R1 \leftarrow M[A] + M[B]$ |
| 2. | ADD | R2, C, D | ; $R2 \leftarrow M[C] + M[D]$ |
| 3. | MUL | X, R1, R2 | ; $M[X] \leftarrow R1 * R2$ |

Example: Evaluate $(A+B) * (C+D)$

Two-Address

- | | | | |
|----|-----|--------|-----------------------------|
| 1. | MOV | R1, A | ; $R1 \leftarrow M[A]$ |
| 2. | ADD | R1, B | ; $R1 \leftarrow R1 + M[B]$ |
| 3. | MOV | R2, C | ; $R2 \leftarrow M[C]$ |
| 4. | ADD | R2, D | ; $R2 \leftarrow R2 + M[D]$ |
| 5. | MUL | R1, R2 | ; $R1 \leftarrow R1 * R2$ |
| 6. | MOV | X, R1 | ; $M[X] \leftarrow R1$ |

Instruction Formats



Example: Evaluate $(A+B) * (C+D)$ One-Address

- | | | | |
|----|-------|---|-----------------------------|
| 1. | LOAD | A | ; $AC \leftarrow M[A]$ |
| 2. | ADD | B | ; $AC \leftarrow AC + M[B]$ |
| 3. | STORE | T | ; $M[T] \leftarrow AC$ |
| 4. | LOAD | C | ; $AC \leftarrow M[C]$ |
| 5. | ADD | D | ; $AC \leftarrow AC + M[D]$ |
| 6. | MUL | T | ; $AC \leftarrow AC * M[T]$ |
| 7. | STORE | X | ; $M[X] \leftarrow AC$ |

Example: Evaluate $(A+B) * (C+D)$ RISC

- | | | | |
|----|-------|------------|---------------------------|
| 1. | LOAD | R1, A | ; $R1 \leftarrow M[A]$ |
| 2. | LOAD | R2, B | ; $R2 \leftarrow M[B]$ |
| 3. | LOAD | R3, C | ; $R3 \leftarrow M[C]$ |
| 4. | LOAD | R4, D | ; $R4 \leftarrow M[D]$ |
| 5. | ADD | R1, R1, R2 | ; $R1 \leftarrow R1 + R2$ |
| 6. | ADD | R3, R3, R4 | ; $R3 \leftarrow R3 + R4$ |
| 7. | MUL | R1, R1, R3 | ; $R1 \leftarrow R1 * R3$ |
| 8. | STORE | X, R1 | ; $M[X] \leftarrow R1$ |

ARM Data Types



- ❑ Supports: 8 (byte), 16 (halfword), 32 (word) bits

Halfword and word accesses should be half word & word aligned respectively

Nonaligned access alternatives

- Default
 - Treated as truncated
 - Bits[1:0] treated as zero for word
 - Bit[0] treated as zero for halfword
 - Load single word instructions rotate right word aligned data transferred by non word-aligned address one, two or three bytes Alignment checking
- Data abort signal indicates alignment fault for attempting unaligned access
- Unaligned access
- Processor uses one or more memory accesses to generate transfer of adjacent bytes transparently to the programmer

ARM Data Types contnd.



Unsigned integer interpretation supported for all types

Twos-complement signed integer interpretation supported for all types

Majority of implementations do not provide floating-point hardware

- Saves power and area
- Floating-point arithmetic implemented in software
- Optional floating-point coprocessor
- Single- and double-precision IEEE 754 floating point data types
- (1) load or store a 32-bit word or an 8-bit unsigned byte, and
- (2) load or store a 16-bit unsigned halfword,
- and load and sign extend a 16-bit halfword or an 8-bit byte

ARM Operation Types



- ❑ The ARM architecture provides a large collection of operation types
- ❑ The following are the principal categories:
 - ✓ Load and store instructions: only load and store instructions access memory locations; arithmetic and logical instructions are performed only on registers and immediate values encoded in the instruction
 - ✓ A Branch instructions: Allows a conditional branch forwards or backwards up to 32 MB.
 - ✓ Data-processing instructions: includes logical instructions (AND, OR, XOR), add and subtract instructions, and test and compare instructions
 - ✓ Multiply instructions: The integer multiply instructions operate on word or halfword operands and can produce normal or long results

ARM Operation Types contnd.



- ✓ Parallel addition and subtraction instructions
- ✓ Extend instructions: There are several instructions for unpacking data by sign or zero extending bytes to halfwords or words, and halfwords to words.
- ✓ Status register access instructions: ARM provides the ability to read and also to write portions of the status register.

ARM Condition Codes



Condition code flags

Condition code register / status register

N (negative)

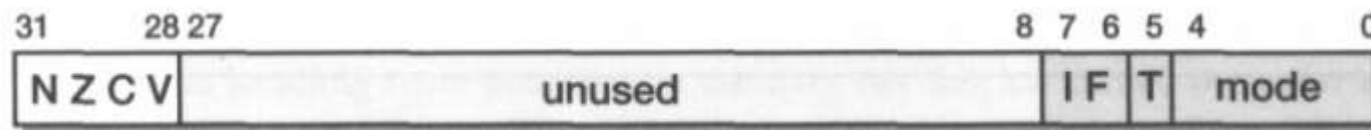
Z (zero)

V (overflow)

C (carry)

Different instructions affect different flags

The Current Program Status Register (CPSR)



Using Registers



- ❑ Registers are faster
- ❑ Shorter instructions
 - The number of registers is smaller (e.g. 32 registers need 5 bits)
- ❑ Potential speedup
- ❑ Minimize the frequency with which data is moved back and forth between the memory and processor registers.

Conditional Branch Instructions



Example:

- A: 1 1 1 1 0 0 0 0
- B: 0 0 0 1 0 1 0 0

$$\begin{array}{r} \text{A:} \quad 1\ 1\ 1\ 1\ 0\ 0\ 0\ 0 \\ +(-\text{B}): \quad 1\ 1\ 1\ 0\ 1\ 1\ 0\ 0 \\ \hline \quad 1\ 1\ 0\ 1\ 1\ 1\ 0\ 0 \end{array}$$

$\text{C} = 1$ (Carry flag, indicated by an orange arrow from the carry-out of the addition)

$\text{Z} = 0$ (Zero flag, indicated by a brown bracket under the result bits 1-7)

$\text{S} = 1$ (Sign flag, indicated by a blue arrow from the sign bit of the result)

$\text{V} = 0$ (Overflow flag)

Overflow can only happen when adding two numbers of the same sign and getting a different sign. So, to detect overflow we don't care about any bits except the sign bits. Ignore the other bits.

ARM Conditions for Conditional Instruction Execution



Code	Symbol	Condition Tested	Comment
0000	EQ	$Z = 1$	Equal
0001	NE	$Z = 0$	Not equal
0010	CS/HS	$C = 1$	Carry set/unsigned higher or same
0011	CC/LO	$C = 0$	Carry clear/unsigned lower
0100	MI	$N = 1$	Minus/negative
0101	PL	$N = 0$	Plus/positive or zero
0110	VS	$V = 1$	Overflow
0111	VC	$V = 0$	No overflow
1000	HI	$C = 1 \text{ AND } Z = 0$	Unsigned higher
1001	LS	$C = 0 \text{ OR } Z = 1$	Unsigned lower or same
1010	GE	$N = V$ $[(N = 1 \text{ AND } V = 1)$ $\text{OR } (N = 0 \text{ AND } V = 0)]$	Signed greater than or equal
1011	LT	$N \neq V$ $[(N = 1 \text{ AND } V = 0)$ $\text{OR } (N = 0 \text{ AND } V = 1)]$	Signed less than
1100	GT	$(Z = 0) \text{ AND } (N = V)$	Signed greater than
1101	LE	$(Z = 1) \text{ OR } (N \neq V)$	Signed less than or equal
1110	AL	—	Always (unconditional)
1111	—	—	This instruction can only be executed unconditionally

Expression Evaluation



❑ Infix notation: A binary operator appears between the operands

✓ $a + (b \times c)$, will yield a different result from $(a + b) \times c$

❑ Reverse Polish, or postfix, notation

$a + b$ becomes $a b +$

$a + (b \times c)$, becomes $a b c \times +$

$(a + b) \times c$ becomes $a b + c \times$

✓ Regardless of the complexity of an expression, no parentheses are required when using reverse Polish

✓ An expression in this form is easily evaluated using a stack

❑ Rules

1. If the element is a variable or constant, push it onto the stack.

2. If the element is an operator, pop the top two items of the stack, perform the operation, and push the result.

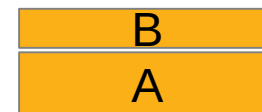
Instruction Formats

TOS = TOS (operation)(TOS-1)

Example: Evaluate $(A+B) * (C+D)$

Zero-Address

1. PUSH A ; $TOS \leftarrow A$
2. PUSH B ; $TOS \leftarrow B$
3. ADD ; $TOS \leftarrow (A + B)$
4. PUSH C ; $TOS \leftarrow C$
5. PUSH D ; $TOS \leftarrow D$
6. ADD ; $TOS \leftarrow (C + D)$
7. MUL ; $TOS \leftarrow (C+D)*(A+B)$
8. POP X ; $M[X] \leftarrow TOS$



Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

Zero-Address

1. PUSH A ; $TOS \leftarrow A$
2. PUSH B ; $TOS \leftarrow B$
3. ADD ; $TOS \leftarrow (A + B)$
4. PUSH C ; $TOS \leftarrow C$
5. PUSH D ; $TOS \leftarrow D$
6. ADD ; $TOS \leftarrow (C + D)$
7. MUL ; $TOS \leftarrow (C+D)*(A+B)$
8. POP X ; $M[X] \leftarrow TOS$

D
C
A+B

Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

Zero-Address

1. PUSH A ; $TOS \leftarrow A$
2. PUSH B ; $TOS \leftarrow B$
3. ADD ; $TOS \leftarrow (A + B)$
4. PUSH C ; $TOS \leftarrow C$
5. PUSH D ; $TOS \leftarrow D$
6. ADD ; $TOS \leftarrow (C + D)$
7. MUL ; $TOS \leftarrow (C+D)*(A+B)$
8. POP X ; $M[X] \leftarrow TOS$

C+D
A+B

Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

Zero-Address

1. PUSH A ; $TOS \leftarrow A$
2. PUSH B ; $TOS \leftarrow B$
3. ADD ; $TOS \leftarrow (A + B)$
4. PUSH C ; $TOS \leftarrow C$
5. PUSH D ; $TOS \leftarrow D$
6. ADD ; $TOS \leftarrow (C + D)$
7. MUL ; $TOS \leftarrow (C+D)*(A+B)$
8. POP X ; $M[X] \leftarrow TOS$

$(C+D)*(A+B)$



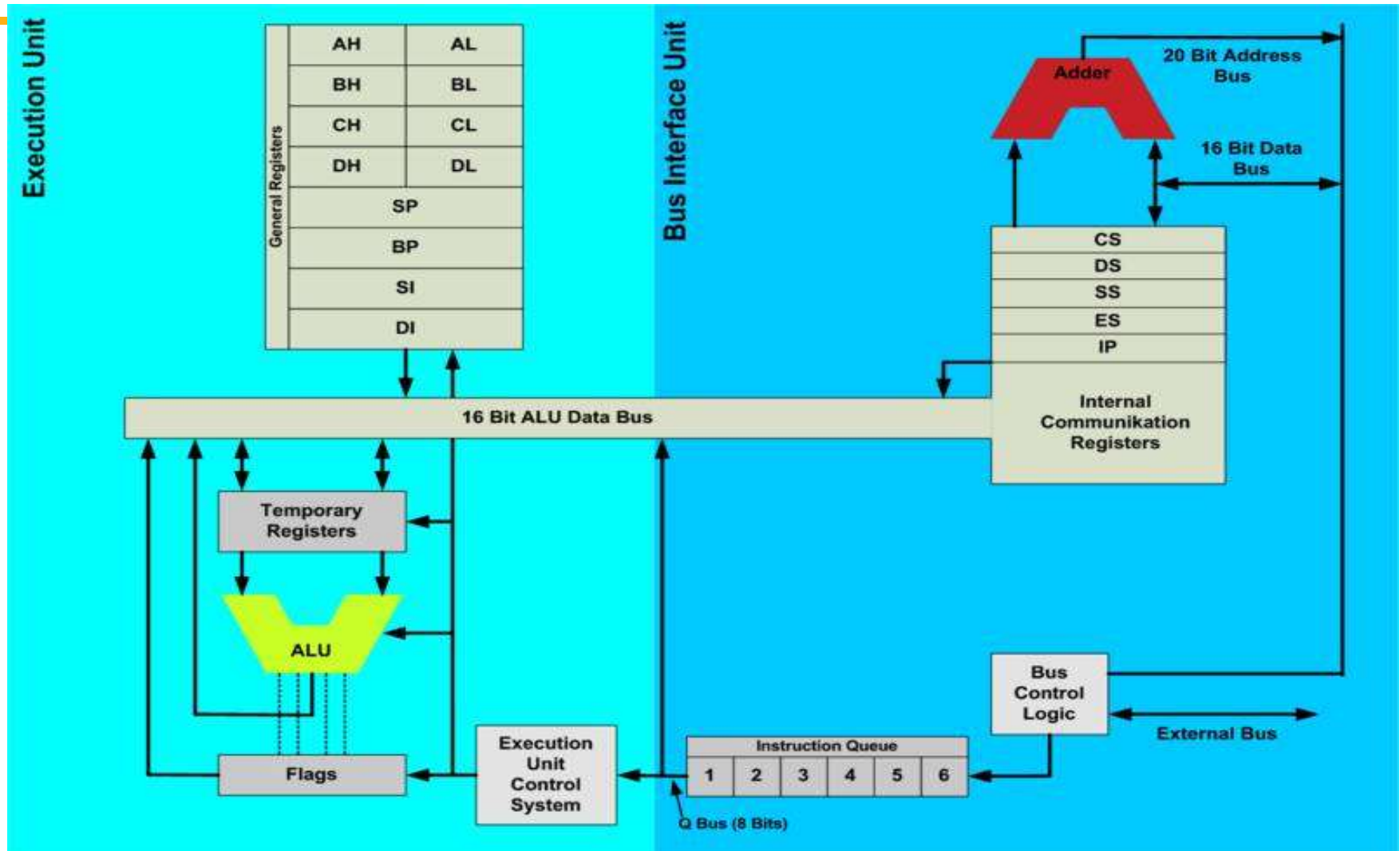
BITS Pilani

Pilani | Dubai | Goa | Hyderabad



Addressing Modes

• 8086 Intel Microprocessor Architecture





Generating Memory Addresses

How to specify the address of branch target?

Can we give the memory operand address directly in a single Add instruction in the loop?

Use a register to hold the address of NUM1; then increment by 4 on each pass through the loop.

Addressing Modes



Implied

- AC is implied in “ADD M[AR]” in “One-Address” instr.
- TOS is implied in “ADD” in “Zero-Address” instr.

Immediate

- The use of a constant in “MOV R1, 5”, i.e. $R1 \leftarrow 5$

Register

- Indicate which register holds the operand

Addressing Modes

Register Indirect

- Indicate the register that holds the number of the register that holds the operand

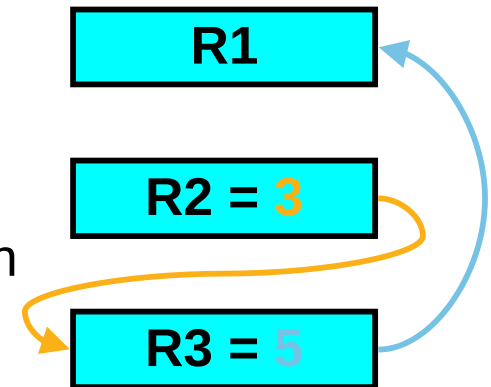
MOV R1, (R2)

Autoincrement / Autodecrement

- Access & update in 1 instr.

Direct Address

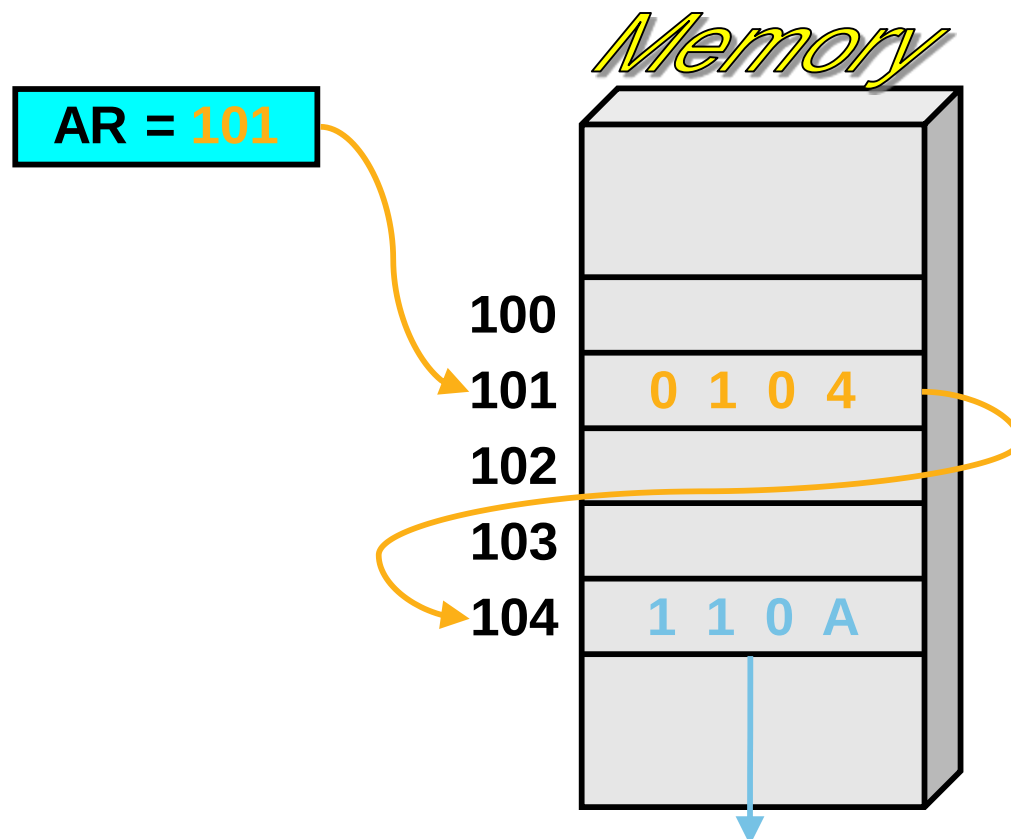
- Use the given address to access a memory location



Addressing Modes

Indirect Address

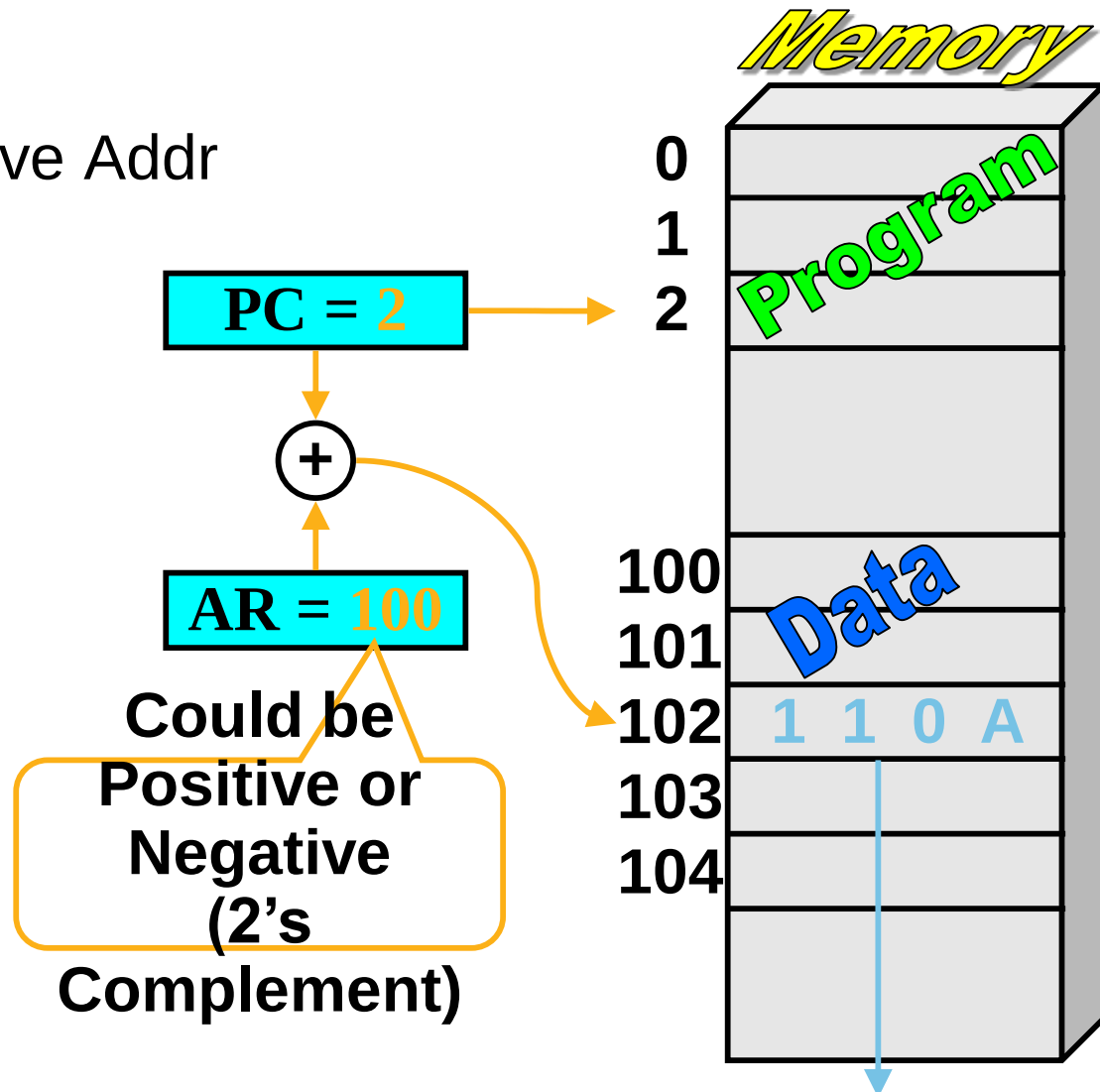
- Indicate the memory location that holds the address of the memory location that holds the data



Addressing Modes

Relative Address

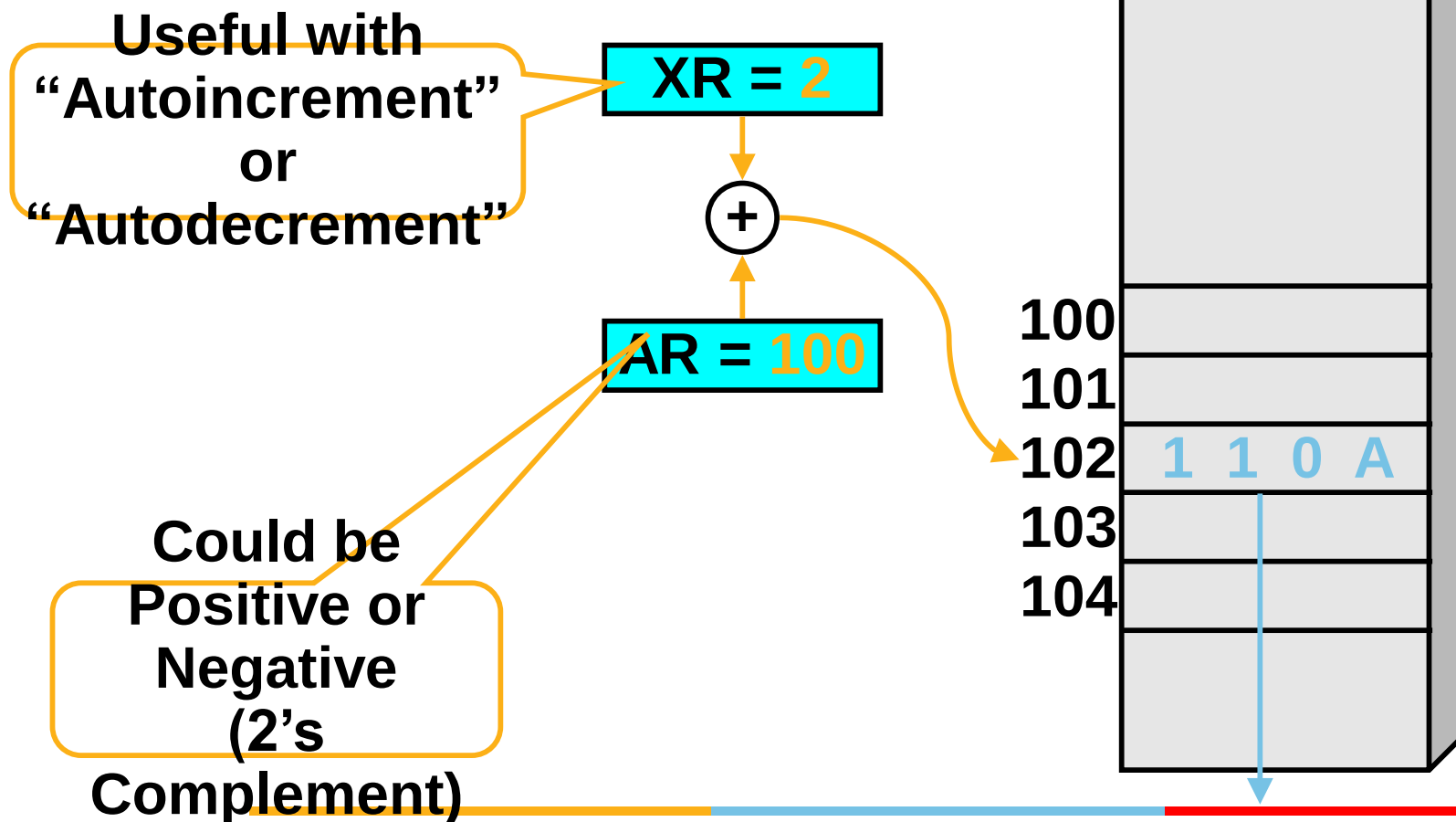
- $EA = PC + \text{Relative Addr}$



Indexed

- $EA = \text{Index Register} + \text{Relative Addr}$

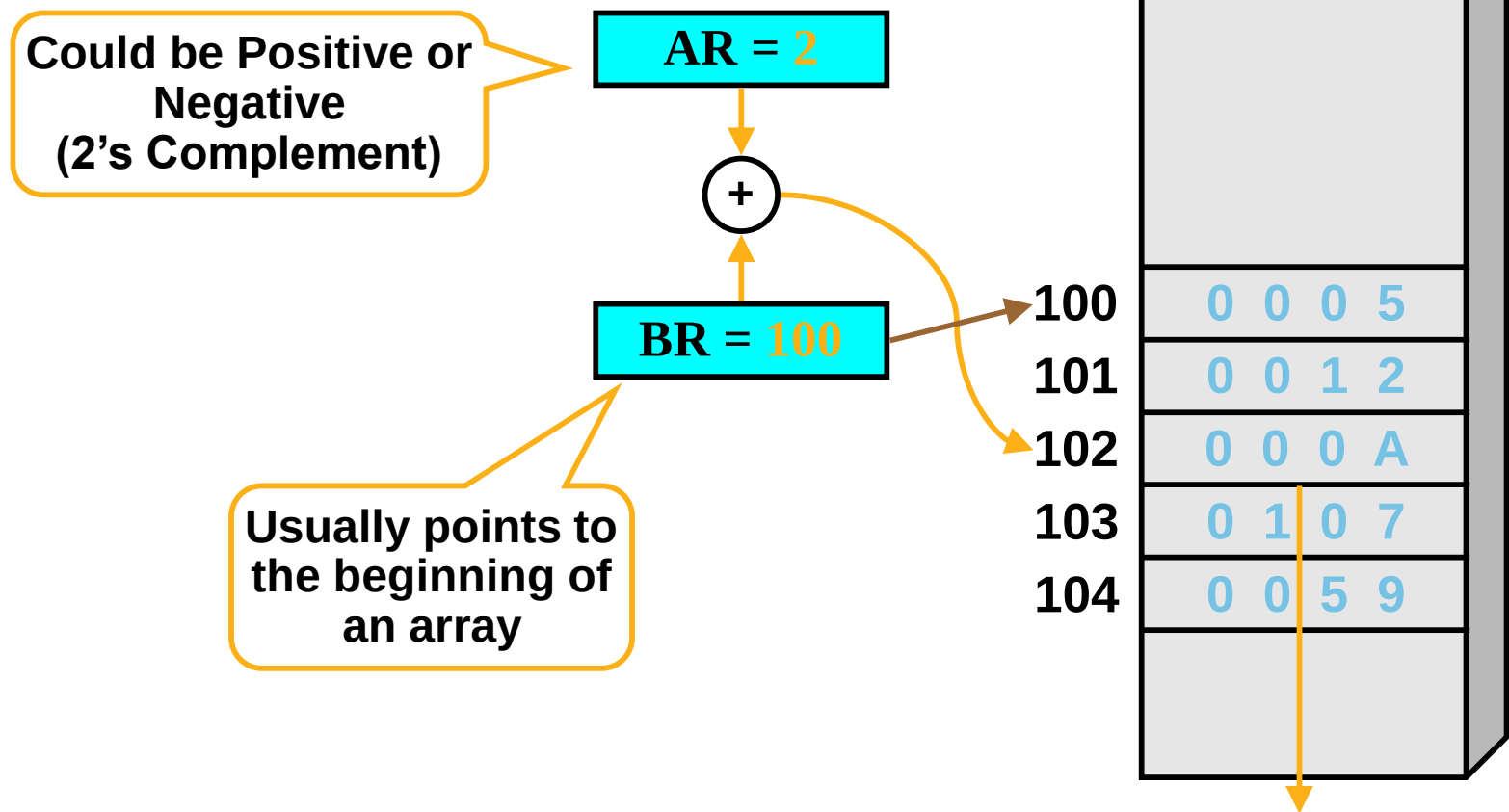
Memory



Addressing Modes

Base Register

EA = Base Register + Relative Addr

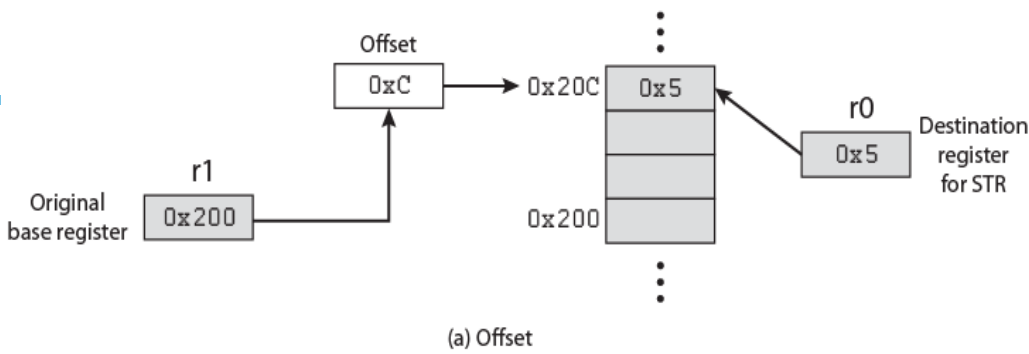




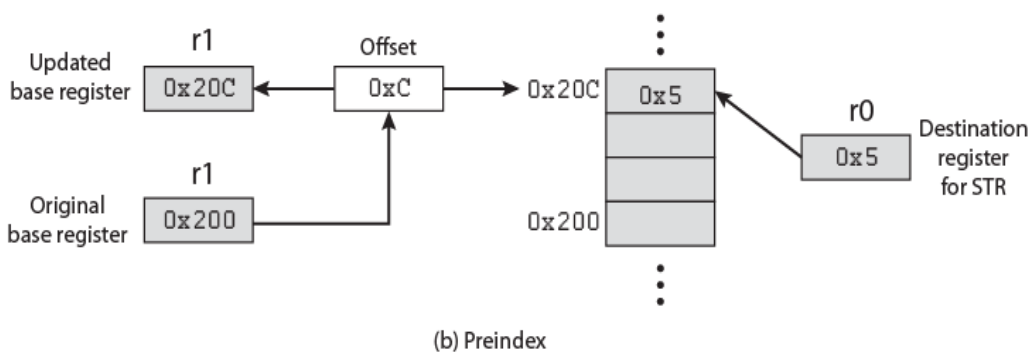
ARM Indexing Methods

- ❑ RISC machine, unlike a CISC machine has simple and relatively straightforward set of addressing modes
- ❑ ARM architecture departs somewhat from this tradition by providing a relatively rich set of addressing modes.

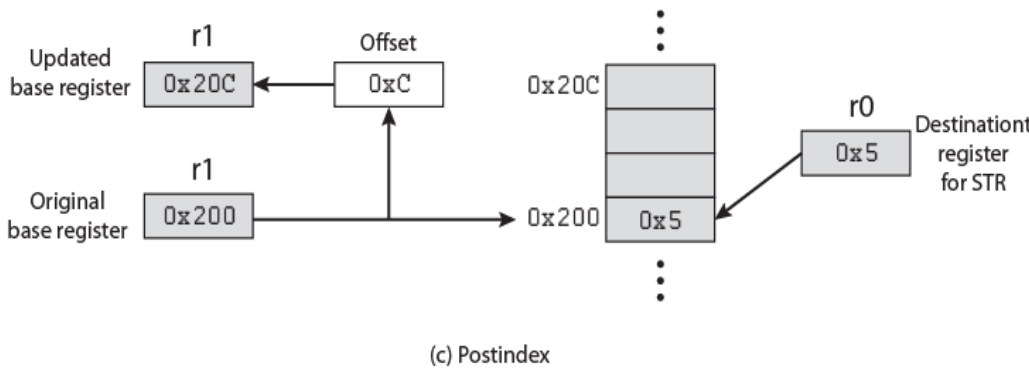
```
STRB r0, [r1, #12]
```



```
STRB r0, [r1, #12]!
```



```
STRBv r0, [r1], #12
```



ARM Addressing Modes

Load/Store



- ❑ Only instructions that reference memory
- ❑ Indirectly through base register plus offset
- ❑ Offset
 - ✓ Offset added to or subtracted from base register contents to form the memory address
- ❑ Preindex
 - ✓ Memory address is formed as for offset addressing
 - ✓ Memory address also written back to base register
 - ✓ So base register value incremented or decremented by offset value

ARM Addressing Modes

Load/Store



☐ Postindex

- ✓ Memory address is base register value
- ✓ Offset added or subtracted

Result written back to base register

- ☐ Base register acts as index register for preindex and postindex addressing
- ☐ Offset either immediate value in instruction or another register
- ☐ If register scaled register addressing available
- ✓ Offset register value scaled by shift operator
- ✓ Instruction specifies shift size

ARM Data Processing Instruction Addressing & Branch Instructions



☐ Data Processing

- ✓ Register addressing
 - Value in register operands may be scaled using a shift operator
- ✓ Or mixture of register and immediate addressing

☐ Branch

- ✓ Immediate
- ✓ Instruction contains 24 bit value
- ✓ Shifted 2 bits left
 - On word boundary
 - Effective range +/-32MB from PC.

ARM Load/Store Multiple Addressing

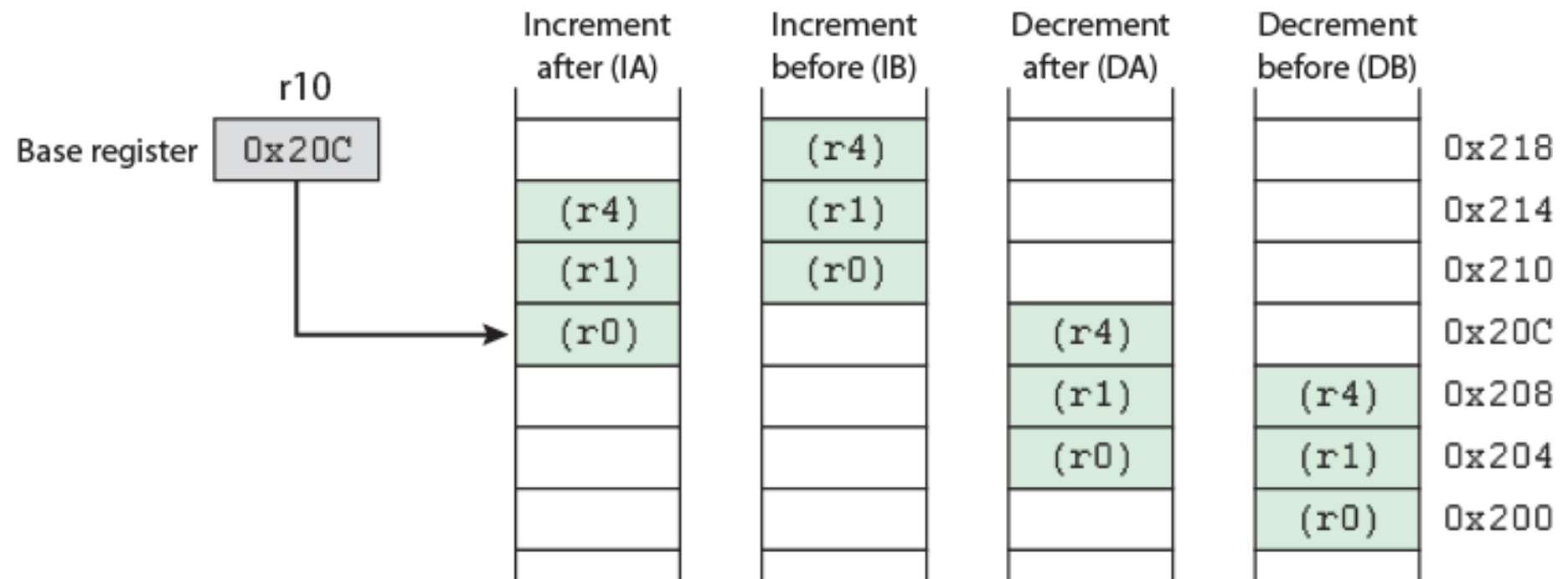


- ❑ Load/store subset of general-purpose registers
- ❑ 16-bit instruction field specifies list of registers
- ❑ Sequential range of memory addresses
- ❑ Increment after, increment before, decrement after, and decrement before
- ❑ Base register specifies main memory address
- ❑ Incrementing or decrementing starts before or after first memory access

ARM Load/Store Multiple Addressing



```
LDMxx r10, {r0, r1, r4}  
STMxx r10, {r0, r1, r4}
```



Instruction Formats



- ❑ Layout of bits in an instruction
- ❑ Includes opcode
- ❑ Includes (implicit or explicit) operand(s)
- ❑ Usually more than one instruction format in an instruction set

Instruction Length

- ❑ Affected by and affects:
 - ✓ Memory size
 - ✓ Memory organization
 - ✓ Bus structure
 - ✓ CPU complexity
 - ✓ CPU speed
- ❑ Trade off between powerful instruction repertoire and saving space

Allocation of Bits



- ☐ Number of addressing modes
- ☐ Number of operands
- ☐ Register versus memory
- ☐ Number of register sets
- ☐ Address range
- ☐ Address granularity

In class Example-1



Assume an instruction set that uses a fixed 16-bit instruction length. Operand specifiers are 6 bits in length. There are K two-operand instructions and L zero-operand instructions. What is the maximum number of one-operand instructions?

In class Example-1



Design a variable-length opcode to allow all of the following to be encoded in a 36-bit instruction:

- instructions with two 15-bit addresses and one 3-bit register number
- instructions with one 15-bit address and one 3-bit register number
- instructions with no addresses or registers

In class Example-1



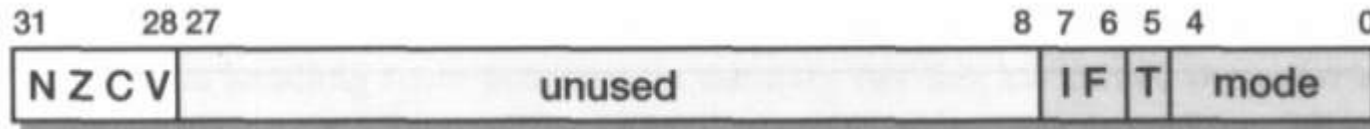
Design a variable-length opcode to allow all of the following to be encoded in a 36-bit instruction:

- instructions with two 15-bit addresses and one 3-bit register number
- instructions with one 15-bit address and one 3-bit register number
- instructions with no addresses or registers

ARM CPSR format

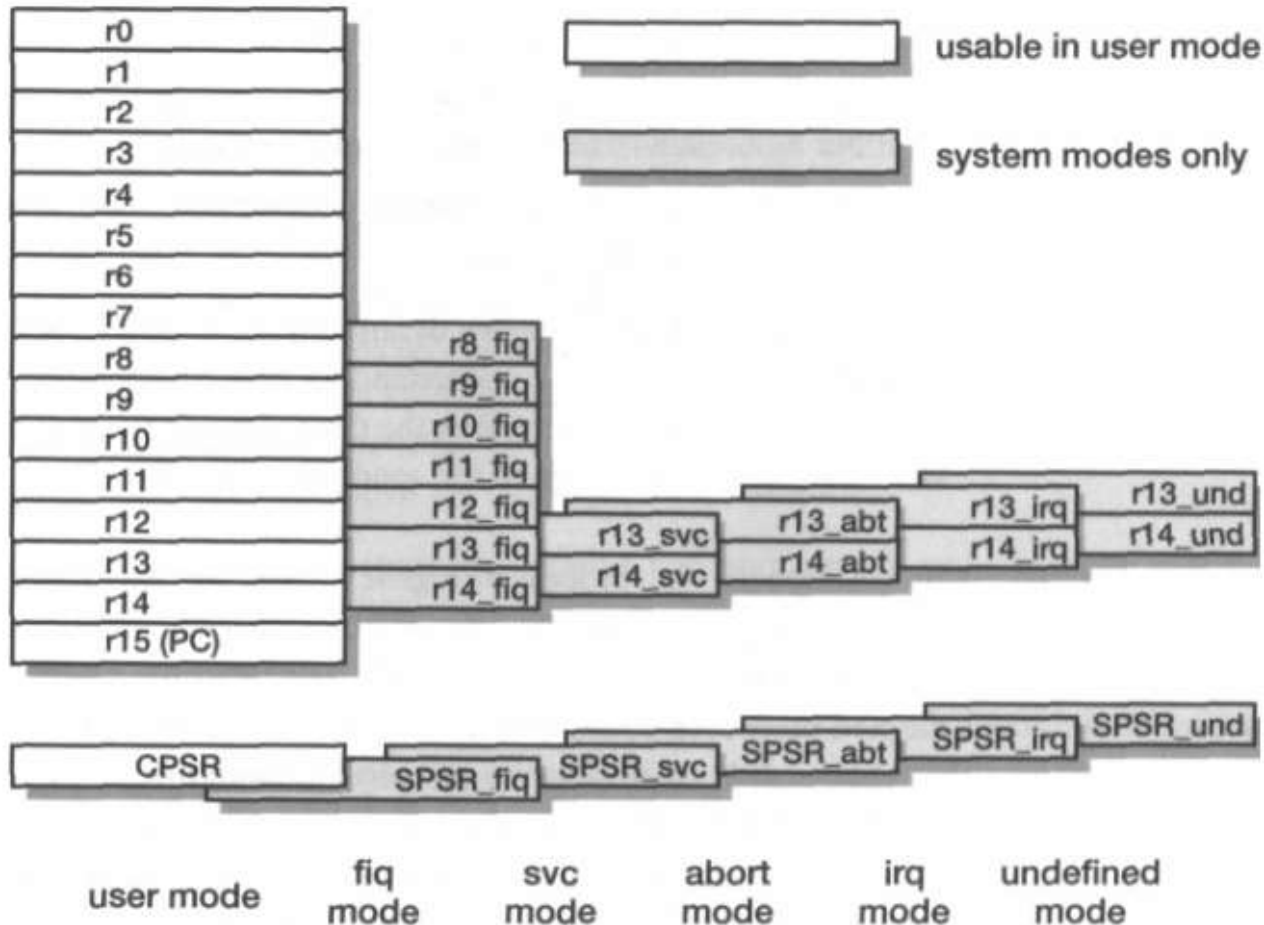


The Current Program Status Register (CPSR)



CPSR[4:0]	Mode	Use	Registers
10000	User	Normal user code	user
10001	FIQ	Processing fast interrupts	_fiq
10010	IRQ	Processing standard interrupts	_irq
10011	SVC	Processing software interrupts (SWIs)	_svc
10111	Abort	Processing memory faults	_abt
11011	Undef	Handling undefined instruction traps	_und
11111	System	Running privileged operating system tasks	user

ARM's visible registers



When writing user-level programs, only the 15 general-purpose 32-bit registers (r0 to r!4), the program counter (r15) and the current program status register (CPSR) need be considered.

The remaining registers are used only for system-level programming and for handling exceptions (for example, interrupt)

Each privileged mode (except system mode) has associated with it a Saved Program Status Register, or SPSR. This register is used to save the state of the CPSR

Exception in ARM



When an exception arises, ARM completes the current instruction

Exception	Mode	Vector address
Reset	SVC	0x00000000
Undefined instruction	UND	0x00000004
Software interrupt (SWI)	SVC	0x00000008
Prefetch abort (instruction fetch memory fault)	Abort	0x0000000C
Data abort (data access memory fault)	Abort	0x00000010
IRQ (normal interrupt)	IRQ	0x00000018
FIQ (fast interrupt)	FIQ	0x0000001C

- ❑ Changes to the operating mode corresponding to the particular exception. •
- ❑ saves the address of the instruction following the exception entry instruction in r14 of the new mode.
- ❑ saves the old value of the CPSR in the SPSR of the new mode address given in Table

- ❑ . disables IRQs by setting bit 7 of the CPSR and, if the exception is a fast interrupt, disables further fast interrupts by setting bit 6 of the CPSR.
- ❑ forces the PC to begin executing at the relevant vector
- ❑ Vector address 0x00000014 is not used. Used for Address exceptions in earlier ARM processors with 26 bit address space

Exception return



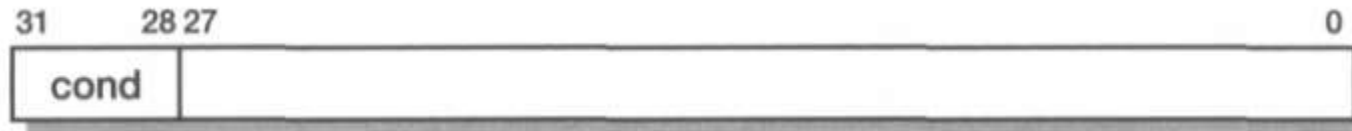
Once the exception has been handled the user task is normally resumed. This requires the handler code to restore the user state exactly as it was when the exception first arose:

- ❑ Any modified user registers must be restored from the handler's stack.
- ❑ The CPSR must be restored from the appropriate SPSR.
- ❑ The PC must be changed back to the relevant instruction address in the user instruction stream.

Exception priorities

1. reset (highest priority);
2. data abort;
3. FIQ;
4. IRQ;
5. prefetch abort;
6. SWI, undefined instruction

ARM Conditional execution



- ❑ Each of the 16 values of the condition field causes the instruction to be executed or skipped according to the values of the N, Z, C and V flags in the CPSR
- ❑ Every ARM instruction mnemonic may be extended by appending the two letters defined in the condition table (Symbols)

ARM Instruction Formats

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
data processing immediate shift	cond			0			0			0			opcode			S	Rn			Rd			shift amount				shift		0	Rm							
data processing register shift	cond			0			0			0			opcode			S	Rn			Rd			Rs			0	shift		1	Rm							
data processing immediate	cond			0			0			1			opcode			S	Rn			Rd			rotate				immediate										
load/store immediate offset	cond			0			1			0			P	U	B	W	L	Rn			Rd			immediate													
load/store register offset	cond			0			1			1			P	U	B	W	L	Rn			Rd			shift amount				shift		0	Rm						
load/store multiple	cond			1			0			0			P	U	S	W	L	Rn			register list																
branch/branch with link	cond			1			0			1			L	24-bit offset																							

S = For data processing instructions, updates condition codes

S = For load/store multiple instructions, execution restricted to supervisor mode

P, U, W = distinguish between different types of addressing_mode

B = Unsigned byte (B==1) or word (B==0) access

L = For load/store instructions, Load (L==1) or Store (L==0)

L = For branch instructions, is return address stored in link register

Branch and Branch with Link binary encoding.



Branch and Branch with Link instructions cause the processor to begin executing instructions from an address computed by sign extending the 24-bit offset specified in the instruction, shifting it left two places to form a word offset, then adding it to the program counter which contains the address of the branch instruction plus eight bytes.

The range of the branch instruction is +/- 32 Mbytes

The Branch with Link variant, which has the L bit (bit 24) set, also moves the address of the instruction following the branch into the link register (r14) of the current processor mode.

This is normally used to perform a subroutine call, with the return being caused by copying the link register back into the PC.

Assembler computes the correct offset under normal circumstances.

Note: PC points eight bytes (two instructions) ahead of the current instruction

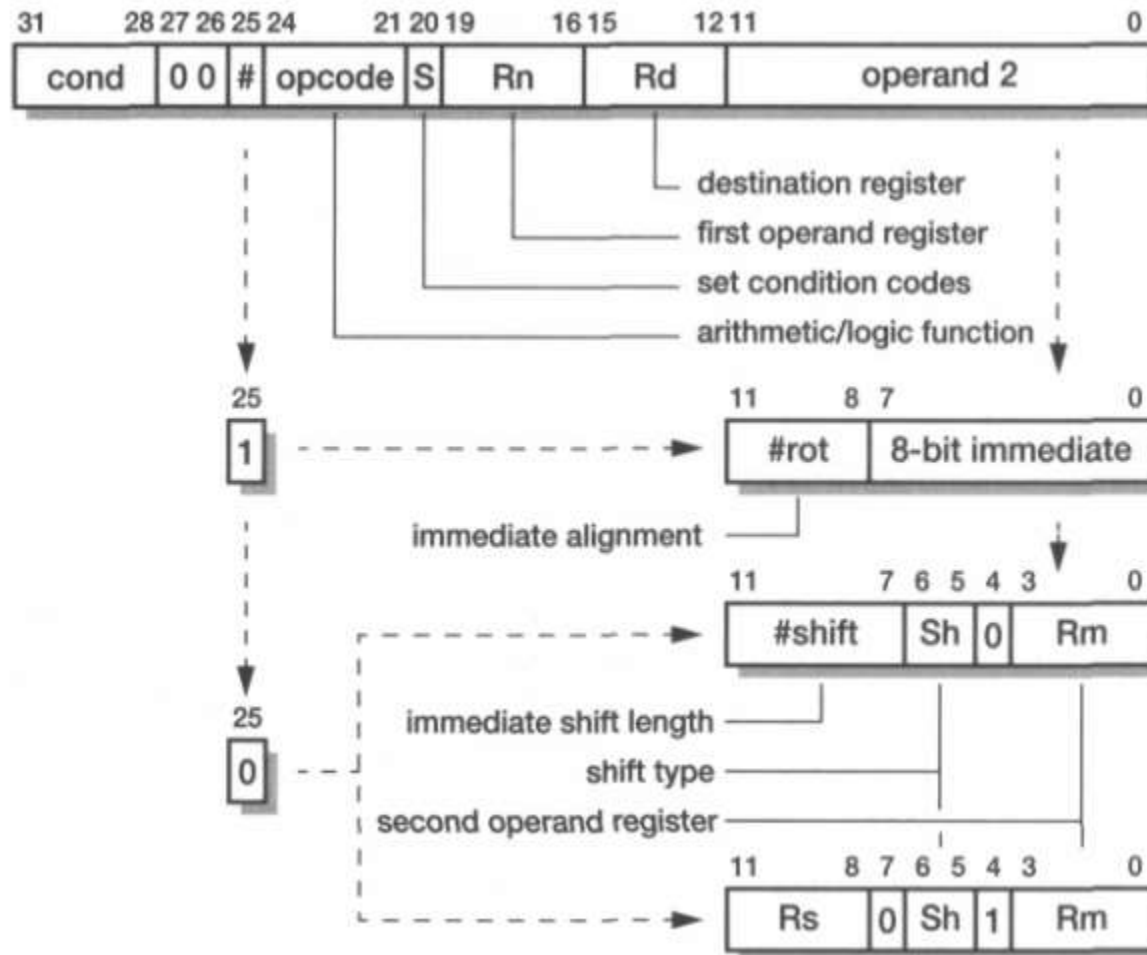
B{L} {condn} <target address>

Data processing instructions



Opcode 124:21)	Mnemonic	Meaning	Effect
0000	AND	Logical bit-wise AND	$Rd := Rn \text{ AND } Op2$
0001	EOR	Logical bit-wise exclusive OR	$Rd := Rn \text{ EOR } Op2$
0010	SUB	Subtract	$Rd := Rn - Op2$
0011	RSB	Reverse subtract	$Rd := Op2 - Rn$
0100	ADD	Add	$Rd := Rn + Op2$
0101	ADC	Add with carry	$Rd := Rn + Op2 + C$
0110	SBC	Subtract with carry	$Rd := Rn - Op2 + C - 1$
0111	RSC	Reverse subtract with carry	$Rd := Op2 - Rn + C - 1$
1000	TST	Test	Set on $Rn \text{ AND } Op2$
1001	TEQ	Test equivalence	Set on $Rn \text{ EOR } Op2$
1010	CMP	Compare	Set on $Rn - Op2$
1011	CMN	Compare negated	Set on $Rn + Op2$
1100	ORR	Logical bit-wise OR	$Rd := Rn \text{ OR } Op2$
1101	MOV	Move	$Rd := Op2$
1110	BIC	Bit clear	$Rd := Rn \text{ AND NOT } Op2$
1111	MVN	Move negated	$Rd := \text{NOT } Op2$

Data processing instructions



Multiply instructions



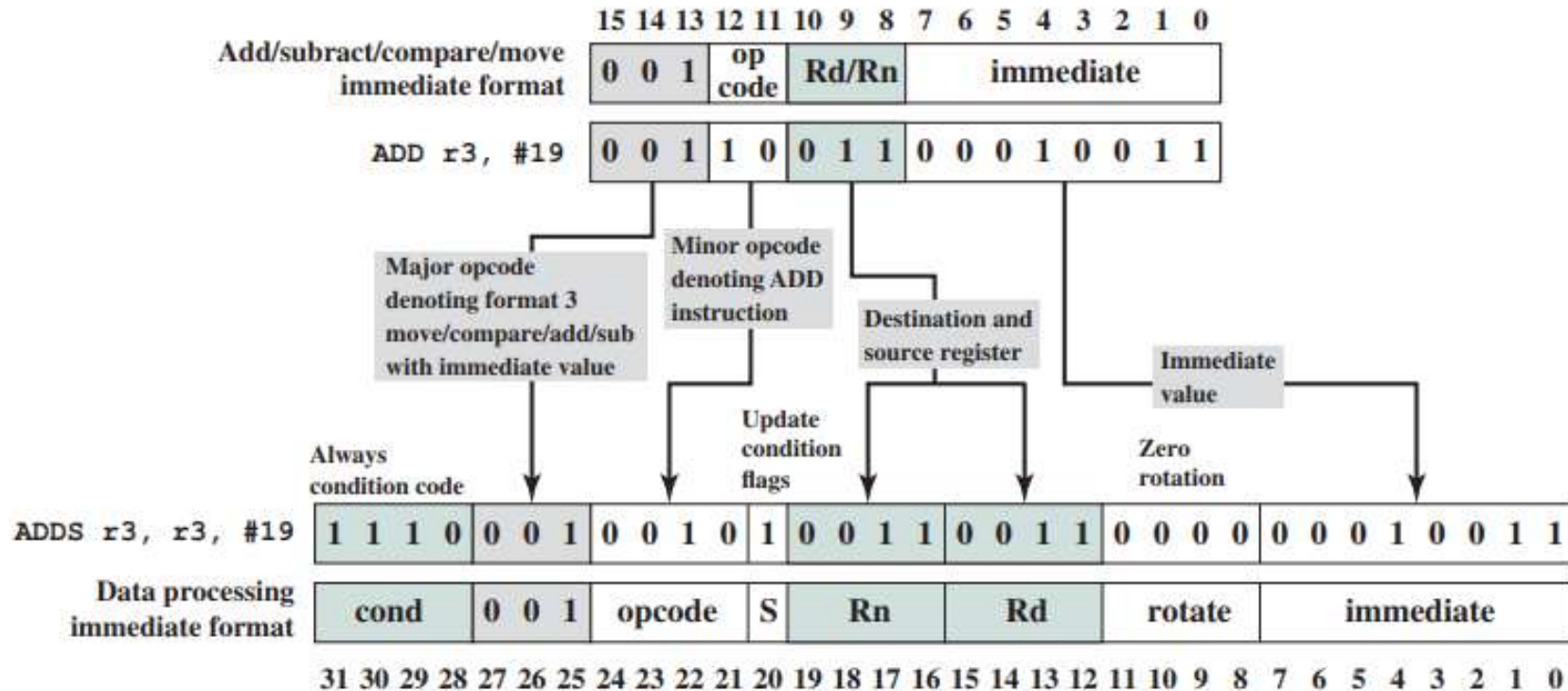
ARM multiply instructions produce the product of two 32-bit binary numbers held in registers. The result of multiplying two 32-bit binary numbers is a 64-bit product

Multiply instructions.



Opcode [23:21]	Mnemonic	Meaning	Effect
000	MUL	Multiply (32-bit result)	$Rd := (Rm * Rs)[31:0]$
001	MLA	Multiply-accumulate (32-bit result)	$Rd := (Rm * Rs + Rn)[31:0]$
100	UMULL	Unsigned multiply long	$RdHi:RdLo := Rm * Rs$
101	UMLAL	Unsigned multiply-accumulate long	$RdHi:RdLo += Rm * Rs$
110	SMULL	Signed multiply long	$RdHi:RdLo := Rm * Rs$
111	SMLAL	Signed multiply-accumulate long	$RdHi:RdLo += Rm * Rs$

Expanding a Thumb ADD Instruction into its ARM Equivalent



The Thumb instruction set is a re-encoded subset of the ARM instruction set. Thumb is designed to increase the performance of ARM implementations that use a 16-bit or narrower memory data bus and to allow better code density than provided by the ARM instruction for both 16-bit and 32-bit processors

Assembler



- ❖ Machines store and understand binary instructions
- ❖ E.g. $N = I + J + K$ initialize $I=2, J=3, K=4$
- ❖ Program starts in location 101
- ❖ Data starting 201
- ❑ Code:
 - ✓ Load contents of 201 into AC
 - ✓ Add contents of 202 to AC
 - ✓ Add contents of 203 to AC
 - ✓ Store contents of AC to 204
 - ✓ Tedious and error prone

Address		Contents		
101	0010	0010	101	2201
102	0001	0010	102	1202
103	0001	0010	103	1203
104	0011	0010	104	3204
201	0000	0000	201	0002
202	0000	0000	202	0003
203	0000	0000	203	0004
204	0000	0000	204	0000

Binary program

Improvements

- ❑ Use hexadecimal rather than binary
 - ✓ Code as series of lines
 - Hex address and memory address
 - ✓ Need to translate automatically using program
- ❑ Add symbolic names or mnemonics for instructions
- ❑ Three fields per line
 - ✓ Location address
 - ✓ Three letter opcode
 - ✓ If memory reference: address
- ❑ Need more complex translation program

Address	Contents
101	2201
102	1202
103	1203
104	3204
201	0002
202	0003
203	0004
204	0000

Hexadecimal program

Address	Instruction	
101	LDA	201
102	ADD	202
103	ADD	203
104	STA	204
201	DAT	2
202	DAT	3
203	DAT	4
204	DAT	0

Symbolic program

Symbolic Addresses



- ❑ First field (address) now symbolic
- ❑ Memory references in third field now symbolic
- ❑ Now have assembly language and need an assembler to translate
- ❑ Assembler used for some systems programming

- Compilers
- I/O routines

Label	Operation	Operand
FORMUL	LDA	I
	ADD	J
	ADD	K
	STA	N
I	DATA	2
J	DATA	3
K	DATA	4
N	DATA	0

Assembly program

Computation of the Formula $N = I + J + K$

Address		Contents		
101	0010	0010	101	2201
102	0001	0010	102	1202
103	0001	0010	103	1203
104	0011	0010	104	3204
201	0000	0000	201	0002
202	0000	0000	202	0003
203	0000	0000	203	0004
204	0000	0000	204	0000

(a) Binary program

Address		Contents
101		2201
102		1202
103		1203
104		3204
201		0002
202		0003
203		0004
204		0000

(b) Hexadecimal program

Address		Instruction	
101	LDA	201	
102	ADD	202	
103	ADD	203	
104	STA	204	
201	DAT	2	
202	DAT	3	
203	DAT	4	
204	DAT	0	

(c) Symbolic program

Label	Operation	Operand
FORMUL	LDA	I
	ADD	J
	ADD	K
	STA	N
I	DATA	2
J	DATA	3
K	DATA	4
N	DATA	0

(d) Assembly program

Addressing Mode In class example 3



Given the following memory values and a one-address machine with an accumulator, what values do the following instructions load into the accumulator?

- Word 20 contains 40.
- Word 30 contains 50.
- Word 40 contains 60.
- Word 50 contains 70.

- a. LOAD IMMEDIATE 20
- b. LOAD DIRECT 20
- c. LOAD INDIRECT 20
- d. LOAD IMMEDIATE 30
- e. LOAD DIRECT 30
- f. LOAD INDIRECT 30

Sol:

- a. 20
- b. 40
- c. 60
- d. 30 e. 50 f. 70

In class example 4



Consider a 16-bit processor in which the following appears in main memory, starting at location 200:

The first part of the first word indicates that this instruction loads a value into an accumulator.

200	Load to AC	Mode
201	500	
202	Next instruction	

The Mode field specifies an addressing mode and, if appropriate, indicates a source register; assume that when used, the source register is R1, which has a value of 400. There is also a base register that contains the value 100. The value of 500 in location 201 may be part of the address calculation. Assume that location 399 contains the value 999, location 400 contains the value 1000, and so on. Determine the effective address and the operand to be loaded for the following address modes:

Contnd.

Determine the effective address and the operand to be loaded for the following address modes



- a. Direct
- b. Immediate
- c. Indirect
- h. Autoindexing with increment using R1
- d. PC relative
- e. Displacement,
- f. Register
- g. Register indirect

200	Load to AC	Mode
201	500	adrs
202	Next instruction	

Mode → R1 and contains 400

Base register 100

Adrs 399 → 999 & 400 → 1000

A	500	1100
B	201	500
C	1100	1700
D	702	1302
E	600	1200
F	R1	400
G	400	1000
H	400	1000

In class example 5



Assume a stack-oriented processor that includes the stack operations PUSH and POP. Arithmetic operations automatically involve the top one or two stack elements. Begin with an empty stack. What stack elements remain after the following instructions are executed?

PUSH 4

PUSH 7

PUSH 8

ADD

PUSH 10

SUB

MUL

Push 4	4
Push 7	7,4
Push 8	8,7,4
ADD	15,4
PUSH 10	10,15,4
SUB	5,4
MUL	20

In class example 6



Compare zero-, one-, two-, and three-address machines by writing programs to compute

for each of the four machines. The instructions available for use are as follows:

$$X = (A + B \times C) / (D - E \times F)$$

0 Address	1 Address	2 Address	3 Address
PUSH M	LOAD M	MOVE($x \leftarrow y$)	MOVE ($x \leftarrow y$)
POP M	STORE M	ADD ($x \leftarrow x + y$)	ADD ($x \leftarrow y + z$)
ADD	ADD M	SUB ($x \leftarrow x - y$)	SUB ($x \leftarrow y - z$)
SUB	SUB M	MUL ($x \leftarrow x \times y$)	MUL ($x \leftarrow y \times z$)
MUL	MUL M	DIV ($x \leftarrow x / y$)	DIV ($x \leftarrow y / z$)
DIV	DIV M		

0 Address	1 Address	2 Address	3 Address
PUSH M	LOAD M	MOVE(x←y)	MOVE (x←y)
POP M	STORE M	ADD (x←x+y)	ADD (x←y +z)
ADD	ADD M	SUB (x←x-y)	SUB (x←y -z)
SUB	SUB M	MUL (x←x X y)	MUL (x←y X Z)
MUL	MUL M	DIV (x←x/y)	DIV (x←y/z)
DIV	DIV M		



```

PUSH A
PUSH B
PUSH C
MUL
ADD
PUSH D
PUSH E
PUSH F
MUL
SUB
DIV
POP X

```

```

LOAD E
MUL F
STORE T
LOAD D
SUB T
STORE T
LOAD B
MUL C
ADD A
DIV T
STO X

```

```

MOV R0, E
MUL R0, F
MOV R1, D
SUB R1, R0
MOV R0, B
MOV R0, C
ADD R0, A
DIV R0, R1
MOV X, R0

```

```

MUL R0, E, F
SUB R0, D, R0
MUL R1, B, C
ADD R1, A, R1
DIV X, R0, R1

```

C
B
A

$$X = (A + B \times C) / (D - E \times F)$$

0 Address	1 Address	2 Address	3 Address
PUSH M	LOAD M	MOVE(x←y)	MOVE (x←y)
POP M	STORE M	ADD (x←x+y)	ADD (x←y +z)
ADD	ADD M	SUB (x←x-y)	SUB (x←y -z)
SUB	SUB M	MUL (x←x X y)	MUL (x←y X Z)
MUL	MUL M	DIV (x←x/y)	DIV (x←y/z)
DIV	DIV M		



```

PUSH A
PUSH B
PUSH C
MUL
ADD
PUSH D
PUSH E
PUSH F
MUL
SUB
DIV
POP X

```

```

LOAD E
MUL F
STORE T
LOAD D
SUB T
STORE T
LOAD B
MUL C
ADD A
DIV T
STO X

```

```

MOV R0, E
MUL R0, F
MOV R1, D
SUB R1, R0
MOV R0, B
MOVUP R0, C
ADD R0, A
DIV R0, R1
MOV X, R0

```

```

MUL R0, E, F
SUB R0, D, R0
MUL R1, B, C
ADD R1, A, R1
DIV X, R0, R1

```

BXC
A

$$X = (A + B \times C) / (D - E \times F)$$

0 Address	1 Address	2 Address	3 Address
PUSH M	LOAD M	MOVE(x←y)	MOVE (x←y)
POP M	STORE M	ADD (x←x+y)	ADD (x←y +z)
ADD	ADD M	SUB (x←x-y)	SUB (x←y -z)
SUB	SUB M	MUL (x←x X y)	MUL (x←y X Z)
MUL	MUL M	DIV (x←x/y)	DIV (x←y/z)
DIV	DIV M		



```

PUSH A
PUSH B
PUSH C
MUL
ADD
PUSH D
PUSH E
PUSH F
MUL
SUB
DIV
POP X

```

```

LOAD E
MUL F
STORE T
LOAD D
SUB T
STORE T
LOAD B
MUL C
ADD A
DIV T
STO X

```

```

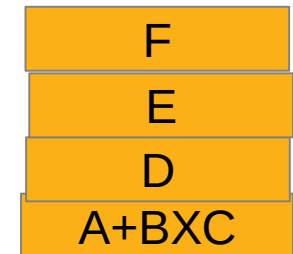
MOV R0, E
MUL R0, F
MOV R1, D
SUB R1, R0
MOV R0, B
MOV R0, C
ADD R0, A
DIV R0, R1
MOV X, R0

```

```

MUL R0, E, F
SUB R0, D, R0
MUL R1, B, C
ADD R1, A, R1
DIV X, R0, R1

```



$$X = (A + B \times C) / (D - E \times F)$$

0 Address	1 Address	2 Address	3 Address
PUSH M	LOAD M	MOVE($x \leftarrow y$)	MOVE ($x \leftarrow y$)
POP M	STORE M	ADD ($x \leftarrow x+y$)	ADD ($x \leftarrow y +z$)
ADD	ADD M	SUB ($x \leftarrow x-y$)	SUB ($x \leftarrow y -z$)
SUB	SUB M	MUL ($x \leftarrow x \times y$)	MUL ($x \leftarrow y \times z$)
MUL	MUL M	DIV ($x \leftarrow x/y$)	DIV ($x \leftarrow y/z$)
DIV	DIV M		



```

PUSH A
PUSH B
PUSH C
MUL
ADD
PUSH D
PUSH E
PUSH F
MUL
SUB
DIV
POP X

```

```

LOAD E
MUL F
STORE T
LOAD D
SUB T
STORE T
LOAD B
MUL C
ADD A
DIV T
STO X

```

```

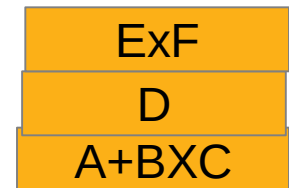
MOV R0, E
MUL R0, F
MOV R1, D
SUB R1, R0
MOV R0, B
MOV R0, C
ADD R0, A
DIV R0, R1
MOV X, R0

```

```

MUL R0, E, F
SUB R0, D, R0
MUL R1, B, C
ADD R1, A, R1
DIV X, R0, R1

```



$$X = (A + B \times C) / (D - E \times F)$$

0 Address	1 Address	2 Address	3 Address
PUSH M	LOAD M	MOVE($x \leftarrow y$)	MOVE ($x \leftarrow y$)
POP M	STORE M	ADD ($x \leftarrow x+y$)	ADD ($x \leftarrow y +z$)
ADD	ADD M	SUB ($x \leftarrow x-y$)	SUB ($x \leftarrow y -z$)
SUB	SUB M	MUL ($x \leftarrow x \times y$)	MUL ($x \leftarrow y \times z$)
MUL	MUL M	DIV ($x \leftarrow x/y$)	DIV ($x \leftarrow y/z$)
DIV	DIV M		



```

PUSH A
PUSH B
PUSH C
MUL
ADD
PUSH D
PUSH E
PUSH F
MUL
SUB
DIV
POP X

```

```

LOAD E
MUL F
STORE T
LOAD D
SUB T
STORE T
LOAD B
MUL C
ADD A
DIV T
STO X

```

```

MOV R0, E
MUL R0, F
MOV R1, D
SUB R1, R0
MOV R0, B
MOV R0, C
ADD R0, A
DIV R0, R1
MOV X, R0

```

```

MUL R0, E, F
SUB R0, D, R0
MUL R1, B, C
ADD R1, A, R1
DIV X, R0, R1

```

D-ExF
A+BX C

$$X = (A + B \times C) / (D - E \times F)$$

0 Address	1 Address	2 Address	3 Address
PUSH M	LOAD M	MOVE($x \leftarrow y$)	MOVE ($x \leftarrow y$)
POP M	STORE M	ADD ($x \leftarrow x+y$)	ADD ($x \leftarrow y +z$)
ADD	ADD M	SUB ($x \leftarrow x-y$)	SUB ($x \leftarrow y -z$)
SUB	SUB M	MUL ($x \leftarrow x \times y$)	MUL ($x \leftarrow y \times z$)
MUL	MUL M	DIV ($x \leftarrow x/y$)	DIV ($x \leftarrow y/z$)
DIV	DIV M		



```

PUSH A
PUSH B
PUSH C
MUL
ADD
PUSH D
PUSH E
PUSH F
MUL
SUB
DIV
POP X

```

```

LOAD E
MUL F
STORE T
LOAD D
SUB T
STORE T
LOAD B
MUL C
ADD A
DIV T
STO X

```

```

MOV R0, E
MUL R0, F
MOV R1, D
SUB R1, R0
MOV R0, B
MOV UP R0, C
ADD R0, A
DIV R0, R1
MOV X, R0

```

```

MUL R0, E, F
SUB R0, D, R0
MUL R1, B, C
ADD R1, A, R1
DIV X, R0, R1

```

$A+B \times C /$
 $D-E \times F$

$$X = (A + B \times C) / (D - E \times F)$$



BITS Pilani
Pilani Campus

Computer Organization & Architecture

Virendra Singh Shekhawat
Department of Computer Science and Information Systems



Module-7 (Lecture-2)

[Ref. Computer Organization and Architecture , 8th Ed. by William Stallings]

Topics: Pipeline Hazards: Data Hazards, Resource Hazards, Control Hazards, RISC
Pipeline

Four Stage Pipeline

- **Fetch:** Read the instruction from the memory
- **Decode:** Decode the instruction and fetch the source operand(s)
- **Execute:** Perform the ALU operation
- **Write:** Store the result in the destination location
- **Note:**
 - *Each stage in pipeline is expected to complete its operation in one clock cycle; hence the clock period should be sufficiently long to complete the task being performed in any stage.*

Stalls/Hazards in Pipeline

- Any condition that causes the pipeline to stall is called Hazard
- Data Hazards
 - Any condition in which either the source or the destination operands of an instruction are not available at the expected time in the pipeline

Data Hazards

- Read After Write Hazard

I1: $R1 = R1 + R2$

I2: $R3 = R1 + 3$

Solution for RAW Dependency

- Hardware Solution
 - Operand Forwarding
- Software Solution
 - Inserting NOP (No- Operation) instructions between I1 and I2

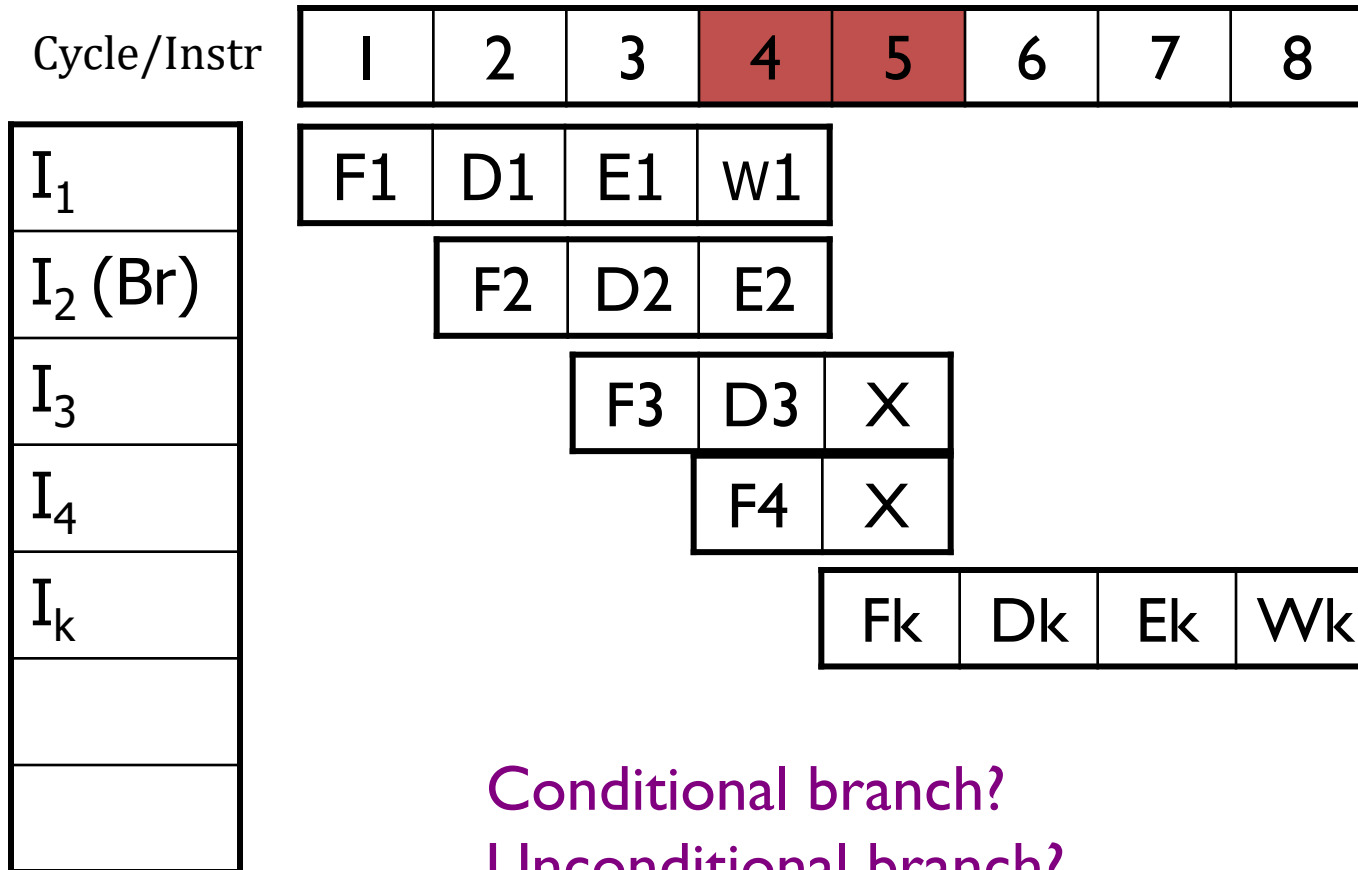
Resource Hazards

- One instruction may need to access memory as part of the **Write stage** while another instruction is being **Fetch**
 - If instruction and data are in same cache unit there is a conflict
 - Called as Structural or Resource conflict or **Resource Hazard**

Instruction/Control Hazards

- If the required instruction is not available-
 - Example: Cache miss for instruction fetch, Branch instruction
 - Called as control hazards or Instruction hazards

Example: Instruction Hazard





Handling Unconditional Branch

- Branch will always be taken
- Compute branch target address as soon as possible
 - Needs extra hardware to do it at Decode stage (e.g. Four stage pipeline)
- Many processors employ fetch unit that can fetch instructions before they are needed

Handling Conditional Branches-1



- The decision to the branch can't be made until the execution of that instruction
- **Multiple Streams**
 - Have two pipelines
 - Prefetch each branch into a separate pipeline
 - Use appropriate pipeline
- **Drawbacks**
 - Leads to bus & register contention
 - Multiple branches lead to further pipelines being needed
 - Used in IBM 370/168

Handling Conditional Branches-2



- **Pre-fetch Branch Target**

- Studies suggest that conditional branches are taken more than 50% of time
- Branch target prefetching gives better result
- Used by IBM 360/91

- **Loop Buffer**

- Very fast memory maintained by fetch stage of pipeline keeps n most recently fetched instructions, sequentially
- Check buffer before fetching from memory
- Very good for small loops or jumps
- Used in CRAY-1

Review Questions

- Consider the following sequence of instructions:
I1: add R1, R0, #20 ($R1 \leftarrow R0 + 20$)
I2: mul R2, R3, #2 ($R2 \leftarrow R3 * 2$)
I3: and R4, R1, R2 ($R4 \leftarrow R1 \text{ and } R2$)
I4: add R5, R4, R2 ($R5 \leftarrow R4 + R2$)
- These instructions are executed in a computer that has a four stage pipeline (**F**etch, **D**ecode, **E**xecute, **W**rite) discussed in this class. Assume that all stages for all instructions requires one cycle each, except the **E**xecution stage of multiply instruction which requires two cycles.
- Draw a diagram to describe the operation being performed by each pipeline stage during each clock cycle. Show the stalls in pipeline, if any.

Solution

Sol:

Instr/Clock Cycles	1	2	3	4	5	6	7	8	9	10	11	12
I1	F1	D1	E1	W1								
I2		F2	D2	E2	E2	W2						
I3			F3	X	X	X	D3	E3	W3			
I4							F4	X	X	D4	E4	W4

I1: add R1, R0, #20 ($R1 \leftarrow R0 + 20$)

I2: mul R2, R3, #2 ($R2 \leftarrow R3 * 2$)

I3: and R4, R1, R2 ($R4 \leftarrow R1 \text{ and } R2$)

I4: add R5, R4, R2 ($R5 \leftarrow R4 + R2$)

Prediction Based Algorithms for Handling Conditional Branches

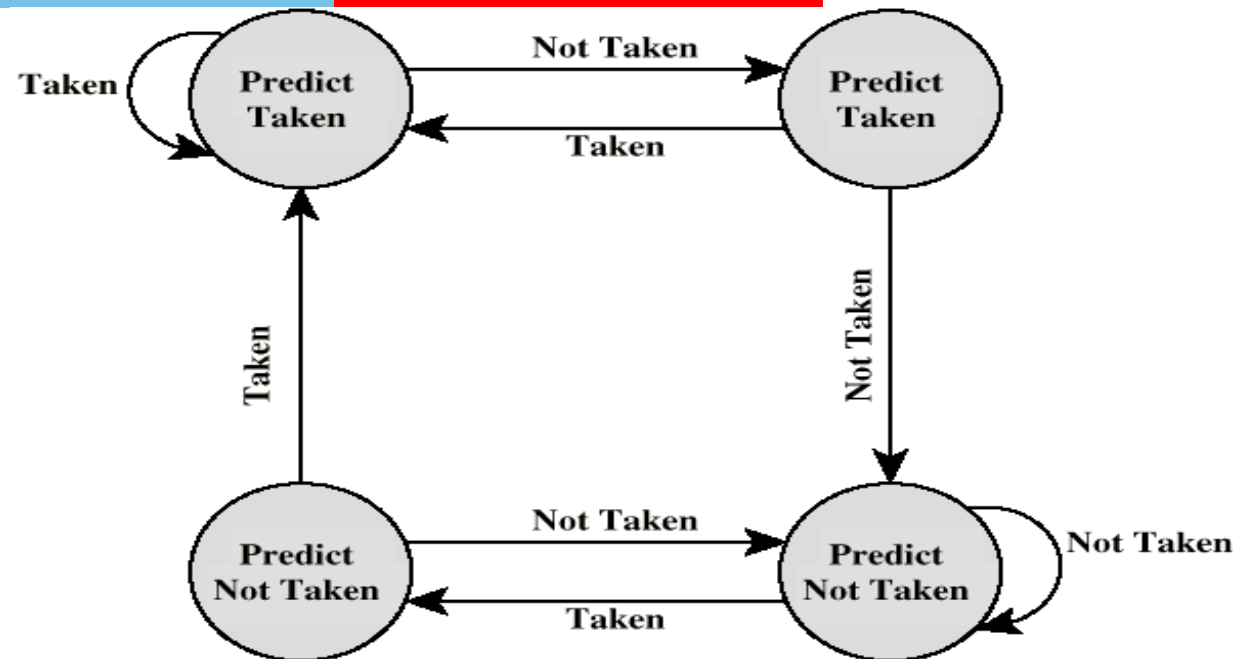


- **Predict never taken**
 - Assume that jump will not happen. Always fetch next instruction
- **Predict always taken**
 - Assume that jump will happen. Always fetch target instruction
- **Predict by opcode**
 - Some instructions are more likely to result in a jump than others. Can get up to 75% success
- **Taken/Not taken switch**
 - Based on previous history.

1-Bit Branch Predictor



Two Bit Branch Predictor



Delayed Branch

- Do not take jump until you have to...
- Rearrange instructions
- Example

LOOP	Shift_left	R1
	Decrement	R2
	Brp	LOOP
NEXT	Add	R1,R3

LOOP	Decrement	R2
	Brp	LOOP
	Shift_left	R1
NEXT	Add	R1,R3

Reordered Instructions

The location following a branch instruction is called a branch delay slot
The objective is to utilize the slot by putting useful instruction.

RISC Pipelining

- Most instructions are register to register
 - I: Instruction fetch
 - E: Execute
 - ALU operation with register input and output
- For load and store
 - I: Instruction fetch
 - E: Execute
 - Calculate memory address
 - D: Memory
 - Register to memory or memory to register operation

RISC Pipeline Example

Load $rA \leftarrow M$
 Load $rB \leftarrow M$
 Add $rC \leftarrow rA + rB$
 Store $M \leftarrow rC$
 Branch X

I	E	D									
			I	E	D						
						I	E				
								I	E	D	
										I	E

Load $rA \leftarrow M$
 Load $rB \leftarrow M$
 NOOP
 Add $rC \leftarrow rA + rB$
 Store $M \leftarrow rC$
 Branch X
 NOOP

I	E	D					
	I	E	D				
		I	E				
			I	E			
				I	E	D	
					I	E	
						I	E

Four Stage RISC Pipelining

- E stage usually involves an ALU operation, it may be longer. So we can divide into two stages
 - E1: Register file read
 - E2: ALU operation and register write

```

Load    rA ← M
Load    rB ← M
NOOP
NOOP
Add      rC ← rA + rB
Store   M ← rC
Branch X
NOOP
NOOP
    
```

I	E ₁	E ₂	D							
	I	E ₁	E ₂	D						
		I	E ₁	E ₂						
			I	E ₁	E ₂					
				I	E ₁	E ₂				
					I	E ₁	E ₂	D		
						I	E ₁	E ₂		
							I	E ₁	E ₂	
								I	E ₁	E ₂

Summary



- Pipeline Hazards
 - Data Hazards
 - Resource Hazards
 - Instruction Hazards
 - Branch Prediction Algorithms
- RISC Pipeline as an Example

Thank You!



BITS Pilani
Pilani Campus

Computer Organization & Architecture

Virendra Singh Shekhawat
Department of Computer Science and Information Systems



Module-7 (Lecture-3)

[Ref. Computer Organization and Architecture , 8th Ed. by William Stallings]

Topics: Instruction Level and Machine Level Parallelism, Superscalar Pipeline and Superscalar Processor Design Elements, Hazards and Solutions



Introduction

- Scalar pipeline improves performance
 - By overlapping various instruction stages
 - Unable to exploit Instruction Level Parallelism (ILP) completely!
- A new term called **Superscalar** is coined (1987) to exploit ILP of a program
-



What is Superscalar?

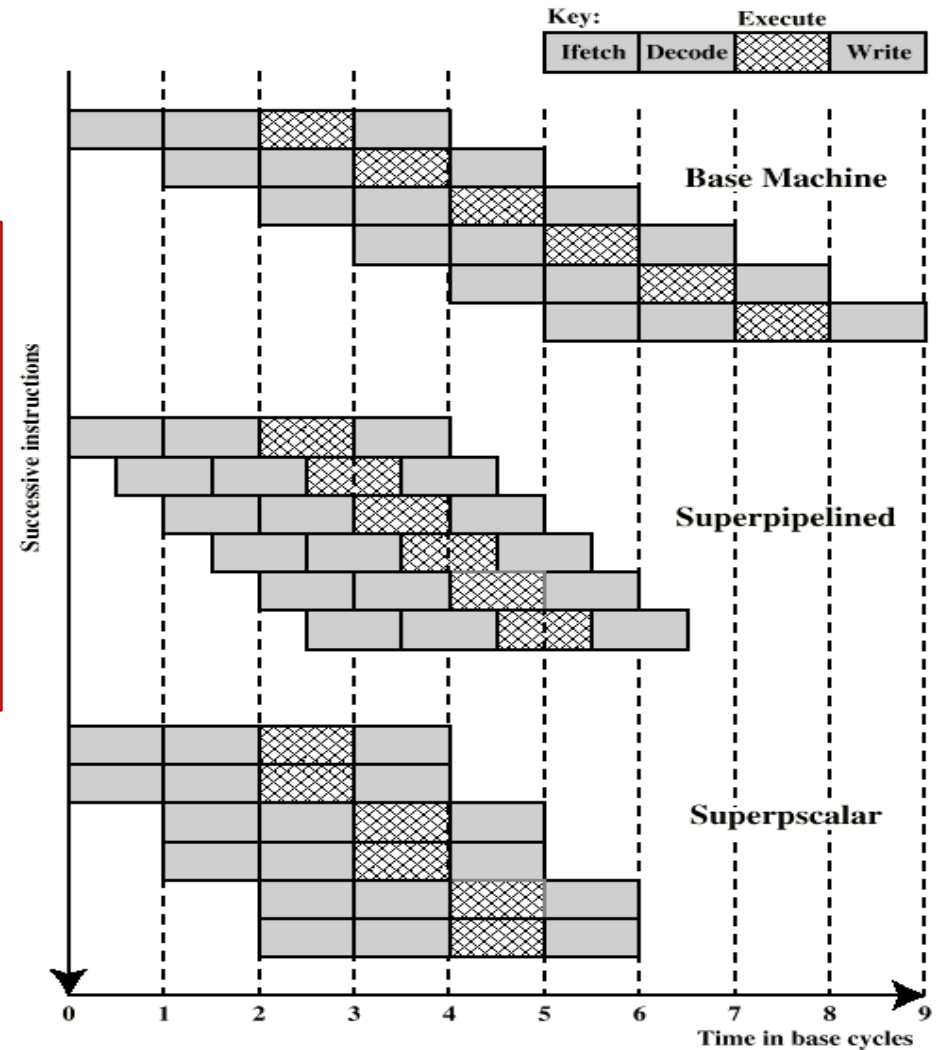
- Multiple independent instruction pipelines are used to execute instructions independently and concurrently
- Common instructions (arithmetic, load/store, conditional branch) initiated simultaneously and executed independently
- Improve the performance of the execution of instructions operated on scalar quantities
- Equally applicable to RISC & CISC but in practice usually RISC

Super Scalar vs. Super Pipeline



Super Pipeline

- Many pipeline stages need less than half a clock cycle
- Double internal clock speed gets two tasks per external clock cycle





Limitations

- Superscalar approach depends on the ability to execute multiple instructions in parallel
- How to maximize the ILP
 - Compiler based optimization
 - Hardware techniques
- Limited by following dependencies or hazards
 - Instruction / Procedural
 - Resource
 - Data Hazards
 - RAW, WAR, WAW



Procedural/Instruction Hazards

- Can not execute instructions after a branch in parallel with instructions before a branch
- Also, if instruction length is not fixed, instructions have to be decoded (partially) to find out how many fetches are needed
 - This prevents simultaneous fetches
- So superscalar techniques are applicable to a RISC or RISC like architecture

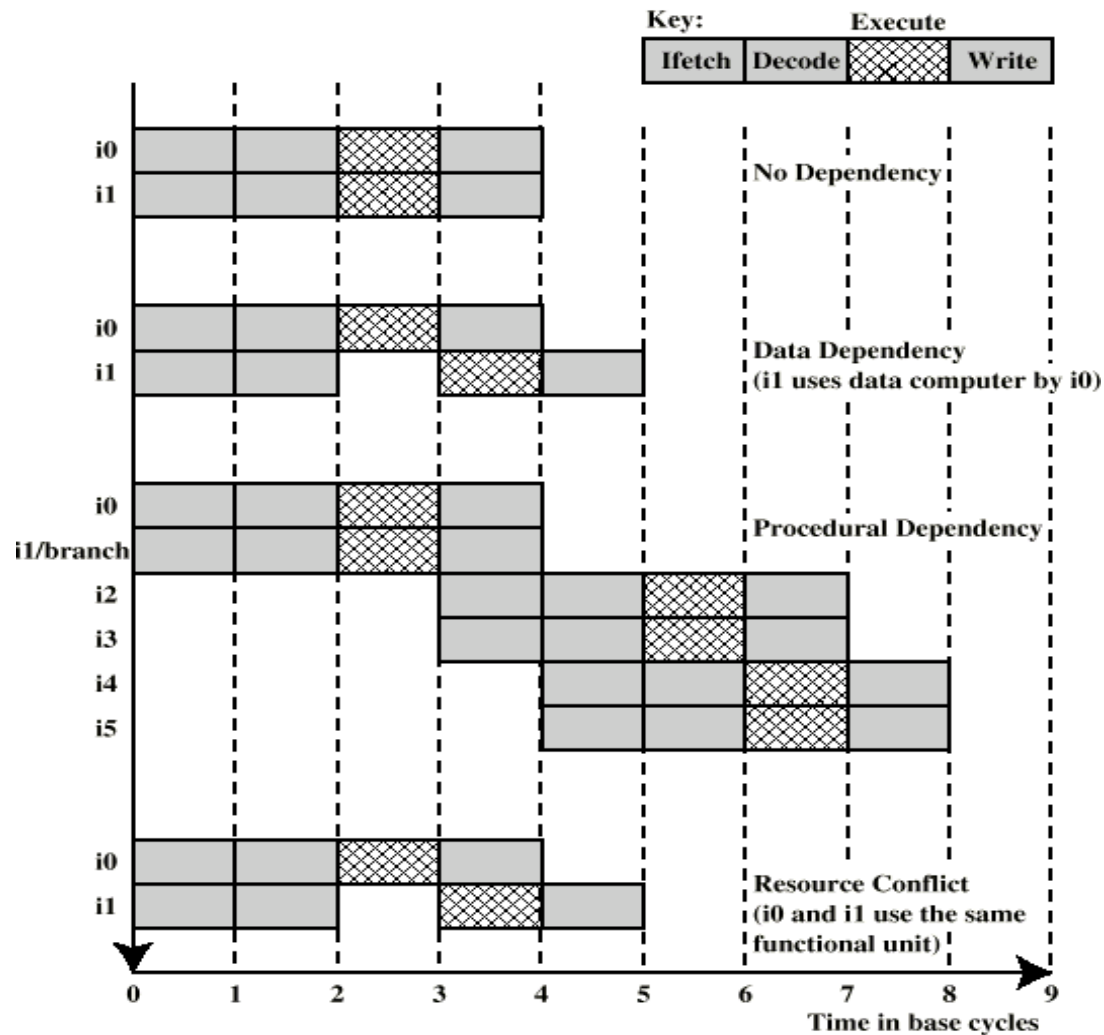
Resource Conflict

- Two or more instructions requiring access to the same resource at the same time
 - e.g. two arithmetic instructions
- Solution
 - Can duplicate resources. e.g. have two arithmetic units

Data Hazards/Conflicts

- Example Program
 - $R3 = R3 + R5$; (I1)
 - $R4 = R3 + 1$; (I2)
 - $R3 = R5 + 1$; (I3)
 - $R7 = R3 + R4$ (I4)
- Identify the hazards among above instructions
 - I1 and I2 ?
 - I3 and I4 ?
 - I2 and I3 ?
 - I1 and I3 ?

Effect of Hazards/ Dependencies



Superscalar Processor Design Issues



- **Instruction level parallelism**
 - If instructions in a sequence are independent, execution can be overlapped
 - Degree of instruction level parallelism is governed by data and procedural dependency
- **Machine Parallelism**
 - Ability to take advantage of instruction level parallelism
 - Governed by number of parallel pipelines
- **Note**
 - *A program may not have enough instruction level parallelism to take full advantage of machine parallelism*



Instruction Issue Policy

- **Instruction Issue**
 - Refer to the process of initiating instruction execution in the processor's functional units
 - Occurs when instruction moves from the **decode stage** to the first **execute stage**
- **Instruction Issue Policy**
 - Refer to the protocol used to issue instructions
 - Processor look ahead to locate instructions that can be brought into the pipeline and executed
- **Ordering issues**
 - Order in which instructions are **fetches, executed and change contents of registers and memory**



- In-order issue with in-order completion
- In-order issue with out-of-order completion
- Out-of-order issue with out-of-order completion



Example: Issue Policies-1

- Assumptions

- A super Scalar Pipeline capable of fetching and decoding two instructions at a time
- The next two instructions must wait until the pair of decode pipeline stages has cleared
- Having three separate functional units (two integer arithmetic and one floating point arithmetic)
- Two instances of the write back pipeline stage

Example: Issue Policies-2

- Constraints
 - I1 requires two cycles to execute
 - I3 and I4 conflict for the same functional unit
 - I5 depends on the value produced by I4
 - I5 and I6 conflict for a functional unit

In-Order Issue In-Order Completion



Decode		Execute			Write		Cycle
11	12						1
13	14	11	12				2
13	14	11					3
	14			13	11	12	4
15	16			14			5
	16		15		13	14	6
			16				7
					15	16	8

In-Order Issue Out-of-Order Completion



- It improves the performance of instructions that requires multiple cycles
 - E.g. I2 is allowed to run to completion prior to I1. Results I3 completed earlier i.e. saving one cycle (Better than previous)

Decode		Execute			Write		Cycle
I1	I2						1
I3	I4	I1	I2				2
	I4	I1		I3	I2		3
I5	I6			I4	I1	I3	4
	I6		I5		I4		5
			I6		I5		6
					I6		7

Out-of-Order Issue Out-of-Order Completion



- Decouple decode stage from execution stage by introducing a buffer called *instruction window*
- Since instructions have been decoded, processor can look ahead

Decode		Window	Execute			Write		Cycle
11	12							1
13	14	11,12	11	12				2
15	16	13,14	11		13	12		3
		14,15,16		16	14	11	13	4
		15		15		14	16	5
						15		6

Register Renaming

- When out of order techniques are used
 - WAW and WAR hazards occur because register contents may not reflect the correct ordering from the program
 - WAW and WAR dependencies are known as storage conflicts
- Compilers use optimized registers allocation which leads to more storage conflicts
- Register Renaming is used to deal with this
 - Similar to resource duplication

Register Renaming

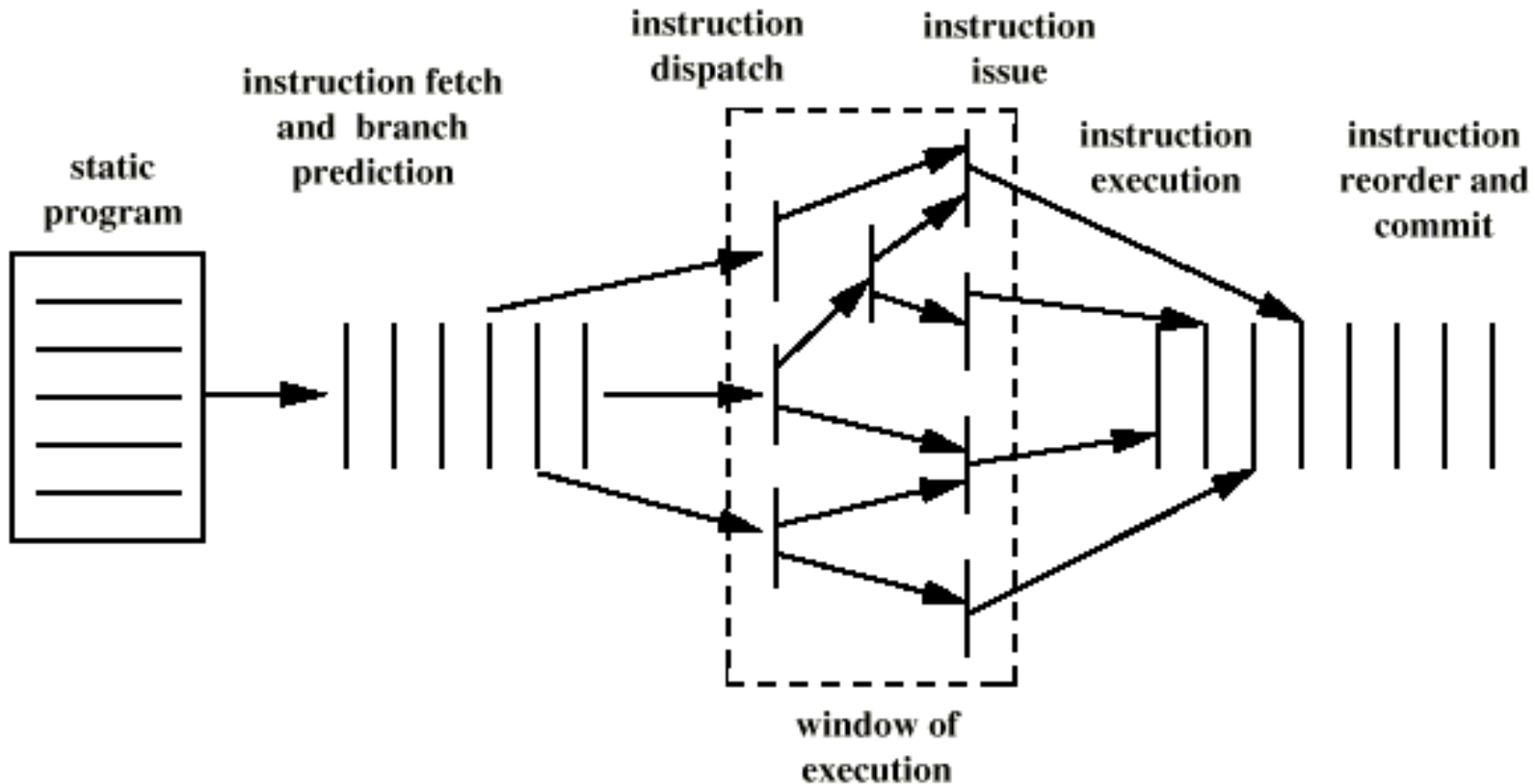
- $R3b := R3a + R5a$ (I1)
- $R4b := R3b + 1$ (I2)
- $R3c := R5a + 1$ (I3)
- $R7a := R3c + R4b$ (I4)

$R3 := R3 + R5;$ (I1)
 $R4 := R3 + 1;$ (I2)
 $R3 := R5 + 1;$ (I3)
 $R7 := R3 + R4$ (I4)

- Inference

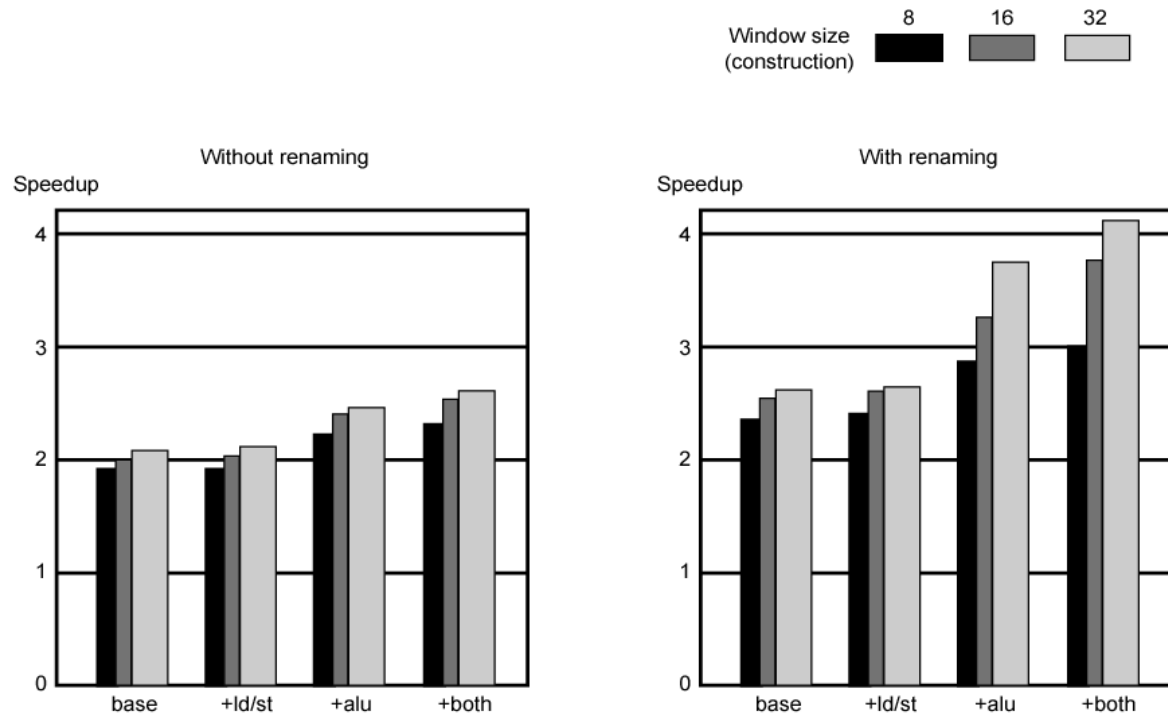
- The same original register reference in several different instructions may refer to different actual registers, if different values are needed

Superscalar Execution



Machine Parallelism

- To enhance the performance of the Super Scalar Processors
 - Duplication of Resources, Out of order instruction issue
 - Register Renaming



Review Questions

- How instruction level parallelism is different from machine parallelism?
- Identify the various data hazards among the following instructions?
 - I1: $R1 = 20$
 - I2: $R1 = R3 + R4$
 - I3: $R2 = R4 - 10$
 - I4: $R4 = R1 + R2$
 - I5: $R2 = R1 + 40$



Summary

- Instruction level Parallelism vs. Machine level parallelism
- Superscalar Pipeline's Performance Limitation Factors
- Superscalar Processor Design Issues
- Register Renaming
 - Eliminates WAR and WAW hazards

Thank You!



BITS Pilani
Pilani Campus



COA The memory

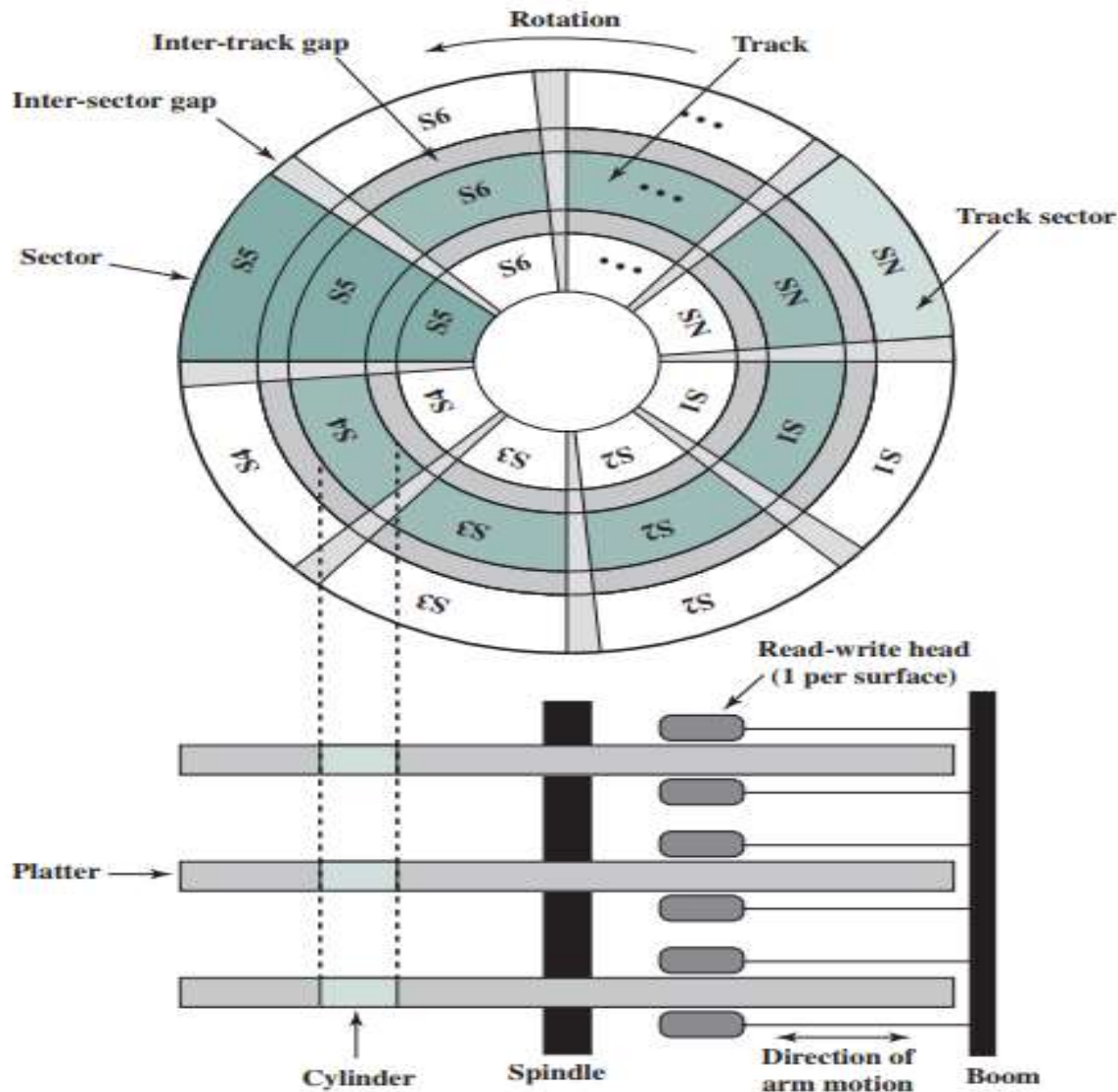
- ❑ Interesting information
- ✓ the 1st drive with a capacity 1 GByte , produced
- ✓ by IBM in 1980
- ✓ weighed 250 kg
- ✓ cost \$40,000
- ✓ size of a kitchen appliance

Magnetic disk

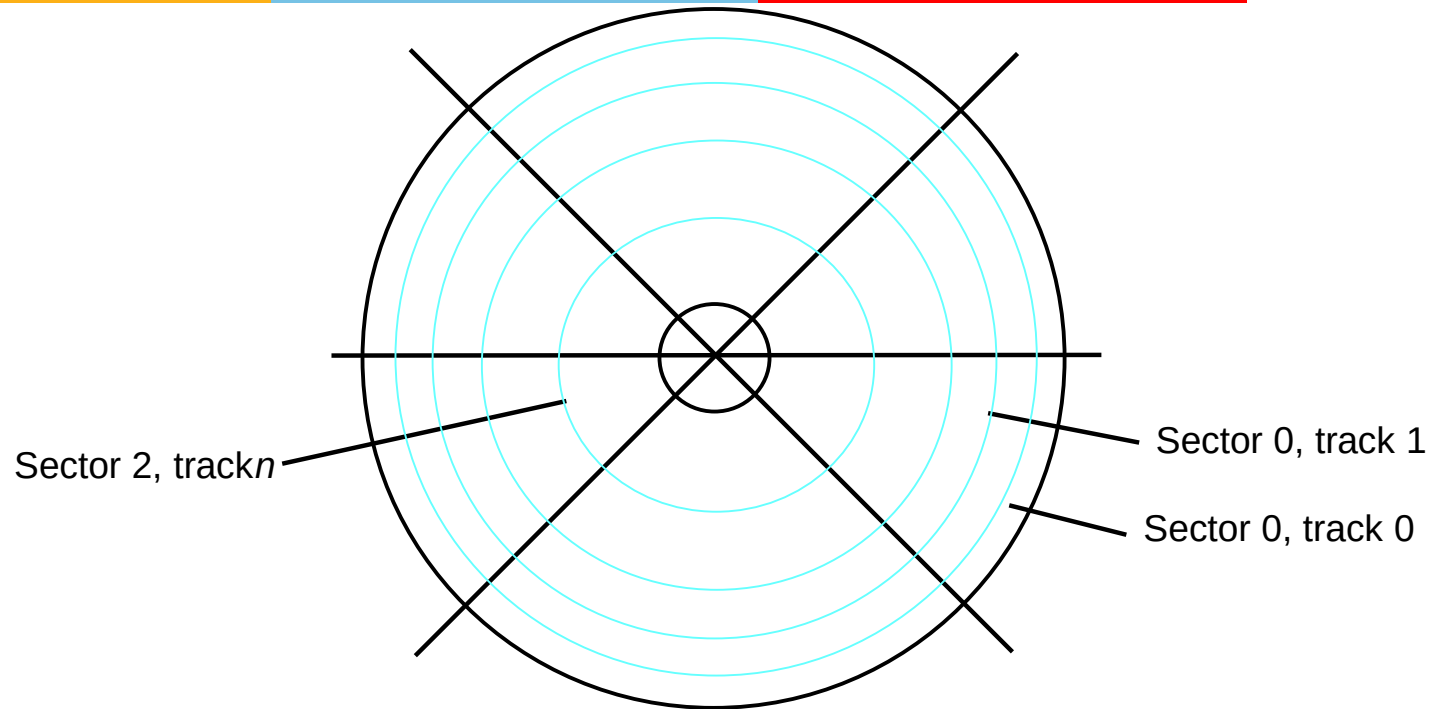


- ❑ A disk is a circular platter constructed of nonmagnetic material, called the substrate, coated with a magnetizable material.
- ❑ Traditionally, the substrate has been an aluminum or aluminum alloy material. More recently, glass substrates have been introduced.
 - ✓ The glass substrate has a number of benefits
 - ✓ Improvement in the uniformity of the magnetic film surface to increase disk reliability
 - ✓ A significant reduction in overall surface defects to help reduce read-write errors
 - ✓ Better stiffness to reduce disk dynamics
 - ✓ Greater ability to withstand shock and damage

Disk Data Layout



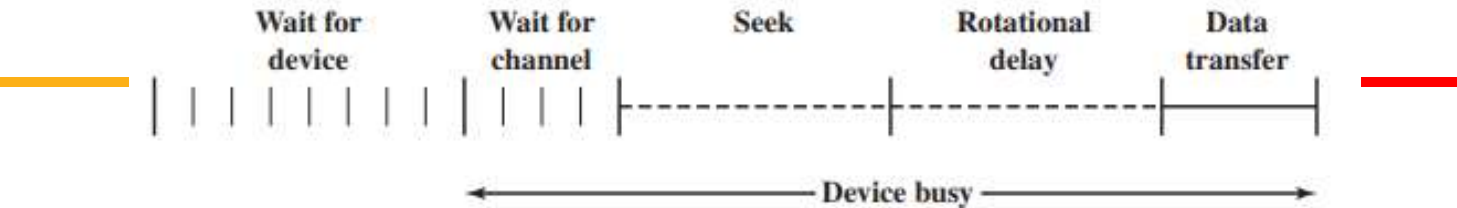
Organization of Data on a Disk



Organization of one surface of a disk.

The set of corresponding tracks on all surfaces of a stack of disk form a cylinder
The data are accessed by specifying the surface number, the track and the sector number

Disk Performance Parameter



- ❑ Seek time: is the time required to move the disk arm to the required track.
- ❑ Rotational delay: Disks, other than floppy disks, rotate at speeds ranging from 3600 rpm (for handheld devices such as digital cameras) up to, as of this writing, 20,000 rpm;
- ❑ Transfer time : The transfer time to or from the disk depends on the rotation speed of the disk $T = b / rN$

T = transfer time ; b = number of bytes to be transferred; N = number of bytes on a track ; r = rotation speed, in revolutions per second

$$T_{\text{total}} = T_s + 1/2r + b/rN$$

A timing comparison



Consider a disk with an advertised average seek time of 4 ms, rotation speed of 15,000 rpm, and 512-byte sectors with **500 sectors per track**. Suppose that we wish to read a file consisting of 2500 sectors for a total of 1.28 Mbytes

First, let us assume that the file is stored as compactly as possible on the disk. That is, the file occupies all of the sectors on 5 adjacent tracks (5 tracks * **500 sectors/track** = 2500 sectors). This is known as sequential organization

Average seek 4 ms; Average rotational delay 2 ms

Read 500 sectors = $b/rN = (512 \times 500 \times 60) / (15000 \times 500 \times 512) = 4 \text{ ms}$; Thus Total = 10 ms

Suppose that the remaining tracks can now be read with essentially no seek time.

That is, the I/O operation can keep up with the flow from the disk. Then, at most, we need to deal with rotational delay for the four remaining tracks. Thus each successive track is read in $2 + 4 = 6 \text{ ms}$.

To read the entire file, Total time = $10 + (4 * 6) = 34 \text{ ms} = 0.034 \text{ seconds}$

A timing comparison



Consider a disk with an advertised average seek time of 4 ms, rotation speed of 15,000 rpm, and 512-byte sectors with 500 sectors per track. Suppose that we wish to read a file consisting of 2500 sectors for a total of 1.28 Mbytes

Now let us calculate the time required to read the same data using random access rather than sequential access;

that is, accesses to the sectors are distributed randomly over the disk.

For each sector, we have

Average seek 4 ms

Rotational delay 2 ms

Read 1 sector = $b/rN = (512 \times 60) / (15000 \times 500 \times 512) = 0.008$ ms

Which is 6.008 ms

Total time = $2500 \times 6.008 = 15,020$ ms = 15.02 second



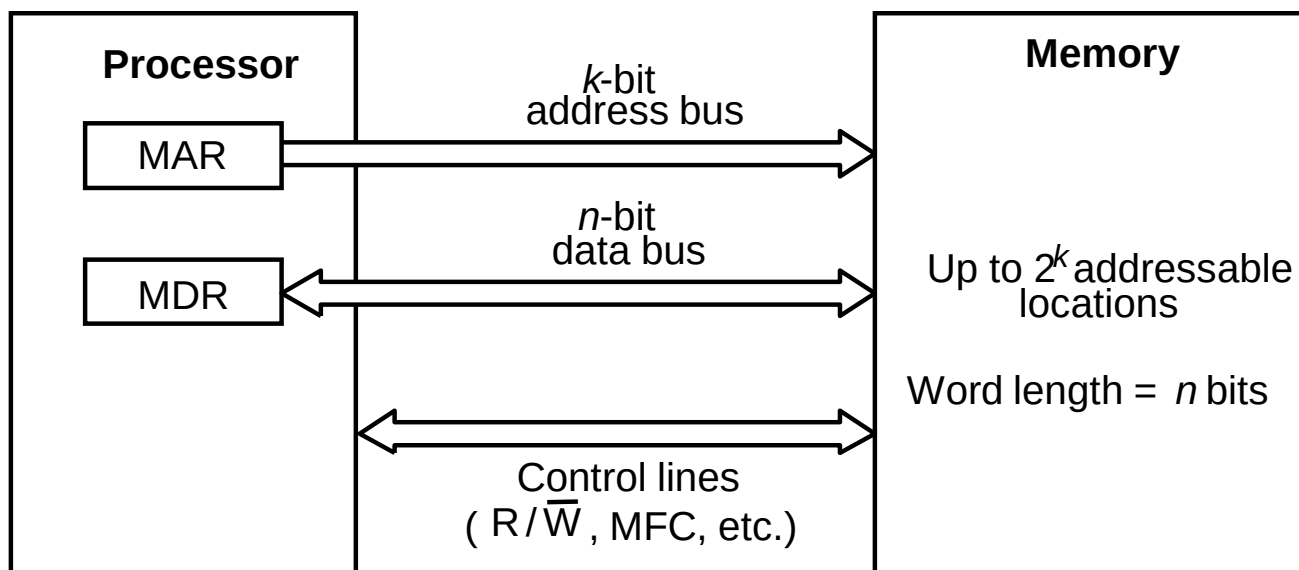
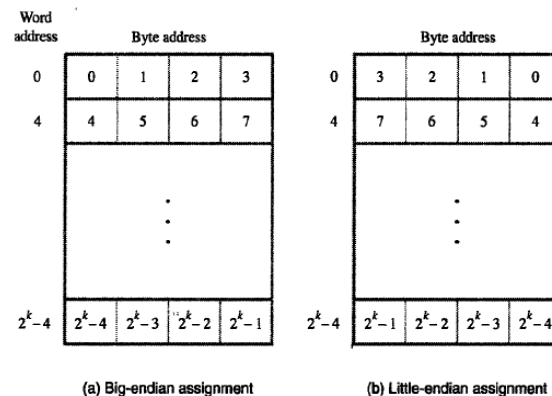
Access Time

- ❑ Seek Time: 5 to 8 ms(Time required to move the read/write head to proper track)
- ❑ Rotational delay or latency time(time to reach the correct sector over the track:
 - ✓ On average This is the time for half a rotation of disk
- ❑ Access Time is the sum of these delays
- ❑ Disk capacity(of a 3.5 inch diameter)
 - ✓ 20 Data recording surface
 - ✓ 15000 tracks / surface
 - ✓ 400 sectors per track
 - ✓ each sector contains 512 bytes of data
 - ✓ $20 \times 15000 \times 400 \times 512 \cong 60 \times 10^9 = 60 \text{ GBytes}$
- ❑ Access time
 - ✓ Typical Average seek time 6 ms
 - ✓ rotate at 10,000 rpm so latency = $\frac{60}{10000 \times 2} = 3ms$
 - ✓ Access time = 9ms

Some basic concepts



- Maximum size of the Main Memory is determined by the addressing scheme
- byte-addressable(big endian or... Little endian)
- CPU-Main Memory Connection



Big Endian or Little Endian

Word address	Byte address			
0	0	1	2	3
4	4	5	6	7
2^k-4	2^k-4	2^k-3	2^k-2	2^k-1

(a) Big-endian assignment

	Byte address			
0	3	2	1	0
4	7	6	5	4
2^k-4	2^k-1	2^k-2	2^k-3	2^k-4

(b) Little-endian assignment

16 bit address capable of $2^{16} = 64k$ Memory locations

What about with 32 bit instructions?

$2^{32} = 4$ G memory locations

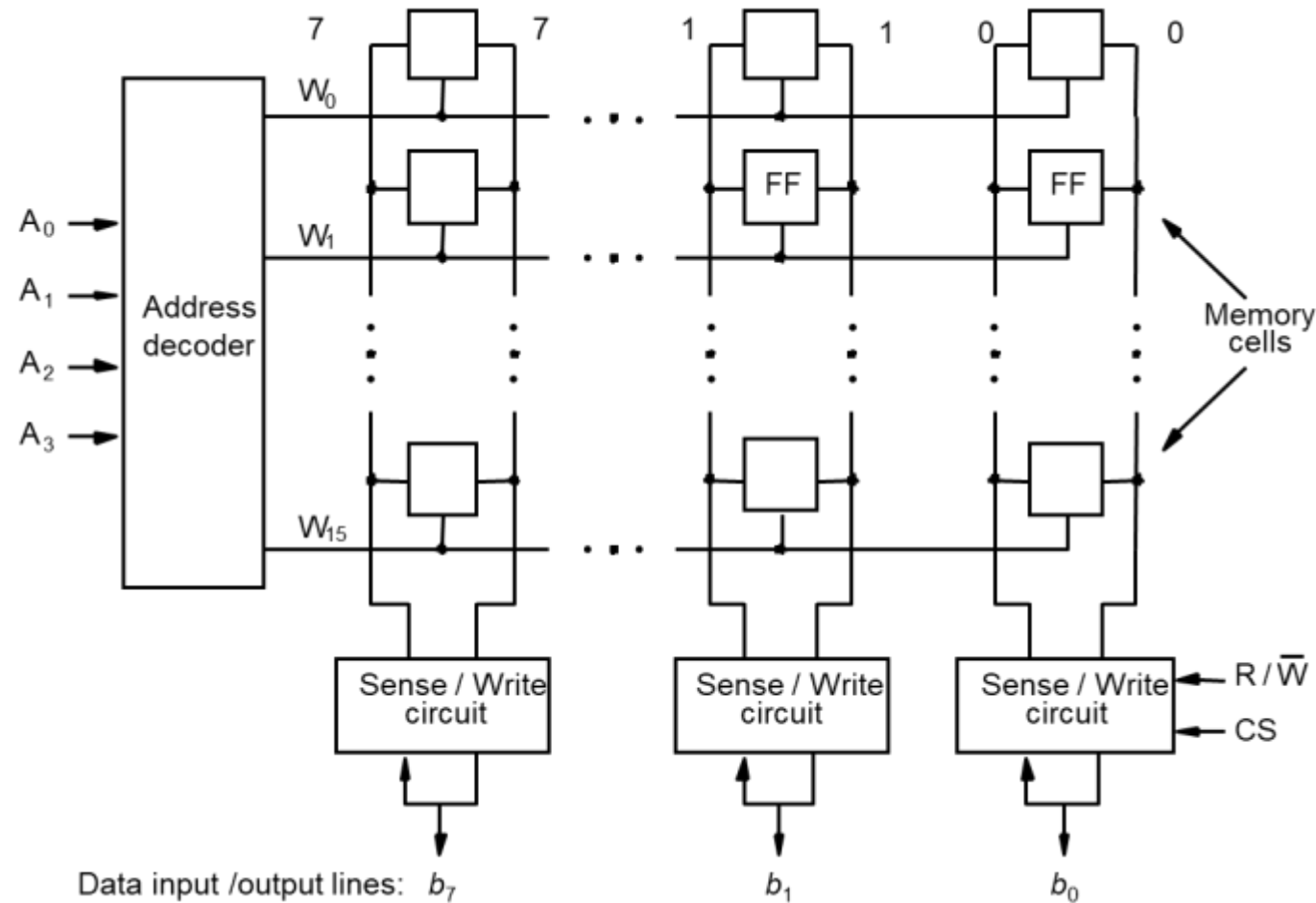
Memory timing definitions



- ❑ Memory Access time: The time between initiation of a task and its completion for example time elapsed between Read and MFC (memory function complete)
- ❑ Memory Cycle time: the minimum time delay between two successive memory operations , eg: the time between two successive write operations
- ❑ Memory cycle time is usually larger than the Memory access time

Internal organization of memory chips

16 X 8 organization





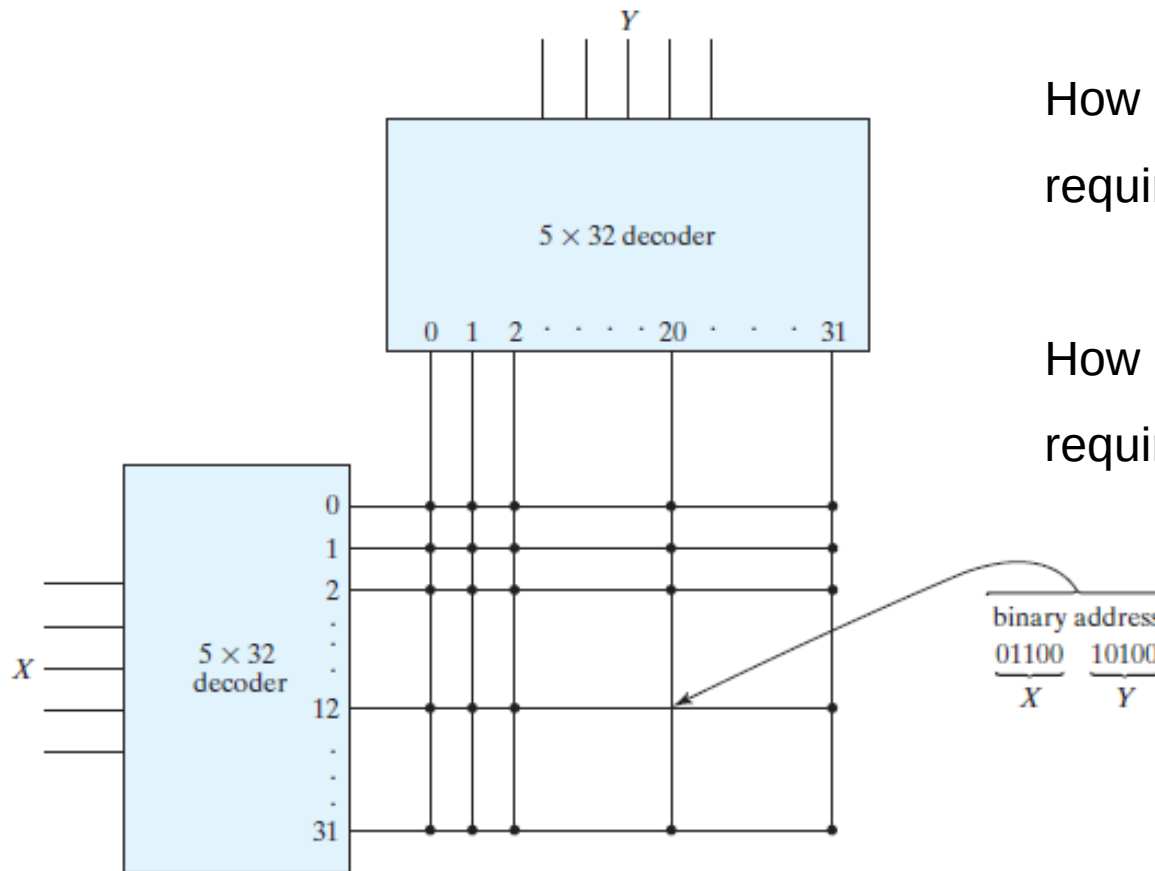
- Each memory cell can hold one bit of information.
- Memory cells are organized in the form of an array.
- One row is one memory word.
- All cells of a row are connected to a common line, known as the “word line”.
- Word line is connected to the address decoder.
- Sense/write circuits are connected to the data input/output lines of the memory chip.
- The memory circuit stores 128 bits
- Requires 14 external connections for address data and control lines
- & another two for Vdd and Grnd

Address is 404 for a 1K-word memory.

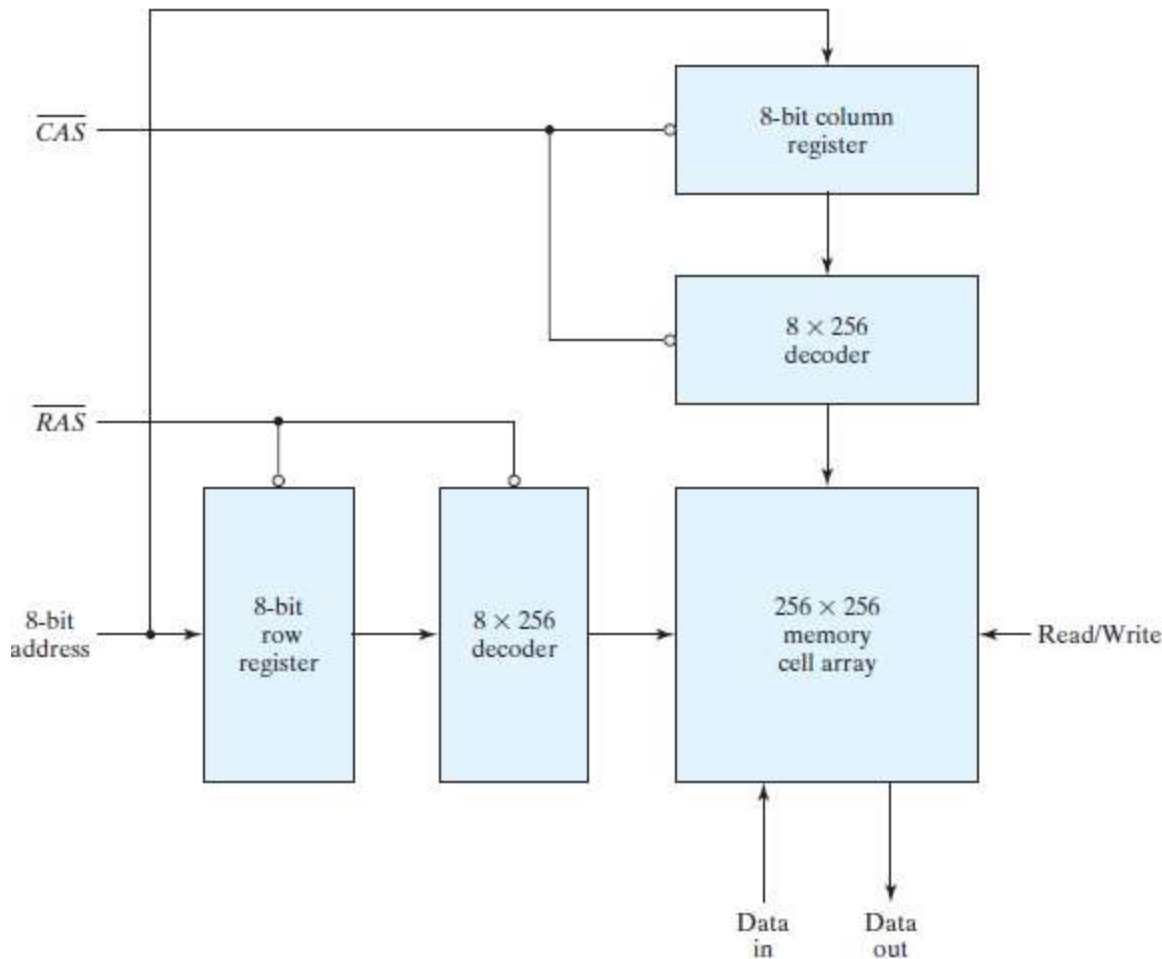
Comparison wrt to Decoder requirement

How many AND(NAND) gates required for 10 to 1k decoder?

How many AND(NAND) gates required for 5 to 32k decoder?



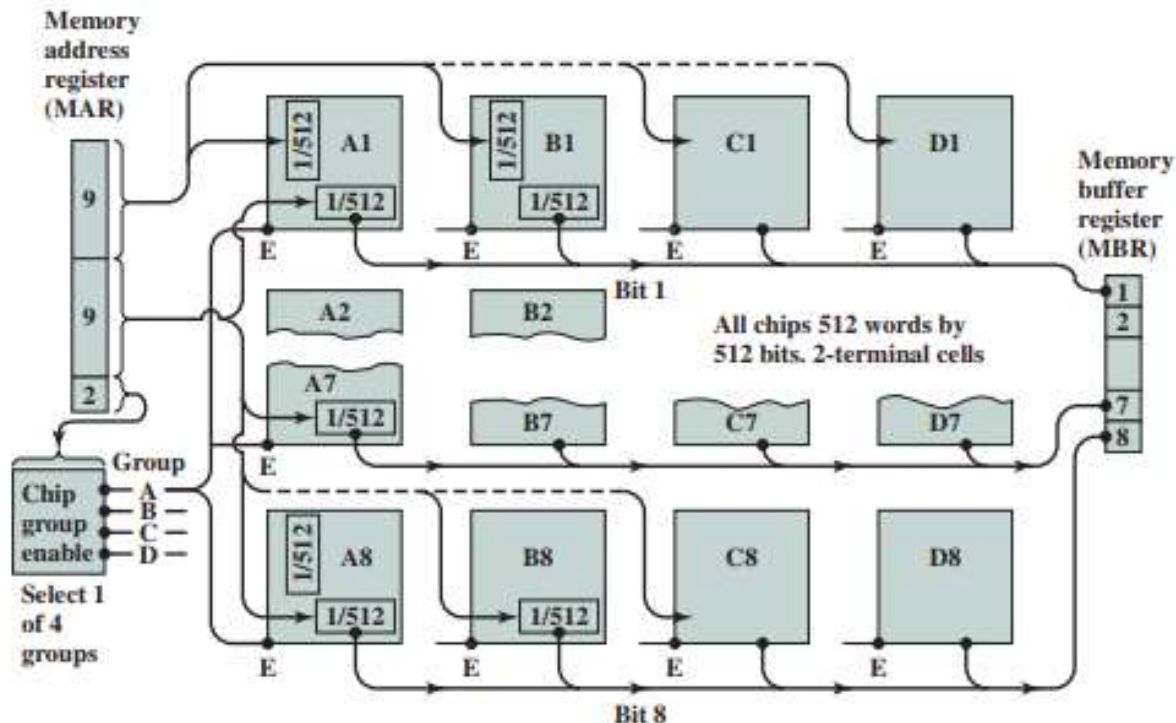
Address multiplexing for a 64K DRAM



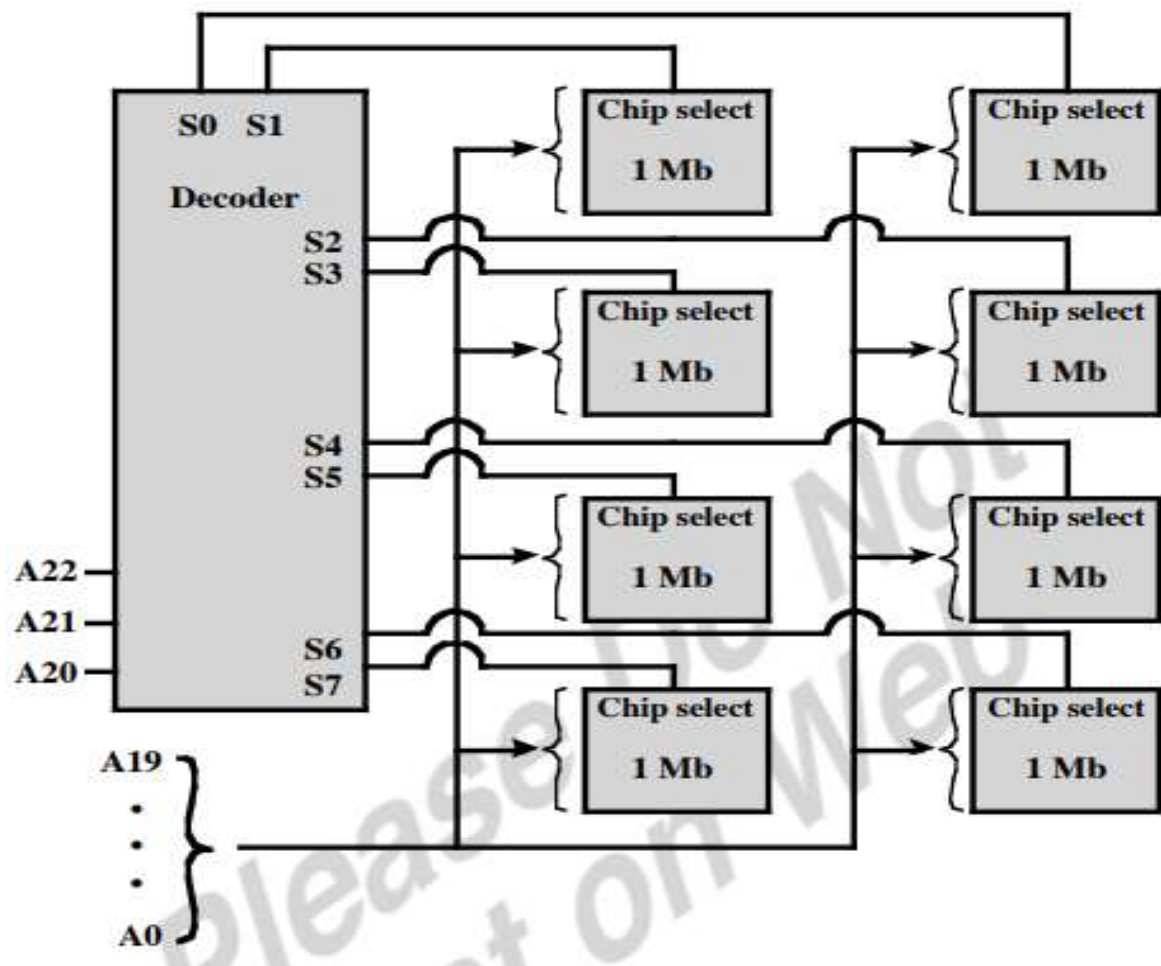
- ✓ After a time equivalent to the **settling time of the row selection**, RAS goes back to the 1 level.
- ✓ **Registers** are used to store the addresses of the row and column.
- ✓ **CAS** must **go back to the 1 level** before initialing another memory operation.

Memory organisation

Figure indicates how to construct a module of chips that can store 1 MB based on a group of four 256-Kbyte chips. Let's say this module of chips is packaged as a single 1-MB chip, where the word size is 1 byte. Give a high-level chip diagram of how to construct an 8-MB computer memory using eight 1-MB chips. Be sure to show the address lines in your diagram and what the address lines are used for.



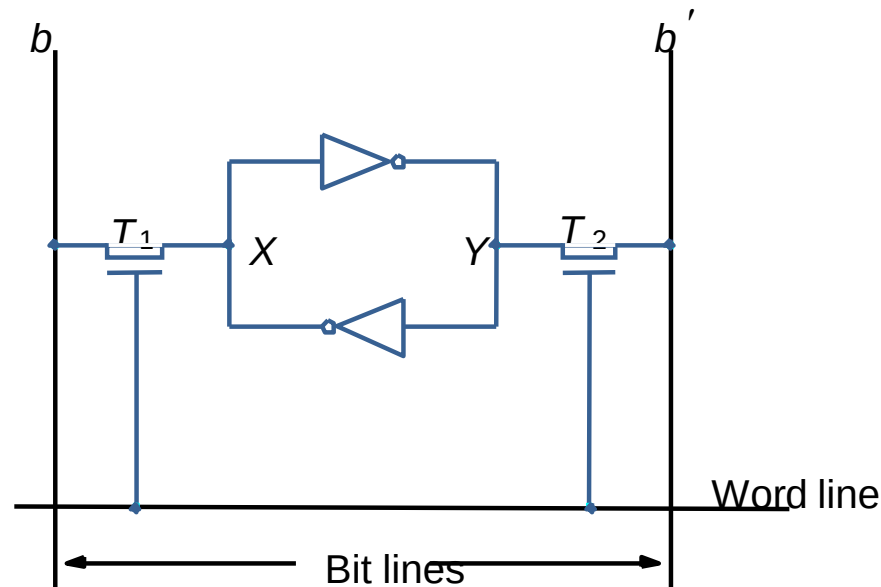
Memory organisation



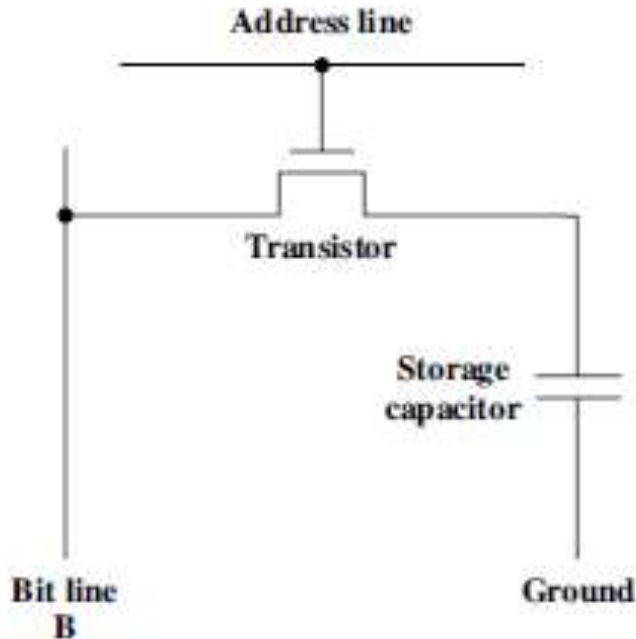
SRAM Cell



- Two transistor inverters are cross connected to implement a basic flip-flop.
- The cell is connected to one word line and two bits lines by transistors T1 and T2
- When word line is at ground level, the transistors are turned off and the latch retains its state
- Read operation: In order to read state of SRAM cell, the word line is activated to close switches T1 and T2. Sense/Write circuits at the bottom monitor the state of b and b'



DRAMS



- ❑ The address line is activated when the bit value from this cell is to be read or written
- ❑ The transistor acts as a switch
- ❑ In a read operation the sense amplifier connected to bit line decides whether the voltage is above some threshold
- ❑ Then it drives the bit line to the full voltage representing 1
- ❑ for zero the bit line is grounded thus discharging the capacitor completely



- **Static RAMs (SRAMs):**
 - Consist of circuits that are capable of retaining their state as long as the power is applied.
 - Volatile memories, because their contents are lost when power is interrupted.
 - Access time of static RAMs are in the range of few nanoseconds.
 - However, the cost is usually high.
- **Dynamic RAMs (DRAMs):**
 - Do not retain their state indefinitely. So called dynamic
 - Contents must be periodically refreshed.
 - Contents may be refreshed while accessing them for reading.



DRAM operation

Consider a dynamic RAM that must be given a refresh cycle 64 times per ms.
Each refresh operation requires 150 ns; a memory cycle requires 250 ns.

What percentage of the memory's total operating time must be given to refreshes?

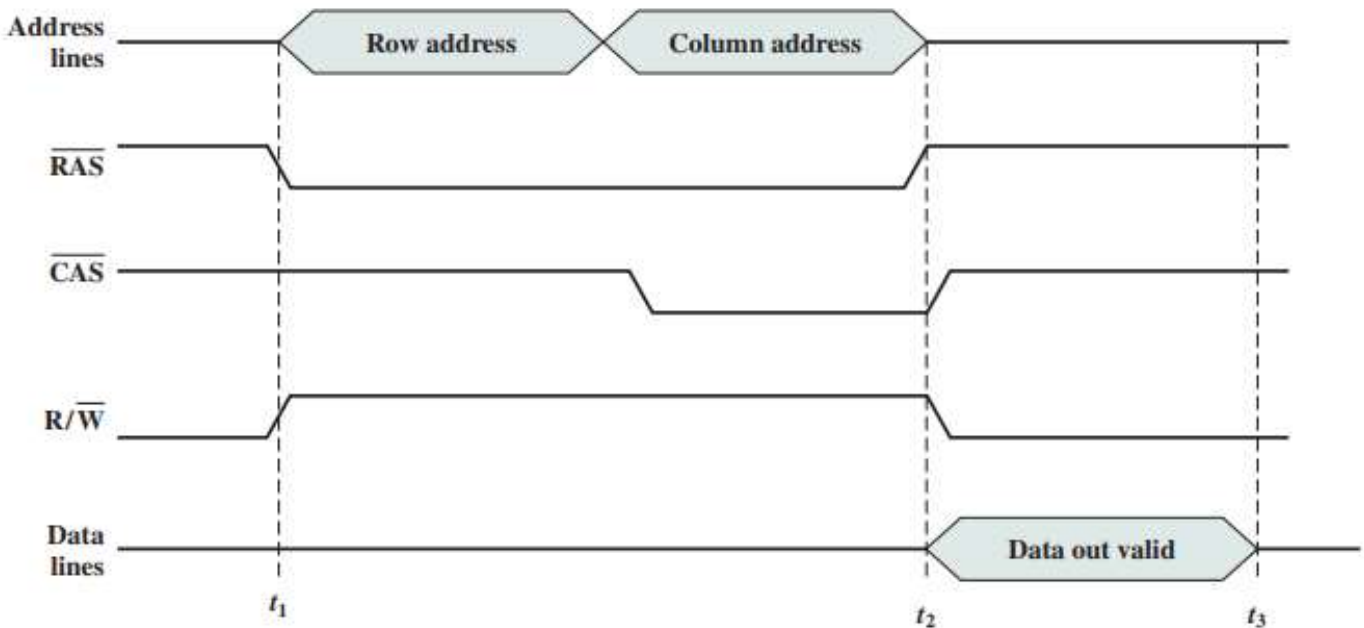
DRAM timing



Figure shows a simplified timing diagram for a DRAM read operation over a bus. The access time is considered to last from t_1 to t_2 . Then there is a recharge time, lasting from t_2 to t_3 , during which the DRAM chips will have to recharge before the processor can access them again.

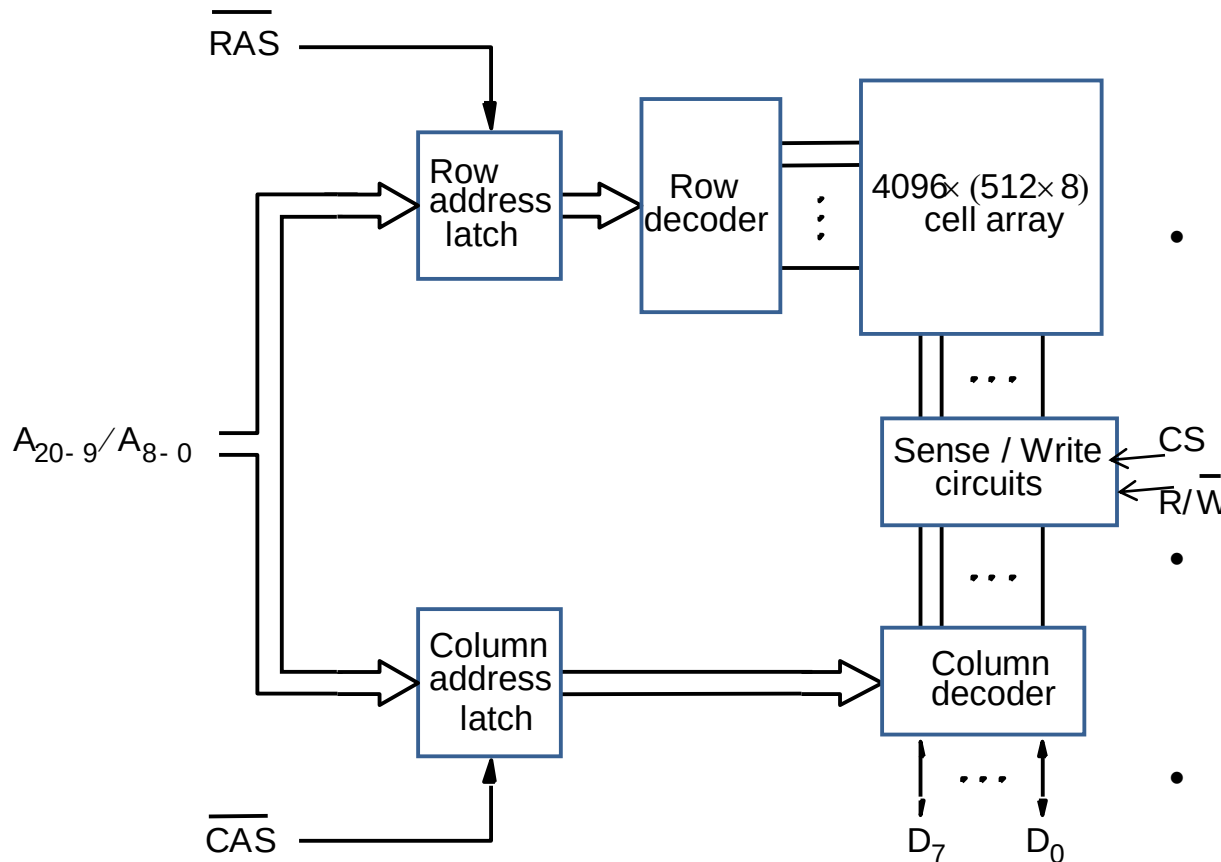
- a. Assume that the access time is 60 ns and the recharge time is 40 ns. What is the memory cycle time? What is the maximum data rate this DRAM can sustain, assuming a 1-bit output?
- b. Constructing a 32-bit wide memory system using these chips yields what data transfer rate?

DRAM timing



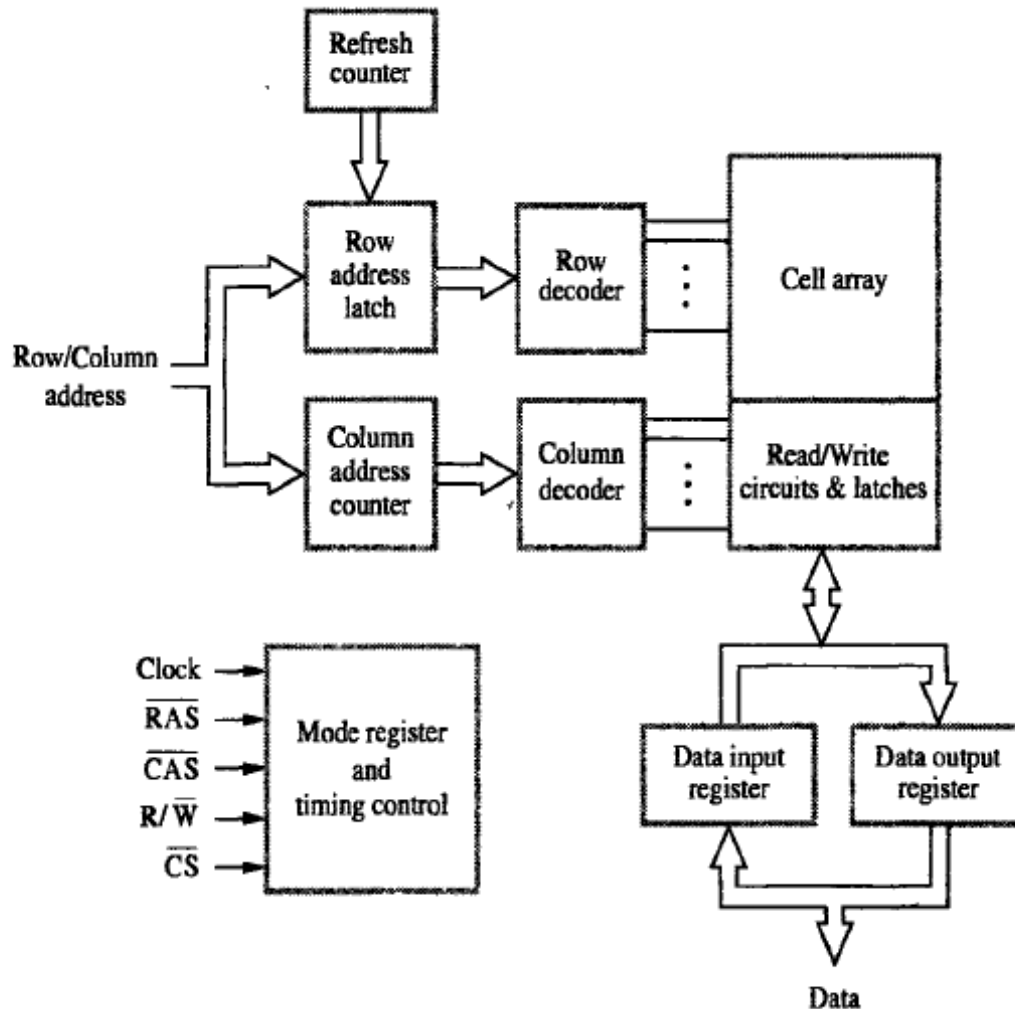
Internal organization of 2M X 8 dynamic memory chip

Arranged in 4k X 4k



- **Each row can store 512 bytes. 12 bits to select a row, and 9 bits to select a group in a row. Total of 21 bits.**
- **First apply the row address, RAS signal latches the row address. Then apply the column address, CAS signal latches the address.**
- **Timing of the memory unit is controlled by a specialized unit which generates RAS and CAS.**
- **This is asynchronous DRAM**

Synchronous DRAMS



- Operation is directly synchronized with processor clock signal.
- The outputs of the sense circuits are connected to a latch.
- During a Read operation, the contents of the cells in a row are loaded onto the latches.
- During a refresh operation, the contents of the cells are refreshed without changing the contents of the latches.
- Data held in the latches correspond to the selected columns are transferred to the output.
- For a burst mode of operation, successive columns are selected using column address counter and clock. CAS signal need not be generated externally. A new data is placed during rising edge of the clock



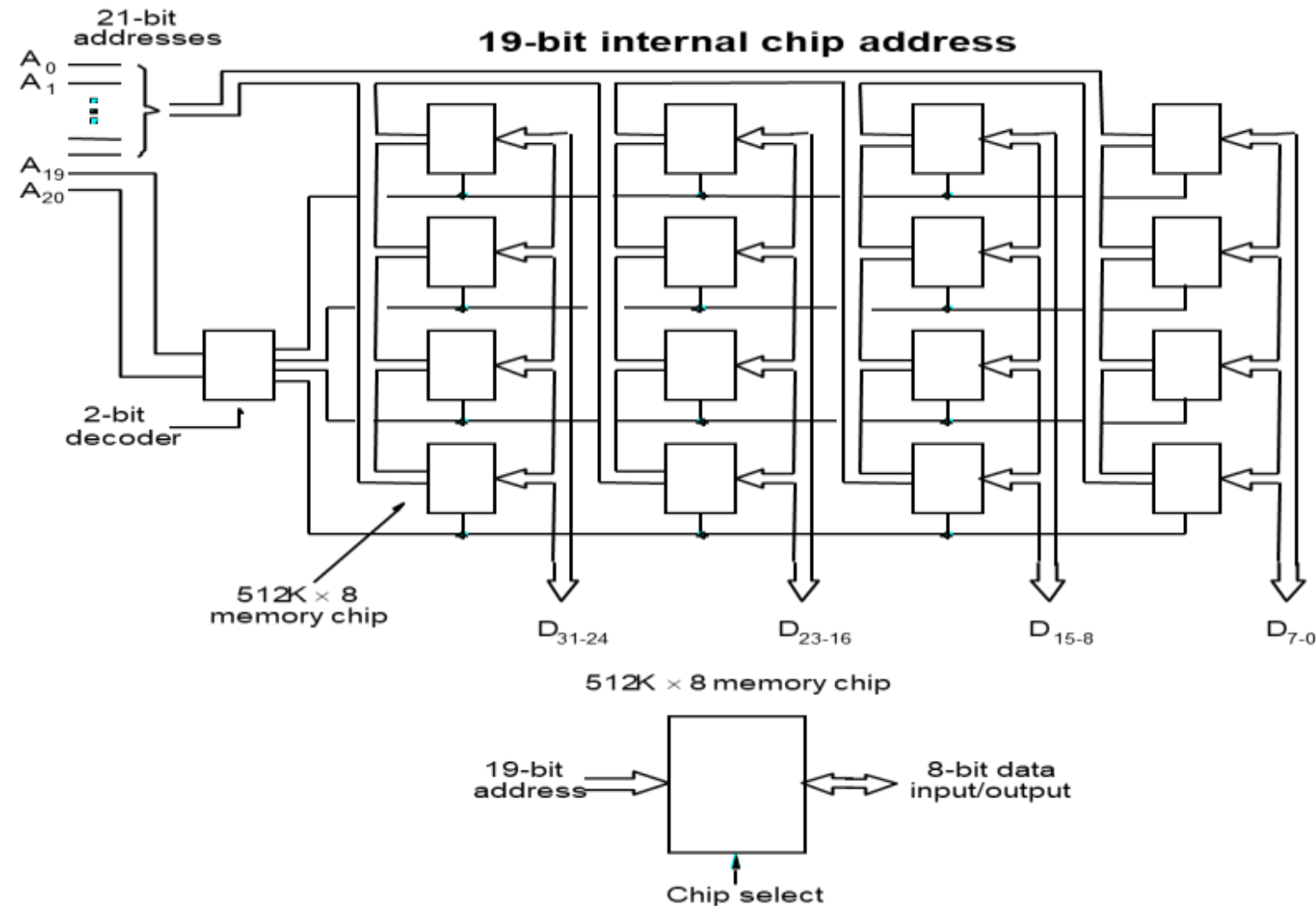
Start here 12/04/2024

- ✓ *Implement a memory unit of 2M words of 32 bits each.*
- ✓ *Use 512kx8 static memory chips.*
- ✓ *Each column consists of 4 chips.*
- ✓ *Each chip implements one byte data.*
- ✓ *A chip is selected by setting its chip select control line to 1.*
- ✓ *Selected chip places its data on the data output line, outputs of other chips are in high impedance state.*
- ✓ *21 bits to address a 32-bit word.*
- ✓ *High order 2 bits are needed to select the row, by activating the four Chip Select signals.*
- ✓ *19 bits are used to access specific byte locations inside the selected chip.*

Static memories implementation of 2M X 32 bits

Implemented as

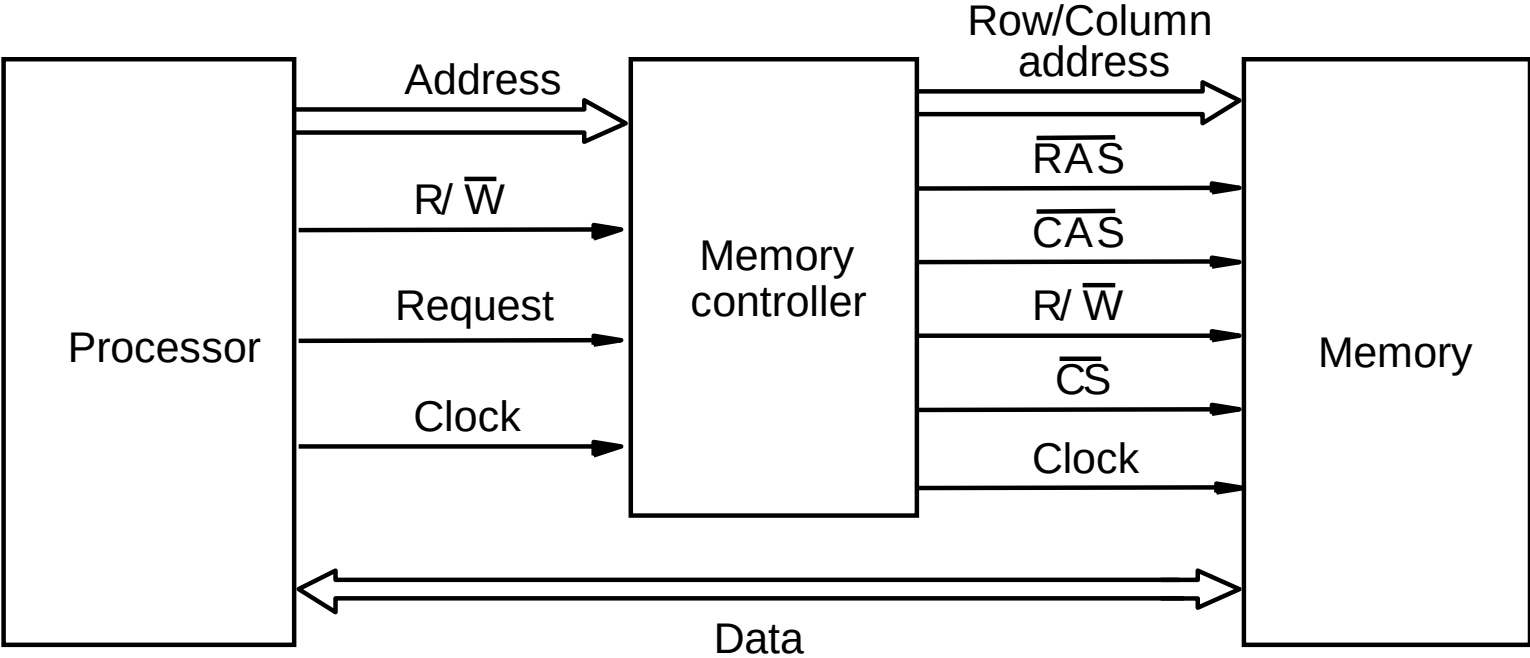
512k X 8 X 4



Latency, Bandwidth, and DDRSDRAMs



- ❑ Memory latency is the time it takes to transfer a word of data to or from memory
- ❑ Memory bandwidth is the number of bits or bytes that can be transferred in one second.
- ❑ DDRSDRAMs: on both edges
Cell array is organized in two banks





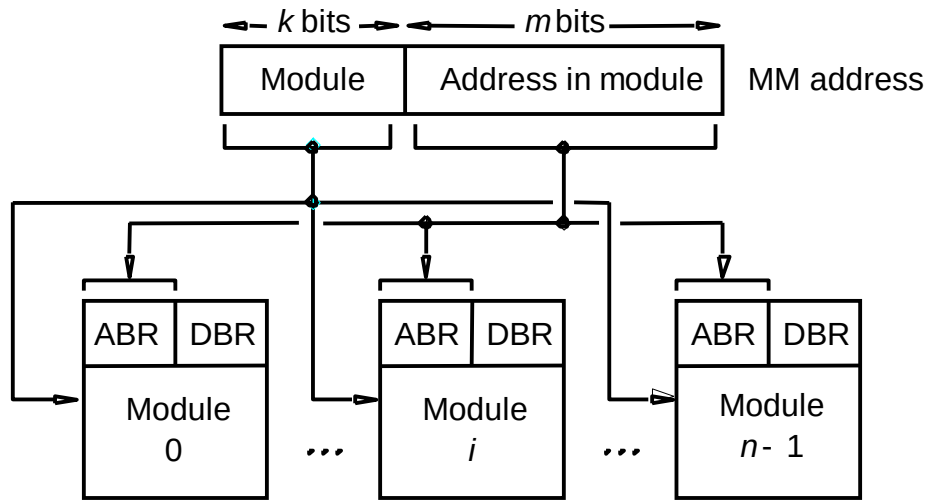
Refreshing overhead

- ❑ When used with DRAM chips which do not have a self refreshing capability Memory controller has to provide all the information needed for refreshing
- ❑ Consider a memory of $8k = 8192$ Rows
- ❑ 4 cycles to refresh each row then $8192 \times 4 = 32768$ cycles to refresh all rows
- ❑ In typical SDRAMs a typical refresh period is 64 ms
- ❑ at a clock rate of 133Mhz
- ❑ time needed to refresh all rows = $32768 / 133\text{MHz} = 0.246\text{ms}$
- ❑ overhead = $0.246 / 64 = 0.0038 = 0.4\%$ total time available for accessing memory



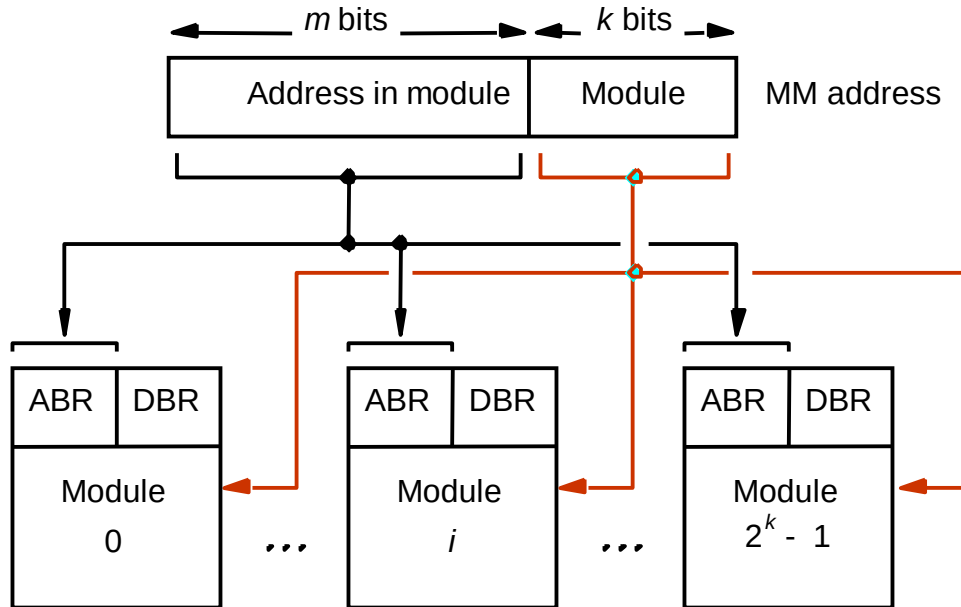
- Divides the memory system into a number of **memory modules**. Each module has its own address buffer register (ABR) and data buffer register (DBR).
- Arranges addressing so that successive words in the address space are placed in different modules.
- When requests for memory access involve consecutive addresses, the access will be to different modules.
- Since parallel access to these modules is possible, the average rate of fetching words from the Main Memory can be increased.

Methods of address layouts



- *Consecutive words are placed in a module.*
- *High-order k bits of a memory address determine the module.*
- *Low-order m bits of a memory address determine the word within a module.*
- *When a block of words is transferred from main memory to cache, only one module is busy at a time.*

Methods of address layouts(Interleaving)



- *Consecutive words are located in consecutive modules.*
- *Consecutive addresses can be located in consecutive modules.*
- *While transferring a block of data, several memory modules can be kept busy at the same time.*

Example



9	0000
8	0001
7	0010
6	0011
5	0100
4	0101
3	0110
2	0111
1	1000
15	1001
14	1010
13	1011
12	1100
11	1101
10	1110
0	1111

	00
00	9
01	5
10	1
11	12

	01
00	8
01	4
10	15
11	11

	10
00	7
01	3
10	14
11	10

	11
00	6
01	2
10	13
11	0



Example: (Definition of cylinders)

A disk has 24 recording surface has 14000 cylinders. There is an average of 400 sectors per track. Each sector contains 512 Bytes of data

What is the maximum number of bytes that can be stored?

The maximum number of bytes that can be stored on this disk is $24 \times 14000 \times 400 \times 512 = 68.8 \times 10^9$ bytes

What is the data transfer rate in bytes per second at a rotational speed of 7200rpm?

The data transfer rate is $(400 \times 512 \times 7200)/60 = 24.58 \times 10^6$ bytes/s.

The number of cylinders of a disk drive exactly equals the number of **tracks** on a single surface in the drive. It comprises the **same** track number on each platter, spanning all such **tracks** across each platter surface that is able to store data



Example: Contnd

A disk has 24 recording surface has 14000 tracks. There is an average of 400 sectors per track. Each sector contains 512 Bytes of data

Using a 32 bit word , suggest a suitable addressing scheme for the disk, assuming that there are 512 bytes / Sector

Need 9 bits to identify a sector, 14 bits for a track, and 5 bits for a surface.

Thus, a possible scheme is to use address bits A_{8-0} for sector, A_{22-9} for track, and A_{27-23} for surface identification. Bits A_{31-28} are not used.

Example



What is the average time to read or write a 512-byte sector for a typical disk rotating at 15,000 RPM? The advertised average seek time is 4 ms, the transfer rate is 100 MB/sec, and the controller overhead is 0.2 ms. Assume that the disk is idle so that there is no waiting time.

Sol:

Average disk access time = average seek time + average rotational Delay + transfer time + controller overhead

seek The process of positioning a read/write head over the proper track on a disk & *seek time* is the time consumed in it

rotational latency Also called rotational delay.

The time required for the desired sector of a disk to rotate under the read/write head; usually assumed to be half the rotation time.



What is the average time to read or write a 512-byte sector for a typical disk rotating at 15,000 RPM? The advertised average seek time is 4 ms, the transfer rate is 100 MB/sec, and the controller overhead is 0.2 ms. Assume that the disk is idle so that there is no waiting time.

Sol:

Average disk access time = average seek time + average rotational
Delay + transfer time + controller overhead

$$\begin{aligned} &= 4.0 \text{ ms} + 0.5 \text{ rotation} / 15,000 \text{ RPM} + 0.5 \text{ KB} / 100 \text{ MB/sec} + 0.2 \text{ ms} \\ &= 4.0 + 2.0 + 0.005 + 0.2 = 6.2 \text{ ms} \end{aligned}$$

If the measured average seek time is 25% of the advertised average time, then it is

$$1.0 \text{ ms} + 2.0 \text{ ms} + 0.005 \text{ ms} + 0.2 \text{ ms} = 3.2 \text{ ms}$$

Example:



Design a 16-bit memory of total capacity 8192 bits using SRAM chips of size 64 X 1 bit. Give the array configuration of the chips on the memory board showing all required input and output signals for assigning this memory to the lowest address space. The design should allow for both byte and 16-bit word accesses.

Sol:

Number of total addressable locations of 16 bits = $8192/16 = 512$

no of 64 X 1 chips required $8192/64 = 128$ chips

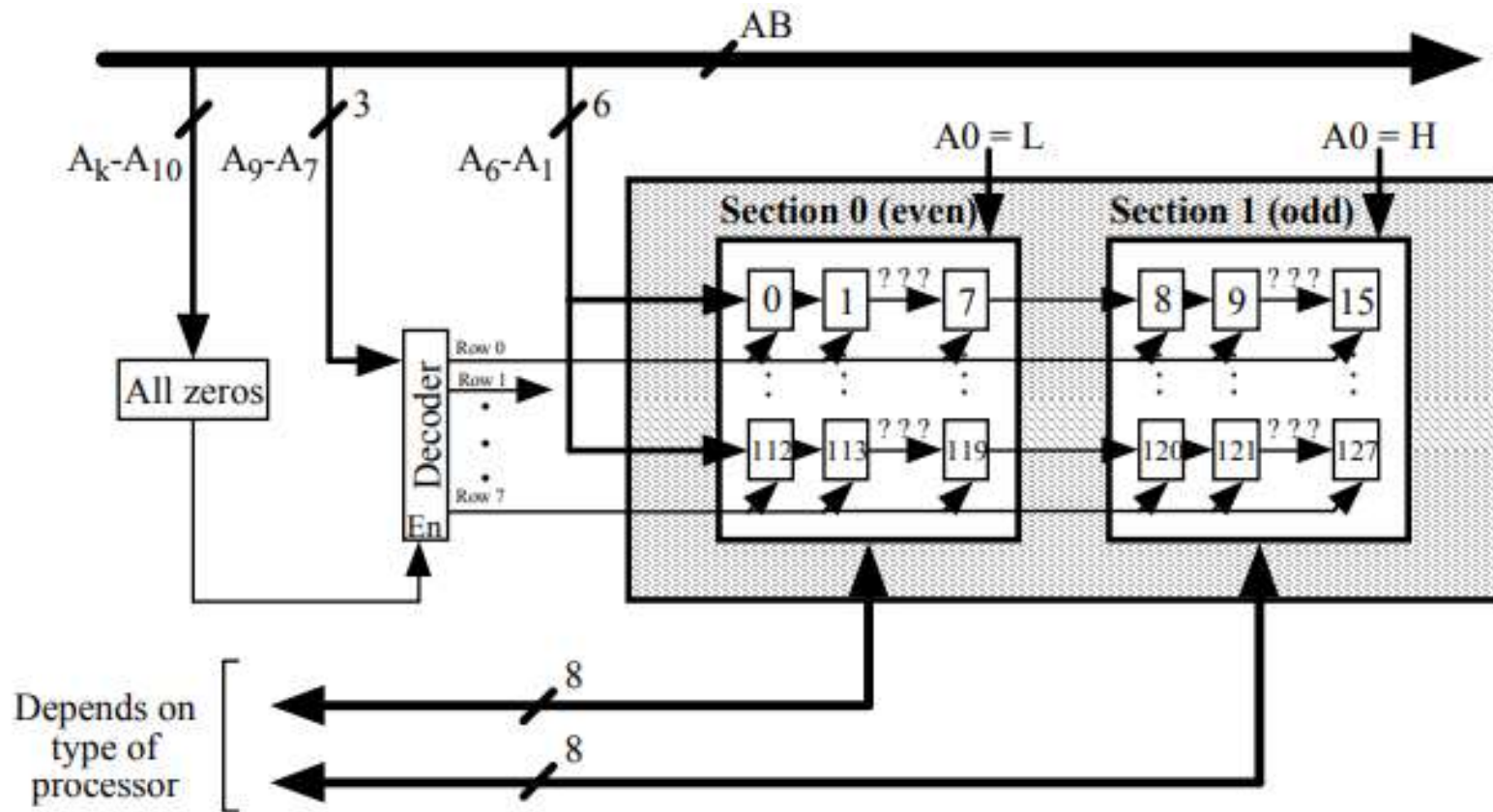
There will be $128/16 = 8$ row of 16 bits

512 can be ranged into 8 rows X 64 columns

For 8 rows no of decoder bits 3

For 64 columns no of decoder bits 6

Solution



Example



Consider a dynamic RAM that must be given a refresh cycle 64 times per ms. Each refresh operation requires 150 ns; a memory cycle requires 250 ns. What percentage of the memory's total operating time must be given to refreshes?

Example



Consider a dynamic RAM that must be given a refresh cycle 64 times per ms. Each refresh operation requires 150 ns; a memory cycle requires 250 ns. What percentage of the memory's total operating time must be given to refreshes?

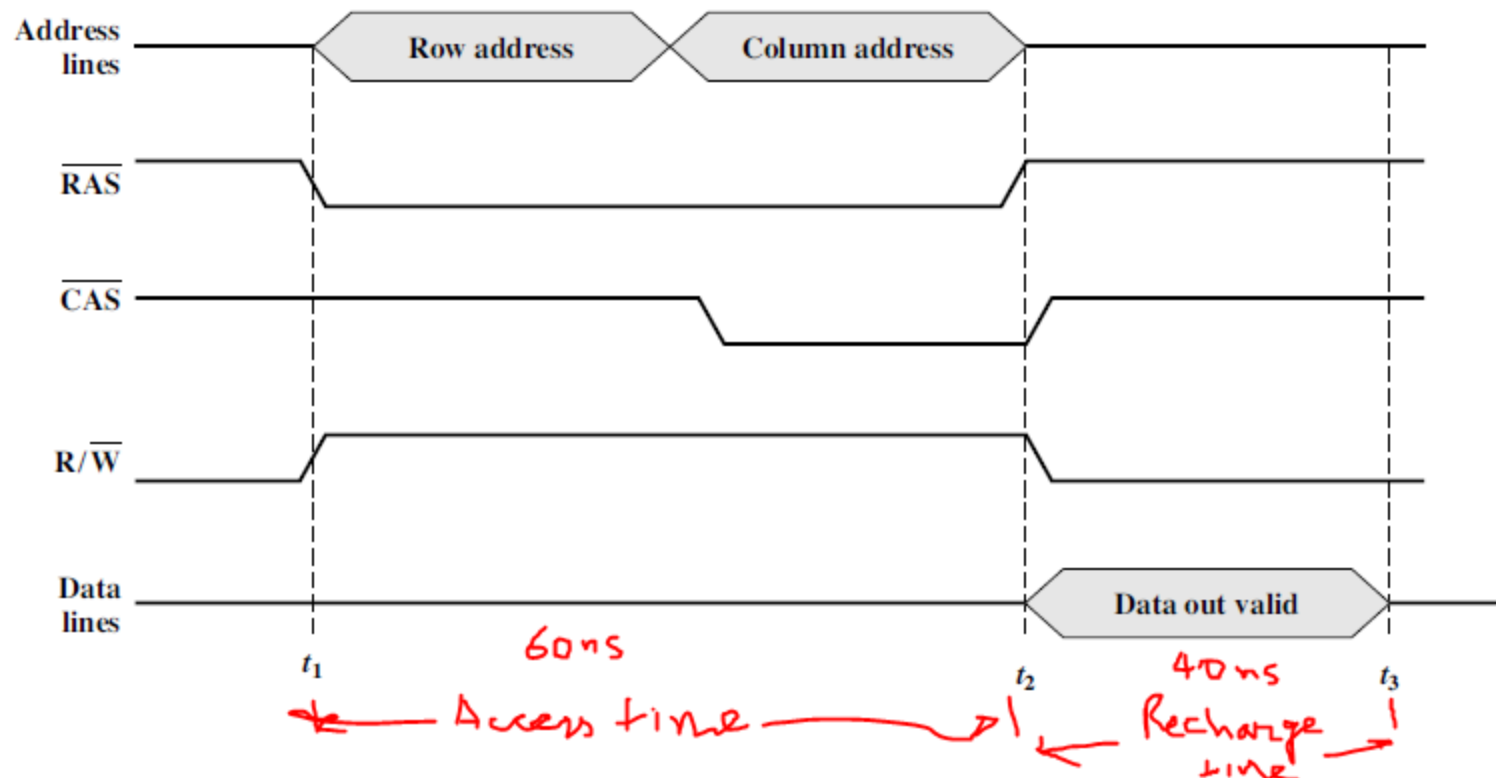
Sol:

Total Time consumed in 1 ms = $64 \times 150 \text{ ns} = 9600 \text{ ns}$
out of 1ms $9600\text{ns}/1\text{ms} \times 100 = 0.96 \% \cong 1\%$

Out of total time of memory cycle of 250 ns

refresh operation consumes $\frac{150\text{ns} \times 100}{250 \text{ ns}} \% = 60\%$

Figure shows a simplified timing diagram for a DRAM read operation over a bus.



What is the memory cycle time?

What is the maximum data rate this DRAM can sustain, assuming a 1-bit output?

What is the memory cycle time?

What is the maximum data rate this DRAM can sustain, assuming a 1-bit output?

Sol: memory cycle time = $(60 + 40) = 100$ ns

What is the memory cycle time?

What is the maximum data rate this DRAM can sustain, assuming a 1-bit output?

Sol: memory cycle time = $(60 + 40) = 100$ ns

1 bit in 100ns so $1/100\text{ns}$ per/ sec = 10Mbps

What is the memory cycle time?

What is the maximum data rate this DRAM can sustain, assuming a 1-bit output?

Sol: memory cycle time = $(60 + 40) = 100$ ns

1 bit in 100ns so $1/100\text{ns}$ per/ sec = 10Mbps

Constructing a 32-bit wide memory system using these chips yields what data transfer rate?



What is the memory cycle time?

What is the maximum data rate this DRAM can sustain, assuming a 1-bit output?

Sol: memory cycle time = $(60 + 40) = 100$ ns

1 bit in 100ns so $1/100\text{ns}$ per/ sec = 10Mbps

Constructing a 32-bit wide memory system using these chips yields what data transfer rate?

Ans $320\text{Mbps} = 320\text{Mbps}/8 = 40\text{MB/s}$



Average Access Time

$$t_{ave} = h_1 C_1 + (1 - h_1) h_2 C_2 + (1 - h_1)(1 - h_2) M$$

Given a 100 MHz machine with a with a miss penalty of 20 cycles, a hit time of 2 cycles, and a miss rate of 5%, calculate the average memory access time (AMAT).



Given a 100 MHz machine with a with a miss penalty of 20 cycles, a hit time of 2 cycles, and a miss rate of 5%, calculate the average memory access time (AMAT).

$$\text{AMAT} = \text{hit_time} + \text{miss_rate} \times \text{miss_penalty}$$

Since the clock rate is 100 MHz, the cycle time is:

$$1/(100 \text{ MHz}) = 10 \text{ ns}$$

which gives

$$\text{AMAT} = 10 \text{ ns} \times (2 + 20 \times 0.05) = 30 \text{ ns}$$

Note: Here we needed to multiply by the cycle time because the hit_time and miss_penalty were given in cycles.



Suppose doubling the size of the cache decrease the miss rate to 3%, but causes the hit time to increase to 3 cycles and the miss penalty to increase to 21 cycles. What is the AMAT of the new machine?



Suppose doubling the size of the cache decrease the miss rate to 3%, but causes the hit time to increase to 3 cycles and the miss penalty to increase to 21 cycles. What is the AMAT of the new machine?

For the new machine:

$$\text{AMAT} = 10 \text{ ns} \times (3 + 21 \times 0.03) = 36.3 \text{ ns}$$



Mass Storage

- Many systems today need to store many terabytes of data
- Don't want to use single, large disk
 - too expensive
 - failures could be dangerous
- Would prefer to use many smaller disks

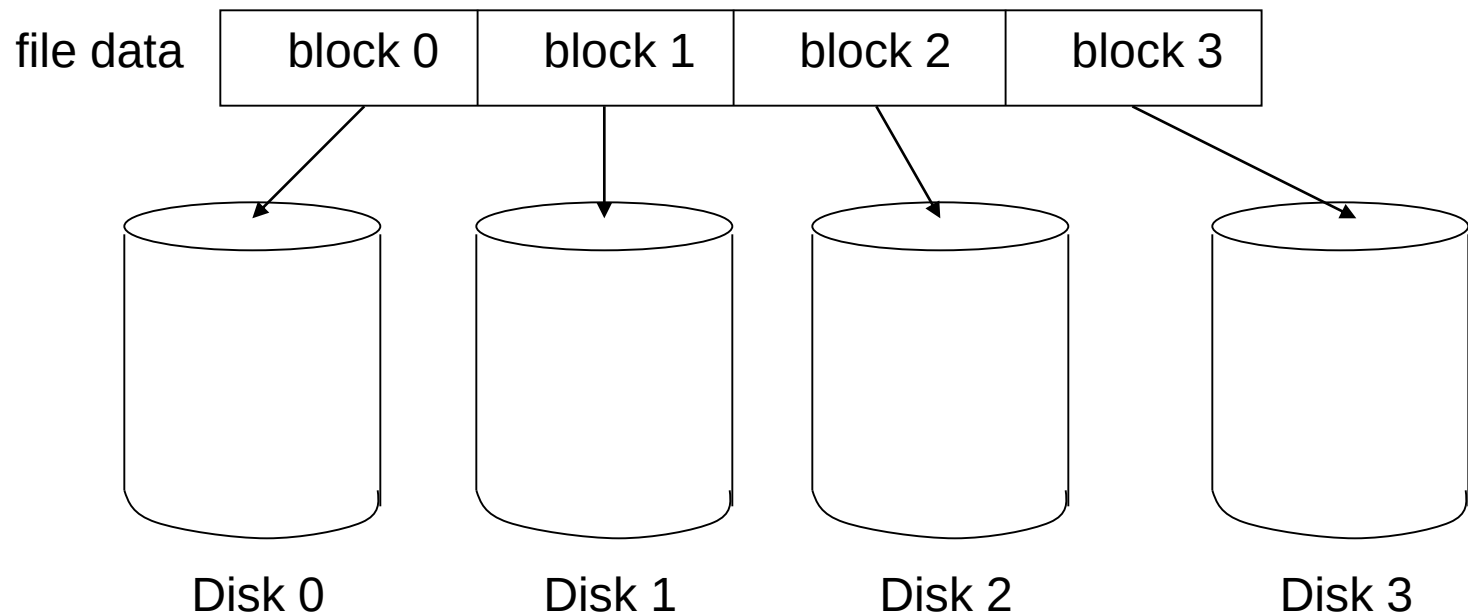
RAID



- Redundant Array of Independent Disks
- Basic idea is to connect multiple disks together to provide
 - large storage capacity
 - faster access to reading data
 - redundant data
- Many different levels of RAID systems
 - differing levels of redundancy, error checking, capacity, and cost

Striping

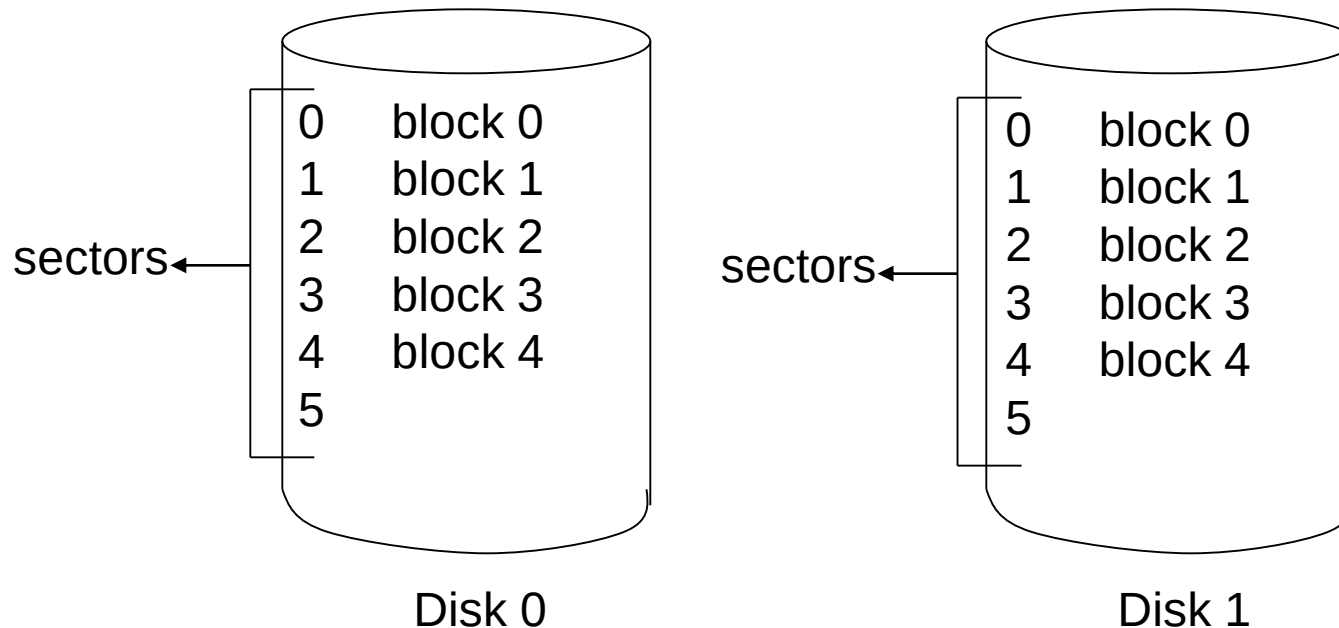
- Take file data and map it to different disks
- Allows for reading data in parallel



MIRRORING

file data

block 0	block 1	block 2	block 3	block 4
---------	---------	---------	---------	---------



Mirroring



- Keep two copies of data on two separate disks
- Gives good error recovery
 - if some data is lost, get it from the other source
- Expensive
 - requires twice as many disks
- Write performance can be slow
 - have to write data to two different spots
- Read performance is enhanced
 - can read data from file in parallel



Parity

- Way to do error checking and correction
- Add up all the bits that are 1
 - if even number, set parity bit to 0
 - if odd number, set parity bit to 1
- To actually implement this, do an exclusive OR of all the bits being considered
- Consider the following 2 bytes

<u>byte</u>	<u>parity</u>
10110011	1
01101010	0

- If a single bit is bad, it is possible to correct it



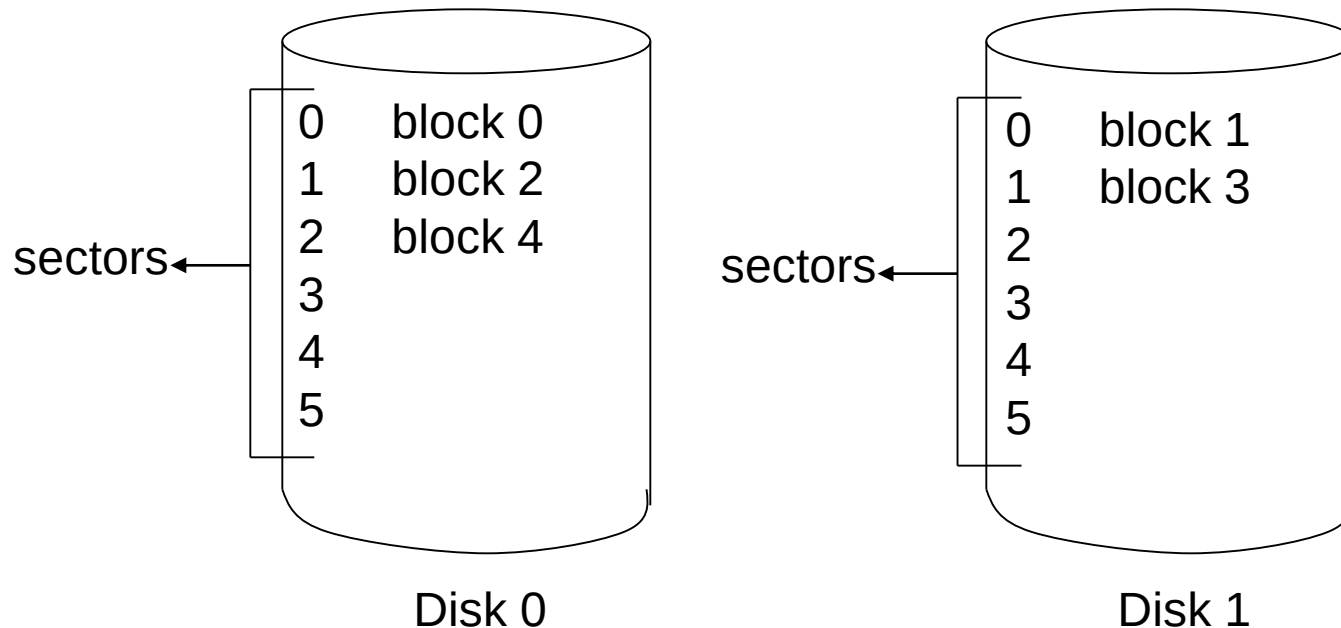
RAID Level-0

- Often called striping
- Break a file into blocks of data
- Stripe the blocks across disks in the system
- Simple to implement
 - $\text{disk} = \text{file block} \% \text{number of disks}$
 - $\text{sector} = \text{file block} / \text{number of disks}$
- provides no redundancy or error detection

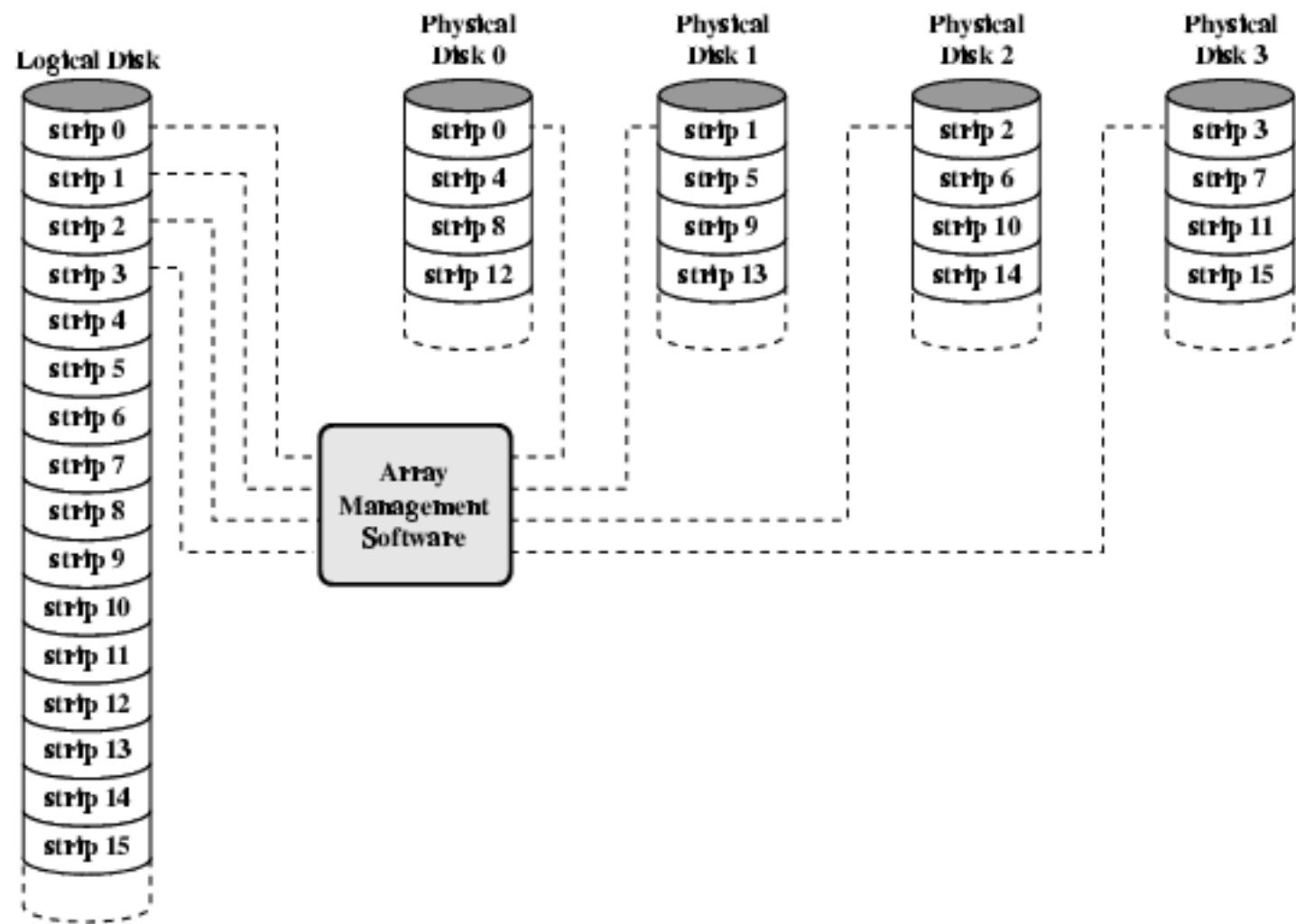
RAID Level-0

file data

block 0	block 1	block 2	block 3	block 4
---------	---------	---------	---------	---------



Data Mapping For RAID 0



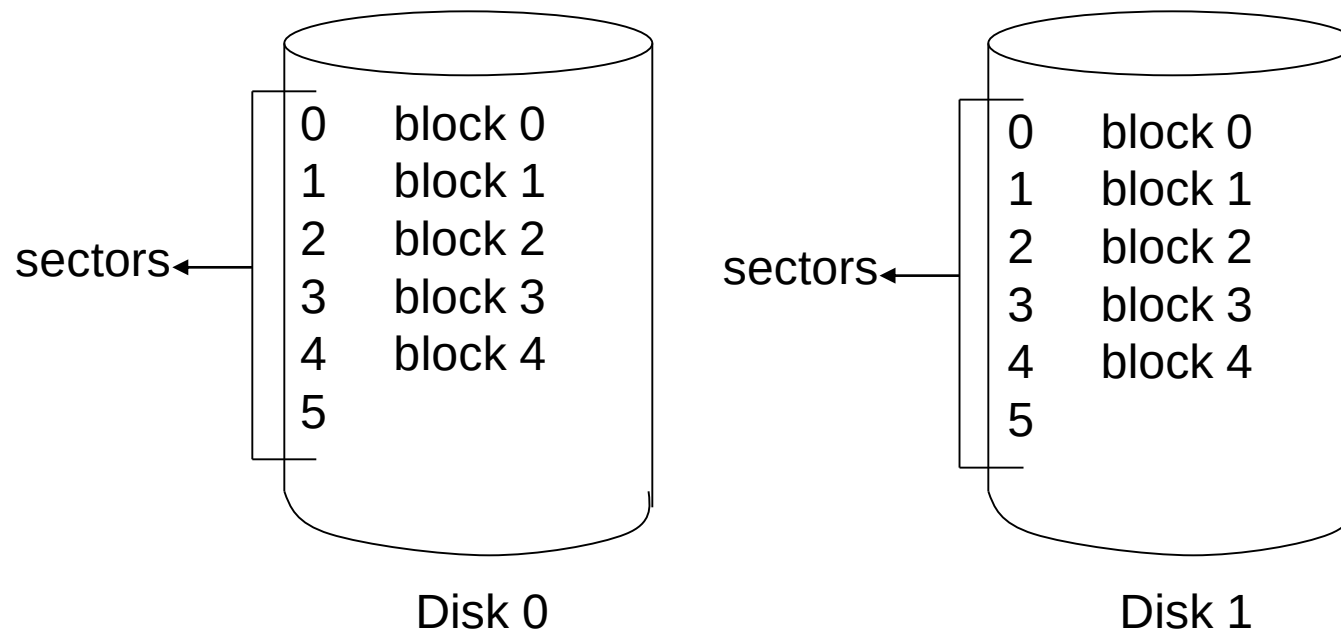
RAID Level-1

- A complete file is stored on a single disk
- A second disk contains an exact copy of the file
- Provides complete redundancy of data
- Read performance can be improved
 - file data can be read in parallel
- Write performance suffers
 - must write the data out twice
- Most expensive RAID implementation
 - requires twice as much storage space

RAID Level-1

file data

block 0	block 1	block 2	block 3	block 4
---------	---------	---------	---------	---------

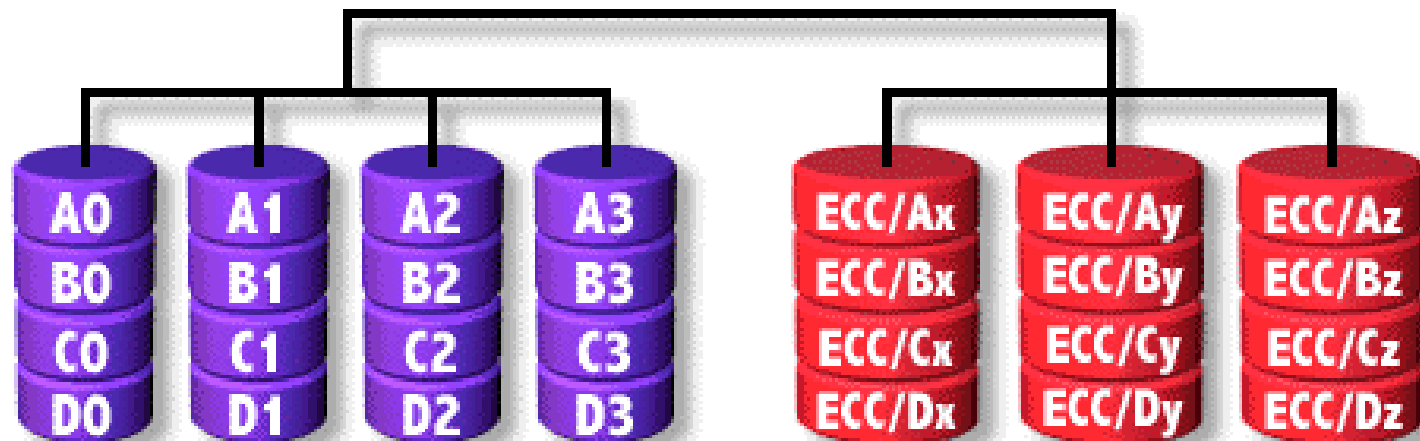


RAID Level 2

- ❑ Uses Bit-level striping with Hamming codes for ECC
- ❑ Number of disks required depends on exact implementation
- ❑ Only fair fault tolerance
- ❑ Advantages
 - Random Read performance=fair
 - Sequential Read Performance=very good
 - Sequential Write performance=good
- ❑ Disadvantages
 - Random Write Performance=poor
 - Requires a complex controller
 - High overhead for check disks
 - Not used in modern systems

RAID Level 2

RAID 2



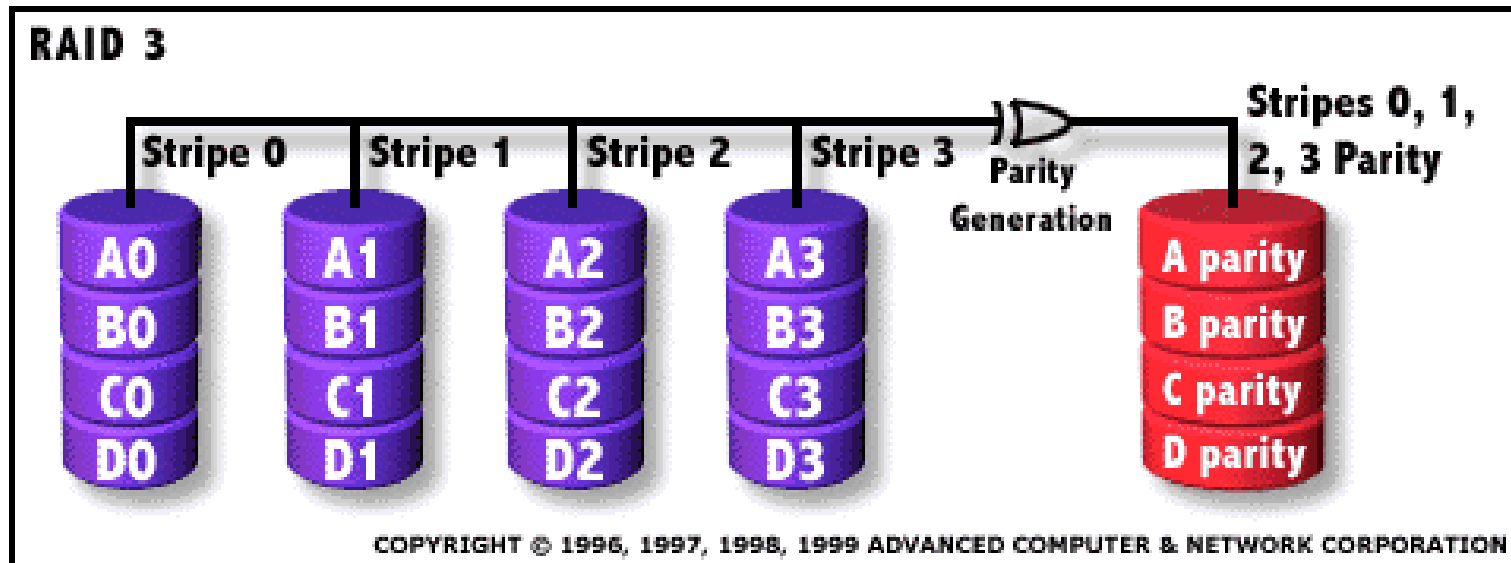
A0 to A3=Word A; B0 to B3 = Word B;
C0 to C3=Word C; D0 to D3 = Word D

ECC/Ax to Az=Word A ECC; ECC/Bx to Bz = Word B ECC;
ECC/Cx to Cz=Word C ECC; ECC/Dx to Dz = Word D ECC

RAID Level 3

- ❑ Uses byte-level striping with dedicated parity
- ❑ Requires minimum three drives to implement
- ❑ Has good fault-tolerance
- ❑ Advantages
 - Random Read Performance=good
 - Sequential Read performance=very good
 - Sequential Write performance=fair to good
 - Lowest overhead for check disks
- ❑ Disadvantages
 - Random Write performance=poor
 - Complex controller design

RAID Level 3

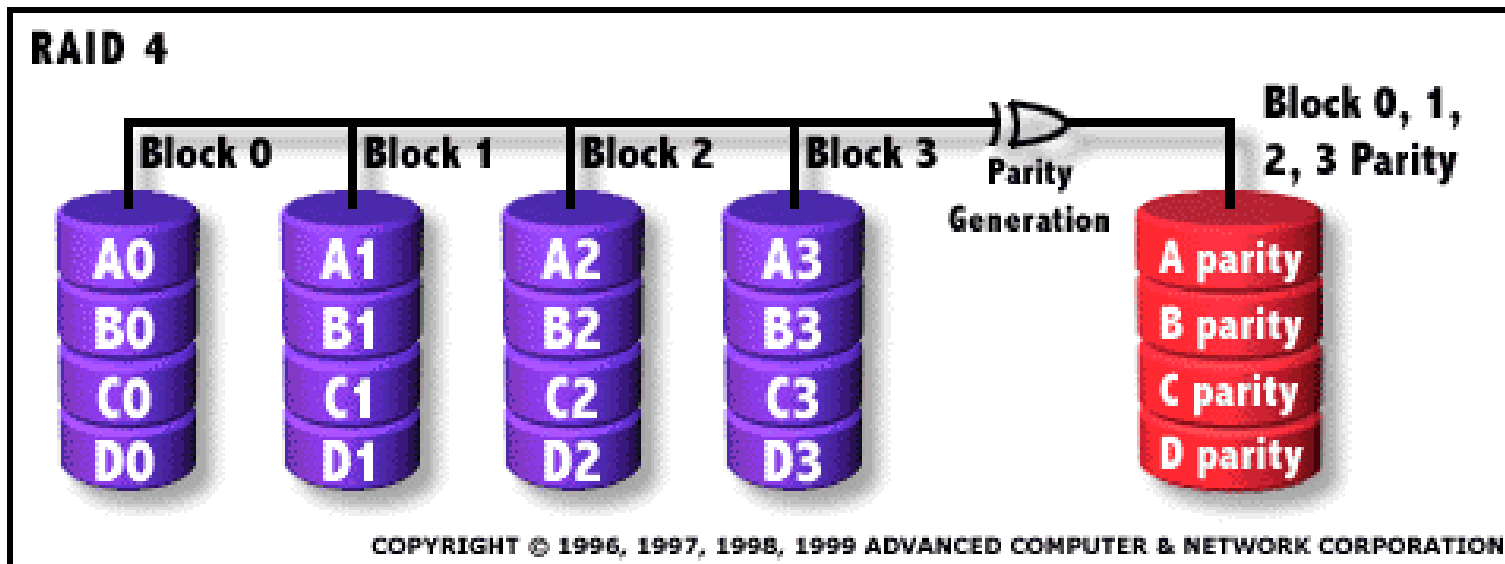




RAID Level 4

- ❑ Uses Block-level striping with dedicated parity
- ❑ Requires minimum of 3 drives to implement
- ❑ Has good fault-tolerance
- ❑ Advantages
 - Random Read Performance=very good
 - Sequential Read and Write performance=good
 - Lowest overhead of check disks
- ❑ Disadvantages
 - Quite complex controller design
 - Random write performance=poor
 - Not commonly used

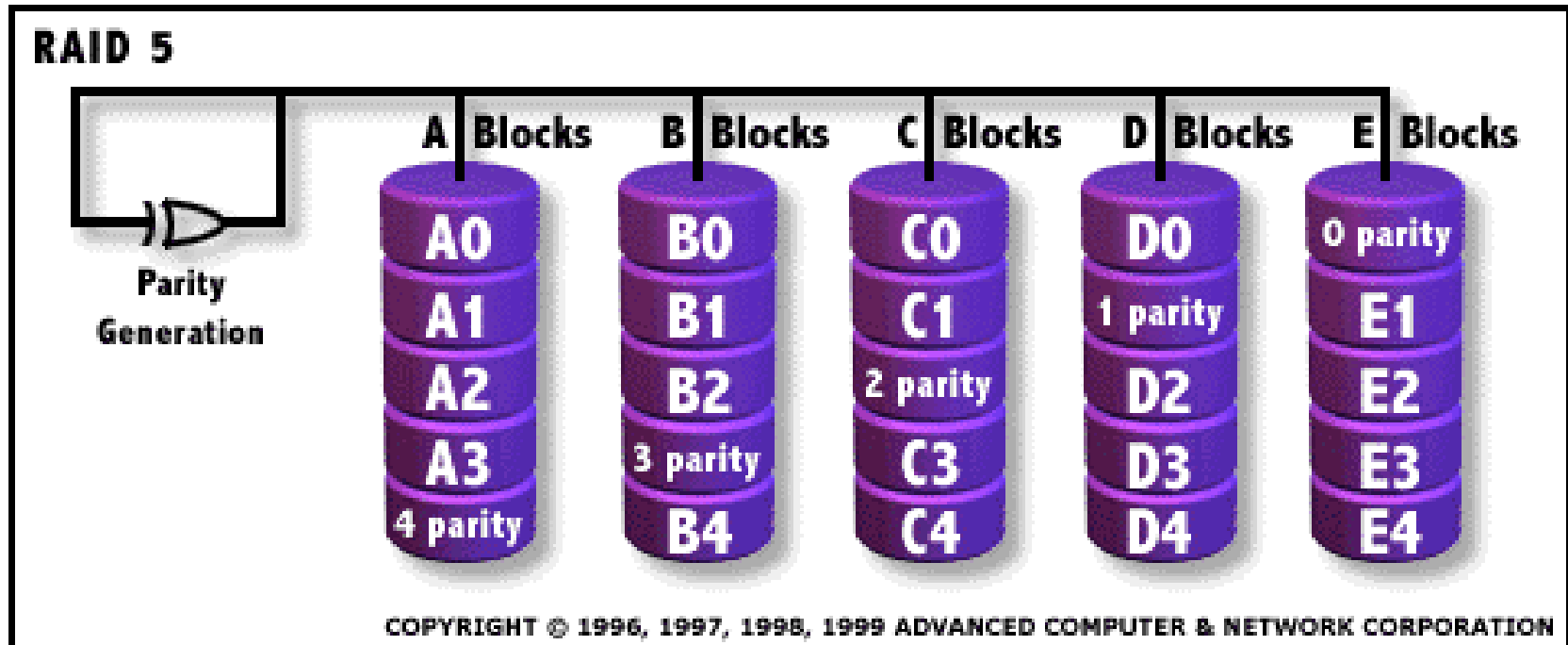
RAID Level 4



RAID Level 5

- ❑ Uses Block-level striping with distributed parity
- ❑ Requires a minimum of 3 drives to implement
- ❑ Advantages
 - Random Read performance=very good
 - Random Write performance=fair
 - Sequential Read and Write performance=good
 - Lowest overhead of check disks
- ❑ Disadvantages
 - Most complex controller design
 - Difficult to rebuild in the event of a disk failure

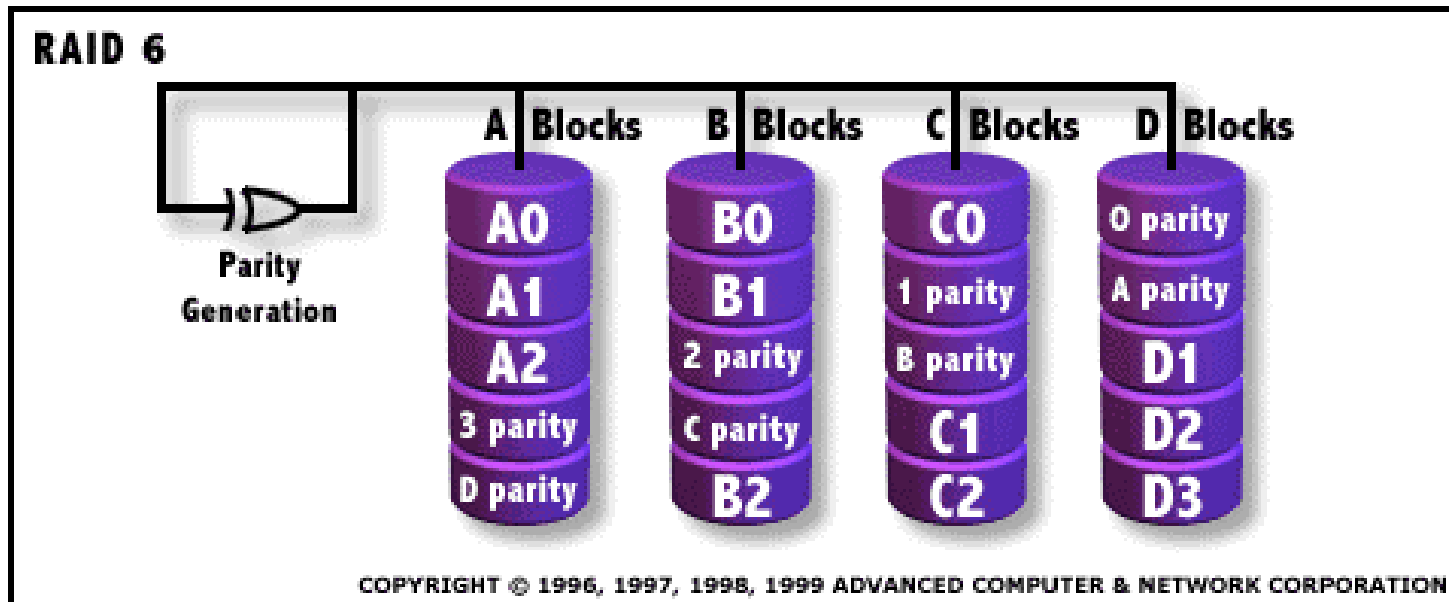
RAID Level 5



RAID 6

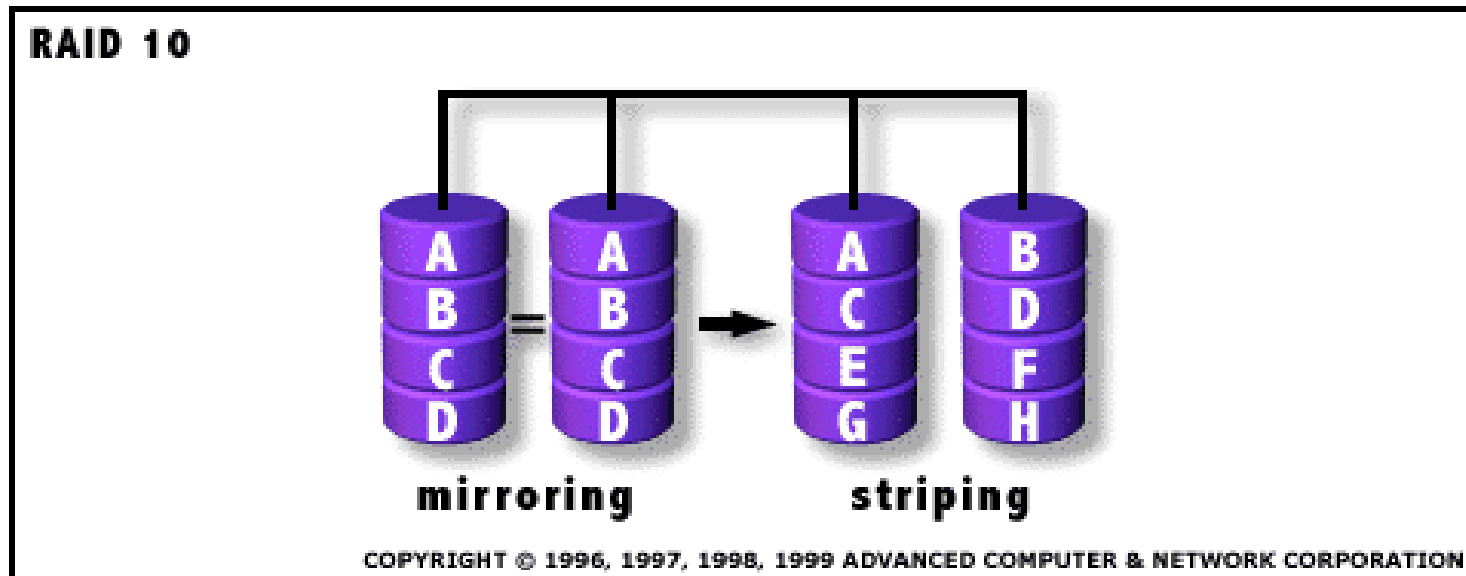
- Two parity calculations
- Stored in separate blocks on different disks
- User requirement of N disks needs N+2
- High data availability
 - Three disks need to fail for data loss
 - Significant write penalty

RAID Level 6



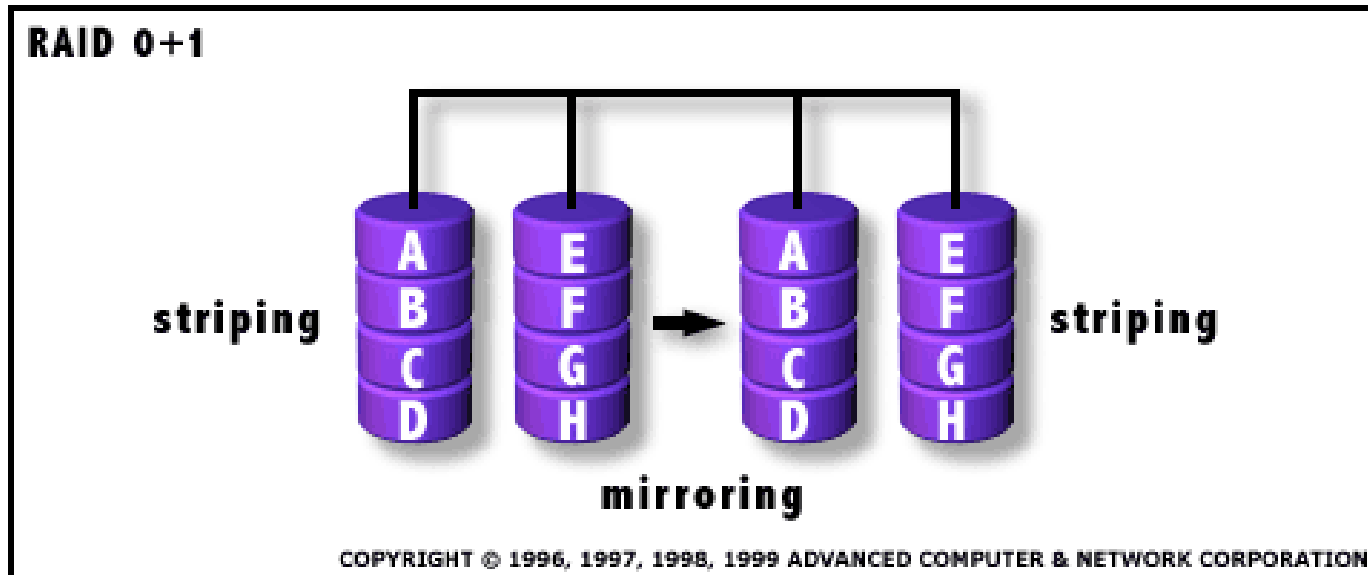
Raid Level 6 uses Block-level striping with *dual* distributed parity

RAID Level 1+0



Uses Mirroring and striping without parity

RAID Level 0+1



Uses Mirroring and striping without Parity



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

R.K.Tiwary



CPU Structure

CPU must:

Fetch Opcode

Interpret instructions (decode)

Fetch data (if needed)

Process data

Write data



Prefetch

- ❑ Fetch accessing main memory
- ❑ Execution usually does not access main memory
- ❑ Can fetch next instruction during execution of current instruction
 - Called instruction prefetch
- ❑ Improved Performance
- ❑ But not doubled:
 - Fetch usually shorter than execution
 - Prefetch more than one instruction?
 - Any jump or branch means that prefetched instructions are not the required instructions
- ❑ Add more stages to improve performance

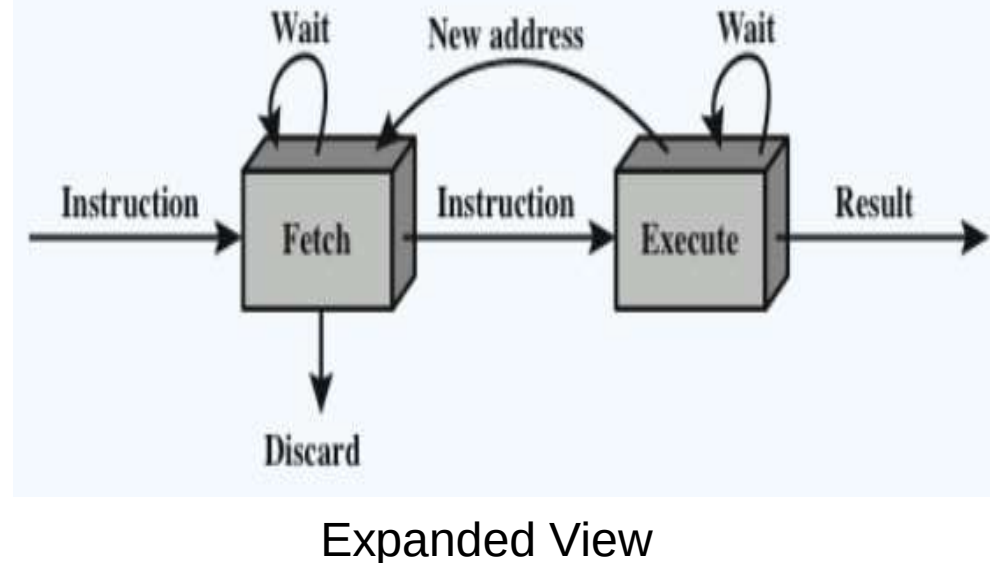
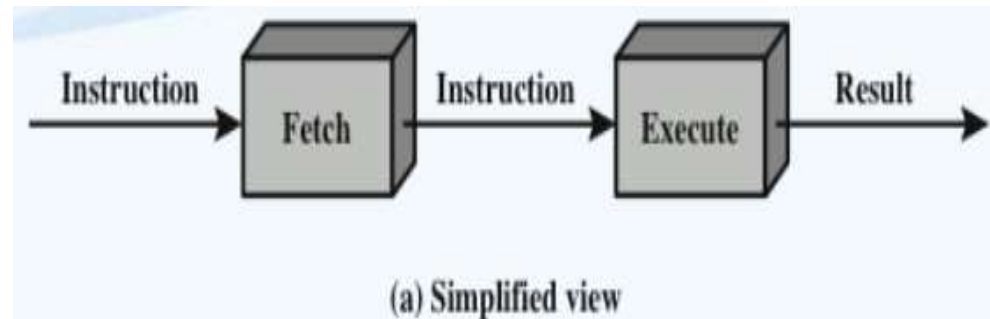
Two-Stage Instruction Pipeline

The pipeline has two independent stages. The first stage fetches an instruction and buffers it.

When the second stage is free, the first stage passes it the buffered instruction.

While the second stage is executing the instruction, the first stage takes advantage of any unused memory cycles to fetch and buffer the next instruction.

This is called instruction prefetch or fetch overlap.

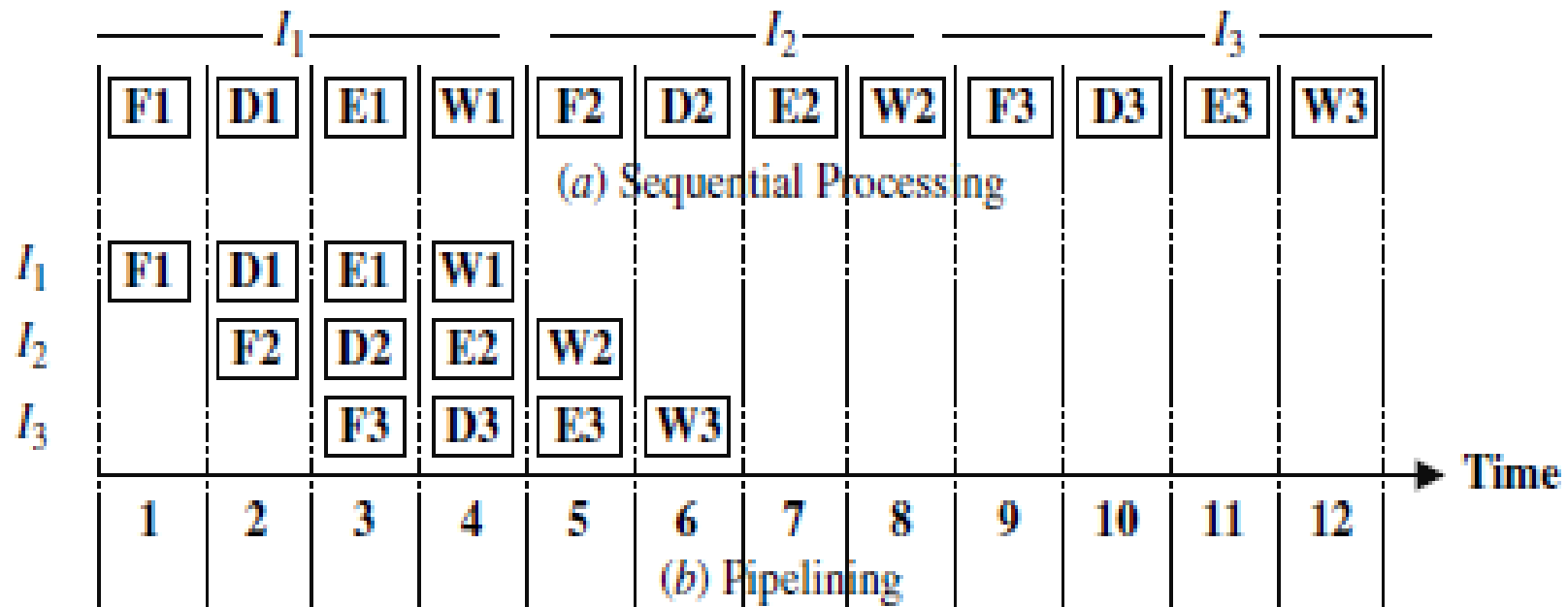


Pipelining is a key implementation technique used to build fast processors. It allows the execution of multiple instructions to overlap in time.

A pipeline within a processor is similar to a car assembly line. Each assembly station is called a *pipe stage* or a *pipe segment*.

The **throughput** of an instruction pipeline is the measure of how often an instruction exits the pipeline.

Pipelining versus sequential processing



Time taken for sequential processing = 12 units



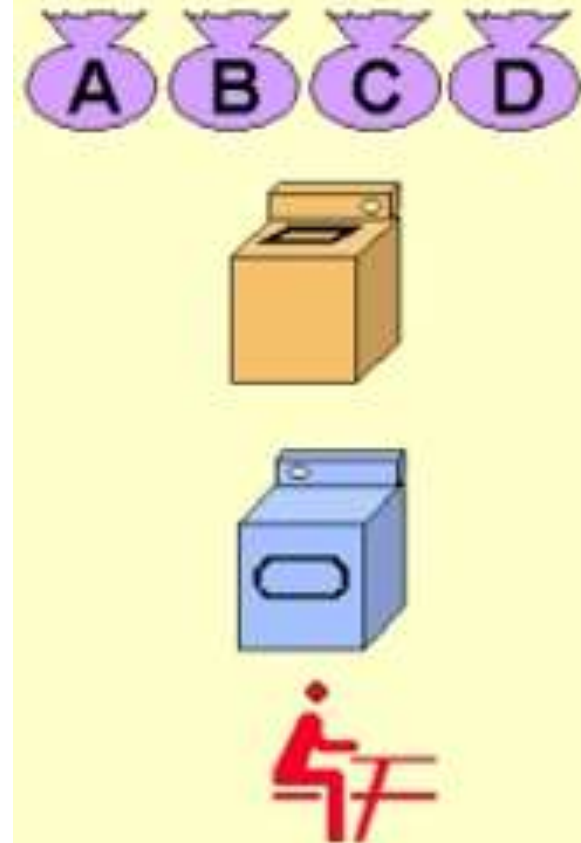
performance measures for the goodness of a pipeline

- ☐ Speed-up $S(n)$
- ☐ Throughput $U(n)$
- ☐ Efficiency $E(n)$

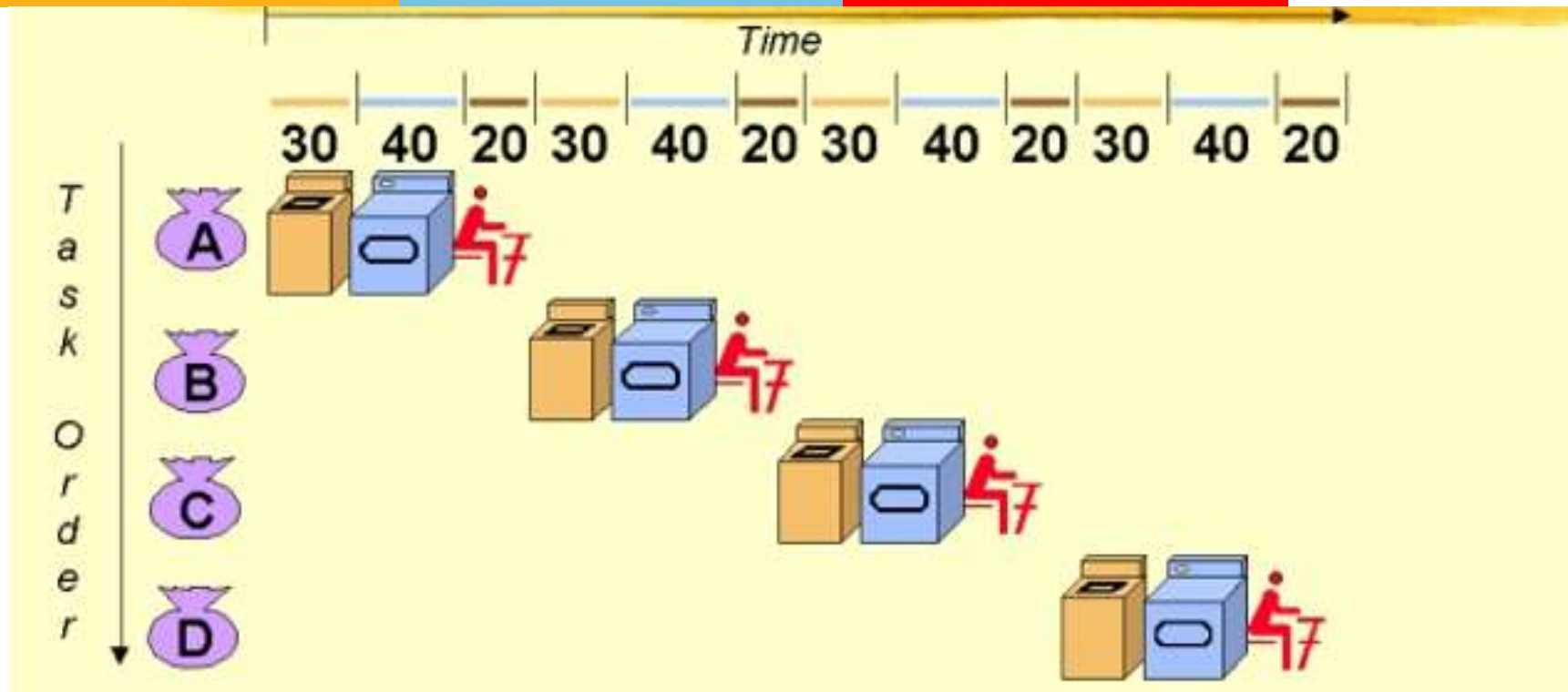
Pipelining : Its Natural



- Laundry Example
- Ann, Brian, Cathy, Dave each have one load of clothes to wash, dry, and fold
 - Washer takes 30 minutes
 - Dryer takes 40 minutes
 - "Folder" takes 20 minutes

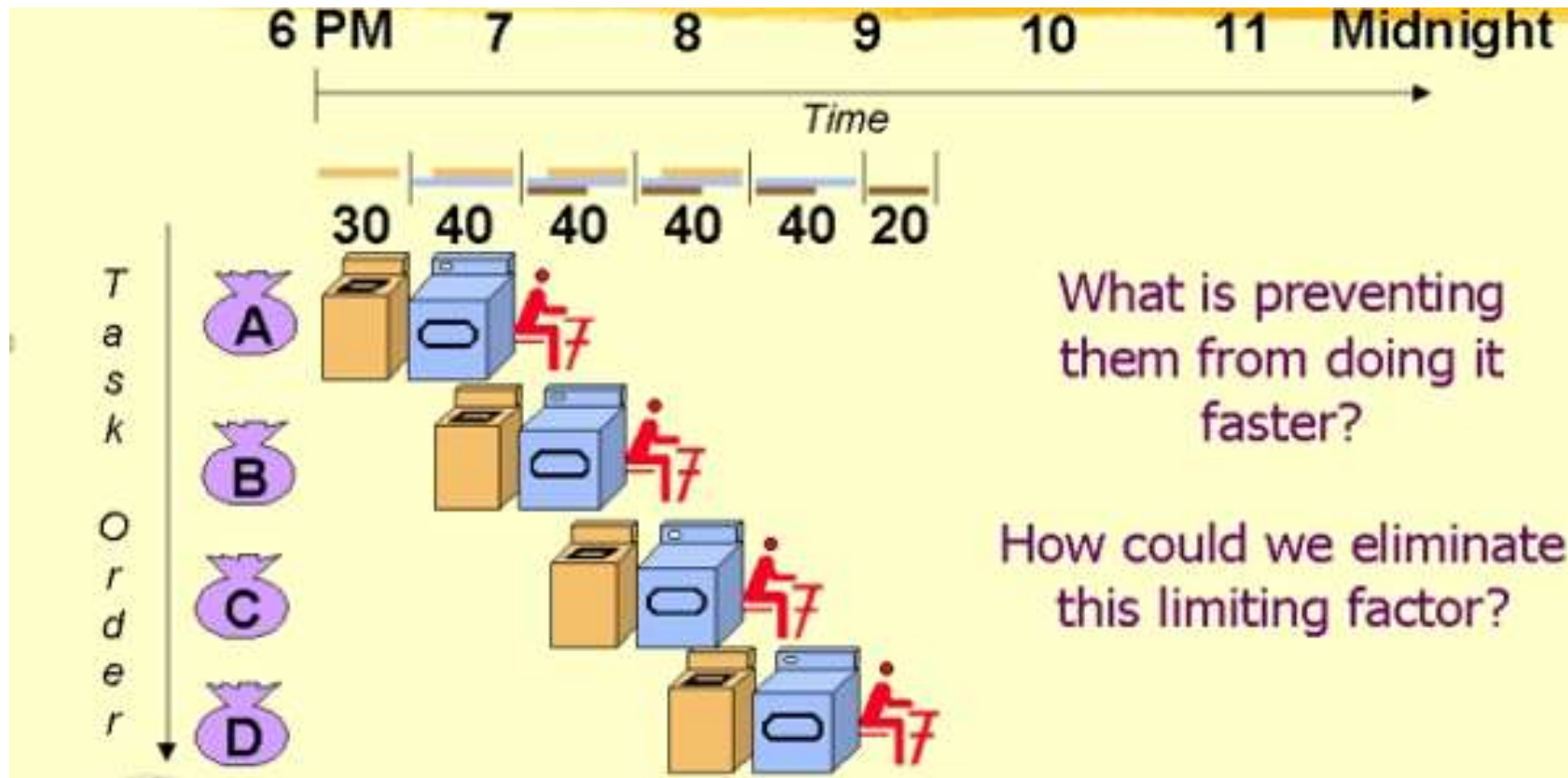


Sequential Laundry



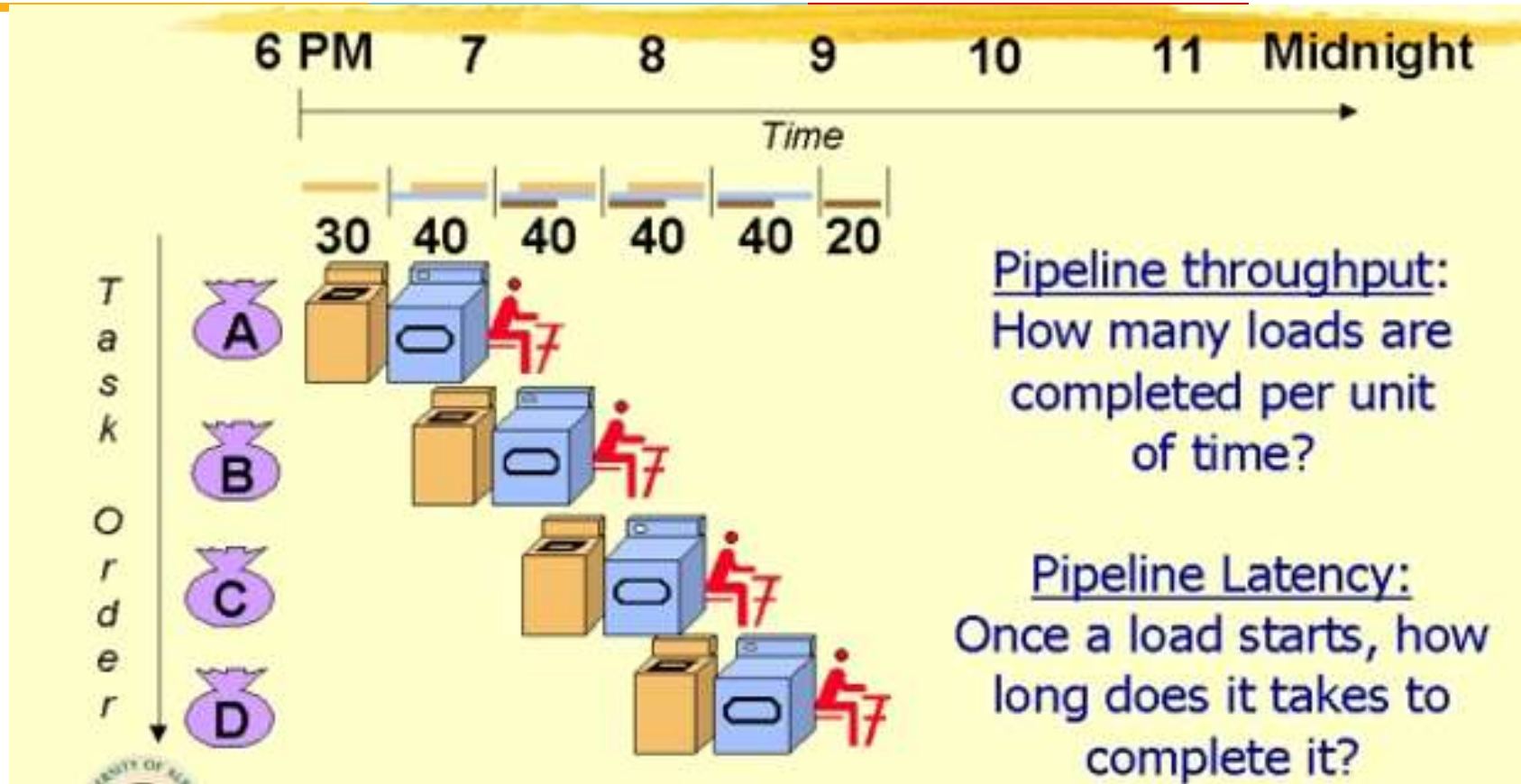
Total Time taken= 6Hrs

Pipelined Laundry Start Work ASAP



Takes only 3.5 Hrs

Pipelined Laundry Start Work ASAP





Additional Stages

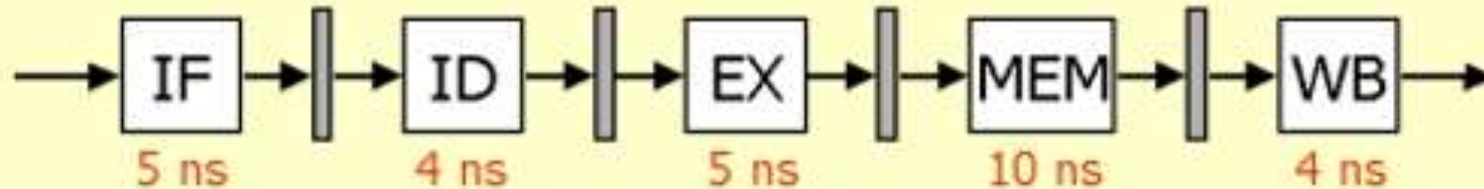
- ❑ Fetch instruction (FI)
 - Read the next expected instruction into a buffer
- ❑ Decode instruction (DI)
 - Determine the opcode and the operand specifiers
- ❑ Calculate operands (CO)
 - Calculate the effective address of each source operand
 - This may involve displacement, register indirect, indirect, or other forms of address calculation
- ❑ Fetch operands (FO)
 - Fetch each operand from memory
 - Operands in registers need not be fetched
- ❑ Execute instruction (EI)
 - Perform the indicated operation and store the result, if any, in the specified destination operand location
- ❑ Write operand or write Back (WO/WB)
 - Store the result in memory

Timing Diagram for Instruction Pipeline Operation

Time →

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO	EI	WO					
Instruction 5					FI	DI	CO	FO	EI	WO				
Instruction 6						FI	DI	CO	FO	EI	WO			
Instruction 7							FI	DI	CO	FO	EI	WO		
Instruction 8								FI	DI	CO	FO	EI	WO	
Instruction 9									FI	DI	CO	FO	EI	WO

Pipeline Throughput and Latency

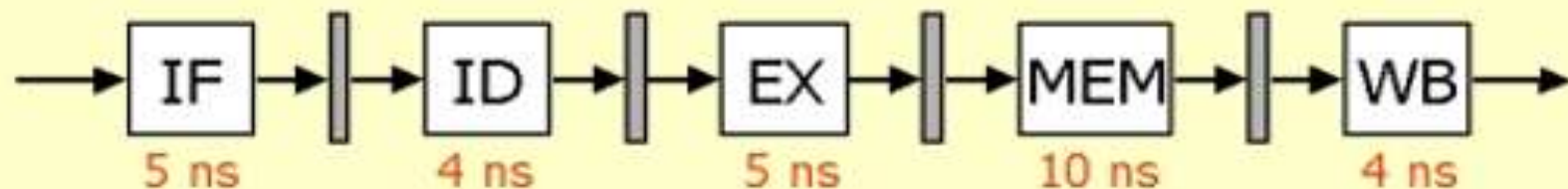


Consider the pipeline above with the indicated delays. We want to know what is the *pipeline throughput* and the *pipeline latency*.

Pipeline throughput: instructions completed per second.

Pipeline latency: how long does it take to execute a single instruction in the pipeline.

Pipeline Throughput and Latency



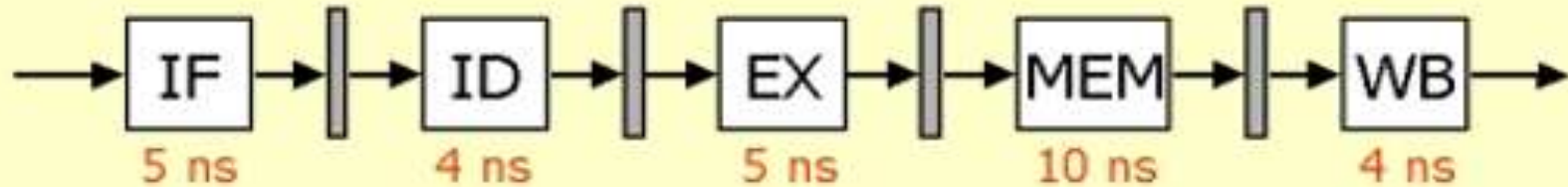
Pipeline throughput: how often an instruction is completed.

$$\begin{aligned} T &= 1_{instr} / \max[lat(IF), lat(ID), lat(EX), lat(MEM), lat(WB)] \\ &= 1_{instr} / \max[5ns, 4ns, 5ns, 10ns, 4ns] \\ &= 1_{instr} / 10ns \end{aligned}$$

Pipeline latency: how long does it take to execute an instruction in the pipeline.

$$L = lat(IF) + lat(ID) + lat(EX) + lat(MEM) + lat(WB)$$

Pipeline Throughput and Latency



Pipeline throughput: how often an instruction is completed.

$$\begin{aligned} T &= 1instr / \max[lat(IF), lat(ID), lat(EX), lat(MEM), lat(WB)] \\ &= 1instr / \max[5ns, 4ns, 5ns, 10ns, 4ns] \\ &= 1instr / 10ns \end{aligned}$$

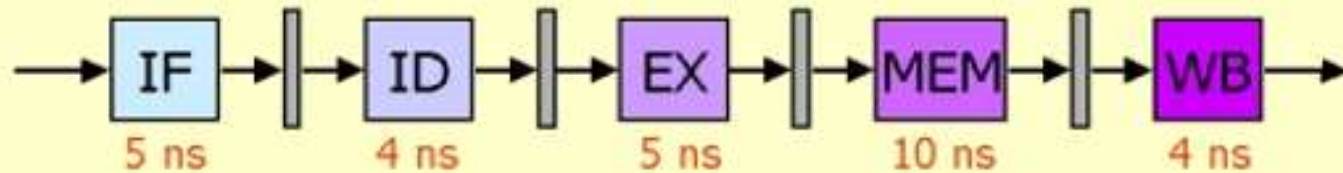
Pipeline latency: how long does it take to execute an instruction in the pipeline.

$$L = lat(IF) + lat(ID) + lat(EX) + lat(MEM) + lat(WB)$$

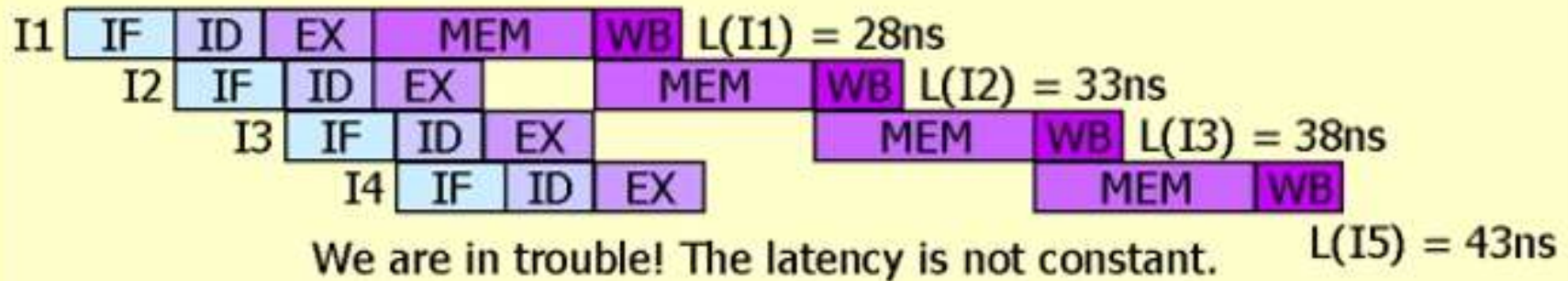
$$= 5ns + 4ns + 5ns + 10ns + 4ns = 28ns$$

Is it correct?

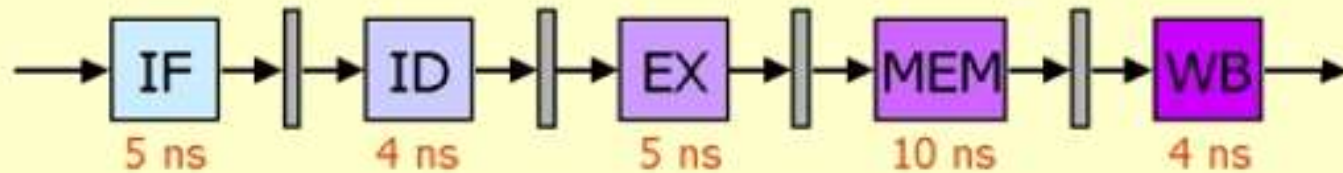
Pipeline Throughput and Latency



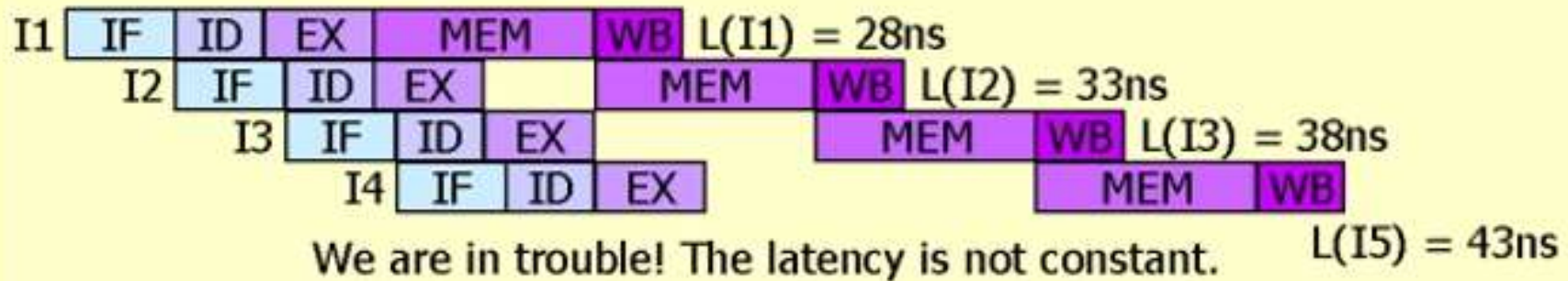
Simply adding the latencies to compute the pipeline latency, only would work for an isolated instruction



Pipeline Throughput and Latency



Simply adding the latencies to compute the pipeline latency, only would work for an isolated instruction

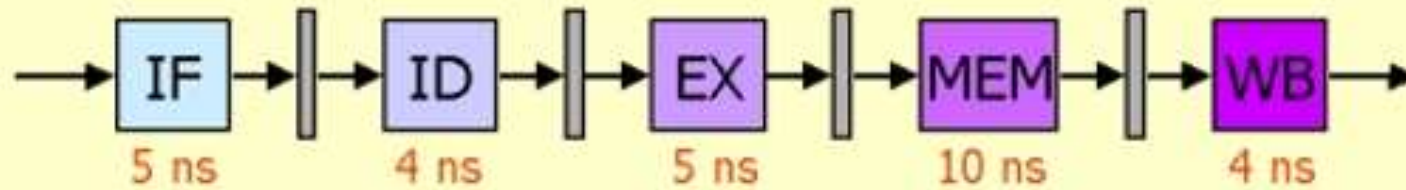


❑ This is an unbalanced pipeline

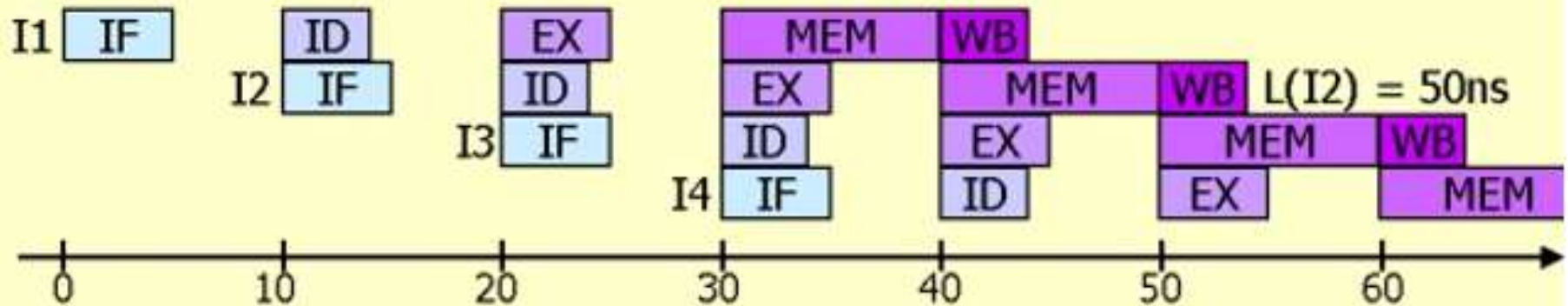
❑ Solution

- ✓ Make every state the same length
- ✓ Longest

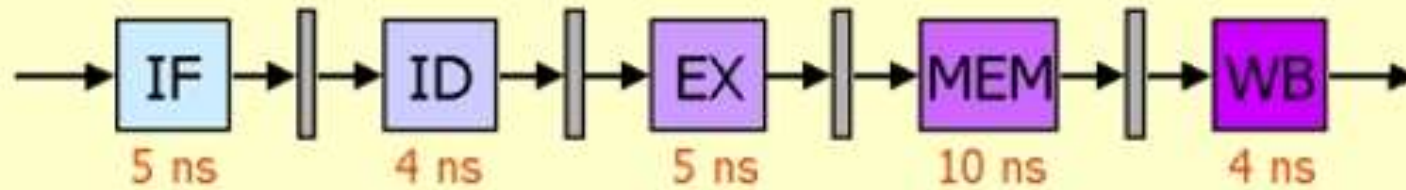
Pipeline Throughput and Latency



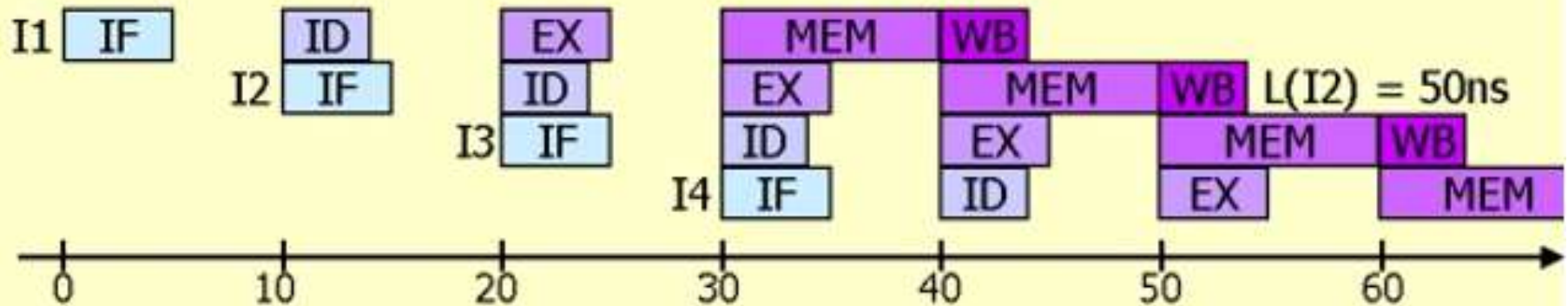
The slowest pipeline state also limits the latency!!



Pipeline Throughput and Latency

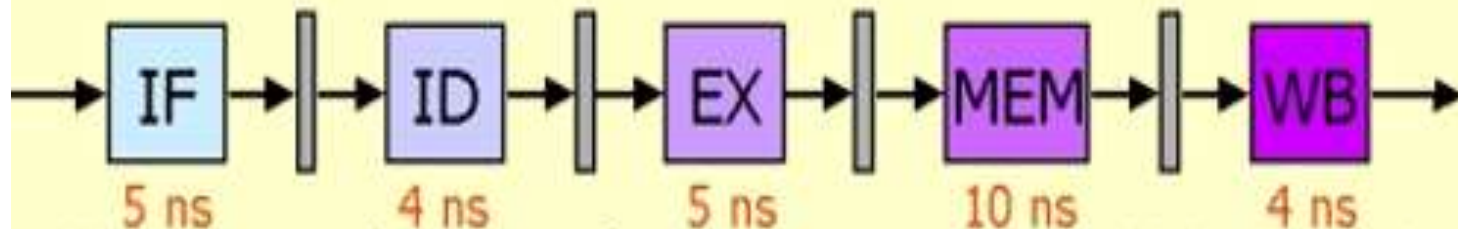


The slowest pipeline state also limits the latency!!



$$L(I_1) = L(I_2) = L(I_3) = L(I_4) = 50$$

Pipeline Throughput and Latency



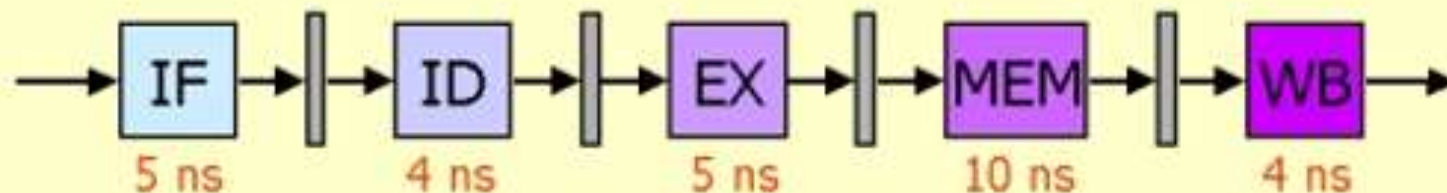
How long does it take to execute 20000 instructions in this pipeline? (disregard bubbles caused by branches, cache misses, and hazards)

$$ExecTime_{pipe} = 20000 \times 10ns = 200000ns = 200\mu s$$

How long would it take using the same modules without pipelining?

$$ExecTime_{non-pipe} = 20000 \times 28ns = 560000ns = 560\mu s$$

Pipeline Throughput and Latency



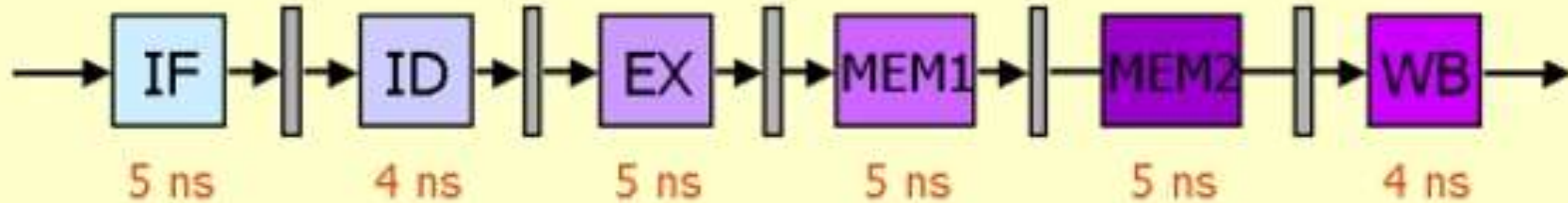
Thus the speedup that we got from the pipeline is:

$$Speedup_{pipe} = \frac{ExecTime_{non-pipe}}{ExecTime_{pipe}} = \frac{560 \mu s}{200 \mu s} = 2.8$$

How can we improve this pipeline design?

We need to reduce the unbalance to increase the clock speed.

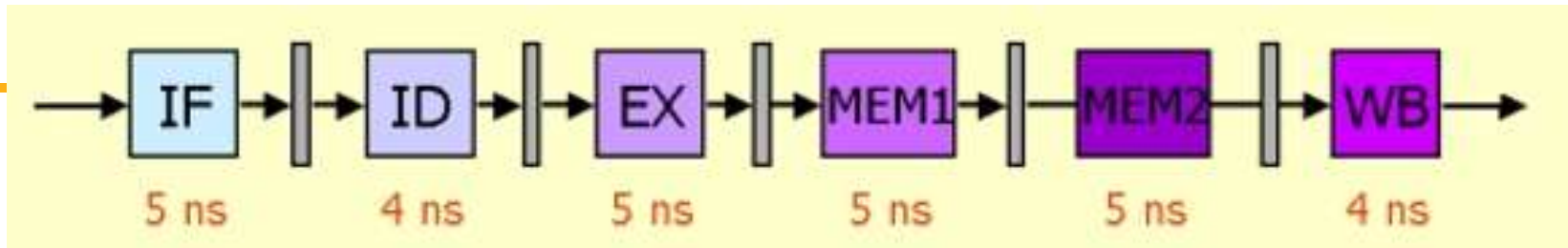
Pipeline Throughput and Latency



Now we have one more pipeline stage, but the maximum latency of a single stage is reduced in half.

$$\begin{aligned} T &= \text{linstr} / \max(\text{lat}(\text{IF}), \text{lat}(\text{ID}), \text{lat}(\text{EX}), \text{lat}(\text{MEM1}), \text{lat}(\text{MEM2}), \text{lat}(\text{WB})) \\ &= \text{linstr} / \max(5\text{ns}, 4\text{ns}, 5\text{ns}, 5\text{ns}, 5\text{ns}, 4\text{ns}) \\ &= \text{linstr} / 5\text{ns} \end{aligned}$$

Pipeline Throughput and Latency



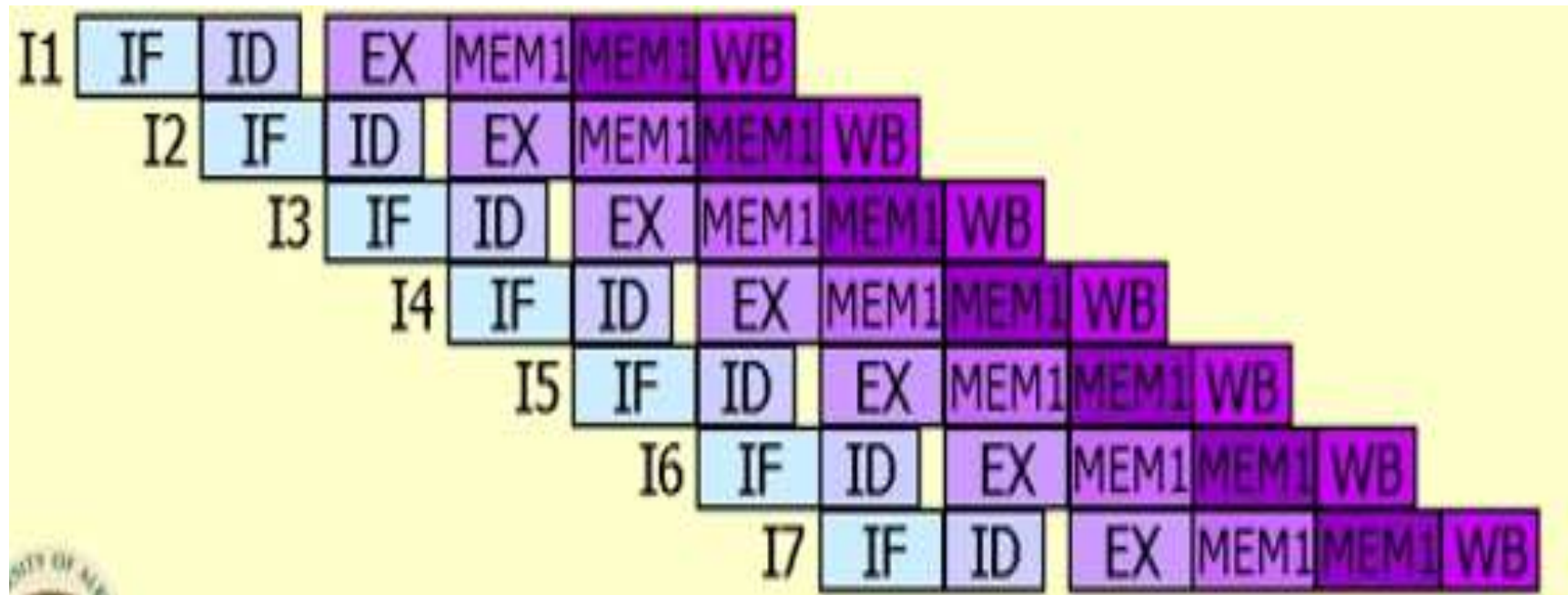
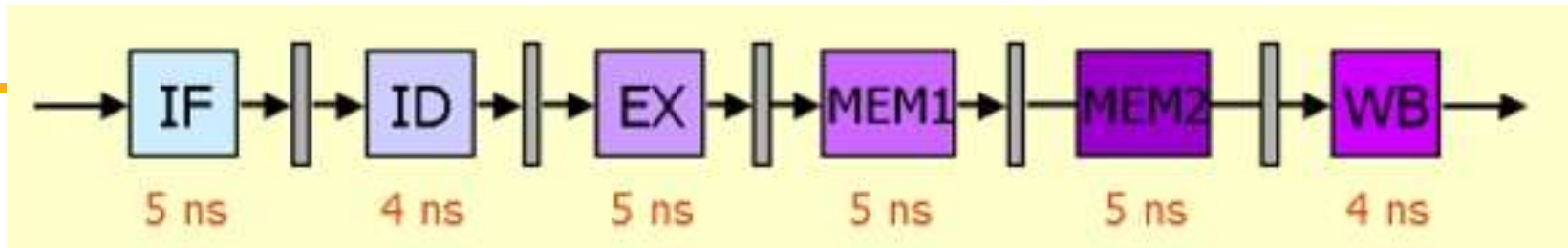
Now we have one more pipeline stage, but the maximum latency of a single stage is reduced in half.

$$\begin{aligned} T &= \text{linstr} / \max(\text{lat}(\text{IF}), \text{lat}(\text{ID}), \text{lat}(\text{EX}), \text{lat}(\text{MEM1}), \text{lat}(\text{MEM2}), \text{lat}(\text{WB})) \\ &= \text{linstr} / \max(5\text{ns}, 4\text{ns}, 5\text{ns}, 5\text{ns}, 5\text{ns}, 4\text{ns}) \\ &= \text{linstr} / 5\text{ns} \end{aligned}$$

The latency of a single Instruction is

$$L = 6 \times 5\text{ns} = 30\text{ns}$$

Pipeline Throughput and Latency





Speed-up $S(n)$

Consider the execution of m tasks (instructions) using n -stages (units) pipeline. As can be seen, $n + m - 1$ time units are required to complete m tasks.

$$\text{Speed-up } S(n) = \frac{\text{Time using sequential processing}}{\text{Time using pipeline processing}} = \frac{m \times n \times t}{(n + m - 1) \times t}$$
$$= \frac{m \times n}{n + m - 1}$$

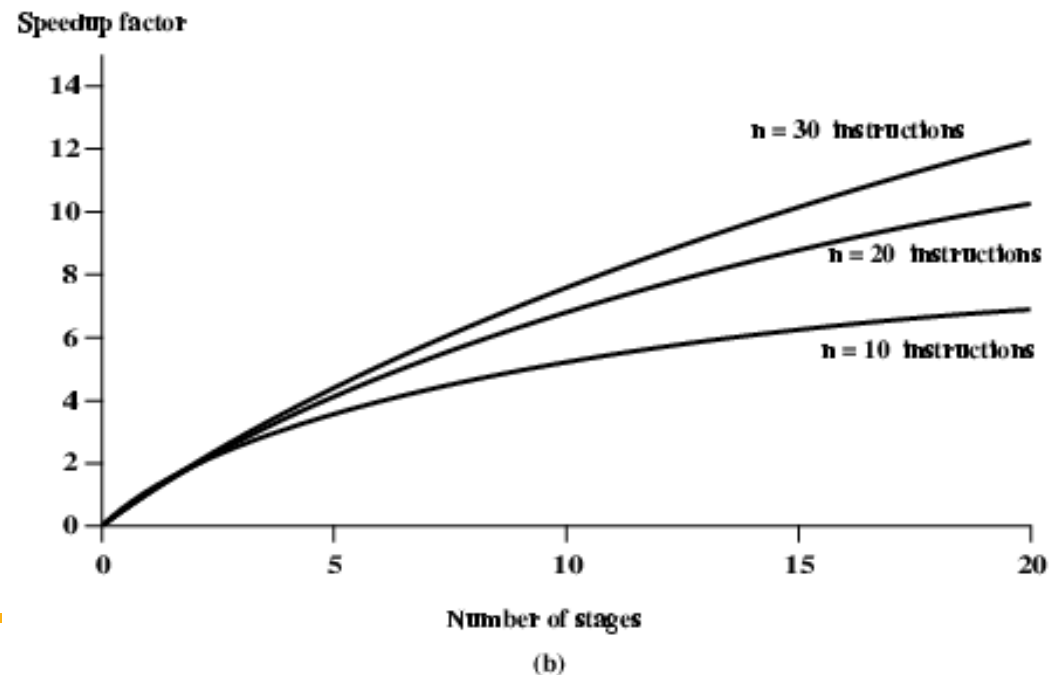
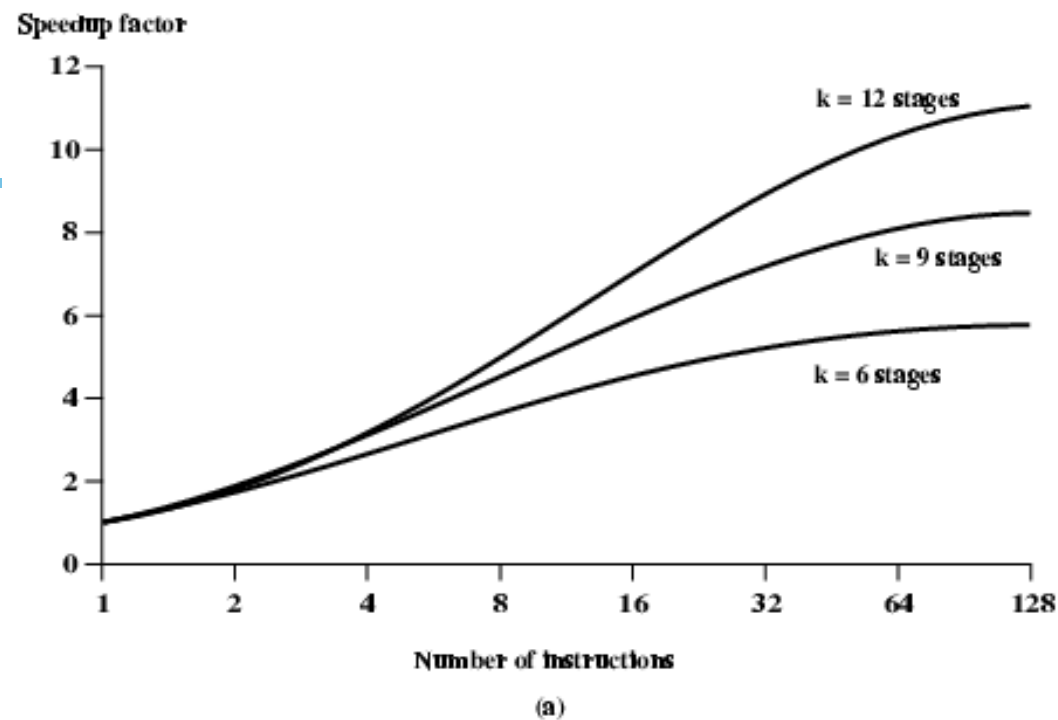
$$\lim_{m \rightarrow \infty} s(n) = n$$

(i.e: n -fold increase in speed is theoretically possible)

Speed-up $S(n)$

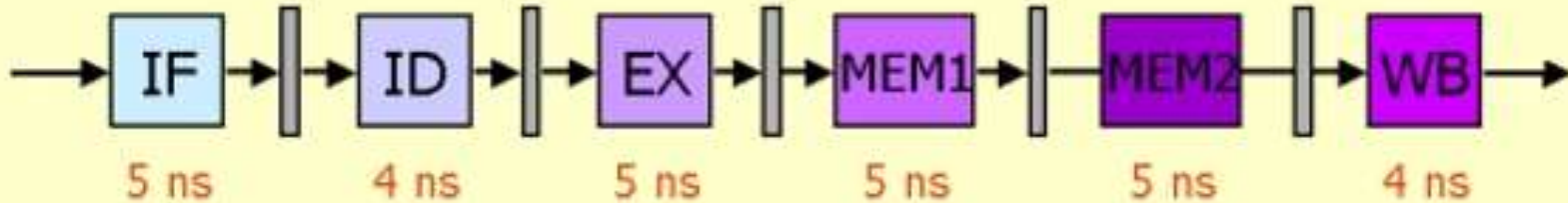
the larger the number of pipeline stages, the greater the potential for speedup.

However, as a practical matter, the potential gains of additional pipeline stages are countered by increases in cost, delays between stages, and the fact that branches will be encountered requiring the flushing of the pipeline



Pipeline Throughput and Latency

Throughput $U(n)$



How long does it take to execute 20000 instructions in this pipeline? (disregard bubbles caused by branches, cache misses, etc, for now)

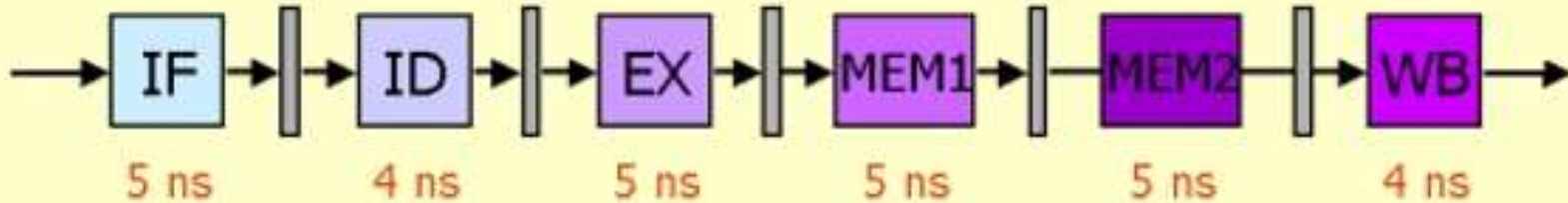
$$ExecTime_{pipe} = 20000 \times 5ns = 100000ns = 100\mu s$$

Thus the speedup that we get from the pipeline is:

$$Speedup_{pipe} = \frac{ExecTime_{non-pipe}}{ExecTime_{pipe}} = \frac{560\mu s}{100\mu s} = 5.6$$

Pipeline Throughput and Latency

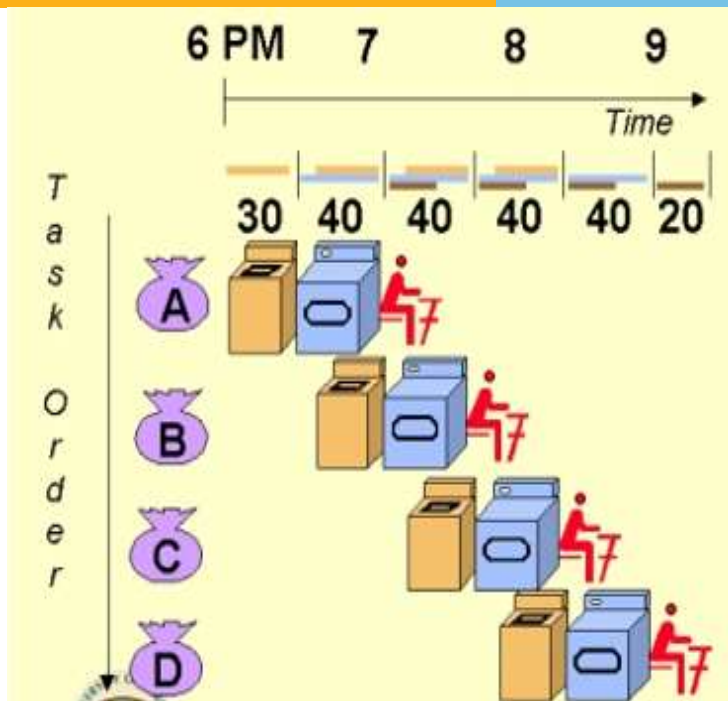
Throughput $U(n)$



What have we learned from this example?

1. It is important to balance the delays in the stages of the pipeline
2. The throughput of a pipeline is $1/\max(\text{delay})$.
3. The latency is $N \times \max(\text{delay})$, where N is the number of stages in the pipeline.

Traditional Pipeline Concept



1. Pipelining doesn't help latency of single task, it helps throughput of entire workload
2. Pipeline rate limited by slowest pipeline stage
3. Multiple tasks operating simultaneously using different resources
4. Potential speedup = Number pipe stages
5. Unbalanced lengths of pipe stages reduces speedup
6. Time to “fill” pipeline and time to “drain” it reduces speedup
7. Stall for Dependences



- ❑ Each pipeline stage is expected to complete in one clock cycle.
- ❑ The clock period should be long enough to let the slowest pipeline stage to complete.
- ❑ Faster stages can only wait for the slowest one to complete.
- ❑ Since main memory is very slow compared to the execution, if each instruction needs to be fetched from main memory, pipeline is almost useless.
- ❑ Fortunately, we have cache.



Pipeline Hazards

Pipeline, or some portion of pipeline, must stall

Also called *pipeline bubble*

Types of hazards

- Resource
- Data
- Control

Resource Hazard Diagram

	Clock cycle								
	1	2	3	4	5	6	7	8	9
Instruction	I1	FI	DI	FO	EI	WO			
	I2		FI	DI	FO	EI	WO		
	I3			FI	DI	FO	EI	WO	
	I4				FI	DI	FO	EI	WO

(a) Five-stage pipeline, ideal case

A resource hazard occurs when two or more instructions that are already in the pipeline need the same resource

The result is that the instructions must be executed in serial rather than parallel for a portion of the pipeline

	Clock cycle								
	1	2	3	4	5	6	7	8	9
Instruction	I1	FI	DI	FO	EI	WO			
	I2		FI	DI	FO	EI	WO		
	I3			Idle	FI	DI	FO	EI	WO
	I4				FI	DI	FO	EI	WO

(b) I1 source operand in memory

A resource hazard is sometimes referred to as a structural hazard



Resource Hazards

- ❑ Two (or more) instructions in pipeline need same resource
- ❑ Executed in serial rather than parallel for part of pipeline
- ❑ E.g. Assume simplified five-stage pipeline: Each stage takes one clock cycle
- ❑ Ideal case is new instruction enters pipeline each clock cycle
- ❑ Assume main memory has single port
- ❑ Assume instruction fetches and data reads and writes performed one at a time
- ❑ Ignore the cache
- ❑ Operand read or write cannot be performed in parallel with instruction fetch
- ❑ Fetch instruction stage must idle for one cycle fetching I3
- ❑ E.g. multiple instructions ready to enter execute instruction phase
- ❑ Single ALU
- ❑ One solution: increase available resources
 - Multiple main memory ports
 - Multiple ALUs

Data Hazard Diagram : RAW hazard

		Clock cycle									
		1	2	3	4	5	6	7	8	9	10
ADD EAX, EBX		FI	DI	FO	EI	WO					
SUB ECX, EAX			FI	DI	Idle		FO	EI	WO		
I3				FI			DI	FO	EI	WO	
I4							FI	DI	FO	EI	WO

Data Hazards : A data hazard occurs when there is a conflict in the access of an operand location



Types of Data Hazard

- ☐ Read after write (RAW), or true dependency
 - ✓ An instruction modifies a register or memory location
 - ✓ Succeeding instruction reads data in memory or register location
 - ✓ Hazard occurs if the read takes place before write operation is complete
- ☐ Write after read (WAR), or anti-dependency
 - ✓ An instruction reads a register or memory location
 - ✓ Succeeding instruction writes to the location
 - ✓ Hazard occurs if the write operation completes before the read operation takes place
- ☐ Write after write (WAW), or output dependency
 - ✓ Two instructions both write to the same location
 - ✓ Hazard occurs if the write operations take place in the reverse order of the intended sequence

Data Hazards



- ✓ Conflict in access of an operand location
- ✓ Two instructions to be executed in sequence
- ✓ Both access a particular memory or register operand
- ✓ If in strict sequence, no problem occurs
- ✓ If in a pipeline, operand value could be updated so as to produce different result from strict sequential execution
- ✓ E.g. x86 machine instruction sequence:
ADD EAX, EBX /* EAX = EAX + EBX
SUB ECX, EAX /* ECX = ECX – EAX
- ✓ ADD instruction does not update EAX until end of stage 5, at clock cycle 5
- ✓ SUB instruction needs value at beginning of its stage 2, at clock cycle 4
- ✓ Pipeline must stall for two clocks cycles
- ✓ Without special hardware and specific avoidance algorithms, results in inefficient pipeline usage



Control Hazard

- ❑ Also known as a branch hazard
- ❑ Occurs when the pipeline makes the wrong decision on a branch prediction
- ❑ Brings instructions into the pipeline that must subsequently be discarded
- ❑ Dealing with Branches:
 - ✓ Multiple streams
 - ✓ Prefetch branch target
 - ✓ Loop buffer
 - ✓ Branch prediction
 - ✓ Delayed branch

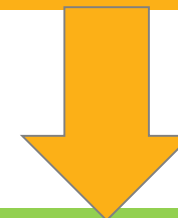


Multiple Streams

A simple pipeline suffers a penalty for a branch instruction because it must choose one of two instructions to fetch next and may make the wrong choice



A brute-force approach is to replicate the initial portions of the pipeline and allow the pipeline to fetch both instructions, making use of two streams



Drawbacks:

- With multiple pipelines there are contention delays for access to the registers and to memory
- Additional branch instructions may enter the pipeline before the original branch decision is resolved

Despite these drawbacks, this strategy can improve performance. Examples of machines with two or more pipeline streams are the IBM 370/168 and the IBM 3033



Prefetch Branch Target

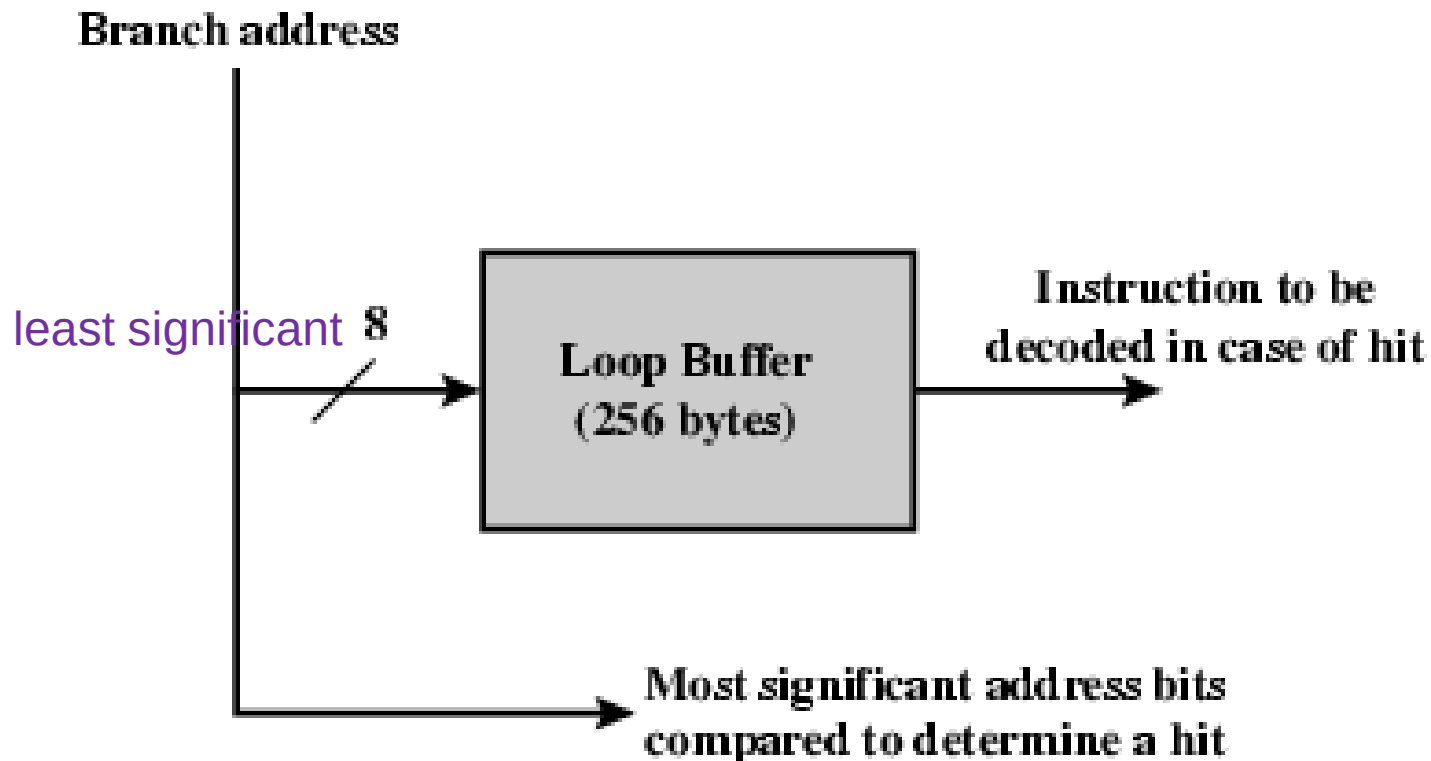
- ❑ When a conditional branch is recognized, the target of the branch is prefetched, in addition to the instruction following the branch
- ❑ Target is then saved until the branch instruction is executed
- ❑ If the branch is taken, the target has already been prefetched
- ❑ IBM 360/91 uses this approach



Loop Buffer

- ❑ Small, very-high speed memory maintained by the instruction fetch stage of the pipeline and containing the n most recently fetched instructions, in sequence
- ❑ Benefits:
 - ✓ Instructions fetched in sequence will be available without the usual memory access time
 - ✓ If a branch occurs to a target just a few locations ahead of the address of the branch instruction, the target will already be in the buffer
 - ✓ This strategy is particularly well suited to dealing with loops
- ❑ Similar in principle to a cache dedicated to instructions
- ✓ Differences:
 - The loop buffer only retains instructions in sequence
 - Is much smaller in size and hence lower in cost

Loop Buffer Diagram



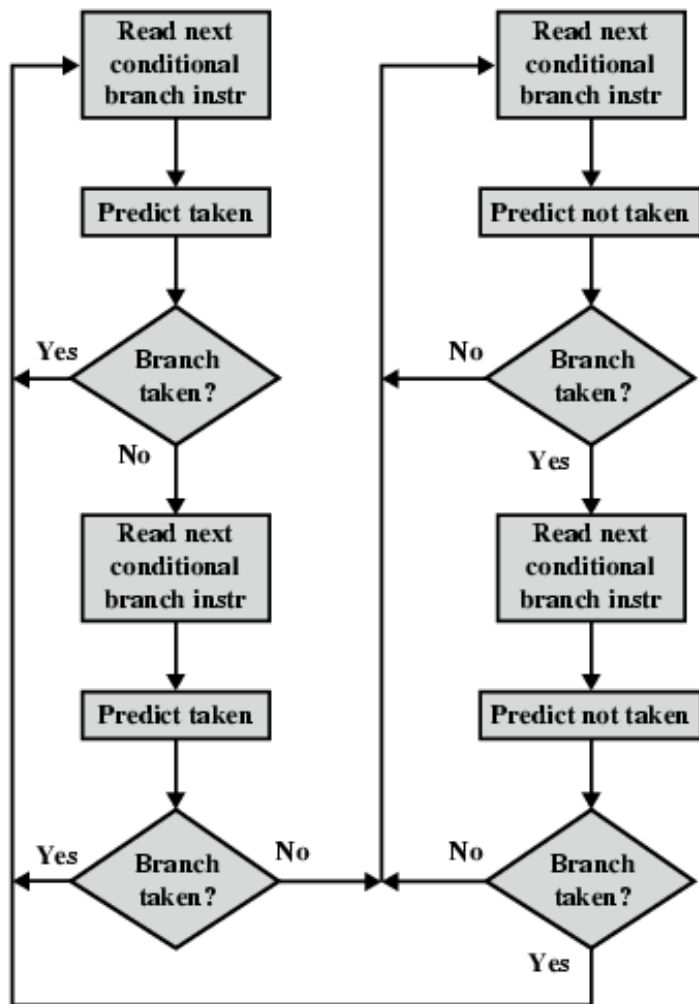
The remaining most significant bits are checked to determine if the branch target lies within the environment captured by the buffer

Branch Prediction

Various techniques can be used to predict whether a branch will be taken:

1. Predict never taken	These approaches are static They do not depend on the execution history up to the time of the conditional branch instruction
2. Predict always taken	
3. Predict by opcode	
Taken/not taken switch	These approaches are dynamic They depend on the execution history
Taken/not taken switch	

Branch Prediction Flowchart



With a single bit, all that can be recorded is whether the last execution of this instruction resulted in a branch or not

. A shortcoming of using a single bit appears in the case of a conditional branch instruction that is almost always taken, such as a loop instruction. With only one bit of history, an error in prediction will occur twice for each use of the loop: once on entering the loop, and once on exiting.

If two bits are used, they can be used to record the result of the last two instances of the execution of the associated instruction





Dynamic Branch Prediction

Why does prediction work?

- Underlying algorithm has regularities
- Data that is being operated on has regularities
- Instruction sequence has redundancies that are artifacts of way that humans/compiler think about problems

Is dynamic branch prediction better than static branch prediction?

- Seems to be
- There are a small number of important branches in programs which have dynamic behavior



Dynamic Branch Prediction

Performance = $f(\text{accuracy, cost of misprediction})$

Branch History Table(BHT): Lower bits of PC address index table of 1-bit values

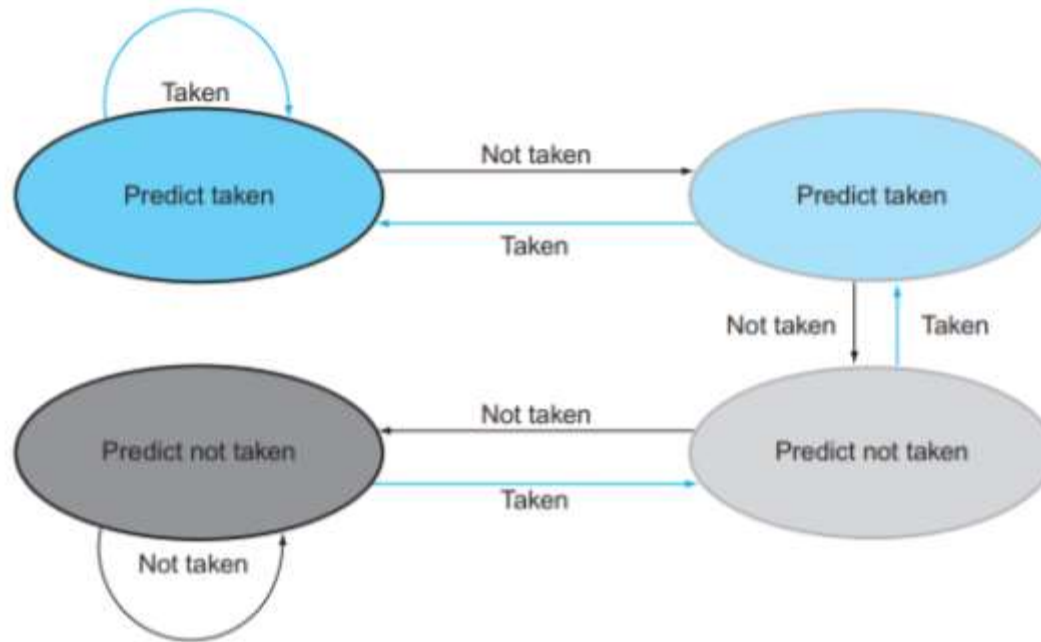
- Says whether or not branch taken last time
- No address check

Problem: in a loop, 1-bit BHT will cause two mispredictions (avg is 9 iterations before exit):

- End of loop case, when it exits instead of looping as before
- First time through loop on *next* time through code, when it predicts exit instead of looping

Dynamic Branch Prediction

Solution: 2-bit scheme where change prediction only if get misprediction *twice*





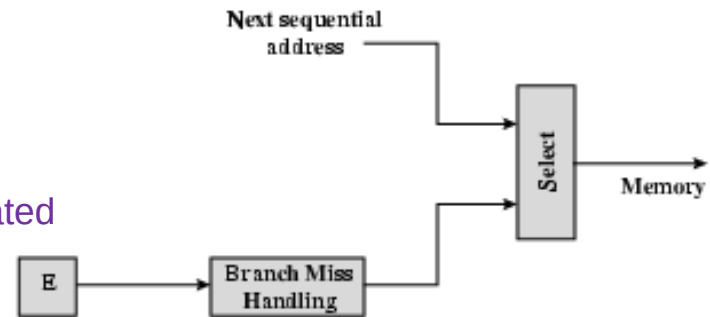
Branch Prediction State Diagram

Dealing With Branches

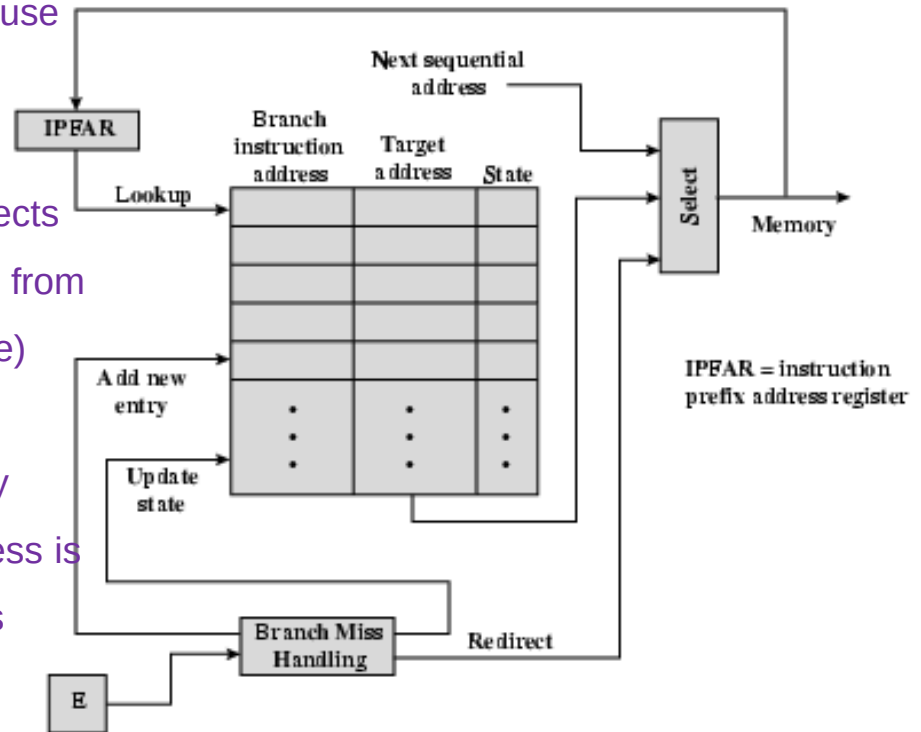
The branch history table is a small cache memory associated with the instruction fetch stage of the pipeline.

Each entry in the table consists of three elements:

- ✓ the address of a branch instruction,
- ✓ some number of history bits that record the state of use of that instruction,
- ✓ and information about the target instruction
- ✓ If a branch is taken, some logic in the processor detects this and instructs that the next instruction be fetched from the target address (in addition to flushing the pipeline)
- ✓ The branch history table is treated as a cache.
- ✓ Each prefetch triggers a lookup in the branch history table. If no match is found, the next sequential address is used for the fetch. If a match is found, a prediction is made based on the state of the instruction:
- ✓ Either the next sequential address or the branch target address is fed to the select logic.



(a) Predict never taken strategy



(b) Branch history table strategy



Intel 80486 Pipelining

Fetch

- From cache or external memory
- Put in one of two 16-byte prefetch buffers
- Fill buffer with new data as soon as old data consumed
- Average 5 instructions fetched per load
- Independent of other stages to keep buffers full

Decode stage 1

- Opcode & address-mode info
- At most first 3 bytes of instruction
- Can direct D2 stage to get rest of instruction

Decode stage 2

- Expand opcode into control signals
- Computation of complex address modes

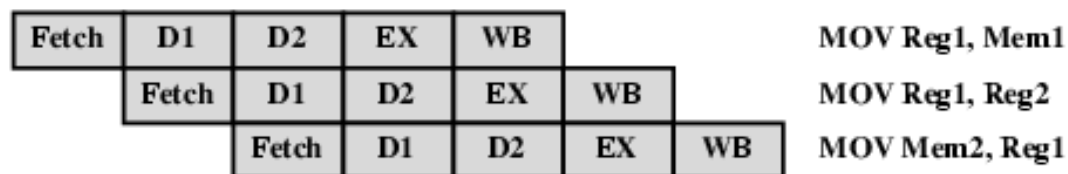
Execute

- ALU operations, cache access, register update

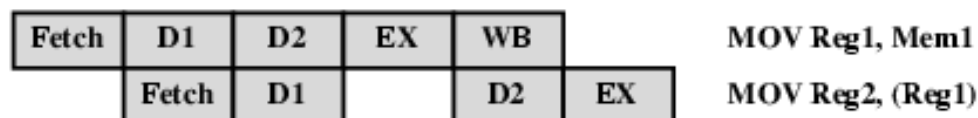
Writeback

- Update registers & flags
- Results sent to cache & bus interface write buffers

80486 Instruction Pipeline Examples



(a) No Data Load Delay in the Pipeline



(b) Pointer Load Delay



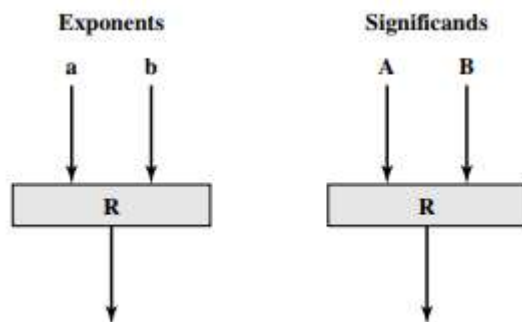
(c) Branch Instruction Timing

Pipelining Example

Pipelining can be applied within the ALU to speed up floating-point operations. Consider the case of floating-point addition and subtraction. In simplified terms, the pipeline could have four stages:

- (1) Compare the exponents;
- (2) Choose the exponent and align the significands;
- (3) Add or subtract significands;
- (4) Normalize the results.

The pipeline can be considered to have two parallel threads, one handling exponents and one handling significands, and could start out like this:

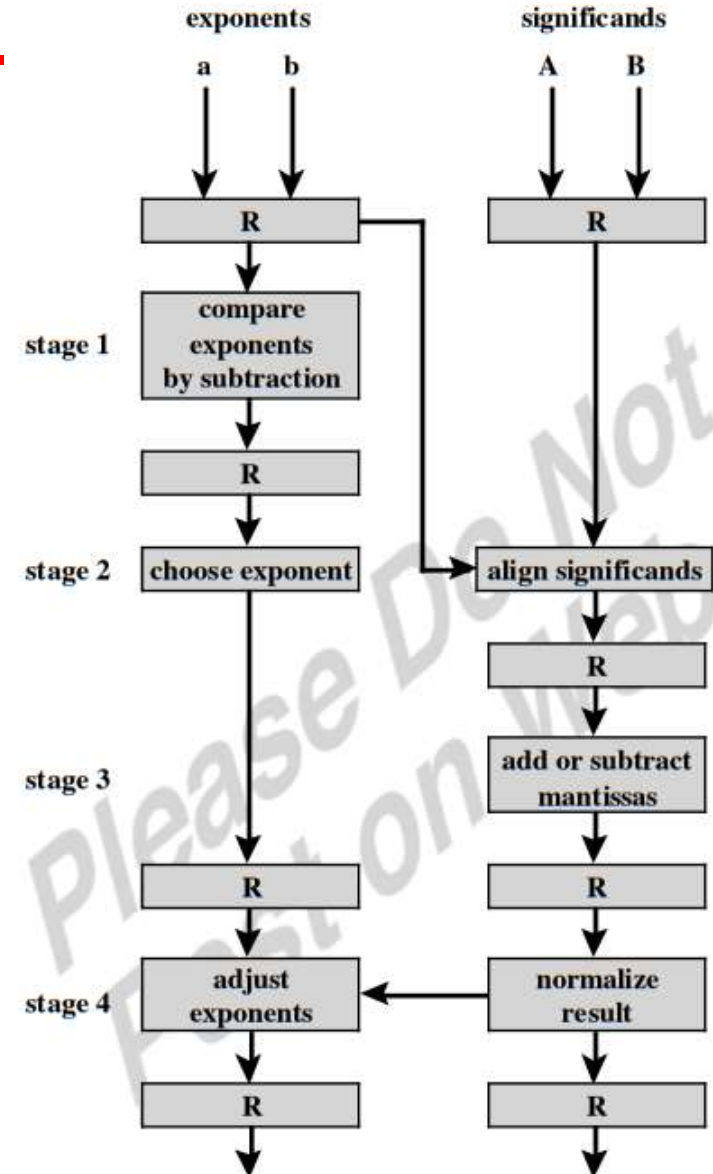


In this figure, the boxes labeled R refer to a set of registers used to hold temporary results. Complete the block diagram that shows at a top level the structure of the pipeline



Pipelining Example

The pipeline can be considered to have two parallel threads, one handling exponents and one handling significands, and could start out like this:





In class exercise

Given a deeply pipelined processor and a branch-target buffer for conditional branches only, assuming a misprediction penalty of 4 cycles and a buffer miss penalty of 3 cycles, 90% hit rate and 90% accuracy, and 15% branch frequency. How much faster is the processor with the BTB vs. a processor that has a fixed 2-cycle branch penalty?

$$\text{Speedup} = \text{CPI}_{\text{noBTB}} / \text{CPI}_{\text{BTB}}$$

$$= (\text{CPI}_{\text{base}} + \text{Stalls}_{\text{noBTB}}) / (\text{CPI}_{\text{base}} + \text{Stalls}_{\text{BTB}})$$

$$\text{Stalls} = \sum \text{Frequency} \times \text{Penalty}$$

$$\text{Stalls}_{\text{noBTB}} = 15\% \times 2 = 0.30$$

$$\begin{aligned} \text{Stalls}_{\text{BTB}} &= \text{Stalls}_{\text{miss-BTB}} + \text{Stalls}_{\text{hit-BTB-correct}} + \text{Stalls}_{\text{hit-BTB-wrong}} \\ &= 15\% \times 10\% \times 3 + 15\% \times 90\% \times 90\% \times 0 + 15\% \times 90\% \times 10\% \times 4 \\ &= 0.097 \end{aligned}$$

$$\text{Speedup} = (1 + 0.3) / (1 + 0.097) = 1.2$$

Example 1

1. Consider a machine which supports the following two instruction schedules for R class and I class instructions. Assume an instruction mix of 60% R class and 40% I class instructions. Assume that IF steps take 25 nanoseconds, MEM steps of instruction execution require 45 nanoseconds and the other steps require 20 nanoseconds.

	0	1	2	3	4
R Class	IF	ID	EX	WB	
I Class	IF	ID	EX	MEM	WB

For a multi-cycle implementation,

- What is the minimum clock cycle time?
- How long does it take to execute 100 instructions in nanoseconds?



Example 1 Sol

1. For a multi-cycle implementation, clock cycle time is the time for the longest stage => 45 ns
2. For a multi-cycle implementation,
$$\text{exec_time} = 100 \times (\text{exec_time R} + \text{exec_time I})$$
$$= 100 \times (\text{cycle time} \times \text{CPI} \times \text{IC}) \text{ R} + (\text{cycle time} \times \text{CPI} \times \text{IC}) \text{ I}$$
$$= 100 \times (4 \times 45 \times 0.6 + 5 \times 45 \times 0.4)$$
$$= 19800\text{ns}$$



Example 2

Eg. Consider an un-pipelined processor. Assume that it has 1-ns clock cycle and that it uses 4 cycles for ALU operations and 5 cycles for branches and 4 cycles for memory operations.

Assume that the relative frequencies of these operations are 50%, 35% and 15% respectively.

Suppose that due to clock skew and set up, pipelining the processor adds 0.15 ns of overhead to the clock. Ignoring any latency impact, how much speed up in the instruction execution rate will we gain from a pipeline?

The average instruction execution time on an un-pipelined processor is

= clock cycle $\times$ Avg. CPI

= 1 ns $\times$ ((0.5 $\times$ 4) + (0.35 $\times$ 5) + (0.15 $\times$ 4))

= 4.35 ns

The avg. instruction execution time on pipelined processor is = 1 ns + 0.2 ns = 1.2 ns

So speed up = 4.35/1.2 = 3.625



Speed Up Equation for Pipelining

$$CPI_{\text{pipelined}} = \text{Ideal CPI} + \text{Average Stall cycles per Inst}$$

$$\text{Speedup} = \frac{\text{Ideal CPI} \times \text{Pipeline depth}}{\text{Ideal CPI} + \text{Pipeline stall CPI}} \times \frac{\text{Cycle Time}_{\text{unpipelined}}}{\text{Cycle Time}_{\text{pipelined}}}$$

For simple RISC pipeline, Ideal CPI = 1:

$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Cycle Time}_{\text{unpipelined}}}{\text{Cycle Time}_{\text{pipelined}}}$$



Example: Dual-port vs. Single-port

Machine A: Dual ported memory (“Harvard Architecture”)

Machine B: Single ported memory, but its pipelined implementation has a 1.05 times faster clock rate

Ideal CPI = 1 for both

Loads are 40% of instructions executed

$$\begin{aligned}\text{SpeedUp}_A &= \text{Pipeline Depth} / (1 + 0) \times (\text{clock}_{\text{unpipe}} / \text{clock}_{\text{pipe}}) \\ &= \text{Pipeline Depth}\end{aligned}$$

$$\begin{aligned}\text{SpeedUp}_B &= \text{Pipeline Depth} / (1 + 0.4 \times 1) \times (\text{clock}_{\text{unpipe}} / (\text{clock}_{\text{unpipe}} / 1.05)) \\ &= (\text{Pipeline Depth} / 1.4) \times 1.05 \\ &= 0.75 \times \text{Pipeline Depth}\end{aligned}$$

$$\text{SpeedUp}_A / \text{SpeedUp}_B = \text{Pipeline Depth} / (0.75 \times \text{Pipeline Depth}) = 1.33$$

Machine A is 1.33 times faster



Evaluating Branch Alternatives

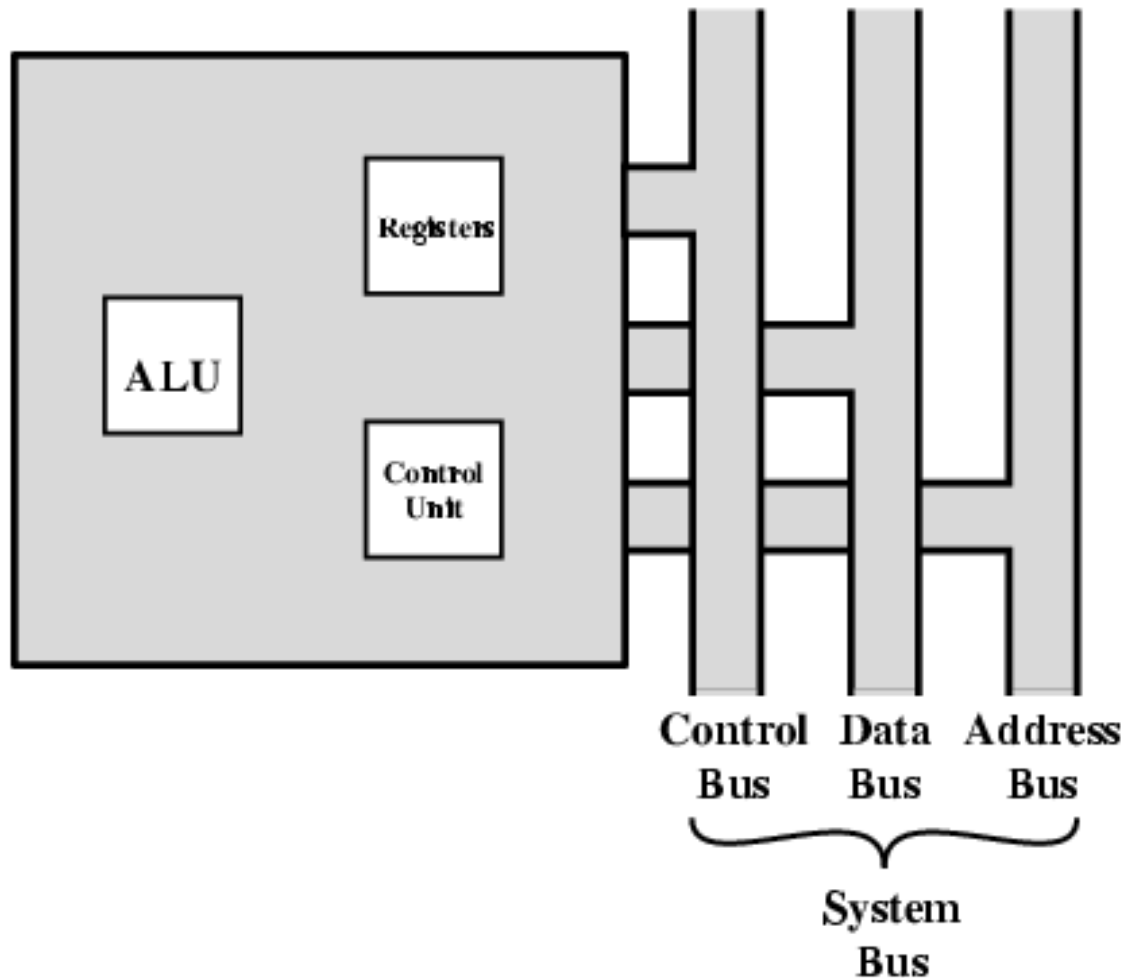
Assume 4% unconditional branch, 6% conditional branch- untaken, 10% conditional branch-taken; Deeper pipeline: target & condition known @ end of 3rd & 4th stage respectively

What's the addition to CPI?

<i>Scheduling</i>	<i>Uncond</i>	<i>Untaken</i>	<i>Taken.</i>
<i>scheme</i>	<i>penalty</i>	<i>penalty</i>	<i>penalty</i>
Stall pipeline	2	3	3
Predict taken	2	3	2
Predict not taken	2	0	3

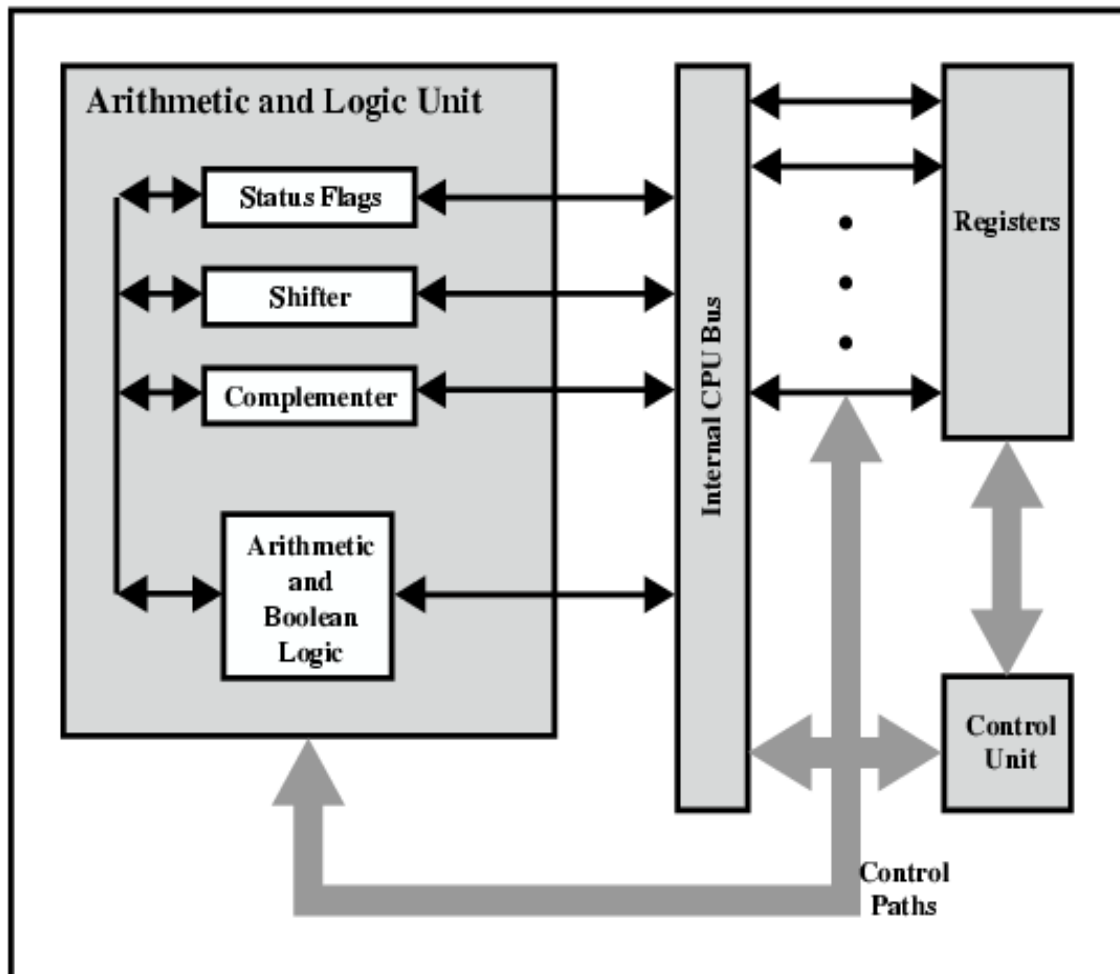
<i>Scheduling</i>	<i>Uncond</i>	<i>Untaken</i>	<i>Taken</i>	<i>All branches</i>
<i>scheme</i>	<i>branches</i>	<i>branches</i>	<i>branches</i>	
	4%	6%	10%	20%
Stall pipeline	0.08	0.18	0.3	0.56
Predict taken	0.08	0.18	0.2	0.46
Predict not taken	0.08	0.00	0.3	0.38

CPU With Systems Bus



- ❑ major components :
 - arithmetic and logic unit (ALU) and a control unit (CU)
- ❑ a minimal internal memory, consisting of a set of storage locations, called registers.

CPU Internal Structure



- ❑ similarity between the internal structure of the computer as a whole and the internal structure of the processor.
- ❑ In both cases, there is a small collection of major elements (**computer**: processor, I/O, memory; **processor**: control unit, ALU, registers) connected by data paths.



Registers

- ❑ CPU must have some working space (temporary storage)
- ❑ Called registers
- ❑ Number and function vary between processor designs
- ❑ One of the major design decisions
- ❑ Top level of memory hierarchy
- ❑ The registers in the processor perform two roles

User-Visible Registers	Control and Status Registers
Enable the machine or assembly language programmer to minimize main memory references by optimizing use of registers	Used by the control unit to control the operation of the processor and by privileged operating system programs to control the execution of programs



User-Visible Registers

Categories:

☐ General purpose

- ✓ Can be assigned to a variety of functions by the programmer

☐ Data

- ✓ May be used only to hold data and cannot be employed in the calculation of an operand address

☐ Address

- ✓ May be somewhat general purpose or may be devoted to a particular addressing mode
- ✓ Examples: segment pointers, index registers, stack pointer

☐ Condition codes

- ✓ Also referred to as flags
- ✓ Bits set by the processor hardware as the result of operation

Referenced by means of the machine language that the processor executes



User-Visible Registers

General-purpose registers can be assigned to a variety of functions by the programmer. Sometimes their use within the instruction set is orthogonal to the operation. That is, any general-purpose register can contain the operand for any opcode. This provides true general-purpose register use. Often, however, there are restrictions. For example, there may be dedicated registers for floating-point and stack operations.

In some cases, general-purpose registers can be used for addressing functions (e.g., register indirect, displacement). In other cases, there is a partial or clean separation between data registers and address registers. Data registers may be used only to hold data and cannot be employed in the calculation of an operand address.



User-Visible Registers

- ❑ **Address registers** may themselves be somewhat general purpose, or they may be devoted to a particular addressing mode. Examples include the following:
 - ✓ **Segment pointers:** In a machine with segmented addressing, a segment register holds the address of the base of the segment. There may be multiple registers: for example, one for the operating system and one for the current process.
 - ✓ **Index registers:** These are used for indexed addressing and may be auto-indexed.
 - ✓ **Stack pointer:** If there is user-visible stack addressing, then typically there is a dedicated register that points to the top of the stack. This allows implicit addressing; that is, push, pop, and other stack instructions need not contain an explicit stack operand.



User-Visible Registers

□ There are several design issues

- ✓ whether to use completely general-purpose registers or to specialize their use
 - this specialization limits the programmer's flexibility.
- ✓ Another design issue is the number of registers
 - affects instruction set design
- ✓ issue of register length
 - Large enough to hold full address
 - Large enough to hold full word
 - Often possible to combine two data registers
 - C programming
 - `double int a;`
 - `long int a;`

User-Visible Registers

- ❑ A final category of registers, which is at least partially visible to the user, holds condition codes (also referred to as flags)
- ❑ Condition code bits are collected into one or more registers. Usually, they form part of a control register. but the programmer cannot alter them.
- ❑ **Condition Codes**

Advantages	Disadvantages
1. Because condition codes are set by normal arithmetic and data movement instructions, they should reduce the number of COMPARE and TEST instructions needed. 2. Conditional instructions, such as BRANCH are simplified relative to composite instructions, such as TEST AND BRANCH. 3. Condition codes facilitate multiway branches. For example, a TEST instruction can be followed by two branches, one on less than or equal to zero and one on greater than zero.	1. Condition codes add complexity, both to the hardware and software. Condition code bits are often modified in different ways by different instructions, making life more difficult for both the microprogrammer and compiler writer. 2. Condition codes are irregular; they are typically not part of the main data path, so they require extra hardware connections. 3. Often condition code machines must add special non-condition-code instructions for special situations anyway, such as bit checking, loop control, and atomic semaphore operations. 4. In a pipelined implementation, condition codes require special synchronization to avoid conflicts.



Control and Status Registers

Four registers are essential to instruction execution:

- Program counter (PC)

- Contains the address of an instruction to be fetched

- Instruction register (IR)

- Contains the instruction most recently fetched

- Memory address register (MAR)

- Contains the address of a location in memory

- Memory buffer register (MBR)

- Contains a word of data to be written to memory or the word most recently read



Program Status Word

A set of bits

Includes Condition Codes

Sign of last result

Zero

Carry

Equal

Overflow

Interrupt enable/disable

Supervisor



Program Status Word (PSW)

- ❑ **Supervisor:** Indicates whether the processor is executing in supervisor or user mode. Certain privileged instructions can be executed only in supervisor mode, and certain areas of memory can be accessed only in supervisor mode.
- ❑ Intel ring zero Kernel mode
- ❑ **Other Registers**
 - ✓ May have registers pointing to:
 - Process control blocks (see O/S)
 - Interrupt Vectors (see O/S)

Example Register Organizations

Data Registers	
D0	
D1	
D2	
D3	
D4	
D5	
D6	
D7	

Address Registers	
A0	
A1	
A2	
A3	
A4	
A5	
A6	
A7	
A7'	

Program Status	
Program Counter	
Status Register	

(a) MC68000

General Registers

AX	Accumulator
BX	Base
CX	Count
DX	Data

Pointer & Index

SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Dest Index

Segment

CS	Code
DS	Data
SS	Stack
ES	Extra

Program Status

Instr Ptr
Flags

(b) 8086

General Registers

EAX	AX
EBX	BX
ECX	CX
EDX	DX

ESP	SP
EBP	BP
ESI	SI
EDI	DI

Program Status

FLAGS Register
Instruction Pointer

(c) 80386 - Pentium II



Example Register Organizations

compare the register organization of comparable systems.

The MC68000 partitions its 32-bit registers into eight data registers and nine address registers.

The eight data registers are used primarily for data manipulation and are also used in addressing as index registers.

The width of the registers allows 8-, 16-, and 32-bit data operations, determined by opcode.

The address registers contain 32-bit (no segmentation) addresses; two of these registers are also used as stack pointers, one for users and one for the operating system, depending on the current execution mode.

Both registers are numbered 7, because only one can be used at a time. The MC68000 also includes a 32-bit program counter and a 16-bit status register.



Example Register Organizations

The Intel 8086 takes a different approach to register organization. Every register is special purpose, although some registers are also usable as general purpose. The 8086 contains four 16-bit data registers that are addressable on a byte or 16-bit basis, and four 16-bit pointer and index registers. The data registers can be used as general purpose in some instructions. In others, the registers are used implicitly. For example, a multiply instruction always uses the accumulator. The four pointer registers are also used implicitly in a number of operations; each contains a segment offset. There are also four 16-bit segment registers. Three of the four segment registers are used in a dedicated, implicit fashion, to point to the segment of the current instruction (useful for branch instructions), a segment containing data, and a segment containing a stack, respectively. These dedicated and implicit uses provide for compact encoding at the cost of reduced flexibility. The 8086 also includes an instruction pointer and a set of 1-bit status and control flags.



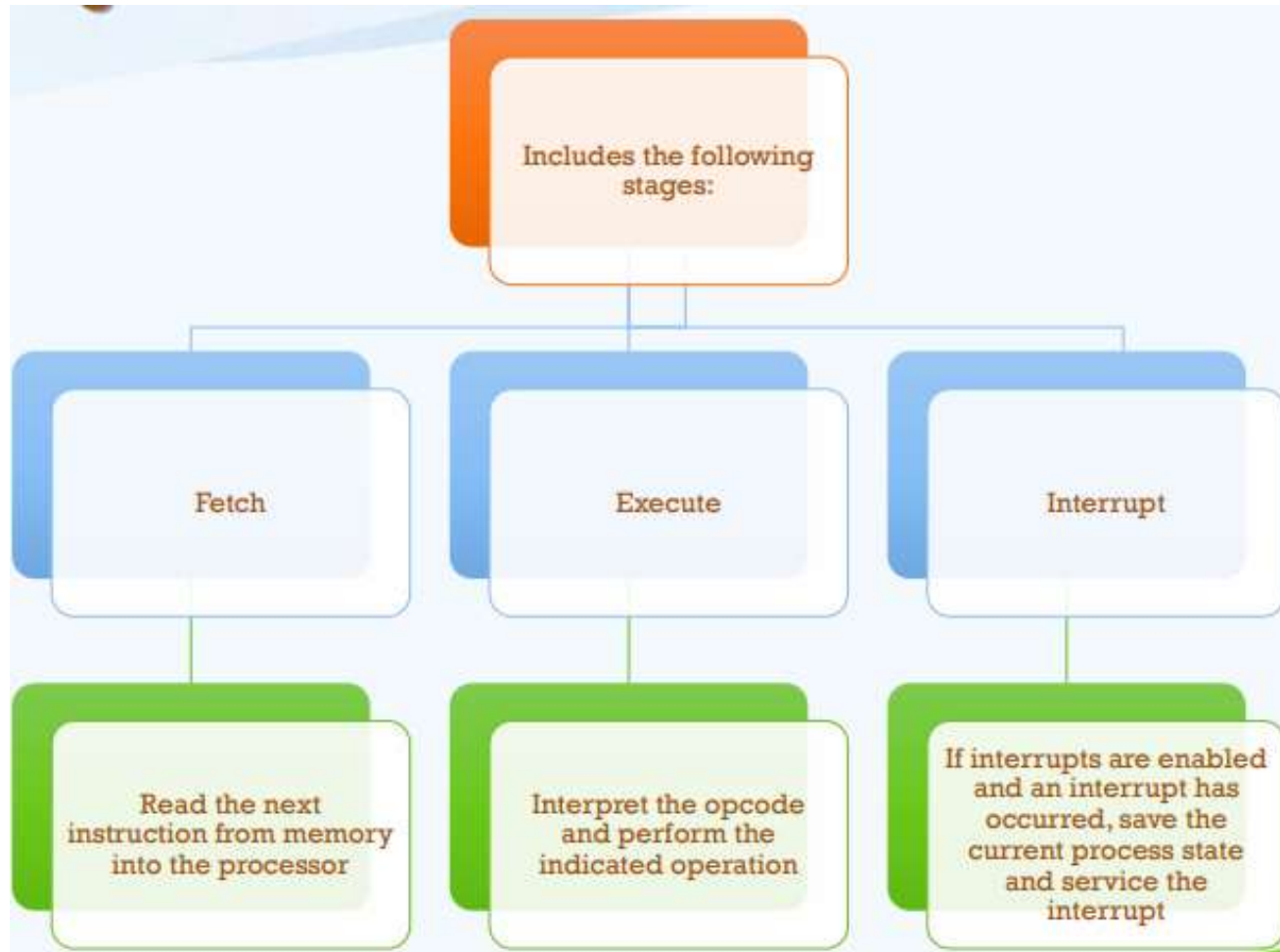
Example Register Organizations

A second instructive point concerning register organization design is illustrated in Figure c. This figure shows the user-visible register organization for the Intel 80386, which is a 32-bit microprocessor designed as an extension of the 8086. The 80386 uses 32-bit registers. However, to provide upward compatibility for programs written on the earlier machine, the 80386 retains the original register organization embedded in the new organization. Given this design constraint, the architects of the 32-bit processors had limited flexibility in designing the register organization.

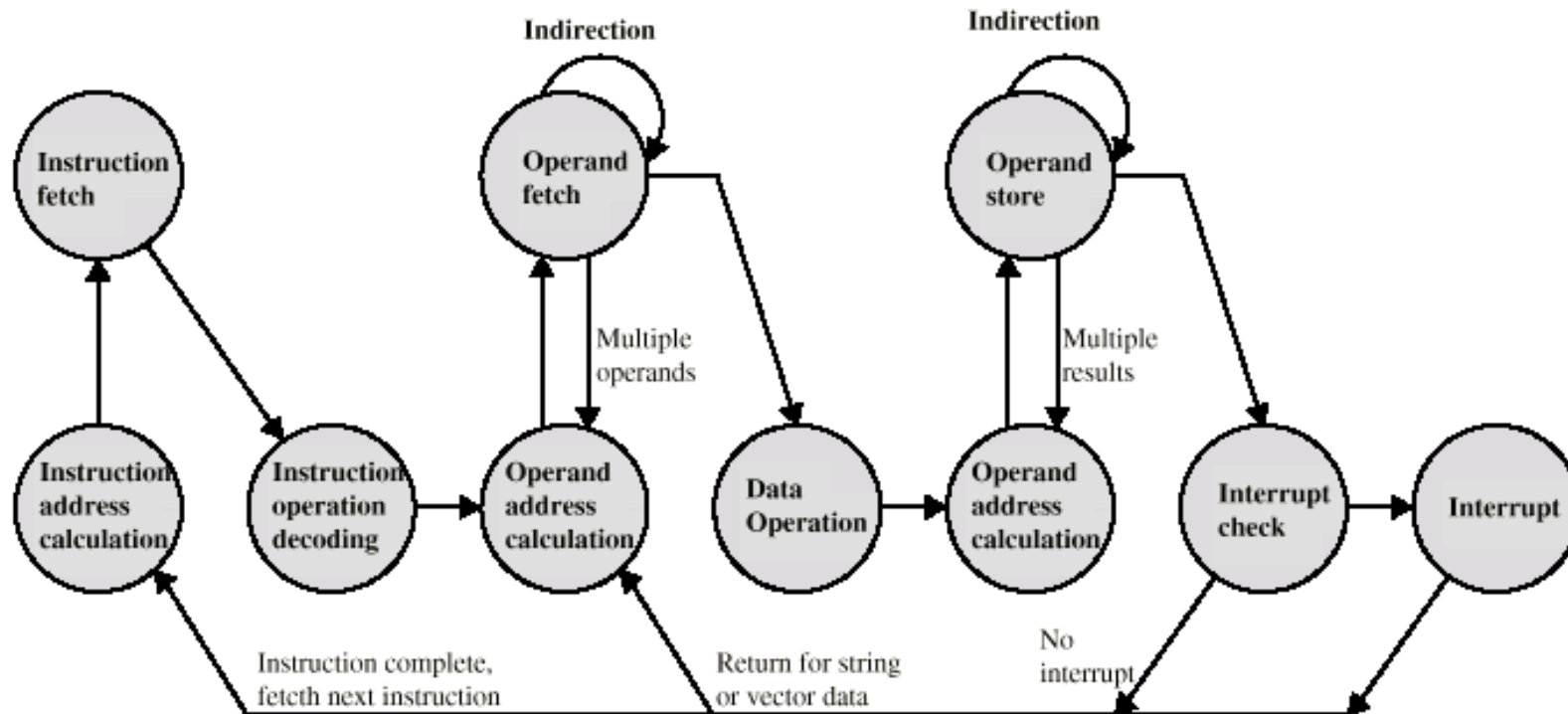
The point of this comparison should be clear. There is no universally accepted philosophy concerning the best way to organize processor registers.

As with overall instruction set design and so many other processor design issues, it is still a matter of judgment and taste.

Instruction Cycle



Instruction Cycle State Diagram



This illustrates more correctly the nature of the instruction cycle. Once an instruction is fetched, its operand specifiers must be identified. Each input operand in memory is then fetched, and this process may require indirect addressing. Register-based operands need not be fetched. Once the opcode is executed, a similar process may be needed to store the result in main memory.





BITS Pilani

Pilani | Dubai | Goa | Hyderabad

R.K.Tiwary



CONTROL UNIT OPERATION

- ❑ The execution of an instruction involves the execution of a sequence of substeps, generally called cycles
 - ✓ The Fetch Cycle
 - ✓ The Indirect Cycle
 - ✓ The Interrupt Cycle
 - ✓ The Execute Cycle
 - ✓ The Instruction Cycle
- ❑ control unit of a processor performs two tasks:
 - ✓ step through a series of micro-operations in the proper sequence
 - ✓ generates the control signals



CONTROL UNIT OPERATION

- ❑ The control signals generated by the control unit cause the opening and closing of logic gates, resulting in the transfer of data to and from registers and the operation of the ALU
- ❑ technique for implementing a control unit
 - ✓ hardwired implementation
 - ✓ Microprogrammed control
- ❑ Micro-operations are a sequence of more fundamental operations which constitute the operation cycle.
- ❑ A single micro-operation generally involves a transfer between registers, a transfer between a register and an external bus, or a simple ALU operation



Micro-Operations

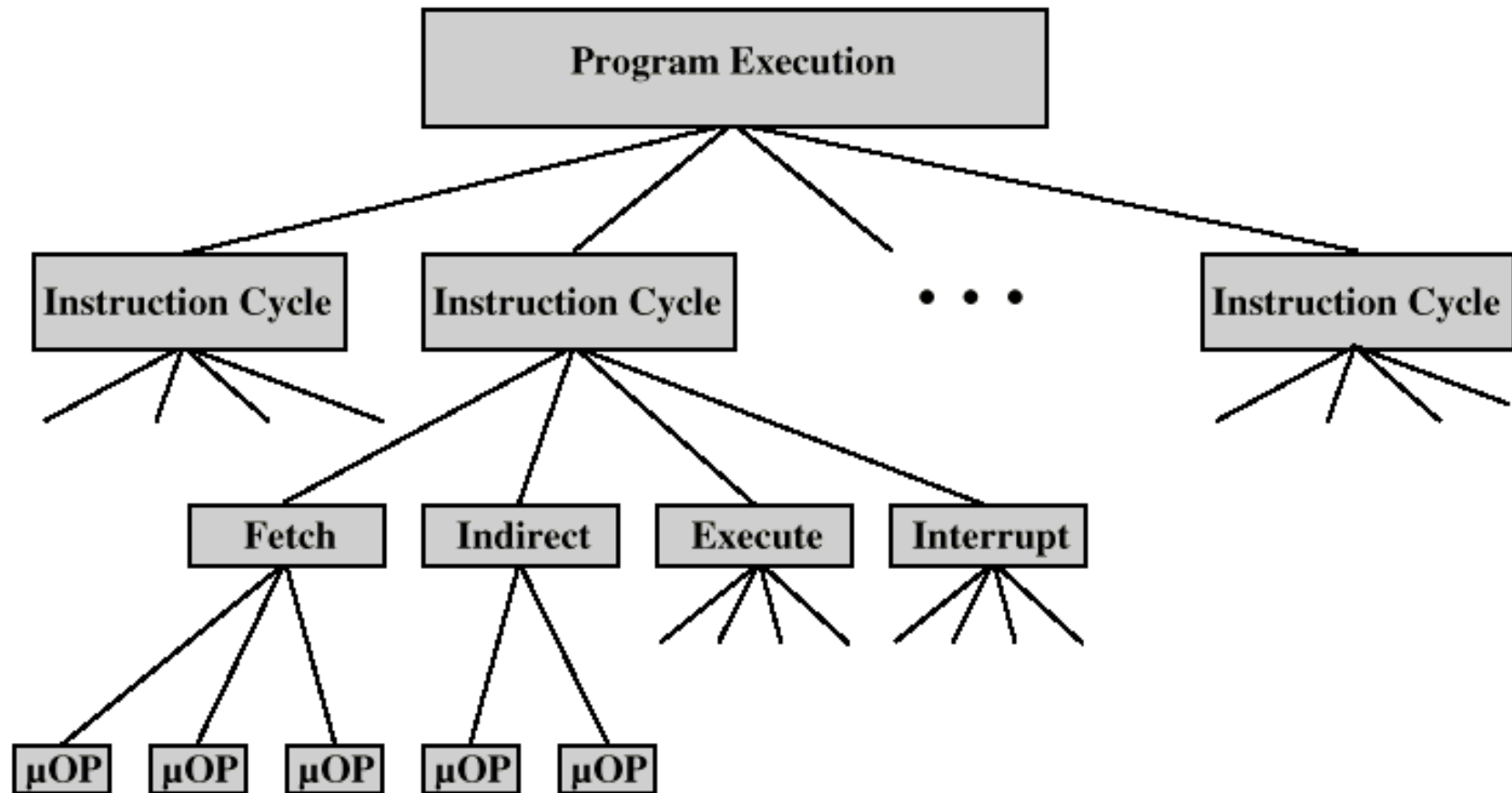
A computer executes a program

Fetch/execute cycle

Each cycle has a number of steps : Called micro-operations

Each step does very little: Atomic operation of CPU
micro-operations needed to perform the subcycles of the instruction cycle.

Constituent Elements of Program Execution





The Fetch Cycle

❑ Memory Address Register (MAR)

- Connected to address bus
- Specifies address for read or write op

❑ Memory Buffer Register (MBR)

- Connected to data bus
- Holds data to write or last data read

❑ Program Counter (PC)

- Holds address of next instruction to be fetched

❑ Instruction Register (IR)

- Holds last instruction fetched

Executing an Instruction

- ❑ Fetch the contents of the memory location pointed to by the PC. The contents of this location are loaded into the IR (fetch phase).

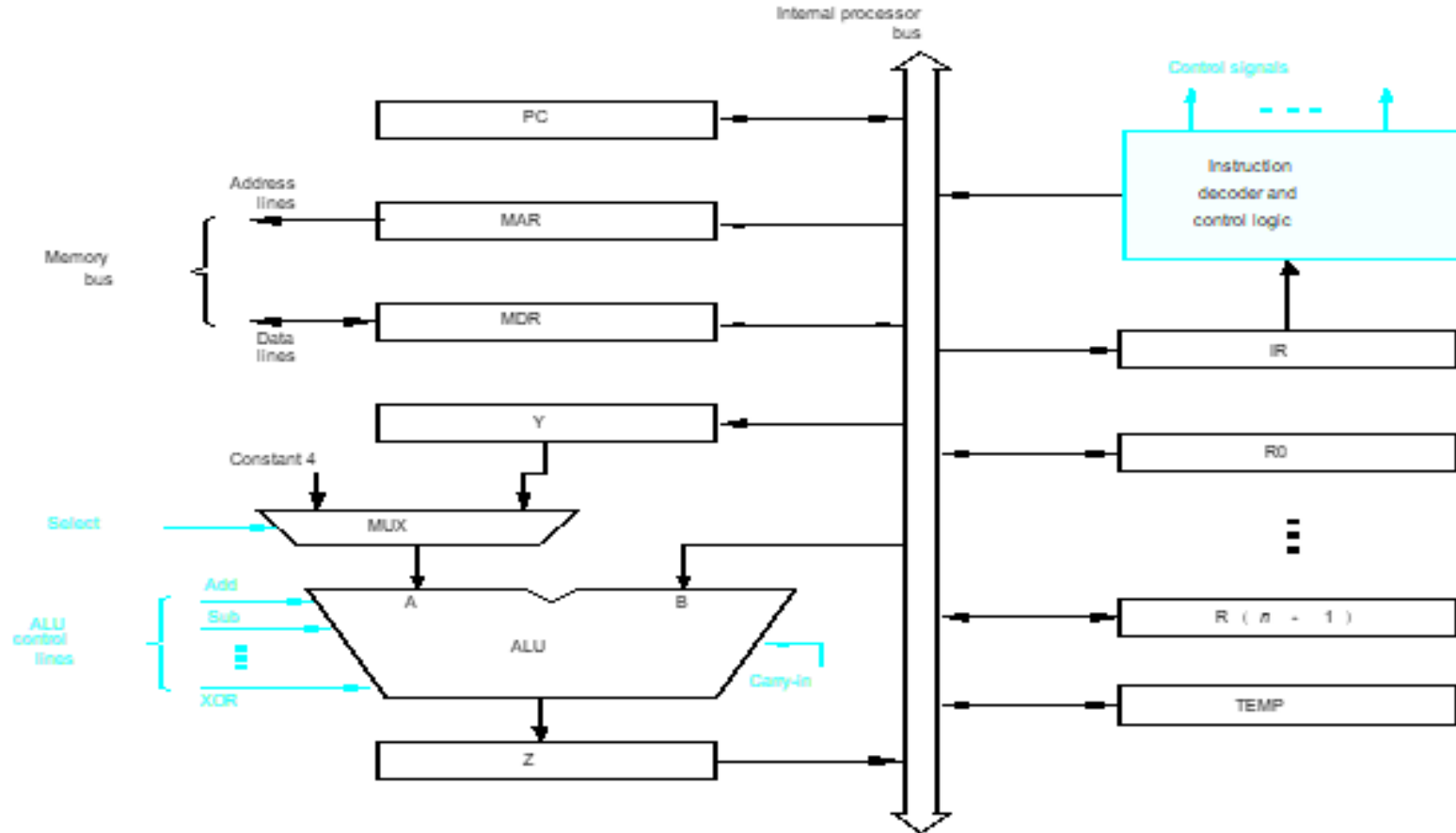
$$IR \leftarrow [[PC]]$$

- ❑ Assuming that the memory is byte addressable, increment the contents of the PC by 4 (Assume 4 Byte instruction).

$$PC \leftarrow [PC] + 4$$

- ❑ Carry out the actions specified by the instruction in the IR (execution phase).

Processor Organization





Internal organization of the processor

- ALU
- Registers for temporary storage
- Various digital circuits for executing different micro operations.(gates, MUX, decoders,counters).
- Internal path for movement of data between ALU and registers.
- Driver circuits for transmitting signals to external units.
- Receiver circuits for incoming signals from external units.

PC:

- ❖ Keeps track of execution of a program

- ❖ Contains the memory address of the next instruction to be fetched and executed.

MAR:

- ❖ Holds the address of the location to be accessed.

- ❖ I/P of MAR is connected to Internal bus and an O/p to external bus.

MDR:

- ❖ Contains data to be written into or read out of the addressed location.

- ❖ IT has 2 inputs and 2 Outputs.

- ❖ Data can be loaded into MDR either from memory bus or from internal processor bus.

The data and address lines are connected to the internal bus via MDR and

MAR





Registers:

- ❖ The processor registers R_0 to R_{n-1} vary considerably from one processor to another.
- ❖ Registers are provided for general purpose used by programmer.
- ❖ Special purpose registers-index & stack registers.
- ❖ Registers Y,Z &TEMP are temporary registers :used by processor during the execution of some instruction.

Multiplexer:

- ❖ Select either the output of the register Y or a constant value 4 to be provided as input A of the ALU.
- ❖ Constant 4 is used by the processor to increment the contents of PC.

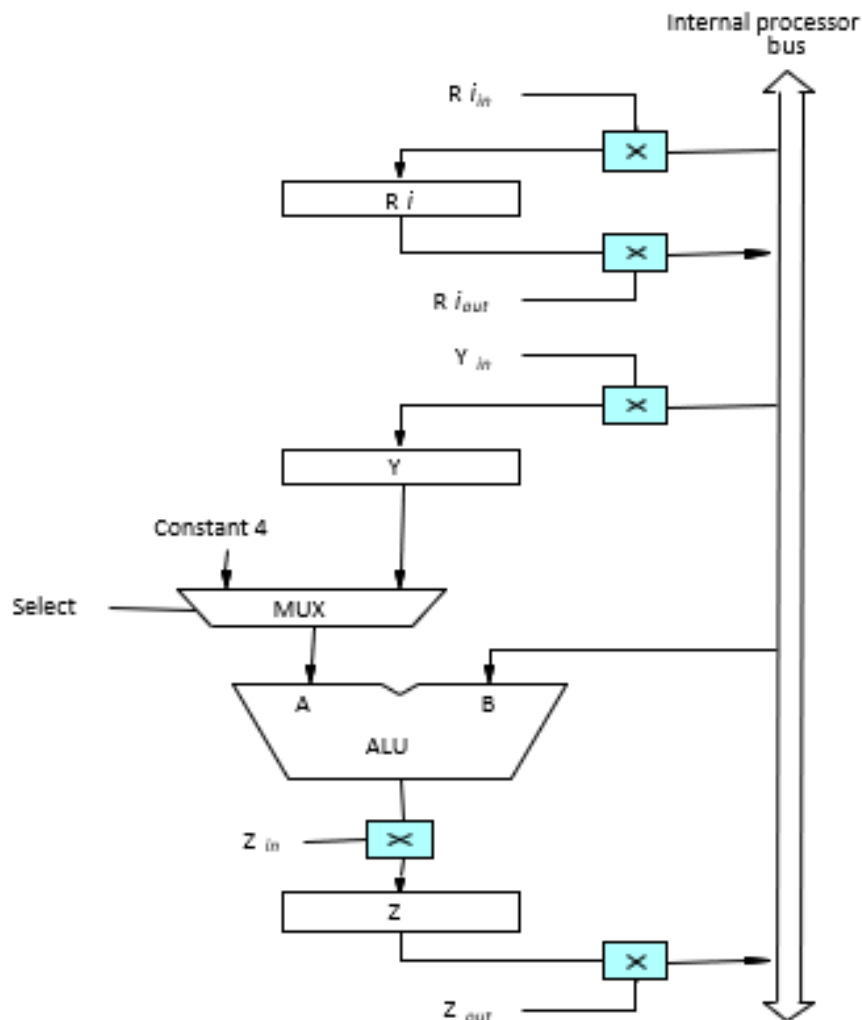
ALU:

Used to perform arithmetic and logical operation.

Data Path:

The registers, ALU and interconnecting bus are collectively referred to as the data path.

Register Transfers



- ❑ The input and output gates for register R_i are controlled by signals in R_{in} and R_{iout} .
- ❑ R_{in} Is set to 1 – data available on common bus are loaded into R_i .
- ❑ R_{iout} Is set to 1 – the contents of register are placed on the bus.
- ❑ R_{iout} Is set to 0 – the bus can be used for transferring data from other registers.



Data transfer between two registers:

EX:

Transfer the contents of R1 to R4.

1. Enable output of register R_1 by setting $R_{1out}=1$. This places the contents of R_1 on the processor bus.
2. Enable input of register R_4 by setting $R_{4in}=1$. This loads the data from the processor bus into register R_4 .



Fetch Sequence (symbolic)

t1: MAR $\leftarrow$ (PC)
t2: MBR $\leftarrow$ (memory)
 PC $\leftarrow$ (PC) +n
t3: IR $\leftarrow$ (MBR)

(tx = time unit/clock cycle)

or

t1: MAR $\leftarrow$ (PC)
t2: MBR $\leftarrow$ (memory)
t3: PC $\leftarrow$ (PC) +n
 IR $\leftarrow$ (MBR)

Assume n byte instruction



Rules for Clock Cycle Grouping

Proper sequence must be followed

- $MAR \leftarrow (PC)$ must precede $MBR \leftarrow (\text{memory})$

Conflicts must be avoided

- Must not read & write same register at same time
- $MBR \leftarrow (\text{memory})$ & $IR \leftarrow (MBR)$ must not be in same cycle

Also: $PC \leftarrow (PC) + n$ involves addition

- Use ALU
- May need additional micro-operations

Sequence of Events, Fetch Cycle

MAR	
MBR	
PC	0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 0
IR	
AC	

(a) Beginning (before t_1)

MAR	0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 0
MBR	
PC	0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 0
IR	
AC	

(b) After first step

MAR	0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 0
MBR	0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0
PC	0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 1
IR	
AC	

(c) After second step

MAR	0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 0
MBR	0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0
PC	0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 1
IR	0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0
AC	

(d) After third step



Interrupt Cycle

t1: MBR $\leftarrow$ (PC)
t2: MAR $\leftarrow$ save-address
 PC $\leftarrow$ routine-address
t3: memory $\leftarrow$ (MBR)

This is a minimum

- May be additional micro-ops to get addresses
- N.B. saving context is done by interrupt handler routine, not micro-ops



The Execute Cycle

Different for each instruction

e.g. ADD R1,X - add the contents of location X to Register

1 , result in R1

t1: $MAR \leftarrow (IR_{\text{address}})$

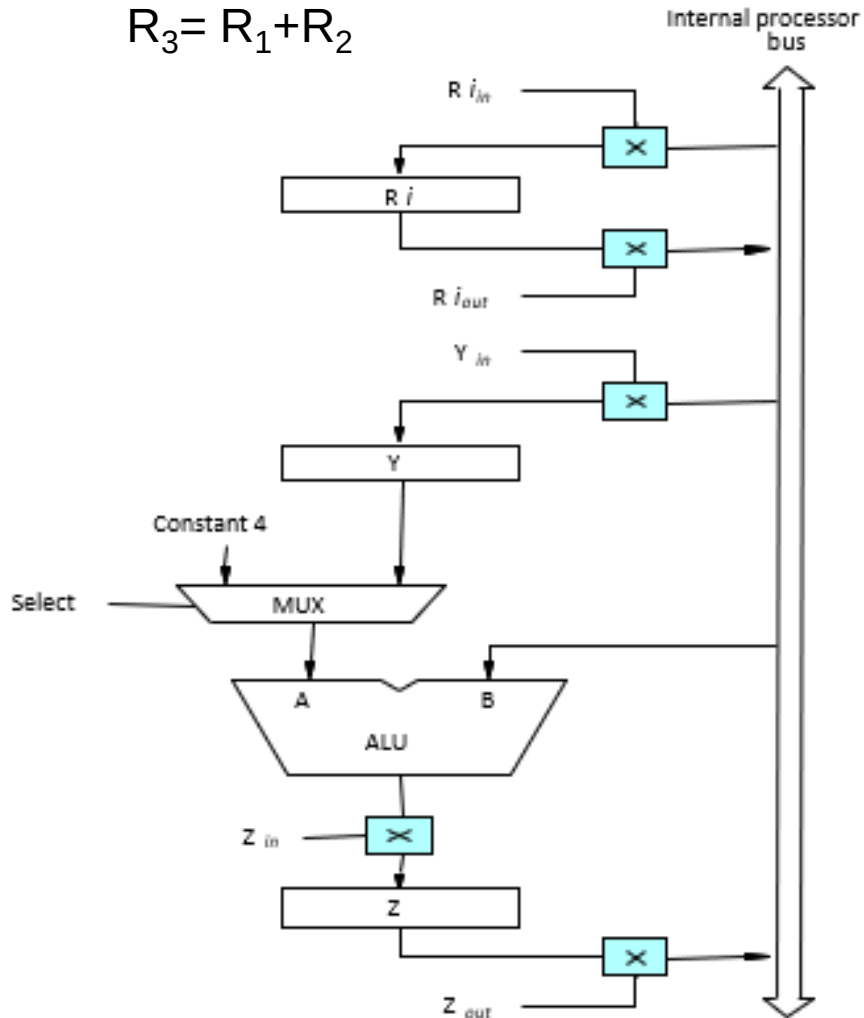
t2: $MBR \leftarrow (\text{memory})$

t3: $R1 \leftarrow R1 + (MBR)$

Note no overlap of micro-operations

Performing an Arithmetic or Logic Operation

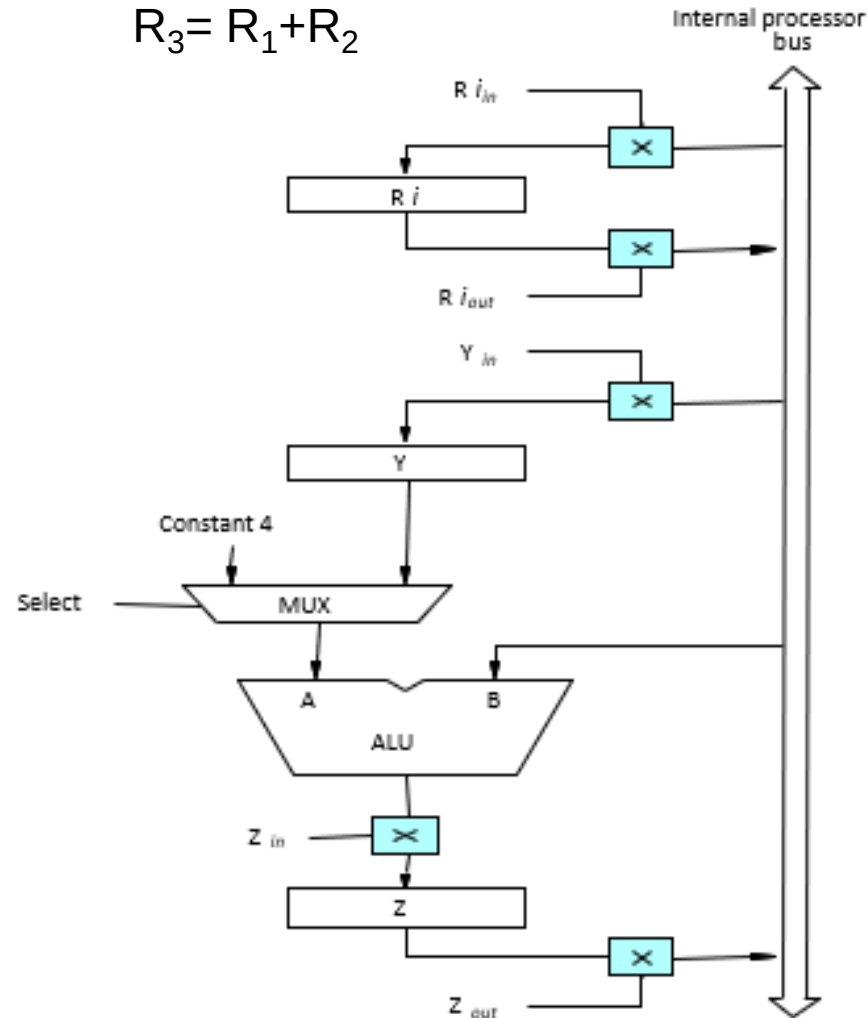
$$R_3 = R_1 + R_2$$



- ❑ The ALU is a combinational circuit that has no internal storage.
- ❑ ALU gets the two operands from MUX and bus. The result is temporarily stored in register Z.
- ❑ What is the sequence of operations to add the contents of register R_1 to those of R_2 and store the result in R_3 ?
 1. R_{1out} , Y_{in}
 2. R_{2out} , SelectY, Add, Z_{in}
 3. Z_{out} , R_{3in}

Performing an Arithmetic or Logic Operation

$$R_3 = R_1 + R_2$$



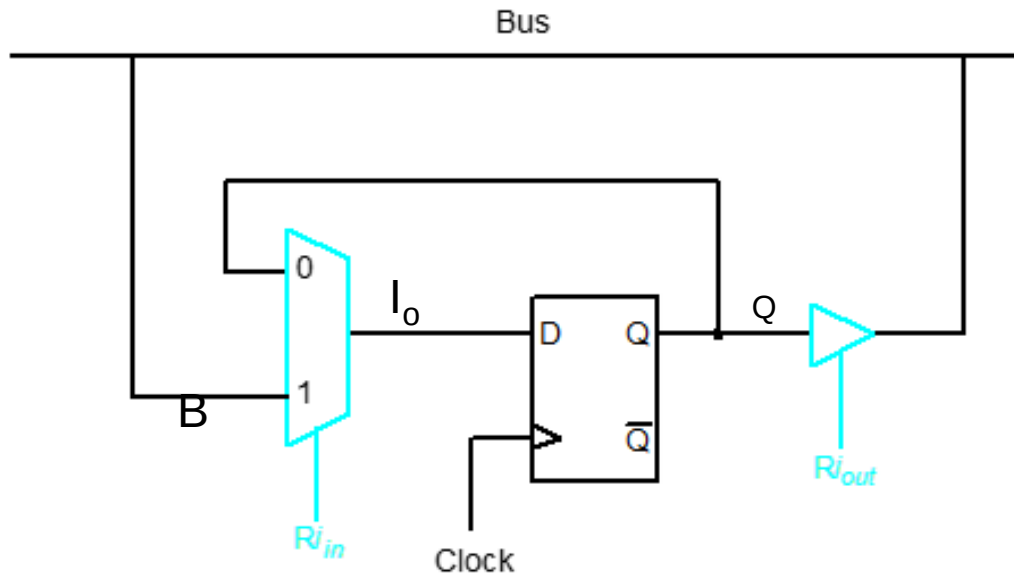
Step 1: Output of the register R_1 and input of the register Y are enabled, causing the contents of R_1 to be transferred to Y .

Step 2: The multiplexer's select signal is set to select Y causing the multiplexer to gate the contents of register Y to input A of the ALU. At the same time contents of R_2 are gated into the bus and hence to input B .

Step 3: The contents of Z are transferred to the destination register R_3 .

Register Transfers

Input and output gating for one register bit.

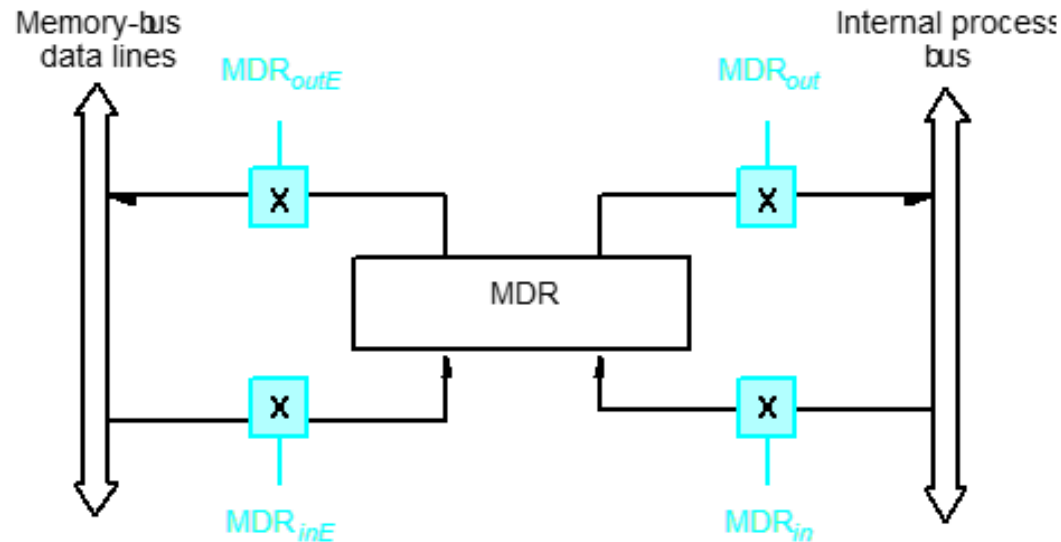


Rin	clk	Q
0	↑	Q
1	↑	B

When $R_{out}=1$ $B=Q$. otherwise in high impedance state

Fetching a Word from Memory

- Connection and control signals for register MDR





Instruction Cycle

Each phase decomposed into sequence of elementary micro-operations

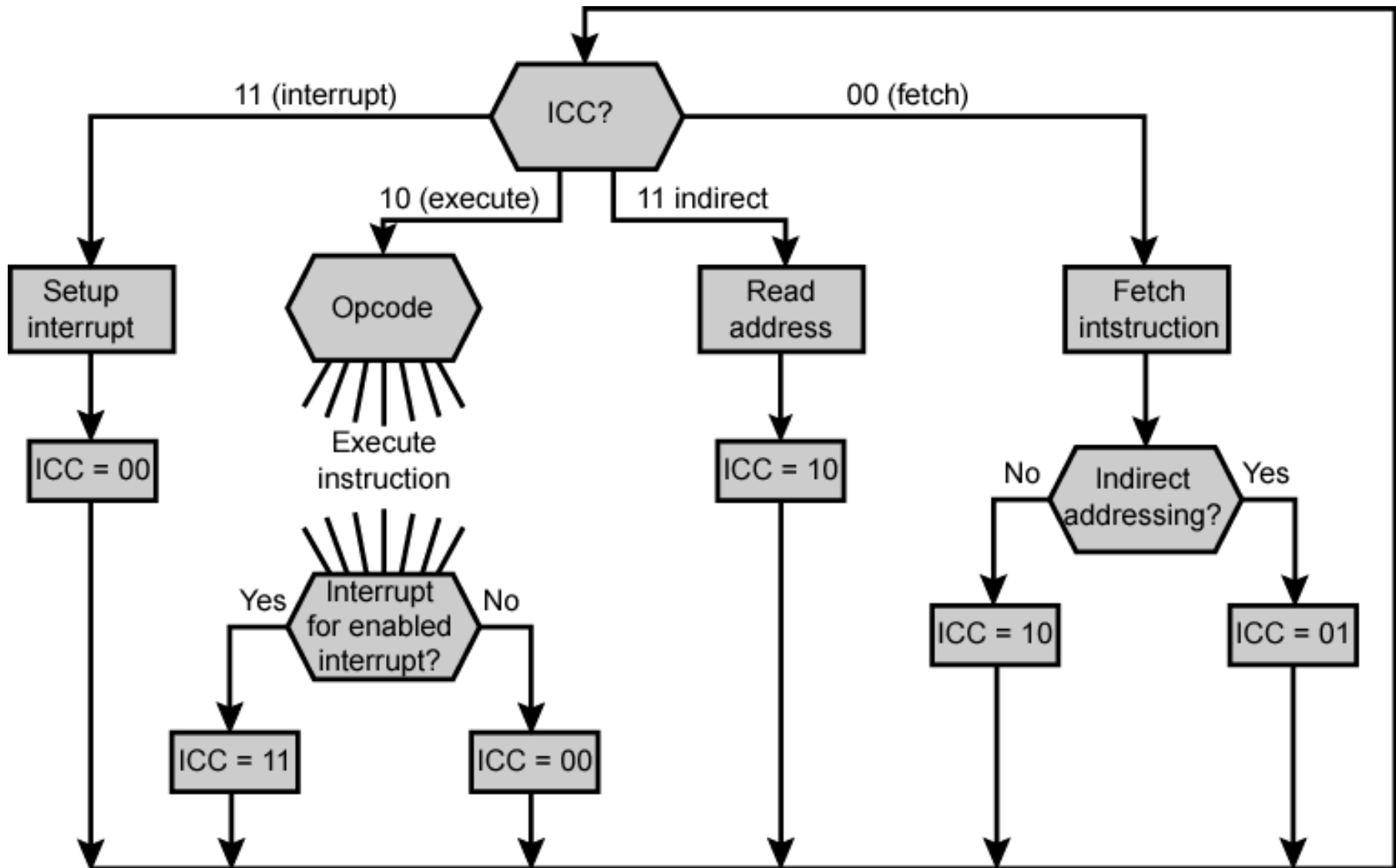
E.g. fetch, indirect, and interrupt cycles Execute cycle

- One sequence of micro-operations for each opcode

Assume new 2-bit register called ICC

- Instruction cycle code (ICC) designates which part of cycle processor is in
 - 00: Fetch
 - 01: Indirect
 - 10: Execute
 - 11: Interrupt

Flowchart for Instruction Cycle



Requirements for CONTROL OF THE PROCESSOR



the following three-step process leads to a characterization of the control unit

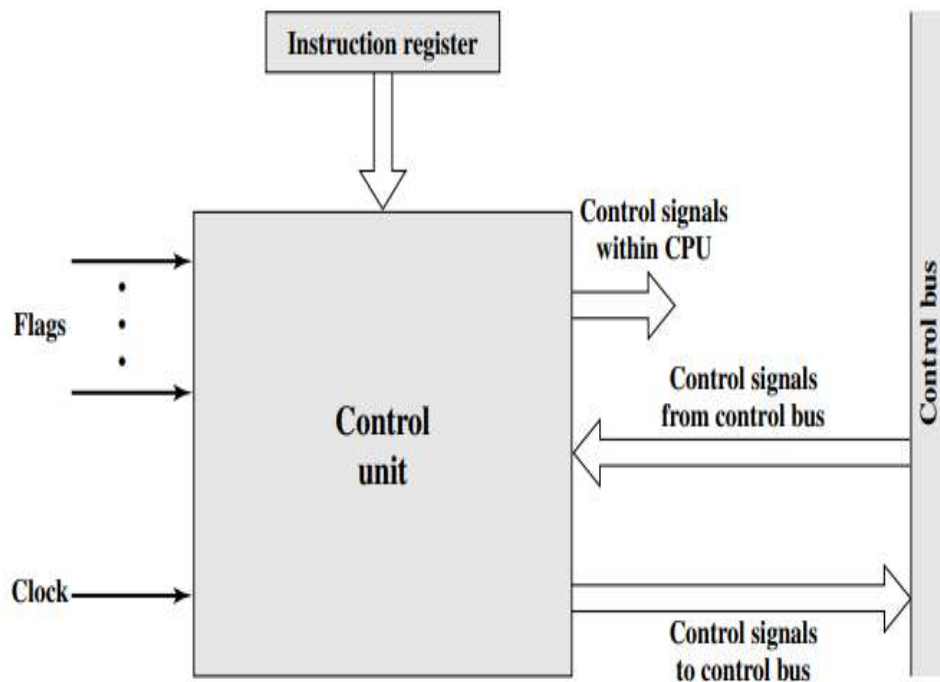
1. Define the basic elements of the processor.
2. Describe the micro-operations that the processor performs.
3. Determine the functions that the control unit must perform to cause the micro-operations to be performed

The control unit performs two basic tasks:

- Sequencing: The control unit causes the processor to step through a series of micro-operations in the proper sequence, based on the program being executed.
- Execution: The control unit causes each micro-operation to be performed

Control Unit

Block Diagram of the Control Unit



Clock

- One micro-instruction (or set of parallel micro-instructions) per clock cycle

Instruction register

- Op-code for current instruction
- Determines which micro-instructions are performed

Flags

- State of CPU
- Results of previous operations

From control bus

- Interrupts
- Acknowledgements



Example Control Signal Sequence - Fetch

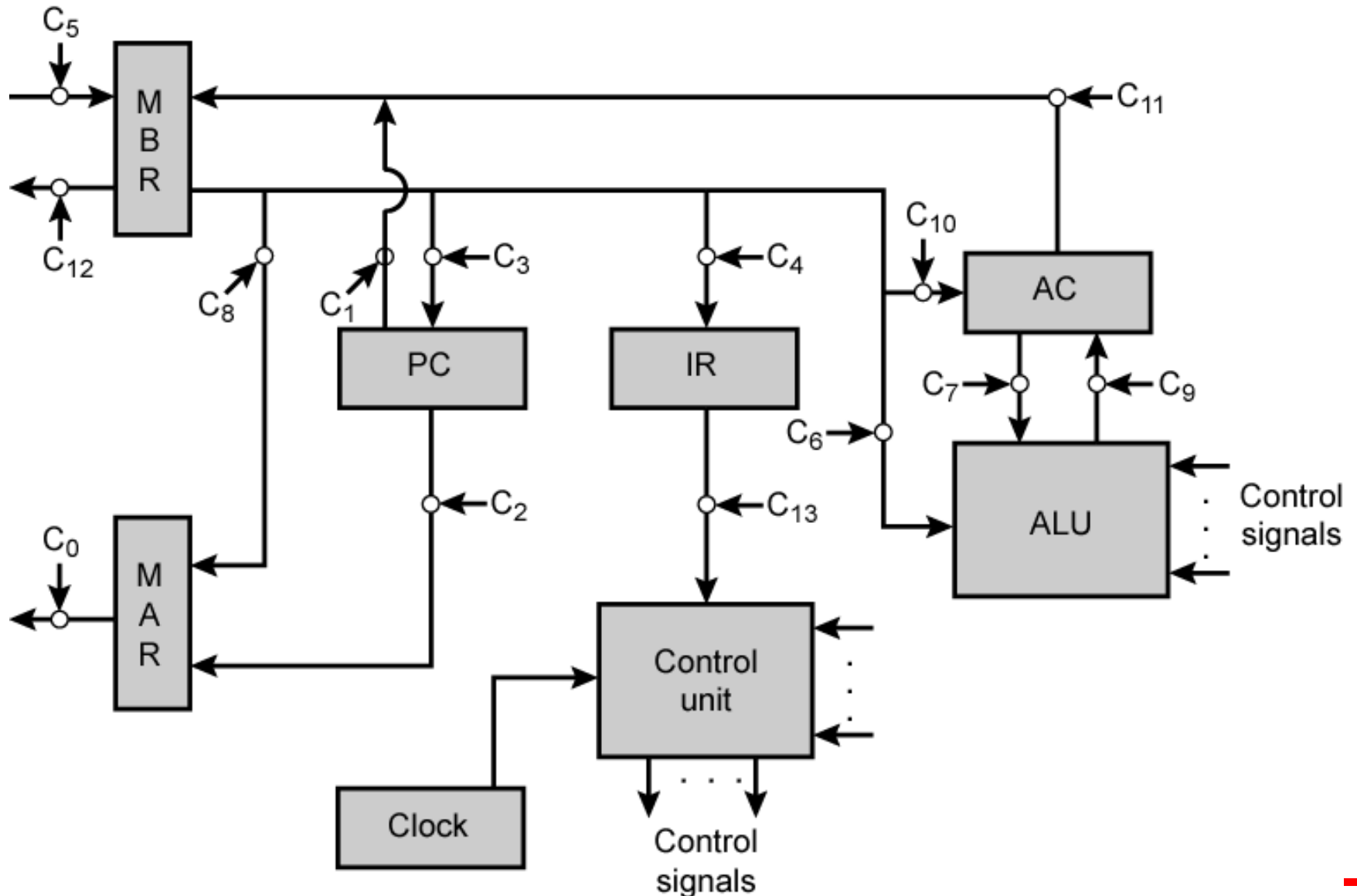
MAR $\leftarrow$ (PC)

- Control unit activates signal to open gates between PC and MAR

MBR $\leftarrow$ (memory)

- Open gates between MAR and address bus
- Memory read control signal
- Open gates between data bus and MBR

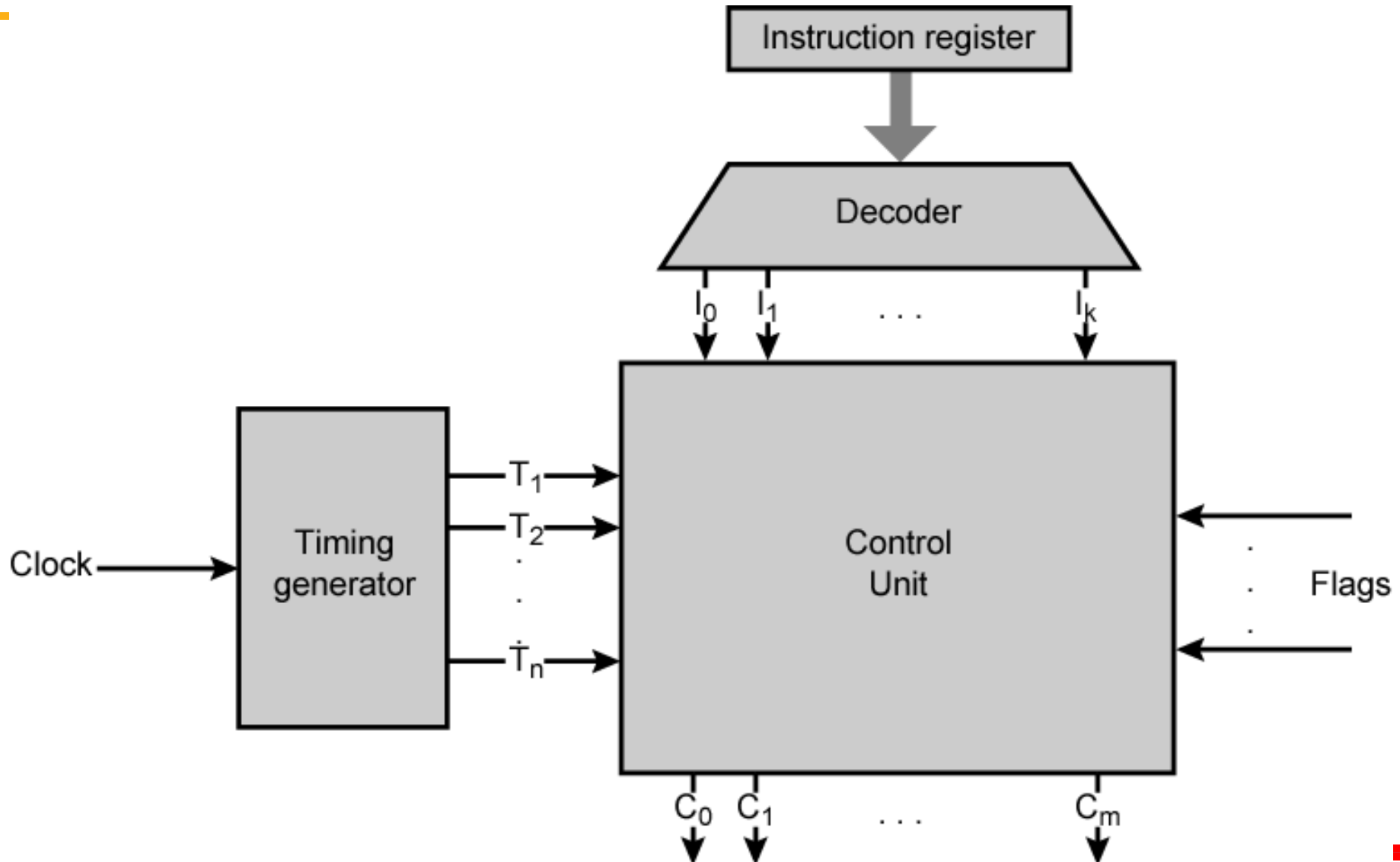
Data Paths and Control Signals



	Micro-operations	Active Control Signals
Fetch:	$t_1: \text{MAR} \leftarrow (\text{PC})$	C_2
	$t_2: \text{MBR} \leftarrow \text{Memory}$ $\text{PC} \leftarrow (\text{PC}) + 1$	C_5, C_R
	$t_3: \text{IR} \leftarrow (\text{MBR})$	C_4
Indirect:	$t_1: \text{MAR} \leftarrow (\text{IR}(\text{Address}))$	C_8
	$t_2: \text{MBR} \leftarrow \text{Memory}$	C_5, C_R
	$t_3: \text{IR}(\text{Address}) \leftarrow (\text{MBR}(\text{Address}))$	C_4
Interrupt:	$t_1: \text{MBR} \leftarrow (\text{PC})$	C_1
	$t_2: \text{MAR} \leftarrow \text{Save-address}$ $\text{PC} \leftarrow \text{Routine-address}$	
	$t_3: \text{Memory} \leftarrow (\text{MBR})$	C_{12}, C_W

C_R = Read control signal to system bus.
 C_W = Write control signal to system bus.

Control Unit with Decoded Inputs





Hardwired control Signal

Let us consider a single control signal, C₅. This signal causes data to be read from the external data bus into the MBR

PQ = 11 Instruction C; PQ = 10 EC ;PQ = 01 Indirect Cycle; PQ = 00 FC

Then the following Boolean expression defines C₅ $C_5 = \bar{P} \cdot \bar{Q} \cdot T_2 + \bar{P} \cdot Q \cdot T_2$

This expression is not complete. C₅ is also needed during the execute cycle.

For our simple example, let us assume that there are only three instructions that read from memory: LDA, ADD, and AND. Now we can define C₅

$$C_5 = \bar{P} \cdot \bar{Q} \cdot T_2 + \bar{P} \cdot Q \cdot T_2 + P \cdot \bar{Q} \cdot (LDA + ADD + AND) \cdot T_2$$

This same process could be repeated for every control signal generated by the processor

Hardwired Implementation (1)



Control unit inputs

Flags and control bus

- Each bit means something

Instruction register

- Op-code causes different control signals for each different instruction
- Unique logic for each op-code
- Decoder takes encoded input and produces single output
- n binary inputs and 2^n outputs

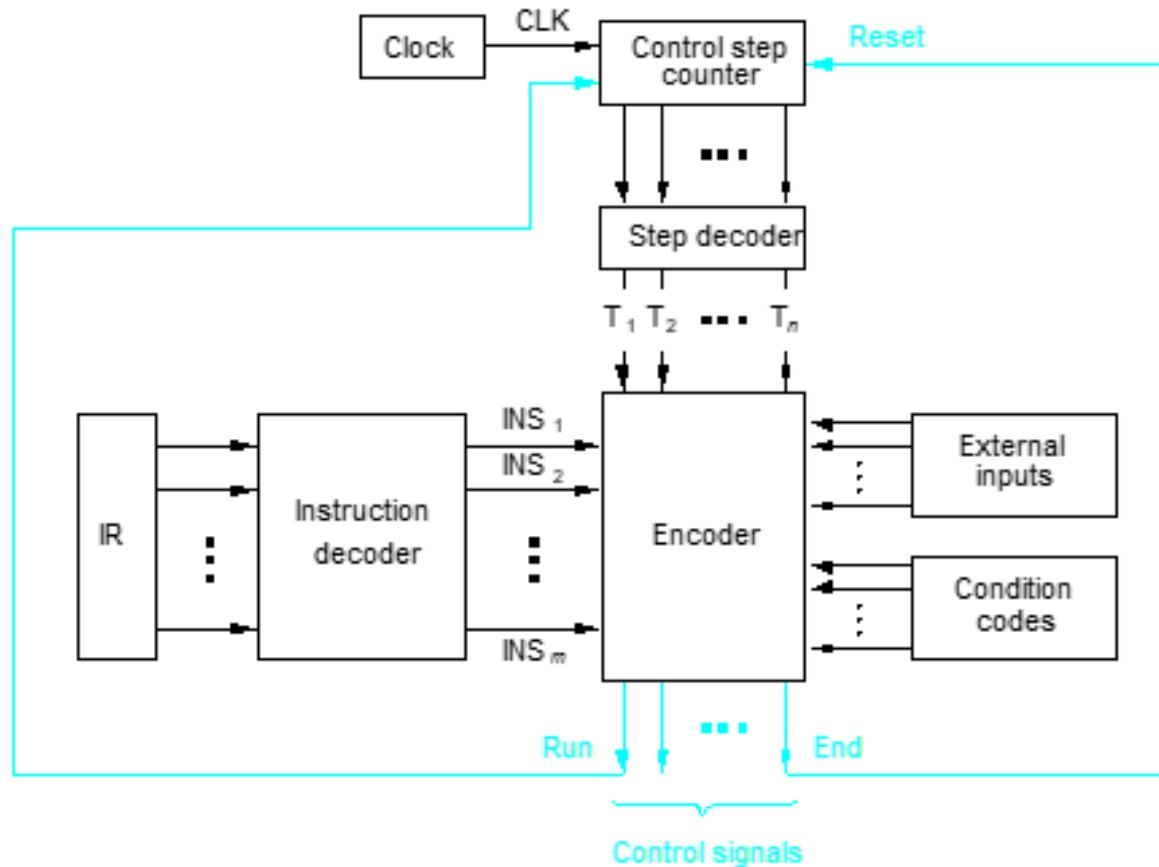
Hardwired Implementation (2)



Clock

- Repetitive sequence of pulses
- Useful for measuring duration of micro-ops
- Must be long enough to allow signal propagation
- Different control signals at different times within instruction cycle
- Need a counter with different control signals for t_1 , t_2 etc.

Detailed Block Description





Generating Z_{in}

Control sequence for an unconditional branch instruction

Step Action

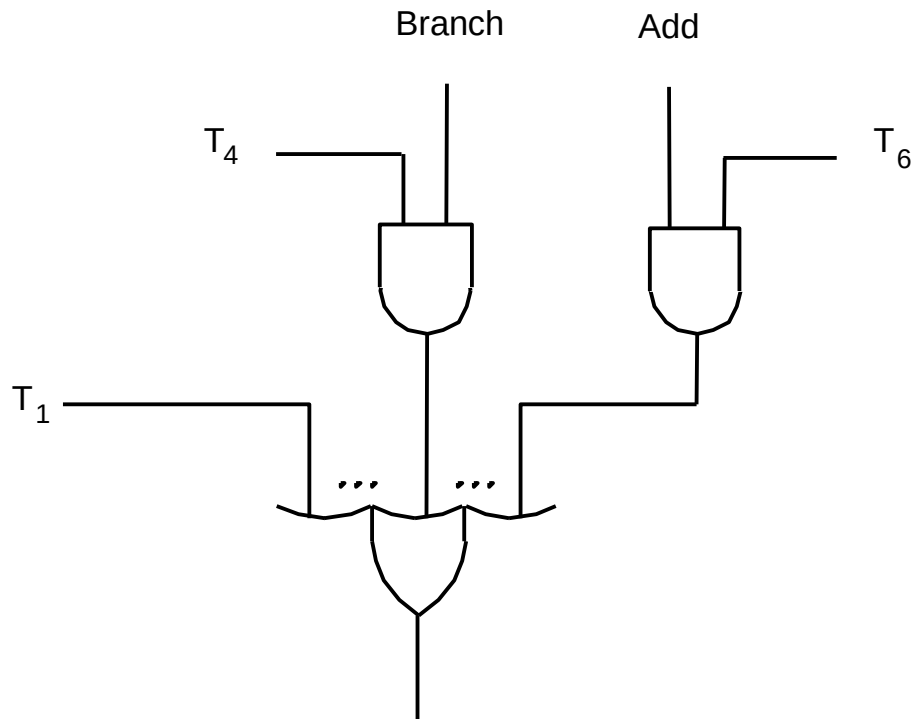
1	PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
2	Z_{out} , PC_{in} , Y_{in} , WMF C
3	MDR_{out} , IR_{in}
4	Offset-field-of- IR_{out} , Add, Z_{in}
5	Z_{out} , PC_{in} , End

Add (R3), R1

Step Action

1	PC_{out} , MAR_{in} , Read, select constant4, $PC+4$, Z_{in}
2	Z_{out} , PC_{in} , Y_{in} , WMF C
3	MDR_{out} , IR_{in}
4	$R3_{out}$, MAR_{in} , Read
5	$R1_{out}$, Y_{in} , WMF C
6	MDR_{out} , SelectY, Add, Z_{in}
7	Z_{out} , $R1_{in}$, End

Generating Z_{in}



$$Z_{in} = T_1 + T_6 \cdot \text{ADD} + T_4 \cdot \text{BR} + \dots$$



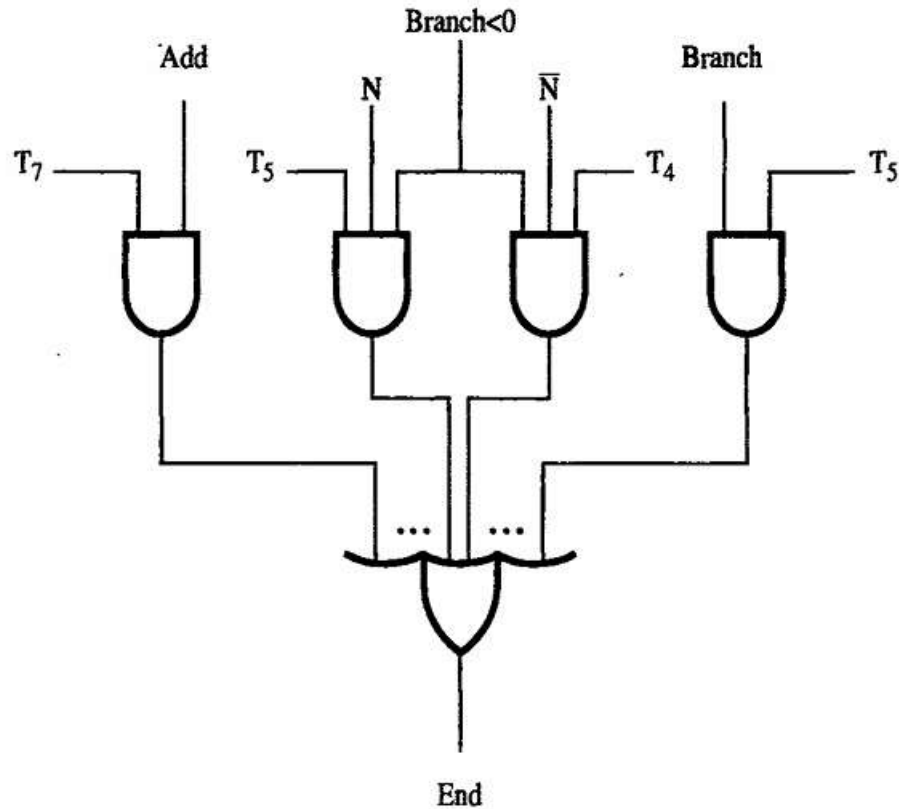
Problems With Hard Wired Designs

- ❑ Complex sequencing & micro-operation logic
- ❑ Difficult to design and test
- ❑ Inflexible design
- ❑ Difficult to add new instructions

Generating End

$$\text{End} = T_7 \cdot \text{ADD} + T_5 \cdot \text{BR} + (T_5 \cdot N + T_4 \cdot N) \cdot \text{BRN} + \dots$$

Control Step counter to its starting value

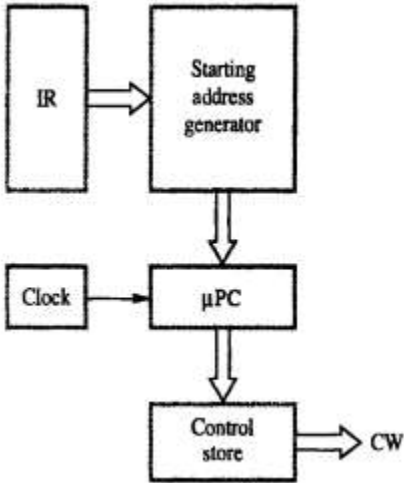


Microprogrammed Control

Control signals are generated by a program similar to machine language programs.
 Control Word (CW); Microroutine; individual microinstruction

Micro - instruction	PC _{in}	PC _{out}	MAR _{in}	Read	MDR _{out}	IR _{in}	Y _{in}	Select	Add	Z _{in}	Z _{out}	R1 _{out}	R1 _{in}	R3 _{out}	WMFC	End
1	0	1	1	1	0	0	0	1	1	1	0	0	0	0	0	0
2	1	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0
3	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0
4	0	0	1	1	0	0	0	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	1	0	0	0	0	1	0	0	1	0
6	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1

Add R1,(R3)



Step	Action
1	PC _{out} , MAR _{in} , Read,select constant4, PC+4, Z _{in}
2	Z _{out} , PC _{in} , Y _{in} , WMF C
3	MDR _{out} , IR _{in}
4	R3 _{out} , MAR _{in} , Read
5	R1 _{out} , Y _{in} , WMF C
6	MDR _{out} , SelectY, Add, Z _{in}
7	Z _{out} , R1 _{in} , End



Micro-programmed Control Implementation

- ❑ Use sequences of instructions to control complex operations
Called micro-programming or firmware
- ❑ All the control unit does is generate a set of control signals
- ❑ Each control signal is on or off
- ❑ Represent each control signal by a bit
- ❑ Have a control word for each micro-operation
- ❑ Have a sequence of control words for each machine code instruction
- ❑ Add an address to specify the next micro-instruction, depending on conditions



Micro-programmed Control Implementation

Today's large microprocessor

- Many instructions and associated register-level hardware
- Many control points to be manipulated

This results in control memory that

- Contains a large number of words
 - corresponding to the number of instructions to be executed
- Has a wide word width
 - Due to the large number of control points to be manipulated



Micro-program

- ❑ Word Length Based on 3 factors
 - Maximum number of simultaneous micro-operations supported
 - The way control information is represented or encoded
 - The way in which the next micro-instruction address is specified

- ❑ Types

Each micro-instruction specifies single (or few) micro-operations to be performed

- (*vertical* micro-programming)

Each micro-instruction specifies many different micro-operations to be performed in parallel

- (*horizontal* micro-programming)



Micro-program

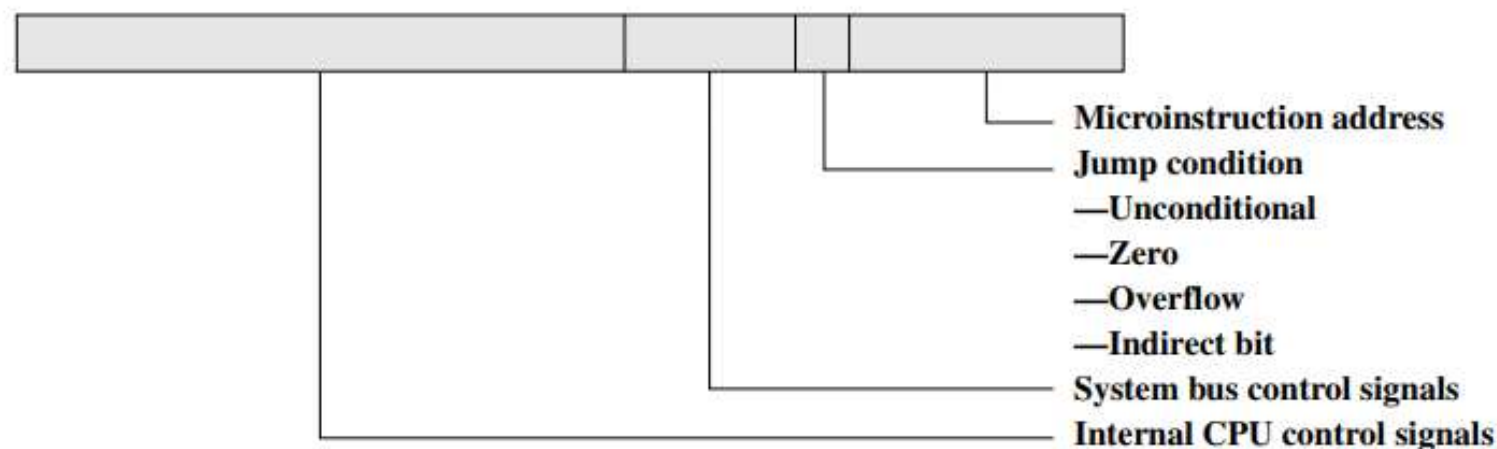
☐ Vertical Micro-programming

- ✓ Width is narrow
- ✓ n control signals encoded into $\log_2 n$ bits
- ✓ Limited ability to express parallelism
- ✓ Considerable encoding of control information requires external memory word decoder to identify the exact control line being manipulated

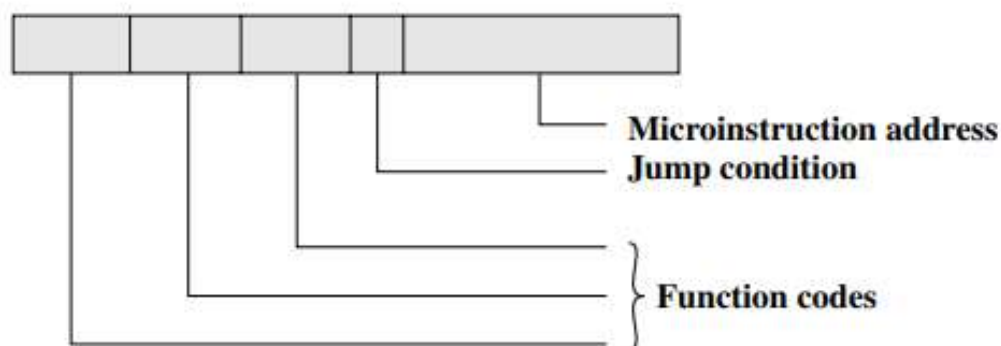
☐ Horizontal Micro-programming

- ✓ Wide memory word
- ✓ High degree of parallel operations possible
- ✓ Little encoding of control information

Typical Microinstruction Formats



(a) Horizontal microinstruction

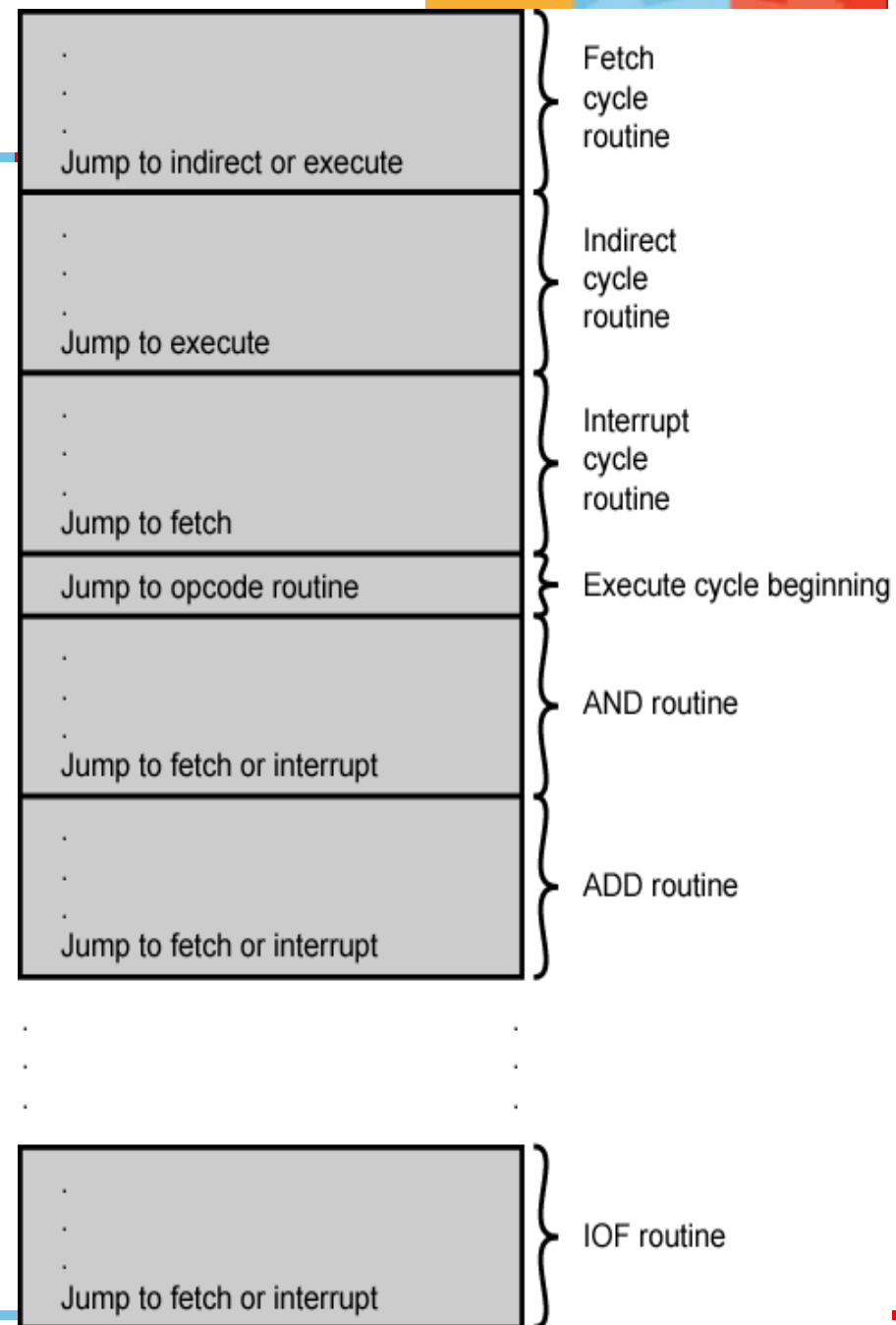


(b) Vertical microinstruction

Micro-program Organization of Control Memory

Compromise

- ❑ Divide control signals into disjoint groups
- ❑ Implement each group as separate field in memory word
- ❑ Supports reasonable levels of parallelism without too much complexity



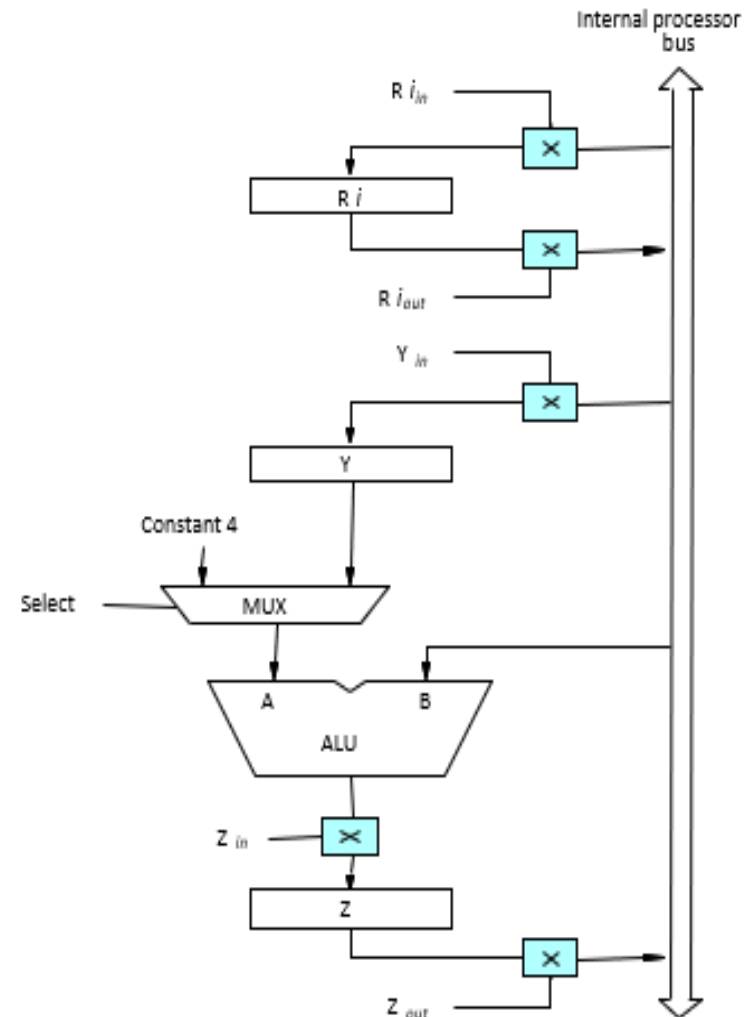
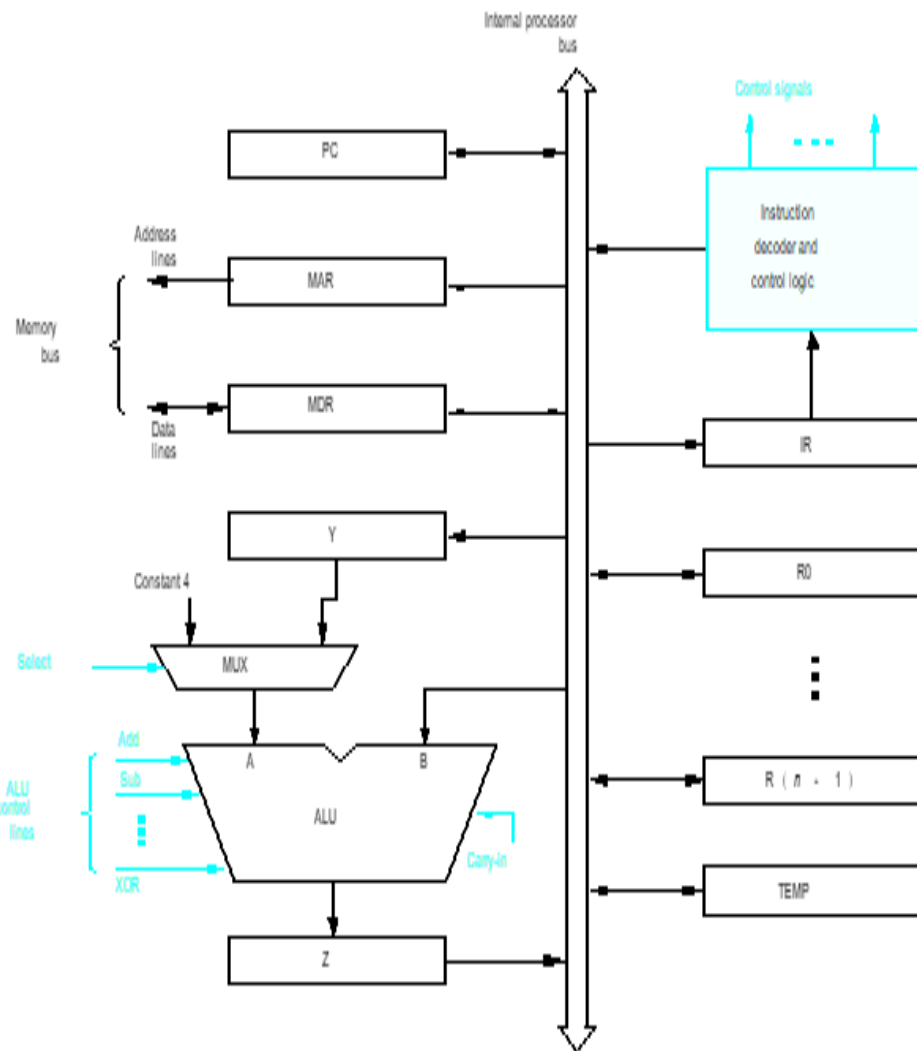


Execution of a Complete Instruction (self study)

- ❑ Add R1, (R3)
- ❑ Fetch the instruction
- ❑ Fetch the first operand (the contents of the memory location pointed to by R3)
- ❑ Perform the addition
- ❑ Load the result into R1

Execution of a Complete Instruction

Add R1, (R3)



Data Flow, Fetch Cycle

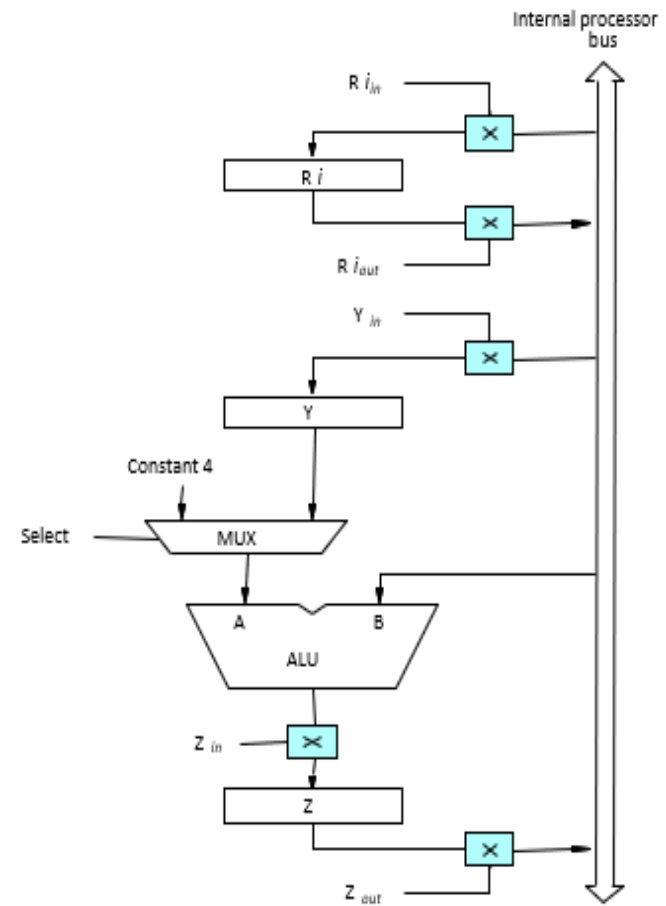
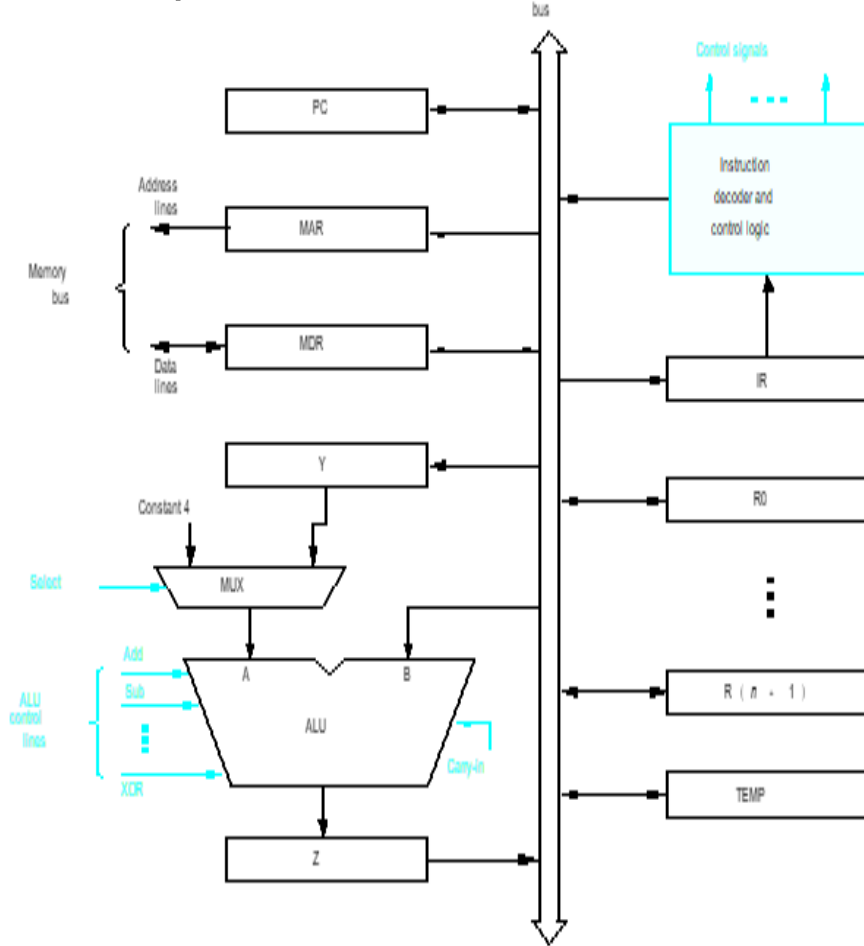
Add R1, (R3)

Step	Action
1	PC _{out} , MAR _{in} , Read, select constant4, PC+4, Z _{in}
2	Z _{out} , PC _{in} , Y _{in} , WMF C
3	MDR _{out} , IR _{in}
4	R3 _{out} , MAR _{in} , Read
5	R1 _{out} , Y _{in} , WMF C
6	MDR _{out} , SelectY, Add, Z _{in}
7	Z _{out} , R1 _{in} , End

WMFC: Wait for memory function check to accommodate the variability in response time, The addressed device sets this signal to 1 to indicate that contents of the specified location has been read

Example

Write the sequence of control steps needed to add the immediate number to R1 as per the bus structure shown



Example

1. PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
2. Z_{out} , PC_{in} , Y_{in} , WMFC
3. MDR_{out} , IR_{in}
4. P_{Cout} , MAR_{in} , Read, Select4, Add, Z_{in}
5. Z_{out} , PC_{in} , Y_{in}
6. $R1_{out}$, Y_{in} , WMFC
7. MDR_{out} , SelectY, Add, Z_{in}
8. Z_{out} , $R1_{in}$, End

Execute Cycle (ISZ)

ISZ X - increment and skip if zero

The content of location X is incremented by 1. If the result is 0, the next instruction is skipped. A possible sequence of micro-operations is

- t1: $MAR \leftarrow (IR_{address})$
- t2: $MBR \leftarrow (memory)$
- t3: $MBR \leftarrow (MBR) + 1$
- t4: $memory \leftarrow (MBR)$
- if $(MBR) == 0$ then $PC \leftarrow (PC) + 1$

Notes:

- if is a single micro-operation
- Micro-operations done during t4



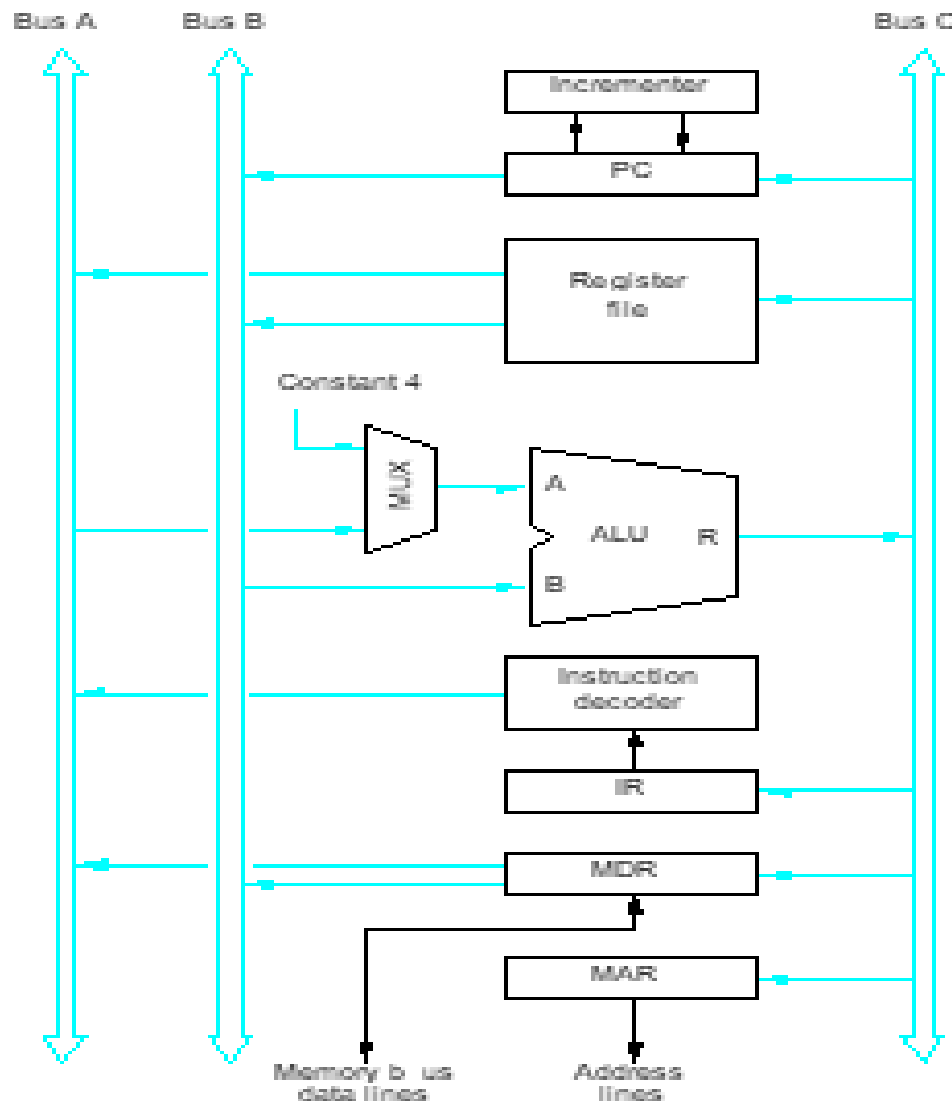
Execute Cycle (BSA)

BSA X - Branch and save address

- Address of instruction following BSA is saved in X
- Execution continues from X+1
- t1: $MAR \leftarrow (IR_{\text{address}})$
- $MBR \leftarrow (PC)$
- t2: $PC \leftarrow (IR_{\text{address}})$
- $\text{memory} \leftarrow (MBR)$
- t3: $PC \leftarrow (PC) + 1$

The address in the PC at the start of the instruction is the address of the next instruction in sequence. This is saved at the address designated in the IR. The latter address is also incremented to provide the address of the instruction for the next instruction cycle

Multiple-Bus Organization



- Allow the contents of two different registers to be accessed simultaneously and have their contents placed on buses A and B.
 - Allow the data on bus C to be loaded into a third register during the same clock cycle.
 - Incrementer unit.
 - ALU simply passes one of its two input operands unmodified to bus C
- control signal: $R=A$ or $R=B$

Add R6, R4, R5



Step Action

- 1 PC_{out} , $R=B$, MAR_{in} , Read, IncPC
 - 2 WMFC
 - 3 MDR_{outB} , $R=B$, IR_{in}
 - 4 $R4_{outA}$, $R5_{outB}$, SelectA, Add, $R6_{in}$, End
-



BITS Pilani

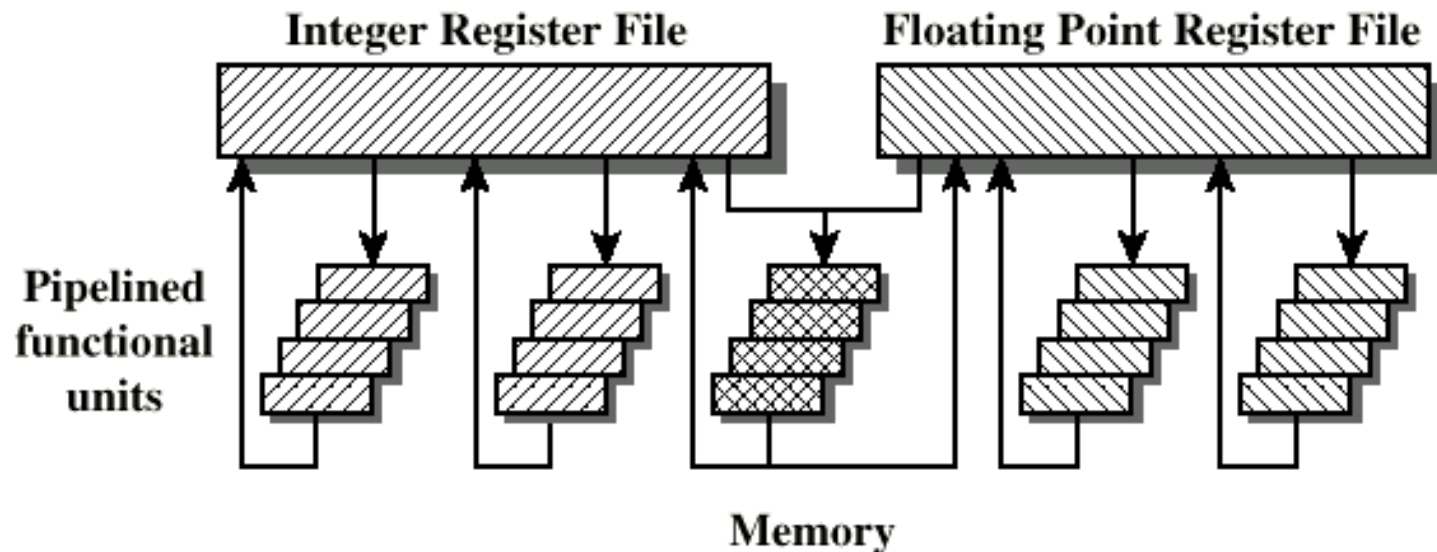
Pilani | Dubai | Goa | Hyderabad

R.K.Tiwary

What is Superscalar?



- ❑ Common instructions (arithmetic, load/store, conditional branch) can be initiated and executed independently
- ❑ Equally applicable to RISC & CISC
- ❑ In practice usually RISC

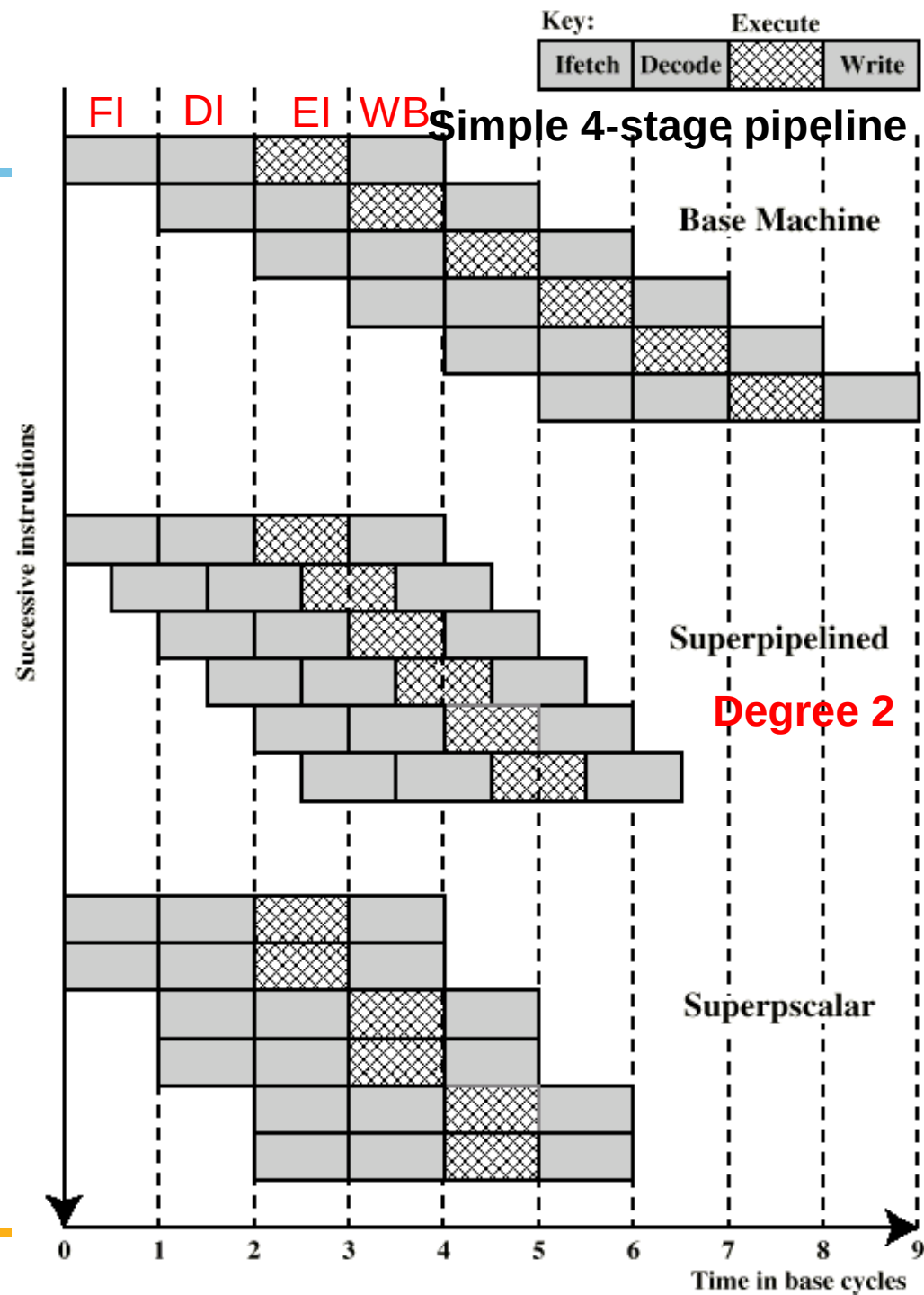


In this example, two integer, two floating-point, and one memory (either load or store) operations can be executing at the same time.

Superscalar v Superpipelined

Superpipelined

- ✓ Many pipeline stages need less than half a clock cycle
- ✓ Double internal clock speed gets two tasks per external clock cycle
- ✓ Superscalar allows parallel fetch execute





Superscalar approach

Instruction level parallelism

Compiler based optimisation

Hardware techniques

Limited by

- True data dependency
- Procedural dependency
- Resource conflicts
- Output dependency
- Antidependency

Effect of Dependencies

❑ True Data Dependency

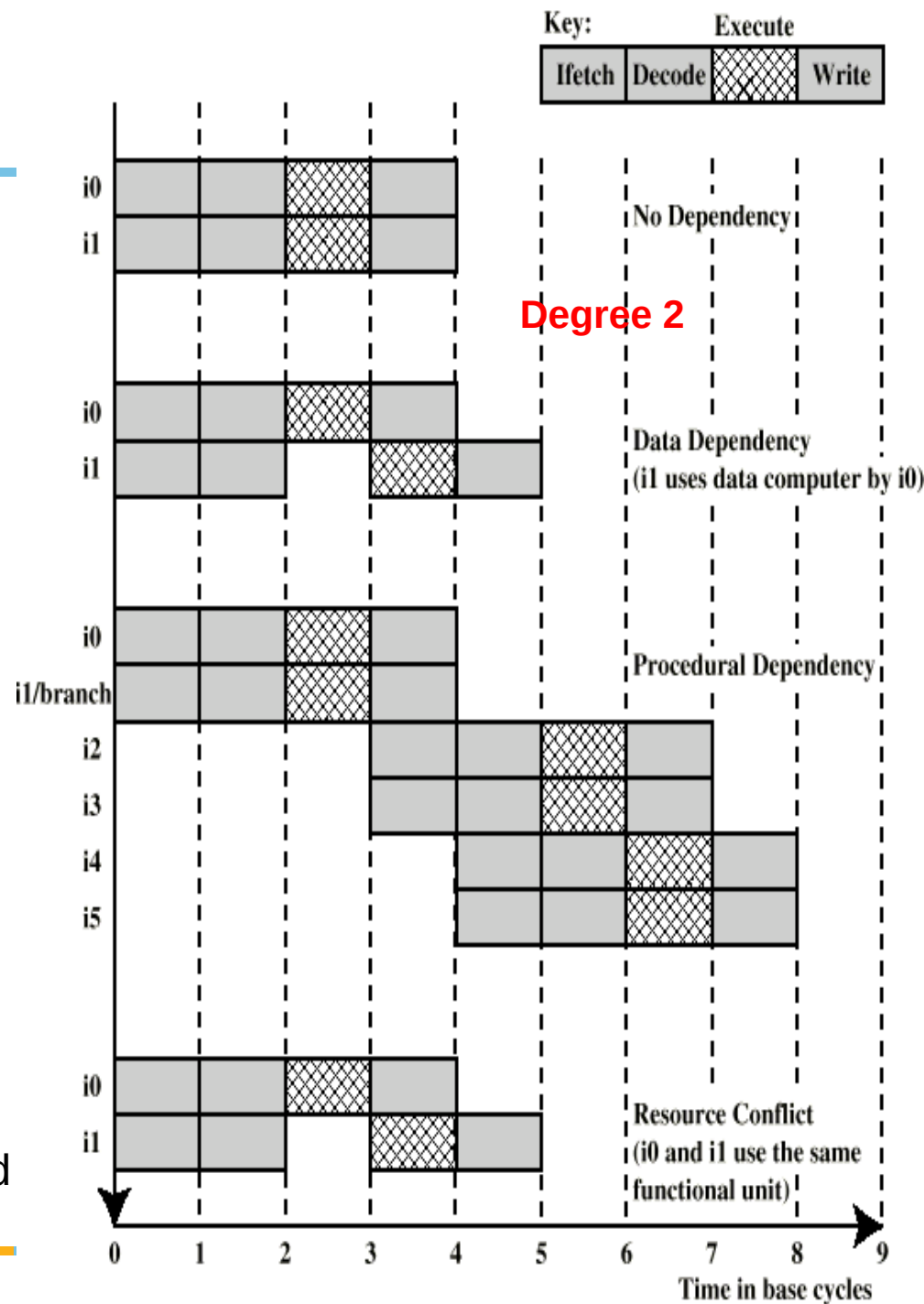
ADD r1, r2 ($r1 := r1 + r2;$)

MOVE r3, r1 ($r3 := r1;$)

- ✓ Can fetch and decode second instruction in parallel with first
- ✓ Can NOT execute second instruction until first is finished

❑ Procedural Dependency

- ✓ Can not execute instructions after a branch in parallel with instructions before a branch
- ✓ Also, if instruction length is not fixed, instructions have to be decoded to find out how many fetches are needed
- ✓ This prevents simultaneous fetches



Effect of Dependencies

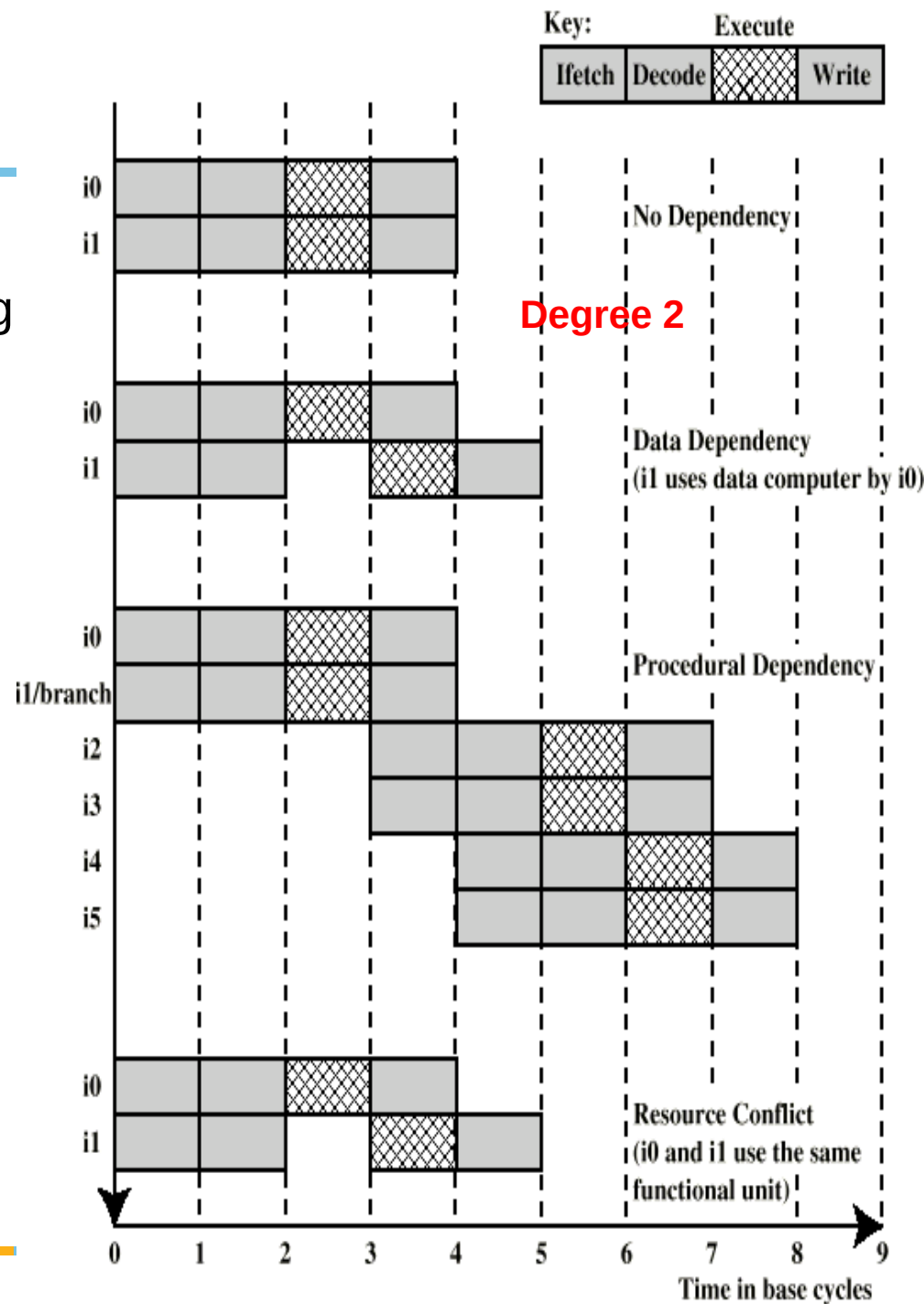
❑ Resource Conflict

- ✓ Two or more instructions requiring access to the same resource at the same time

➤ e.g. two arithmetic instructions

Can duplicate resources

➤ e.g. have two arithmetic units





Dependency due to memory Cycle

MOV EAX, eff ; load register EAX with the contents of
effective memory address eff

MOV EBX, EAX ;load EBX with the contents of EAX

❑ For RISC processor

- ✓ two or more cycles when the load is a cache hit
- ✓ tens or even hundreds of cycles for a cache miss
- ✓ sol: for the compiler to reorder instructions
- ❑ This scheme is less effective in the case of a superscalar pipeline:
 - ✓ The independent instructions executed during the load are likely to be executed on the first cycle of the load, leaving the processor with nothing to do until the load completes



Design Issues: Instruction-Level Parallelism and Machine Parallelism

❑ Instruction level parallelism

- ✓ Instructions in a sequence are independent
- ✓ Execution can be overlapped
- ✓ Governed by data and procedural dependency
- ✓ Example

Independent

Load R1 $\leftarrow$ R2

Add R3 $\leftarrow$ R3, "1"

Add R3 $\leftarrow$ R3, "1"

Showing Dependency

Add R4 $\leftarrow$ R3, R2

Add R4 $\leftarrow$ R4, R2

Store [R4] $\leftarrow$ R0

❑ Machine Parallelism

- ✓ Ability to take advantage of instruction level parallelism
- ✓ Governed by number of parallel pipelines



Instruction Issue Policy

- ❑ Instruction issue policy to refer to the protocol used to issue instructions
- ❑ Three types of orderings are important
 - ✓ Order in which instructions are fetched
 - ✓ Order in which instructions are executed
 - ✓ Order in which instructions change registers and memory
 - ✓ the processor must accommodate the various dependencies and conflicts.



superscalar instruction issue policies

- ☐ In-order issue with in-order completion
- ☐ In-order issue with out-of-order completion
- ☐ Out-of-order issue with out-of-order completion

In-Order Issue In-Order Completion

- ✓ Issue instructions in the order they occur
- ✓ Not very efficient
- ✓ May fetch >1 instruction
- ✓ Instructions must stall if necessary

In-Order Issue In-Order Completion (Diagram)

- ❑ I1 requires two cycles to execute
- ❑ I3 & I4 conflict for the same functional unit
- ❑ I5 depends on the value produced by I4
- ❑ I5 and I6 conflict for a functional unit

Decode		Execute			Write		Cycle
I1	I2						1
I3	I4	I1	I2				2
I3	I4	I1					3
	I4			I3	I1	I2	4
I5	I6			I4			5
	I6		I5		I3	I4	6
			I6				7
					I5	I6	8

Superscalar Instruction Issue and Completion Policies



In-Order Issue Out-of-Order Completion (Diagram)

Decode		Execute			Write		Cycle
11	12						1
13	14	11	12				2
	14	11		13	12		3
15	16			14	11	13	4
	16		15		14		5
			16		15		6
					16		7

Superscalar Instruction Issue and Completion Policies



In-Order Issue Out-of-Order Completion

Output dependency

- I1: $R3 \leftarrow R3 \text{ op } R5$
- I2: $R4 \leftarrow R3 + 1$
- I3: $R3 \leftarrow R5 + 1$
- I4: $R7 \leftarrow R3 \text{ op } R4$
- I2 depends on result of I1 – True data dependency: Instruction I2 cannot execute before instruction I1
- If I3 completes before I1, the result from I1 will be wrong - output (read-write) dependency
- I4 must wait for I3, because it uses a result produced by I3.
- I3 must complete after I1 to produce the correct output values in I4
- Eg. Of a **Write after write (WAW) dependency**.
- Here more difficult to deal with instruction interrupts and exceptions as at the time of interruption, instructions ahead of the instruction that caused the interrupt must have completed

Out-of-Order Issue Out-of-Order Completion

Decode		Window	Execute			Write		Cycle
11	12							1
13	14	11,12	11	12				2
15	16	13,14	11		13	12		3
		14,15,16		16	14	11	13	4
		15		15		14	16	5
						15		6

- ❑ Decouple decode pipeline from execution pipeline
- ❑ Can continue to fetch and decode until this pipeline is full
- ❑ When a functional unit becomes available an instruction can be executed
- ❑ Since instructions have been decoded, processor can look ahead



Antidependency

Read after-write(RAW) dependency

I1: $R3 \leftarrow R3 \text{ op } R5$

I2: $R4 \leftarrow R3 + 1$

I3: $R3 \leftarrow R5 + 1$

I4: $R7 \leftarrow R3 \text{ op } R4$

- ❑ Instruction I3 cannot complete execution before instruction I2 begins execution and has fetched its operands. This is so because I3 updates register R3, which is a source operand for I2.
- ❑ The term antidependency is used because the constraint is similar to that of a true data dependency, but reversed:



Reduced Instruction Set Computer

❑ Key features

- ✓ Large number of general purpose registers
- ✓ or use of compiler technology to optimize register use
- ✓ Limited and simple instruction set
- ✓ Emphasis on optimising the instruction pipeline
- ✓ All operations on data apply to data in registers
- ✓ The only operations that affect memory are load and store operations
- ✓ The instruction formats are few in number, with all instructions typically being one size.



Basic Pipelining: A "Typical" RISC ISA

- ❑ 32-bit fixed format instruction (3 formats)
 - ✓ ALU instructions, Load/Store, Branches and Jumps
- ❑ 32 32-bit GPR (R0 contains zero, DP take pair)
- ❑ 3-address, reg-reg arithmetic instruction
- ❑ Single address mode for load/store:
 - base + displacement
 - no indirection
- ❑ Simple branch conditions /Jump



A simple implementation of a RISC Instruction Set

Every instruction can be implemented in at most 5 cycles.

- Instruction fetch cycle (IF): send PC to memory and fetch the current instruction; Update the PC to the next PC by adding 4.
- Instruction decode/register fetch cycle (ID): decode the instruction and read the registers. For a branch, do the equality test on the registers, and compute the target address
- Execution/effective address cycle (EX): The ALU operates on the operands prepared in the prior cycle, performing:
 - Memory reference: ALU adds the base register and the offset.
 $\text{LW } 100(\text{R1}) \Rightarrow 100 + \text{R1}$
 - Register-Register ALU instruction: ALU performs the operation specified by the opcode on the values read from the register file. $\text{ADD R1,R2,R3} \Rightarrow \text{R1} + \text{R2}$
 - Register-Immediate ALU instruction: ALU performs the operation specified by the opcode on the first values read from the register file and the immediate. $\text{ADD R1,3,R2} \Rightarrow \text{R1} + 3$



A simple implementation of a RISC Instruction Set

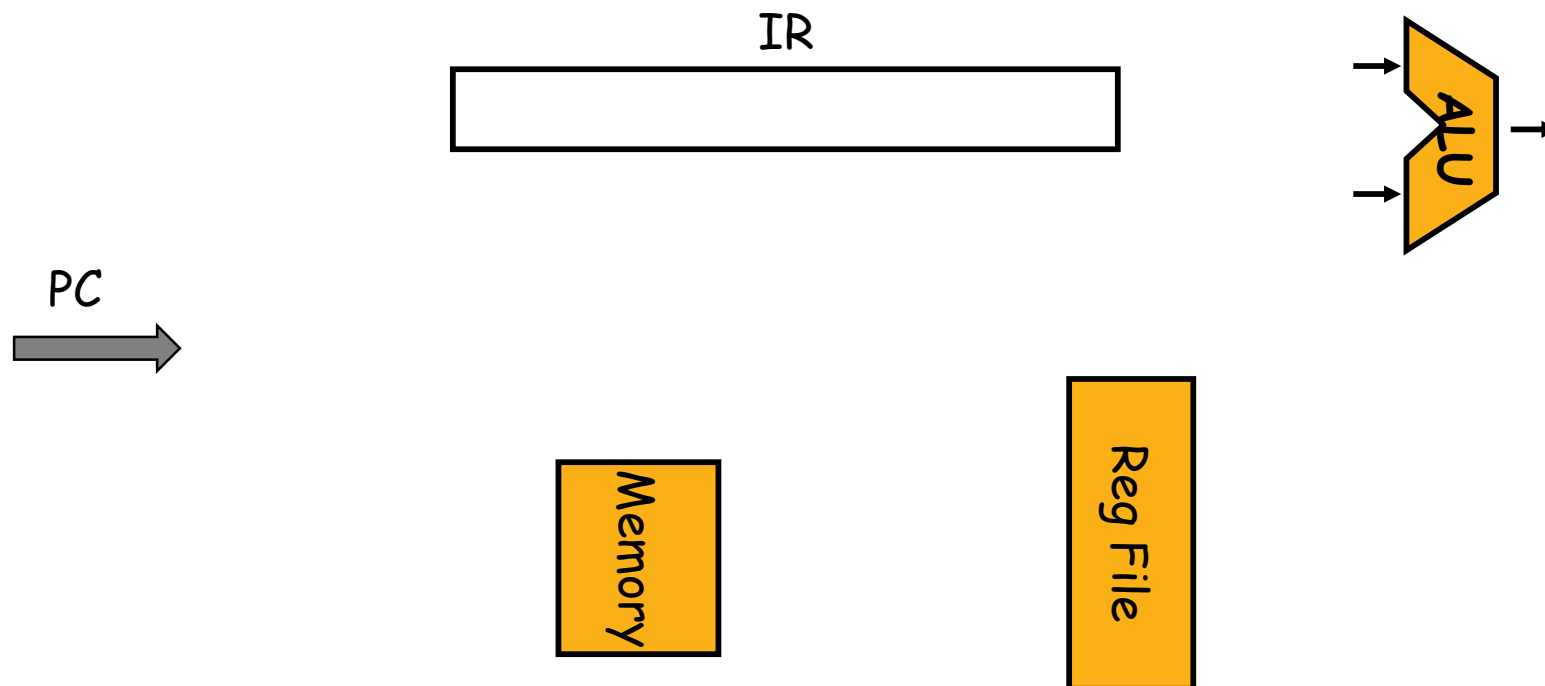
Every instruction can be implemented in at most 5 cycles.

- Memory access (**MEM**). Read the memory using the effective address computed in the previous cycle if the instruction is a load; If it is a store, write the data to the memory
- Write-back cycle (**WB**). For Register-Register ALU instruction or load instruction, write the result into the register file, whether it comes from the memory (for a load) or from the ALU (for an ALU instruction)

Branch instructions require 2 cycles, store instructions require 4 cycles, others require 5 cycles.

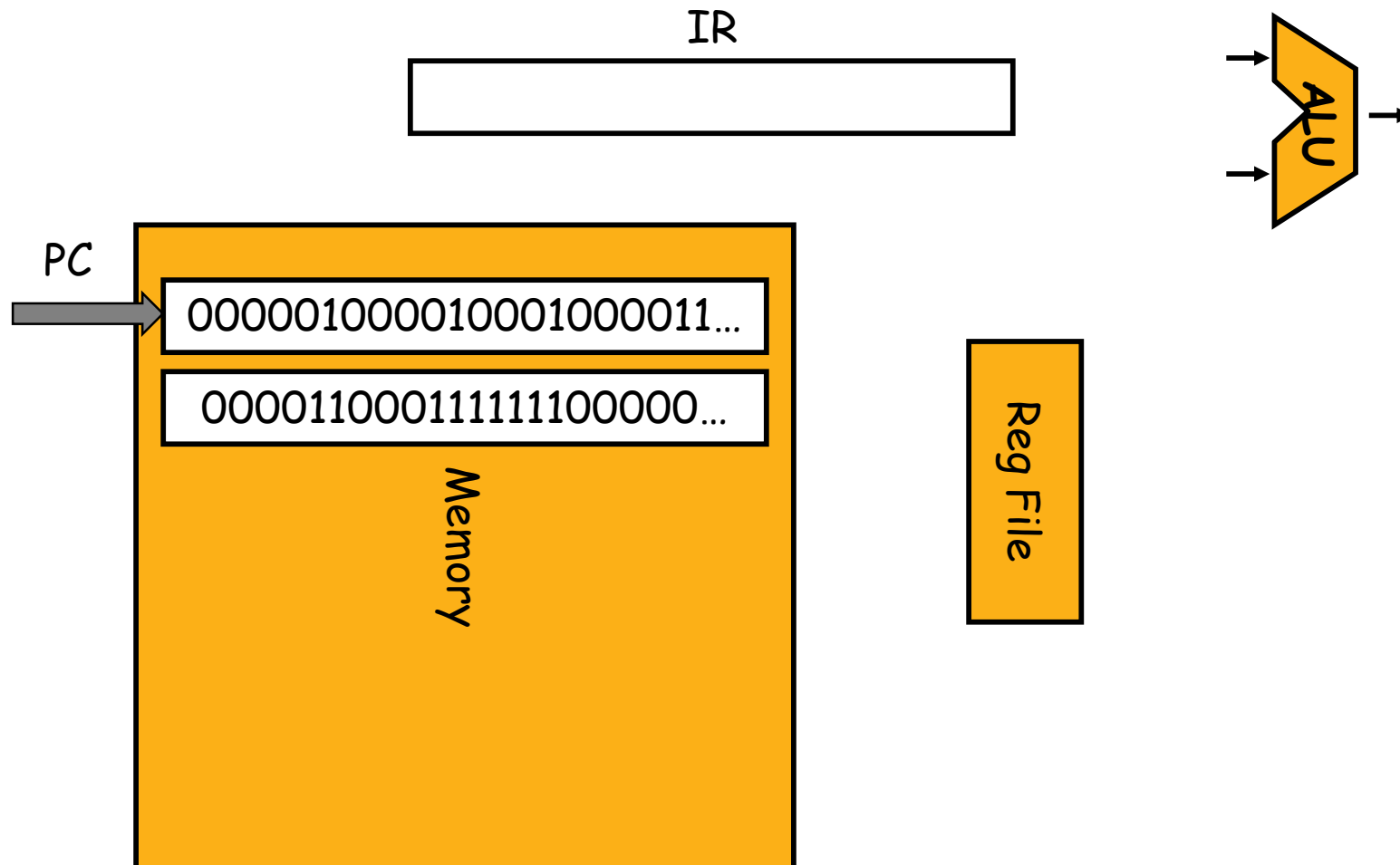
An example of an instruction execution

ADD R3, R1, R2 $\Rightarrow$ $R3 = R1 + R2;$

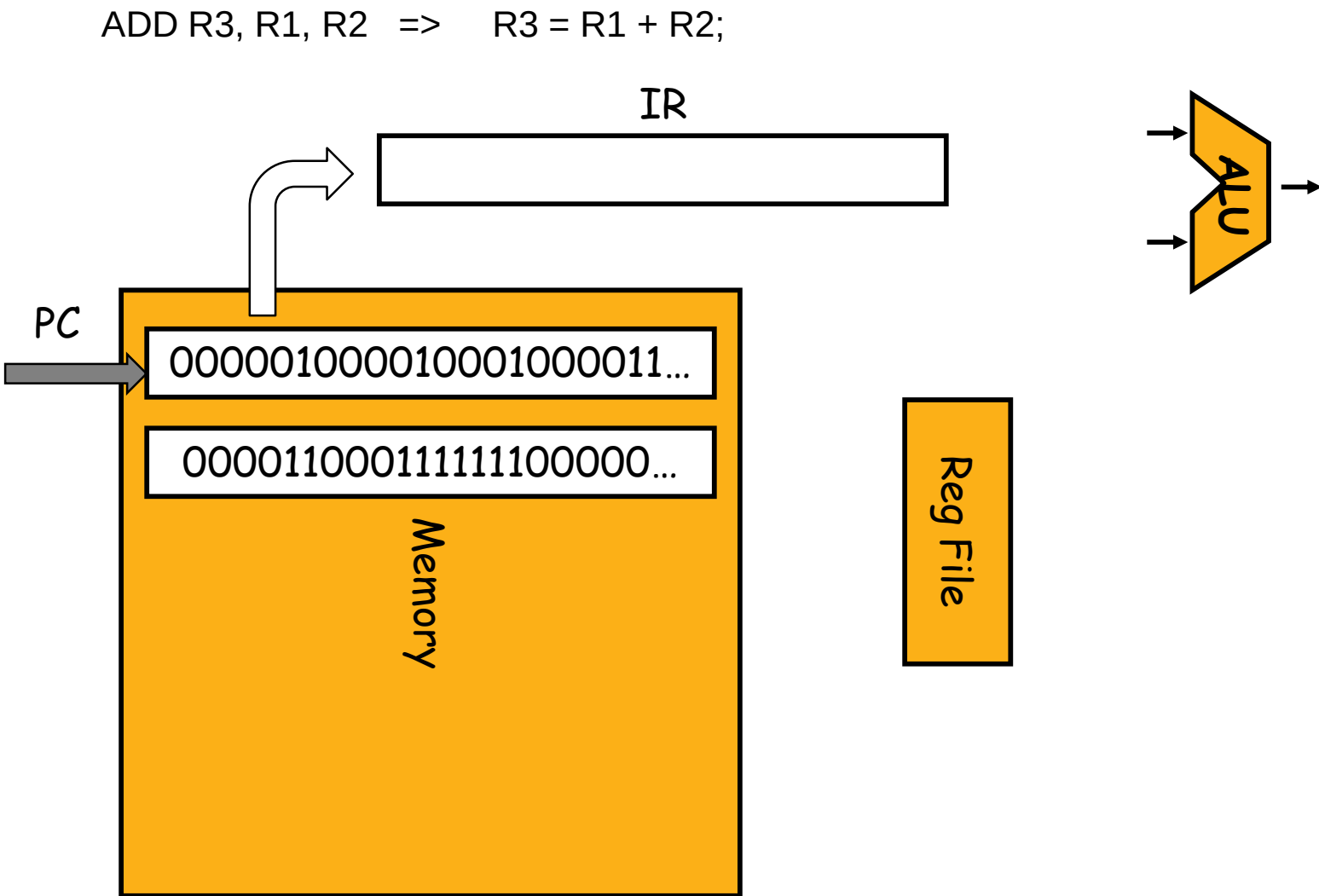


Instruction Fetch (IF)

ADD R3, R1, R2 => R3 = R1 + R2;

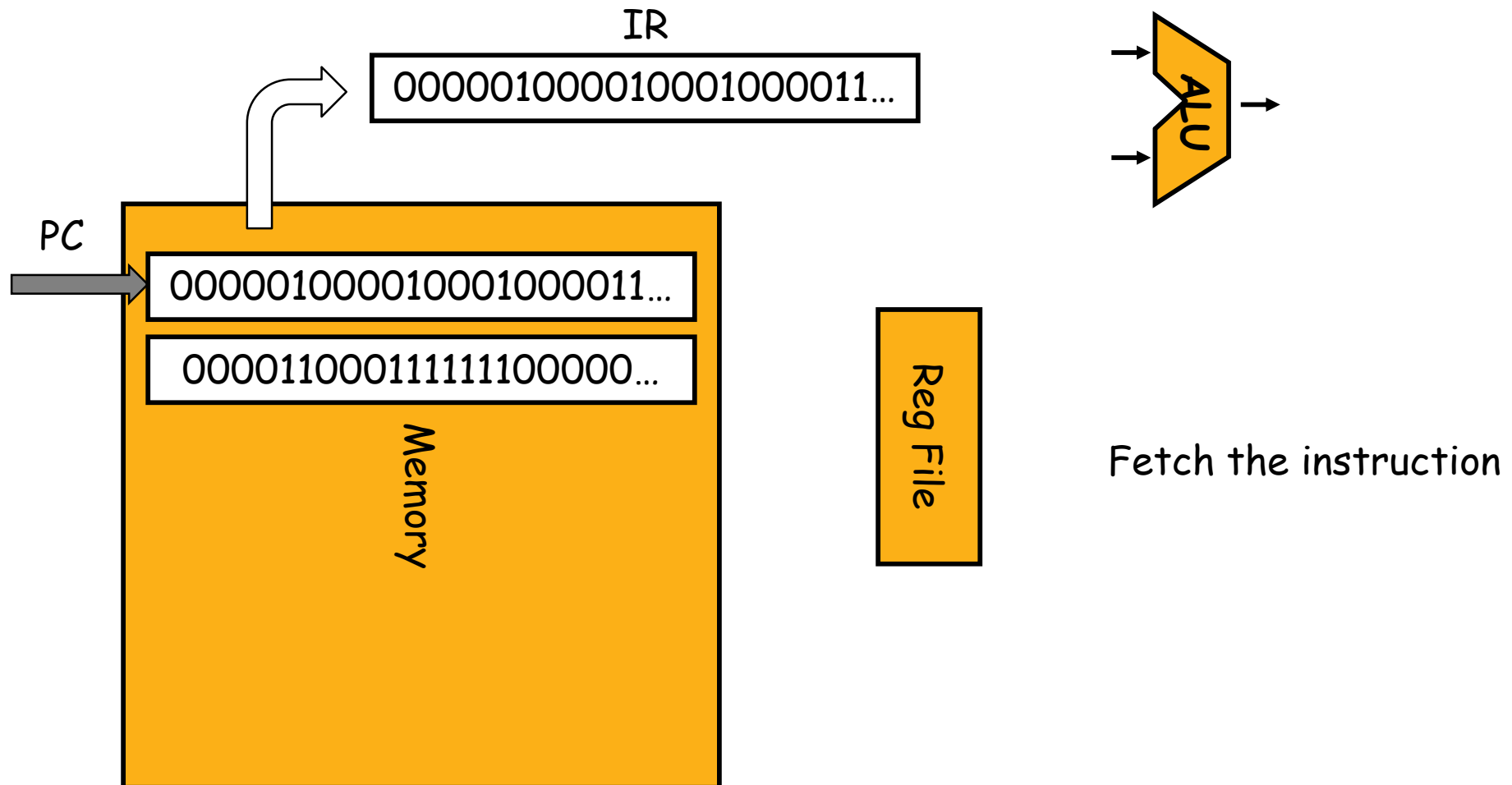


Instruction Fetch (IF)



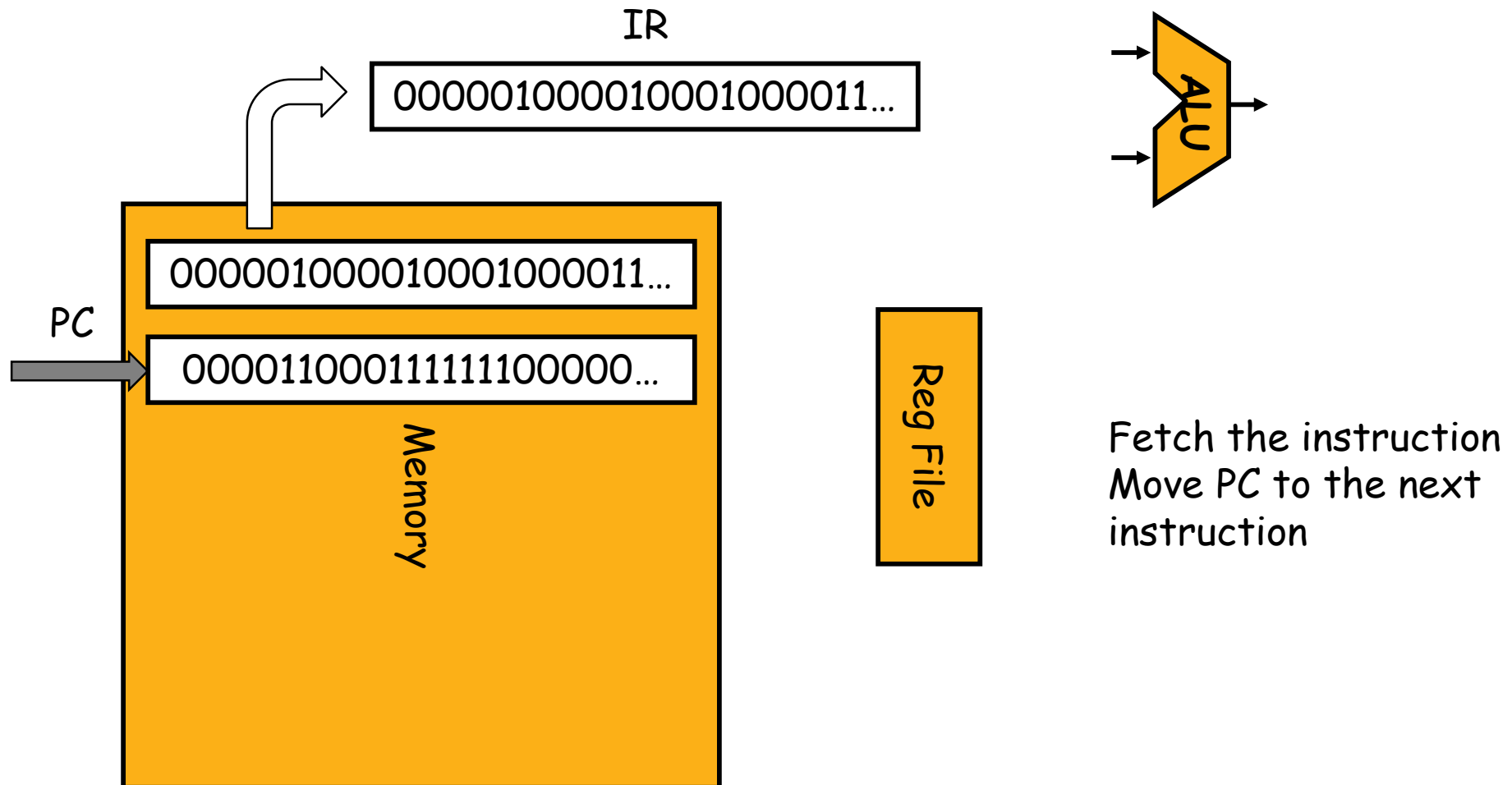
Instruction Fetch (IF)

ADD R3, R1, R2 => R3 = R1 + R2;



Instruction Fetch (IF)

ADD R3, R1, R2 => R3 = R1 + R2;

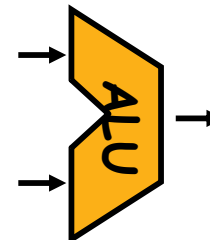


Instruction Decode (ID)

ADD R3, R1, R2 $\Rightarrow$ R3 = R1 + R2;

IR

000001000010001000011...



Reg File

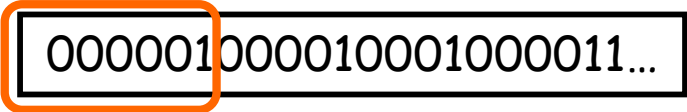
R0	0
R1	x
R2	y
R3	z
	...
R31	...

Memory

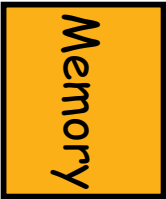
Instruction Decode (ID)

ADD R3, R1, R2 => R3 = R1 + R2;

IR

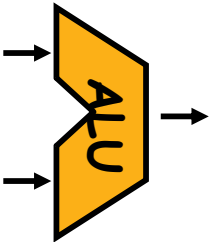


ADD



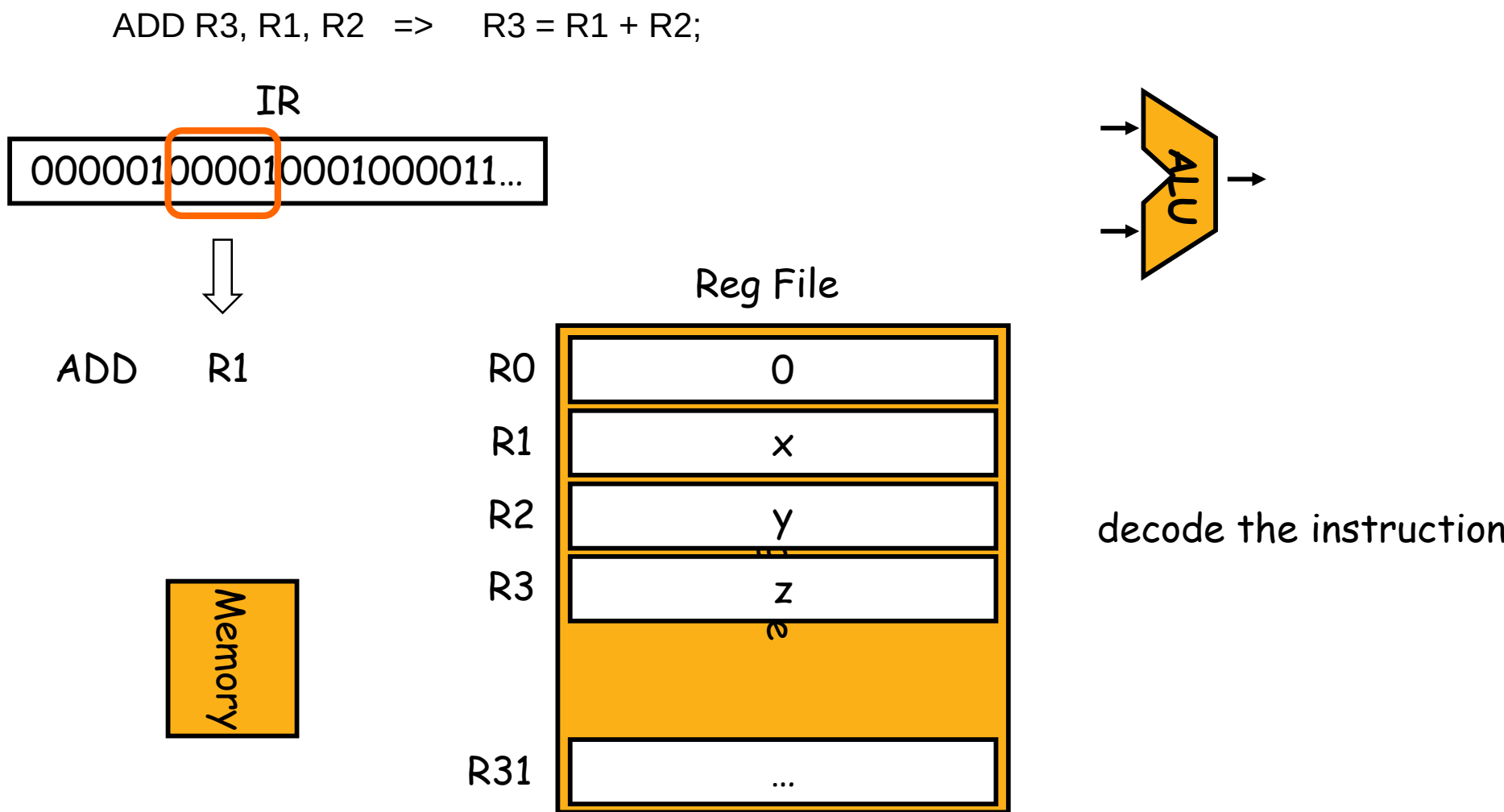
Reg File

R0	0
R1	x
R2	y
R3	z
...	
R31	...



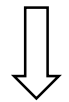
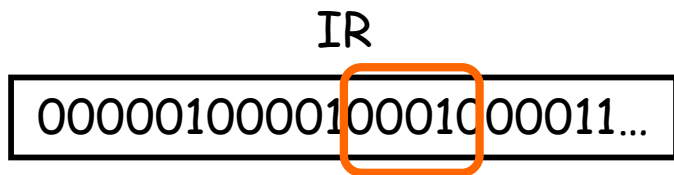
decode the instruction

Instruction Decode (ID)

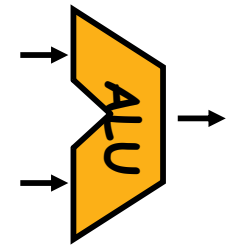
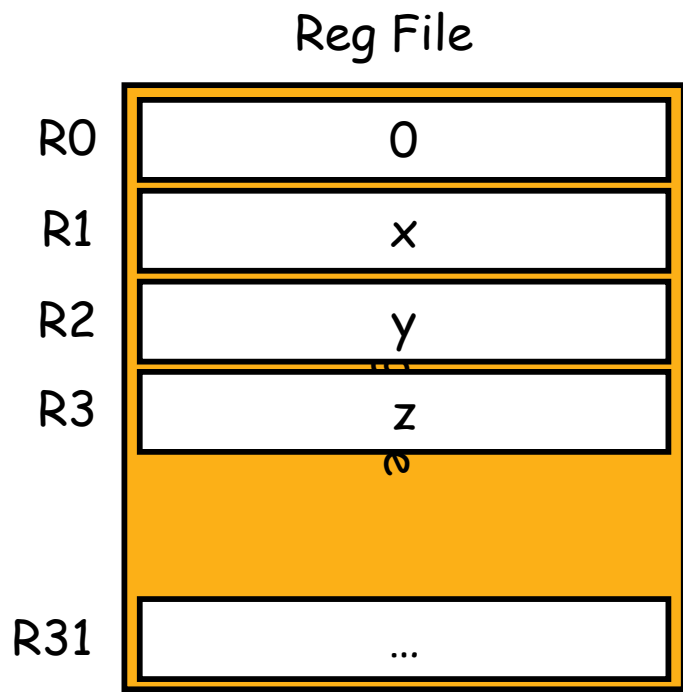
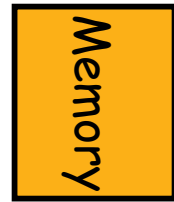


Instruction Decode (ID)

ADD R3, R1, R2 => R3 = R1 + R2;



ADD R1 R2

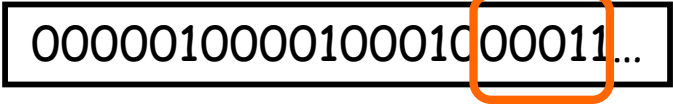


decode the instruction

Instruction Decode (ID)

ADD R3, R1, R2 => R3 = R1 + R2;

IR



ADD R1 R2 R3 R0

R1

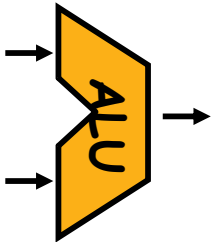
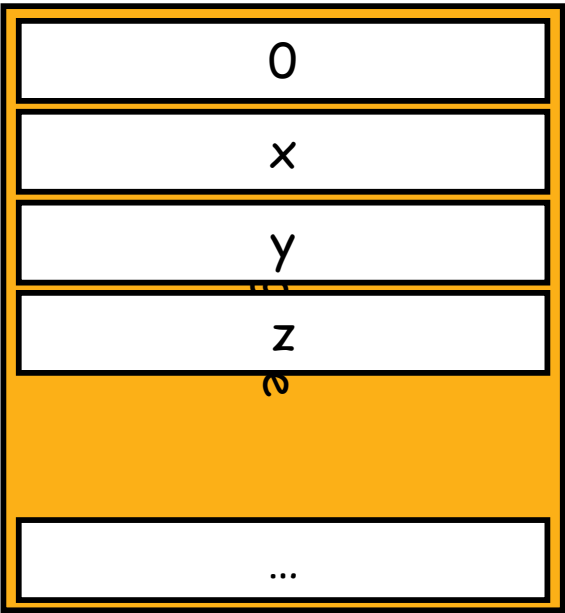
R2

R3

R31



Reg File



Instruction Decode (ID)

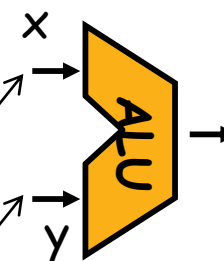
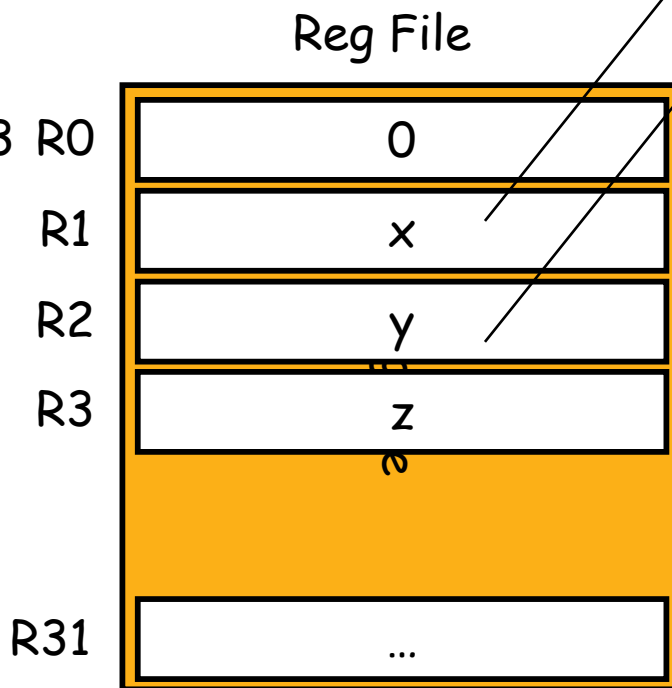
ADD R3, R1, R2 $\Rightarrow$ R3 = R1 + R2;

IR

0000010000100010000011...

ADD R1 R2 R3 R0

Memory



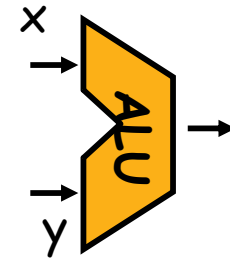
Execution (EX)



ADD R3, R1, R2 => R3 = R1 + R2;

IR

0000010000100010000011...



Memory

Reg File

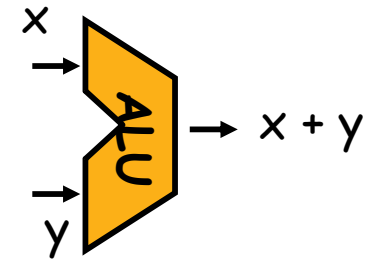
Execution (EX)



ADD R3, R1, R2 $\Rightarrow$ R3 = R1 + R2;

IR

0000010000100010000011...



Memory

Reg File

Execute the operation

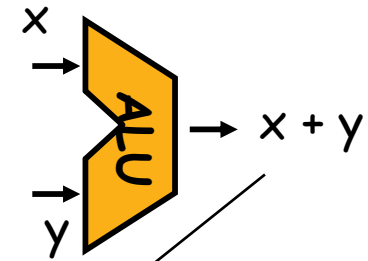
Writeback (WB)



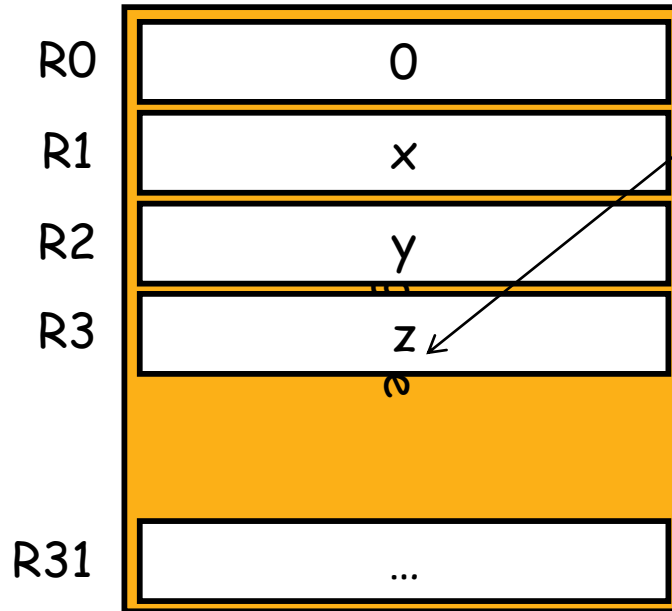
ADD R3, R1, R2 $\Rightarrow$ $R3 = R1 + R2$;

IR

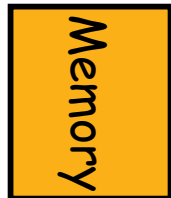
000001000010001000011...



Reg File



Write the result to the register



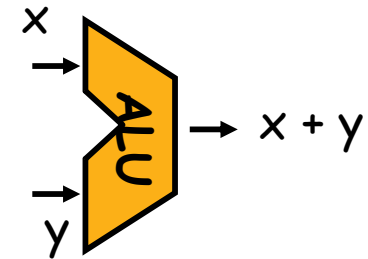
Writeback (WB)



ADD R3, R1, R2 $\Rightarrow$ $R3 = R1 + R2$;

IR

0000010000100010000011...



Reg File

R0	0
R1	x
R2	y
R3	x + y
...	
R31	...

Memory

Write the result to the register